Oracle Database 19c

DB 튜닝

DBTUNING · v2 2026-08-15

목차 ① SQL 부문

- SQL 튜닝 입문과 측정 도구 — ch01 SQL 튜닝이란 · ch02 SQL 실행 과정 · ch03 실행계획 기초 · ch04 SQL 성능 측정
- 데이터 접근·인덱스·통계 — ch05 Full vs Index Scan · ch06 인덱스 설계 · ch07 SARGable SQL · ch08 Optimizer 와 통계(이동)
- 조인·서브쿼리·집합·정렬 — ch09 JOIN 실행 원리 · ch10 Subquery/EXISTS/IN · ch11 UNION/DISTINCT · ch12 SORT/GROUP BY
- 실무 SQL 문제와 SQL 부문 평가 — ch13 페이징 · ch14 실무 SQL 문제 · 심화 30제 V · 최종실습시험 03·04

목차 ② 인스턴스 부문

• 인스턴스 튜닝 개요와 기반 — 브리지 01 · ch15 성능 튜닝 개요 · ch16 설계와 개발(1h) · ch17 방법론과 긴급 대응 · ch18 DB 구성

• 측정 기준과 통계 인프라 — ch19 DB Time · ch20 AWR 인프라 · ch21 히스토그램·확장 통계

• 보고서 판독과 기간 비교 — ch22 AWR 보고서 읽기 · ch23 기간 비교(1h) · ch24 ADDM · ch25 ASH

• V$ 진단·대기 이벤트·모니터링·메모리 구조 — ch26 V$ 절차 · ch27 대기 이벤트 · ch28 상시 모니터링(1h) · ch29 메모리 할당(1h)

• 어드바이저와 SGA — ch30 Advisor 프레임워크 · ch31 버퍼 캐시·리두 버퍼 · ch32 공유 풀·라지 풀

• 결과 캐시·PGA·I/O, 총정리, 종합 평가 — ch33 결과 캐시·PGA · ch34 I/O·OS · 총정리 · 종합 진단 시나리오

SQL 튜닝 입문과 측정 도구

- SQLT ch01 · ch02 · ch03 · ch04
- 측정 도구(DBMS_XPLAN)를 먼저 쥔다

Oracle SQL 튜닝 과정 · ch01

SQL 튜닝이란?

SQL 성능의 정의 · CPU / I/O / Memory · 실행계획 실측 읽기 · IN vs EXISTS 통념 검증 | Oracle Database 19c · 진료비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - SQL 튜닝을 '자원을 덜 쓰는 방법을 실측 근거로 고르는 작업'으로 정의한다
  - SQL 성능을 '빠르다/느리다'가 아니라 자원(CPU·I/O·Memory) 소비량으로 설명한다
  - 실행계획의 Buffers를 논리적 I/O 지표로 읽고 물리적 I/O와 구분한다
  - OMem·1Mem·Used-Mem 컬럼으로 작업 영역(PGA) 사용량을 확인한다
  - 같은 결과를 내는 IN·EXISTS·JOIN+DISTINCT 세 SQL을 작성해 비교한다
  - 세 실행계획의 오퍼레이션 구성을 읽고 자원 소비 차이가 생긴 단계를 지목한다
  - 'EXISTS가 IN보다 빠르다'류의 통념을 실측으로 검증하는 절차를 설명한다
  - Buffers가 같은데 A-Time이 다르면 캐시 상태(물리 I/O)를 의심한다
  - DB 성능 문제의 원인 갈래를 분류하고 SQL 튜닝의 범위를 설정한다
  - 자원 절약이 다차원적임을 이해하고 어떤 자원을 아꼈는지 구분해 판단한다

이 장의 구성 (Agenda)

- 1.1 Why — SQL 튜닝을 왜 배우는가 · 통념의 위험
- 1.2 Concept — SQL 성능의 정의 · 성능 문제의 원인 갈래
- 1.3 Oracle Internals — CPU · I/O · Memory 세 관점
- 1.4 Example — 세 가지 동등 SQL (IN / EXISTS / JOIN+DISTINCT)
- 1.5 Execution Plan — 세 버전 실행계획 실측 비교
- 1.6 Bad SQL — 'EXISTS가 IN보다 빠르다' 통념 검증
- 1.7 Tuning 예고 — 조인 제거와 이후 챕터 로드맵
- 1.8 Benchmark — 자원 관점 종합 비교
- 1.9~1.10 실습 안내 · 요약 (Review)

1.1

SQL 튜닝을 왜 배우는가

- SQL 튜닝의 정의 — 자원과 실측
- 통념이 위험한 이유
- 이 차시의 목표와 범위

SQL 튜닝이란 무엇인가

- SQL 튜닝은 문법을 화려하게 쓰는 기술이 아니다
  - 같은 결과를 내는 여러 방법 중 자원을 덜 쓰는 방법을 골라내는 작업
  - 고르는 근거는 감이나 관행이 아니라 실측이다
- '같은 결과'가 전제라는 점이 중요하다
  - 결과가 달라지면 그것은 튜닝이 아니라 요구사항 변경이다
  - 이번 장의 세 SQL은 결과가 완전히 같은 상태에서 자원만 비교한다
- 튜닝의 산출물은 '더 빠른 SQL'이 아니라 '근거가 붙은 선택'이다
  - 왜 이 방식을 골랐는지 설명할 수 없으면 재현·유지보수가 불가능하다

> **참고:**
> 이 과정의 모든 실행계획·소요 시간·Buffers는 진료비청구심사 실습 DB에서 직접 캡처한 실측값이다.

통념은 왜 위험한가

- 개발자 사이에는 '이 방식이 항상 더 빠르다'류의 통념이 널리 퍼져 있다
  - 예: EXISTS가 IN보다 빠르다 / 서브쿼리보다 조인이 좋다
  - 예: 인덱스를 타면 무조건 빠르다 / DISTINCT는 항상 비싸다
- 통념은 대개 '어떤 버전, 어떤 데이터, 어떤 상황'이라는 조건이 떨어져 나간 결과다
  - 버전이 바뀌면 옵티마이저의 변환 능력이 달라진다
  - 데이터 분포가 바뀌면 유리한 접근 경로가 바뀐다
- 1.6절에서 실측으로 통념 하나를 직접 검증한다

이 차시의 목표와 범위

- 목표 1 — SQL 성능을 판단하는 기준을 자원 관점에서 세운다
- 목표 2 — 판단을 감이 아니라 실측으로 하는 습관을 세운다
- 이 장에서 다루지 않는 것
  - 인덱스 설계, 바인드 변수, 힌트 등 구체적 개선 기법 — 이후 챕터
  - 동시성·락 경합 문제 — 본 과정 범위 밖
- 이 장은 DDL 변경이 없다 — 스키마를 손대지 않고 읽기만 한다
  - 같은 데이터·같은 스키마 위에서 SQL 작성 방식만 바꿔 비교한다

1.2

SQL 성능이란 무엇인가

- 성능의 정의 — 자원 소비량
- 시간 지표가 불안정한 이유
- 성능 문제의 원인 갈래
- SQL 튜닝과 애플리케이션 튜닝

성능의 정의 — 자원 소비량

- SQL 성능은 '빠르다/느리다'가 아니다
  - 얼마만큼의 자원(CPU, I/O, 메모리)을 써서 결과를 만들었는가로 정의한다
- 왜 시간이 아니라 자원인가
  - 같은 시간이 걸려도 자원 소비량이 다르면 동시 사용자가 늘었을 때 체감 성능이 달라진다
  - 혼자 쓸 때 잘 나오던 SQL이 부하가 걸리면 무너지는 전형적인 원인
- 그래서 '몇 초 걸렸다'보다 자원 소비량(Buffers 등)이 더 안정적인 지표다
  - 자원 지표는 서버 상태·캐시 상태에 덜 흔들린다

시간 지표는 왜 흔들리는가

- A-Time(실제 소요 시간)에 섞여 들어오는 요인들
  - 블록이 이미 버퍼 캐시에 있었는가, 디스크에서 새로 읽었는가
  - 그 순간 서버에 다른 부하가 있었는가
  - 같은 SQL을 몇 번째로 실행했는가 (실행 순서)
- 자원 지표는 이런 요인에 상대적으로 덜 흔들린다
  - Buffers는 '몇 개의 블록을 만졌는가'이므로 캐시 여부와 무관하게 유사한 값이 나온다
- 실무 원칙 — 시간은 참고 지표, 자원은 판단 지표
  - 시간이 줄었는데 Buffers가 그대로면 개선이 아니라 캐시 효과일 수 있다

DB 성능 문제의 원인 갈래

- SQL 자체의 접근 경로 문제
  - 불필요한 Full Scan, 비효율적인 조인 순서·조인 방식
- 인덱스 설계 문제
  - 필요한 인덱스가 없거나, 있어도 선두 컬럼이 맞지 않아 쓰이지 못함
- 통계정보·카디널리티 추정 문제
  - 옵티마이저의 행 수 추정이 실제와 크게 어긋나면 잘못된 계획을 고른다
- 동시성 / 락 경합 문제 — 본 과정 범위 밖
  - SQL 자체는 정상인데 대기(wait) 때문에 느린 경우

SQL 튜닝과 애플리케이션 튜닝

- 위험한 가정 — '같은 결과만 나오면 SQL 작성 방식은 상관없다'
  - 애플리케이션 코드 리뷰에서 SQL을 결과 기준으로만 보는 순간 성능은 관리 대상에서 빠진다
- 결과가 완전히 같아도 내부 자원 소비는 크게 달라질 수 있다
  - 1.5절·1.8절 사례에서 세 SQL의 결과 건수는 동일하지만 Buffers는 다르다
- 그래서 코드 리뷰에는 실행계획 확인이 함께 들어가야 한다
  - SQL의 겉모습(길이·문법)만으로는 자원 소비를 예측할 수 없다
  - 이번 장의 세 버전 비교가 그 근거가 된다

1.3

자원의 세 관점 — CPU · I/O · Memory

- CPU 관점 — 연산량
- I/O 관점 — Buffers와 물리 I/O
- Memory 관점 — 작업 영역(PGA)
- 좋은 SQL과 나쁜 SQL의 판단 기준

CPU 관점 — 연산량

- CPU는 '무엇을 얼마나 계산했는가'로 소비된다
  - 조건 비교 — WHERE 절의 필터를 행마다 평가
  - 정렬 — ORDER BY, GROUP BY, DISTINCT 등에서 발생
  - 해시 연산 — 해시 조인·해시 그룹핑에서 해시 값 계산과 탐색
- 실행계획에서는 오퍼레이션의 종류와 처리 행 수로 가늠한다
  - 많은 행을 통과시키는 단계일수록 연산 부담이 커진다
  - 같은 결과라도 오퍼레이션 구성이 다르면 연산량이 달라진다

I/O 관점 — Buffers와 물리 I/O

- 핵심 질문 — 몇 개의 블록을 읽었는가
- Buffers = 논리적 I/O
  - 버퍼 캐시에서 블록을 가져온 횟수까지 포함한 전체 블록 접근 수
  - 실행계획의 Buffers 컬럼으로 단계별·누적으로 확인
- 메모리에 없으면 물리적 I/O(디스크 접근)가 추가로 발생한다
  - 같은 Buffers라도 물리 I/O 여부에 따라 소요 시간은 크게 달라진다
  - 1.6절에서 정확히 이 현상을 실측 데이터로 확인한다

Memory 관점 — 작업 영역(PGA)

- 해시 조인·정렬 등은 세션 전용 작업 영역(PGA)을 사용한다
  - 중간 결과를 쌓아 두어야 하는 연산이 메모리를 요구한다
- 실행계획에서 확인하는 컬럼
  - OMem — 옵티마이저가 추정한 필요 메모리
  - 1Mem — 한 번에 처리(one-pass)하는 데 필요한 메모리 추정
  - Used-Mem — 실제로 사용한 작업 영역 크기
- 작업 영역이 부족하면 임시 공간(TEMP)으로 넘쳐 디스크 I/O가 발생한다
  - 1.8절 종합 비교에서 세 버전의 Used-Mem을 나란히 확인한다

좋은 SQL과 나쁜 SQL을 가르는 기준

- 문법의 길이는 기준이 아니다
  - 짧다고 좋은 SQL이 아니고, 길다고 나쁜 SQL도 아니다
- 판단은 실측 지표로 한다 — 체크리스트
  - ① 실제 접근한 블록 수(Buffers)가 얼마인가
  - ② 사용한 작업 영역(Used-Mem)이 얼마인가
  - ③ 실행계획의 오퍼레이션 구성이 어떠한가 — 불필요한 단계가 있는가
  - ④ 추정(E-Rows)과 실제(A-Rows)가 크게 어긋나지 않는가
- '느낌'이 아니라 실측 — 이 장의 결론이자 과정 전체의 원칙

1.4

실측 소재 — 세 가지 동등 SQL

- 소재와 대상 건수
- A. IN 방식
- B. EXISTS 방식
- C. JOIN + DISTINCT 방식

실측 소재 — 주사제 처방 청구 찾기

- 목표 — 주사제 카테고리 약품을 하나 이상 처방받은 청구 건수 확인
  - MEDICAL_CLAIMS 중 CLAIM_DETAILS에서 DRUG_MASTER.CATEGORY = '주사제'인 건
- 사전 확인 결과
  - 대상 건수 79,342건 — 전체 30만 건의 약 26%
  - 전체의 4분의 1가량을 걸러 내는, 선택도가 낮지 않은 조건
- 같은 목표를 세 가지 방식으로 작성해 비교한다
  - A. IN 서브쿼리 / B. EXISTS 서브쿼리 / C. JOIN + DISTINCT
  - 세 버전의 결과 건수는 완전히 동일하다

A. IN 방식

서브쿼리 결과 집합에 존재하는지를 IN으로 확인한다

```sql
-- A. IN 방식
SELECT COUNT(*)
FROM medical_claims
WHERE claim_id IN (
  SELECT cd.claim_id FROM claim_details cd, drug_master dm
  WHERE cd.drug_code = dm.drug_code AND dm.category = '주사제'
);

-- 서브쿼리는 '주사제를 처방한 청구 번호 목록'을 만들고
-- 바깥 쿼리는 그 목록에 자기 claim_id가 들어 있는지만 확인한다
-- 목록에 같은 claim_id가 여러 번 나와도 결과 건수에는 영향이 없다
```

B. EXISTS 방식

바깥 행마다 조건을 만족하는 행이 존재하는지 확인한다

```sql
-- B. EXISTS 방식
SELECT COUNT(*)
FROM medical_claims mc
WHERE EXISTS (
  SELECT 1 FROM claim_details cd, drug_master dm
  WHERE cd.claim_id = mc.claim_id
    AND cd.drug_code = dm.drug_code
    AND dm.category = '주사제'
);

-- 서브쿼리가 바깥 컬럼(mc.claim_id)을 참조하는 상관 서브쿼리
-- SELECT 목록의 1은 아무 의미가 없다 — 존재 여부만 판정하기 때문
```

C. JOIN + DISTINCT 방식

세 테이블을 직접 조인하고 중복을 제거해 센다

```
-- C. JOIN + DISTINCT 방식
SELECT COUNT(DISTINCT mc.claim_id)
FROM medical_claims mc, claim_details cd, drug_master dm
WHERE mc.claim_id = cd.claim_id
  AND cd.drug_code  = dm.drug_code
  AND dm.category   = '주사제';

-- 서브쿼리 없이 세 테이블을 한 번에 조인한다
-- 조인 결과에는 한 청구가 여러 번 나올 수 있으므로
-- COUNT(DISTINCT ...)로 중복을 제거해야 결과가 A·B와 같아진다
-- SELECT 절이 실제로 필요로 하는 mc 컬럼은 claim_id 하나뿐이다
```

1.5

실행계획 — 세 버전 실측 비교

- A. IN 방식 실행계획과 읽는 법
- B. EXISTS 방식 실행계획
- A와 B의 구조 비교
- C. JOIN+DISTINCT 실행계획의 특이점

A. IN 방식 실행계획

SQL_ID 7z0a37r0usf76 — CLAIM_DETAILS 가 캐시에 없어 Reads 열이 나타난다

Id	Operation	Name	E-Rows	A-Rows	A-Time	Buffers	Reads
0	SELECT STATEMENT			1	00:00:00.10	8659	6709
1	SORT AGGREGATE		1	1	00:00:00.10	8659	6709
2	HASH JOIN RIGHT SEMI		296K	79342	00:00:00.10	8659	6709
3	VIEW	VW_NSO_1	299K	90430	00:00:00.02	6768	6709
4	HASH JOIN		299K	90430	00:00:00.02	6768	6709
5	TABLE ACCESS FULL	DRUG_MASTER	3333	1000	00:00:00.01	91	89
6	TABLE ACCESS FULL	CLAIM_DETAILS	899K	899K	00:00:00.01	6621	6613
7	INDEX FAST FULL SCAN	PK_MEDICAL_CLAIMS	300K	300K	00:00:00.02	1891	0

A 실행계획 읽는 법

- 읽는 순서 — 들여쓰기가 깊은 안쪽부터, 같은 깊이는 위에서 아래로
- 컬럼의 의미
  - E-Rows 옵티마이저의 추정 행 수 / A-Rows 실제 처리한 행 수
  - A-Time 그 단계까지 누적된 실제 소요 시간
  - Buffers 그 단계까지 누적된 논리적 I/O 블록 수
- HASH JOIN RIGHT SEMI — IN 서브쿼리가 세미조인으로 변환된 형태
  - 매칭이 여러 건이어도 결과는 한 건으로만 남는다 (IN의 의미론과 일치)
- 5번 단계 — E-Rows 3333 vs A-Rows 1000, 추정이 실제보다 크다

B. EXISTS 방식 실행계획

SQL_ID 7jgu8vt9cppsb — 구조는 A와 동일, VIEW 이름만 VW_SQ_1. Reads 열이 없다

Id	Operation	Name	E-Rows	A-Rows	A-Time	Buffers
0	SELECT STATEMENT			1	00:00:00.05	8604
1	SORT AGGREGATE		1	1	00:00:00.05	8604
2	HASH JOIN RIGHT SEMI		296K	79342	00:00:00.06	8604
3	VIEW	VW_SQ_1	299K	90430	00:00:00.02	6713
4	HASH JOIN		299K	90430	00:00:00.02	6713
5	TABLE ACCESS FULL	DRUG_MASTER	3333	1000	00:00:00.01	91
6	TABLE ACCESS FULL	CLAIM_DETAILS	899K	899K	00:00:00.01	6621
7	INDEX FAST FULL SCAN	PK_MEDICAL_CLAIMS	300K	300K	00:00:00.01	1891

A와 B — 이름표만 다른 같은 계획

- 두 실행계획에서 다른 것은 VIEW 이름 하나뿐이다
  - A는 VW_NSO_1, B는 VW_SQ_1 — Oracle이 변환 과정에서 붙이는 내부 식별자
  - 오퍼레이션 구성, E-Rows, A-Rows, Buffers는 모두 동일
- 왜 같아지는가 — 옵티마이저의 쿼리 변환
  - IN 서브쿼리와 EXISTS 서브쿼리를 같은 세미조인(HASH JOIN RIGHT SEMI) 형태로 변환
  - 변환 단계에서 두 문장의 문법적 차이가 사라진다
- 그런데 A-Time은 다르다 — 0.10초 vs 0.05초, Buffers는 둘 다 8,604(A 는 물리 읽기 몫 55 가 더해져 8,659)
  - 같은 계획·같은 자원인데 시간만 다르다면 원인은 계획 밖에 있다

C. JOIN + DISTINCT 실행계획

SQL_ID dx438vv8qg166 — MEDICAL_CLAIMS 접근이 보이지 않는다

```
| Id | Operation           | Name          | E-Rows | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT    |               |        |      1 |00:00:00.04 |    6713 |
| 1  |  SORT AGGREGATE     |               |      1 |      1 |00:00:00.04 |    6713 |
| 2  |   VIEW              | VW_DAG_0      |   296K |  79342 |00:00:00.04 |    6713 |
| 3  |    HASH GROUP BY    |               |   296K |  79342 |00:00:00.04 |    6713 |
| 4  |     HASH JOIN       |               |   299K |  90430 |00:00:00.03 |    6713 |
| 5  |      TABLE ACCESS FULL| DRUG_MASTER |   3333 |   1000 |00:00:00.01 |      91 |
| 6  |      TABLE ACCESS FULL| CLAIM_DETAILS|  899K |   899K |00:00:00.02 |    6621 |
```

C의 특이점 — 사라진 MEDICAL_CLAIMS

- FROM 절에 medical_claims mc가 분명히 있는데 실행계획에 나타나지 않는다
  - MEDICAL_CLAIMS에 대한 TABLE ACCESS도, PK_MEDICAL_CLAIMS 인덱스 접근도 없다
- A·B에는 있었던 단계가 통째로 빠졌다
  - A·B의 7번 단계 INDEX FAST FULL SCAN — Buffers 1,891
  - 총 Buffers 차이 8,604 − 6,713 = 1,891, 사라진 단계의 값과 정확히 일치
- 옵티마이저가 테이블 접근 자체를 제거할 수 있다는 뜻
  - 상세 원리(조인 제거)는 ch09 JOIN 실행 원리에서 다룬다

1.6

통념 검증 — EXISTS가 IN보다 빠른가

- 관측 사실 정리
- 원인 추론 — 캐시 상태
- 결론과 벤치마크 방법론

관측 사실 정리

- 사실 ① 실행계획 구조가 동일하다
  - A와 B의 오퍼레이션 구성은 완전히 같고 VIEW 이름만 다르다
  - Oracle 옵티마이저가 IN과 EXISTS를 같은 세미조인 형태로 변환했다
- 사실 ② Buffers가 동일하다
  - A와 B 모두 8,604 — 논리적 I/O 총량이 같다
- 사실 ③ A-Time만 크게 다르다
  - A는 0.10초, B는 0.05초 — A 의 계획에만 Reads 열(6,709)이 있다
- 사실 ④ 시간 차이가 특정 한 단계에 집중되어 있다
  - A의 6번 CLAIM_DETAILS Full Scan 에 Reads 6,613 이 몰려 있다, B의 같은 단계는 Reads 없음

원인 추론 — 캐시 상태

- Buffers가 같은데 시간이 다르다면 물리적 I/O 여부를 의심한다
  - Buffers는 논리적 I/O — 캐시에서 읽었는지 디스크에서 읽었는지 구분하지 않는다
  - 같은 블록 수를 만졌어도 디스크에서 읽으면 훨씬 오래 걸린다
- 실행 순서로 설명이 된다
  - A를 먼저 실행하며 디스크에서 블록을 읽어 버퍼 캐시에 올렸을 가능성이 크다
  - B는 그 직후 실행되어 이미 캐시에 있는 블록을 그대로 재사용했을 가능성이 높다
- 시간 차이가 몰린 지점이 가설과 일치한다
  - Reads 가 몰린 A의 6번 단계는 '처음 읽는 블록'(실습 01 이 읽지 않은 CLAIM_DETAILS)과 일치한다

결론과 벤치마크 방법론

- 결론 — 이번 실측에서 관측된 시간 차이는 EXISTS와 IN의 구조적 차이가 아니다
  - 캐시 상태, 즉 먼저 실행됐는지 나중에 실행됐는지에서 비롯됐다
  - 실행계획 구조 자체는 완전히 동일했다
- 실무 교훈 — 한 번 측정한 결과로 단정하지 않는다
  - 실행 순서를 통제하지 않은 단일 측정은 순서 효과를 그대로 결론에 섞는다
- 공정한 비교를 위한 체크리스트
  - ① 순서를 바꿔 반복 측정한다 ② 캐시 상태를 맞춘다
  - ③ Buffers처럼 캐시에 덜 흔들리는 지표를 함께 본다

1.7~1.8

자원 관점 종합 비교와 예고

- 세 버전 종합 벤치마크
- Buffers와 Memory의 트레이드오프
- 이후 챕터 로드맵

자원 관점 종합 비교

1.8절 Benchmark — 세 버전 실측 요약

```
+-------------------+---------+--------+-----------------------+
| 버전              | Buffers | A-Time | Memory (Used-Mem, 최대)|
+-------------------+---------+--------+-----------------------+
| A. IN             |   8,659 | 0.10초 | 6,651K                |
| B. EXISTS         |   8,604 | 0.05초 | 6,694K                |
| C. JOIN+DISTINCT  |   6,713 | 0.04초 | 12M (HASH GROUP BY)   |
+-------------------+---------+--------+-----------------------+

-- A : EXISTS와 구조 동일, 캐시 미보유 상태에서 측정
-- B : IN과 구조 동일, 캐시 보유 상태에서 측정
-- C : MEDICAL_CLAIMS 접근 자체가 없음
```

Buffers와 Memory의 트레이드오프

- Buffers만 보면 C가 가장 적고 A·B는 동일하다
  - C는 MEDICAL_CLAIMS 접근 단계가 없어 그만큼 블록을 덜 읽었다
- 그런데 Memory는 C가 오히려 더 크다
  - C는 HASH GROUP BY로 중복을 제거하느라 12M의 작업 영역을 썼다
  - A·B는 단순 집계라 6,694K로 더 작았다
- 'Buffers가 적으면 항상 전방위로 유리하다'는 뜻이 아니다
  - 자원 절약은 다차원적이다 — 어떤 자원을 절약했는지 구분해서 봐야 한다
  - 동시 사용자가 많은 환경에서는 작업 영역 사용량이 더 중요한 제약이 될 수 있다

이후 챕터 로드맵

- 이번 장에서 남겨 둔 질문
  - C 버전은 왜 MEDICAL_CLAIMS 접근을 생략할 수 있었는가 → ch09 JOIN 실행 원리
  - E-Rows와 A-Rows는 왜 어긋나는가 → 통계정보·카디널리티 추정 챕터
- 본격적인 개선 기법은 이후 챕터에서 순서대로 다룬다
  - 바인드 변수 사용 여부, 인덱스 설계, 조인 방식 선택 등
- 이번 차시의 범위는 '판단 기준을 세우는 것'까지다
  - 기준 없이 기법부터 배우면 언제 써야 할지를 판단할 수 없다

요약 (Summary)

- SQL 튜닝은 같은 결과를 내는 방법 중 자원을 덜 쓰는 쪽을 실측으로 고르는 작업이다
- SQL 성능은 '몇 초'가 아니라 자원(CPU·I/O·Memory) 소비량으로 판단한다
- I/O는 Buffers(논리적 I/O), Memory는 OMem·1Mem·Used-Mem으로 확인한다
- Buffers가 같아도 A-Time이 다를 수 있다 — 캐시 상태(물리 I/O 여부)를 의심한다
- IN과 EXISTS는 같은 세미조인으로 변환되어 실행계획이 완전히 동일했다
- '이 방식이 항상 빠르다'는 통념은 실측 없이 신뢰하면 안 된다
- 같은 FROM 절이라도 옵티마이저가 불필요한 테이블 접근을 제거할 수 있다 (ch09)
- 자원 절약은 다차원적이다 — Buffers를 아꼈다고 Memory도 아꼈다는 보장은 없다

실습 안내

- 실습문제 10문항 — ch01_문제.md
  - 1~2번 HASH JOIN RIGHT SEMI의 의미와 IN·EXISTS 실행계획 동일성
  - 3번 Buffers가 같은데 A-Time이 다른 이유 추론
  - 4~5번 Buffers 차이가 발생한 단계 지목과 MEDICAL_CLAIMS 접근 생략의 근거
  - 6번 DRUG_MASTER의 E-Rows 추정이 실제와 어긋난 이유
  - 7·9번 코드 리뷰와 실무 판단 — 스타일만으로 성능을 단정할 수 없는 이유
  - 8·10번 Buffers와 Memory의 트레이드오프, 실무 채택 근거 서술
- 실습 전 확인 — 이번 장은 DDL 변경이 없다
  - 인덱스·통계를 임의로 바꾸면 교안의 실행계획이 재현되지 않는다

Oracle SQL 튜닝 과정 · ch02

SQL은 어떻게 실행되는가?

Parse / Bind / Execute / Fetch · 하드 파싱과 소프트 파싱 · SQL_ID와 공유 풀 · 바인드 변수
| Oracle Database 19c · 진료비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - SQL 한 문장이 Parse → Bind → Execute → Fetch 순서로 처리되는 과정을 설명한다
  - 실행계획이 만들어지는 단계가 Parse임을 지목하고 그 근거를 말한다
  - 하드 파싱과 소프트 파싱의 차이를 비용 관점에서 구분한다
  - Oracle이 SQL 텍스트를 해시해 SQL_ID로 식별한다는 원리를 설명한다
  - 리터럴 값만 다른 두 SQL이 서로 다른 SQL_ID를 받는 이유를 설명한다
  - 바인드 변수를 쓴 SQL이 같은 SQL_ID를 유지하는 이유를 설명한다
  - SQL_ID·EXECUTIONS·PARSE_CALLS로 실행계획 재사용 여부를 확인한다
  - 같은 문장 구조인데 E-Rows가 다르게 나온 원인을 통계정보로 설명한다
  - E-Rows가 달라도 Buffers가 같을 수 있는 이유를 접근 경로로 설명한다
  - 바인드 변수의 이점과 부작용을 함께 놓고 사용 여부를 판단한다

이 장의 구성 (Agenda)

- 2.1 Why — SQL 처리 과정을 왜 알아야 하는가
- 2.2 Concept — SQL 처리 4단계 (Parse / Bind / Execute / Fetch)
- 2.3 Oracle Internals — 하드 파싱 · 소프트 파싱 · SQL_ID
- 2.4 Example — 리터럴 SQL과 바인드 변수 SQL
- 2.5 Execution Plan — 리터럴 값에 따른 실측 비교
- 2.6~2.7 Tuning — SQL_ID로 확인하는 파싱 재사용 여부
- 2.8 Benchmark — 파싱 관점 종합 비교
- 2.9~2.10 실습 안내 · 요약 (Review)

2.1

SQL 처리 과정을 왜 알아야 하는가

- 실행계획만 읽어서는 설명되지 않는 문제
- 이번 차시가 답하려는 질문
- 이 차시의 목표와 범위

실행계획을 읽는 것만으로 부족한 이유

- 실행계획을 정확히 읽어도 설명되지 않는 현상이 있다
  - 같은 SQL인데 어떤 때는 빠르고 어떤 때는 느리다
  - SQL 하나하나는 가벼운데 서버 전체 CPU 사용률이 올라간다
- 빠져 있는 질문 — 이 SQL은 언제 다시 최적화되고 언제 재사용되는가
  - 실행계획이 언제 만들어지는지 모르면 공유 풀 부하를 진단할 수 없다
  - 반복적인 하드 파싱은 개별 SQL의 실행계획만 봐서는 드러나지 않는다
- 그래서 이번 장은 무엇이 실행되는가가 아니라 어떻게 실행되는가를 본다
  - 실행계획은 결과물이고, 이번 장의 주제는 그 결과물이 생기는 과정이다

이 차시의 목표와 범위

- 목표 1 — SQL 한 문장이 실행되기까지 거치는 단계를 이해한다
  - Parse / Bind / Execute / Fetch 네 단계의 역할을 각각 구분한다
- 목표 2 — 그 단계 중 어디서 실행계획이 만들어지는지 파악한다
  - 실행계획은 Parse 단계의 산출물이며 이후 단계는 그것을 따라간다
- 소재 — MEDICAL_CLAIMS를 HOSP_ID 리터럴 값으로 조회하는 동일 SQL
  - 같은 SQL을 값만 바꿔 여러 번 실행하며 Parse 단계의 동작을 관찰한다
- 이 장에서 다루지 않는 것
  - 히스토그램의 내부 원리와 바인드 변수 대응 기법 — ch08
  - 본 차시는 DDL 변경이 없다 — 스키마를 그대로 둔 채 관찰만 한다

2.2

SQL 처리 4단계

- Parse — 문법·의미 검사와 실행계획 확보
- Bind · Execute · Fetch의 역할
- 옵티마이저가 동작하는 지점

SQL 처리 4단계 개요

- SQL은 한 번에 실행되지 않고 네 단계를 순서대로 거친다
  - Parse → Bind → Execute → Fetch
- Parse — 문법·의미를 검사하고 실행계획을 확보한다
- Bind — SQL에 바인드 변수가 있으면 그 자리에 실제 값을 대입한다
- Execute — 확보된 실행계획대로 실제 데이터를 처리한다
- Fetch — 처리 결과를 클라이언트로 반환한다
- 네 단계 중 판단이 일어나는 곳은 Parse 하나뿐이다
  - 나머지 단계는 이미 정해진 계획을 그대로 따라갈 뿐이다
  - 그래서 성능 판단의 초점은 이 계획이 어떻게 정해졌는가에 놓인다

Parse 단계에서 일어나는 일

- ① 문법 검사 — SQL 문장이 문법에 맞게 작성되었는가
- ② 의미 검사 — 테이블·컬럼이 실제로 존재하고 접근 권한이 있는가
- ③ 공유 풀 조회 — 이미 같은 SQL이 공유 풀(Shared Pool)에 있는가
  - 없으면 옵티마이저가 새로 실행계획을 생성한다 (하드 파싱)
  - 있으면 기존 실행계획을 그대로 재사용한다 (소프트 파싱)
- ④ 최적화 — 통계정보를 근거로 실행계획을 만든다 (하드 파싱일 때만)
- Parse는 SQL 텍스트 단위로 독립적으로 일어난다
  - 텍스트가 다르면 서로 다른 파싱 과정을 각각 거친다

Bind · Execute · Fetch 단계

- Bind — 바인드 변수 자리에 실제 값을 대입한다
  - :b_hosp 같은 자리표시자에 그때그때의 값이 들어간다
  - 바인드 변수가 없는 SQL에는 이 단계에서 할 일이 없다
- Execute — 만들어진 실행계획대로 실제 데이터를 처리한다
  - 블록을 읽고 필터·조인·정렬을 수행하는 실제 작업이 여기서 일어난다
  - A-Rows·A-Time·Buffers 같은 실측치는 이 단계를 거쳐야 생긴다
- Fetch — 처리된 결과를 클라이언트로 반환한다
  - 결과 건수가 많으면 여러 번에 나누어 반환된다
- 이 세 단계는 계획을 바꾸지 않는다 — 정해진 대로 수행할 뿐이다

옵티마이저는 어디서 동작하는가

- 옵티마이저는 Parse 단계에서 동작한다
  - 그 결과물이 앞 장부터 계속 읽어 온 실행계획이다
- Cost·E-Rows 같은 추정치도 전부 Parse 시점에 계산된다
  - 실행이 끝난 뒤에 다시 계산해 보정하는 값이 아니다
  - 그래서 추정이 빗나가도 그 실행 중에는 계획이 바뀌지 않는다
- EXPLAIN PLAN과 실제 실행의 차이도 여기서 설명된다
  - EXPLAIN PLAN은 네 단계 중 Parse까지만 수행한다
  - 그래서 추정치만 보이고 실측치는 애초에 존재하지 않는다

> **참고:**
> EXPLAIN PLAN이 네 단계 중 Parse까지만 수행한다는 점은 실습문제 7번에서 다시 확인한다.

2.3

하드 파싱과 소프트 파싱

- 하드 파싱의 비용 · 소프트 파싱의 절약
- SQL_ID — 텍스트를 해시해 식별한다
- 리터럴과 바인드 변수의 갈림길

하드 파싱 (Hard Parse)

- 정의 — 공유 풀에 동일한 SQL 텍스트가 없어 처음부터 전부 수행하는 것
  - 문법 검사 → 의미 분석 → 최적화(실행계획 생성)를 모두 거친다
- 비용이 큰 이유
  - 옵티마이저가 여러 실행 방법을 만들어 비교·평가해야 한다
  - 통계정보를 읽어 카디널리티와 Cost를 계산해야 한다
  - 결과를 공유 풀에 등록하는 동안 내부 자원을 점유한다
- 하드 파싱 자체는 잘못이 아니다 — 처음 한 번은 반드시 필요하다
  - 문제가 되는 것은 같은 로직이 값만 바뀌며 반복 하드 파싱되는 경우다

소프트 파싱 (Soft Parse)

- 정의 — 공유 풀에 동일한 SQL 텍스트가 있어 기존 실행계획을 재사용하는 것
- 비용이 작은 이유
  - 가장 비싼 최적화 단계를 건너뛴다 — 이미 만들어진 계획을 그대로 쓴다
  - 통계정보를 다시 읽어 Cost를 계산하는 작업이 없다
- 단, 문법·의미 검사와 공유 풀 조회는 여전히 수행한다
  - 소프트 파싱도 공짜는 아니다 — 하드 파싱보다 쌀 뿐이다
- 판단 기준 — 반복 실행되는 SQL이 소프트 파싱으로 처리되고 있는가
  - 이 질문에 답하려면 SQL_ID가 하나로 유지되는지 확인해야 한다

SQL_ID — Oracle이 SQL을 식별하는 방법

- Oracle은 SQL 텍스트를 해시한 값(SQL_ID)으로 SQL을 식별한다
  - 공유 풀에서 재사용할지 새로 만들지를 이 식별자 하나로 판단한다
- 판단 기준은 오직 텍스트가 일치하는가다
  - 로직이 같아도 텍스트가 다르면 재사용 대상이 아니다
  - Oracle이 두 SQL의 의미가 같다는 것을 알아서 묶어 주지 않는다
- 따라서 텍스트를 바꾸는 모든 요소가 SQL_ID를 바꾼다
  - 리터럴 값은 물론 공백, 줄바꿈, 대소문자, 주석도 텍스트의 일부다
  - 같은 SQL을 팀마다 다른 서식으로 쓰면 그만큼 사본이 늘어난다

리터럴 값이 다르면 완전히 별개의 SQL

- WHERE hosp_id = 1과 WHERE hosp_id = 500은 글자 하나만 다르다
  - 그런데도 SQL 텍스트가 다르므로 SQL_ID가 다르게 부여된다
  - 결과적으로 완전히 별개의 SQL로 취급되어 각각 하드 파싱된다
- 값의 가짓수만큼 서로 다른 SQL이 쌓인다
  - 병원 ID별로 계속 다른 리터럴 값을 쓰면 값마다 SQL_ID가 생긴다
  - 교안의 예시처럼 병원이 1,000곳이면 그만큼의 SQL_ID가 쌓일 수 있다
- 누적되면 두 방향으로 부담이 된다
  - 메모리 — 사실상 같은 로직의 SQL 사본이 공유 풀을 차지한다
  - CPU — 값이 바뀔 때마다 하드 파싱이 다시 일어난다

바인드 변수는 SQL 텍스트를 고정한다

- WHERE hosp_id = :b_hosp — 값이 바뀌어도 SQL 텍스트는 그대로다
  - 같은 SQL_ID를 유지하므로 공유 풀에서 기존 계획을 찾아 쓸 수 있다
  - 최초 1회 하드 파싱한 실행계획을 이후 실행이 재사용한다
- 값은 Parse가 아니라 Bind 단계에서 들어간다
  - 계획을 만드는 시점과 값을 넣는 시점이 분리된다는 뜻이다
  - 그래서 값이 달라져도 계획을 다시 만들 필요가 없다
- 이것이 처리 단계를 먼저 배운 실질적인 이유다
  - 단계가 분리돼 있기 때문에 계획 재사용이라는 개념이 성립한다

2.4

리터럴 SQL과 바인드 변수 SQL

- A. 리터럴 SQL — 값을 문장에 직접 써 넣는 방식
- B. 바인드 변수 SQL — 값을 자리표시자로 분리
- 두 예제를 무엇으로 비교할 것인가

A. 리터럴 SQL

2.4절 Example — 값을 SQL 문장에 직접 써 넣는 방식

```
-- A. 리터럴 SQL
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = 1;
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = 500;

-- 두 문장은 hosp_id 뒤의 값 하나만 다르다
-- 그러나 Oracle이 보기에는 서로 다른 SQL 텍스트다
-- 따라서 각각 별도의 SQL_ID를 받고 각각 하드 파싱된다
```

B. 바인드 변수 SQL

2.4절 Example — 값을 자리표시자로 분리하는 방식

```
-- B. 바인드 변수 SQL
VARIABLE b_hosp NUMBER
EXEC :b_hosp := 1
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = :b_hosp;
EXEC :b_hosp := 500
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = :b_hosp;

-- SELECT 문장의 텍스트는 두 번 모두 완전히 같다
-- 달라지는 것은 :b_hosp에 대입되는 값뿐이다
```

두 예제를 무엇으로 비교할 것인가

- 비교 대상은 결과가 아니라 처리 방식이다
  - A와 B는 같은 조건으로 같은 건수를 세는 SQL이다
  - 달라지는 것은 값을 문장에 넣는가, 자리표시자로 분리하는가뿐이다
- 확인할 지표 세 가지
  - SQL_ID — 오라클이 이것을 몇 개의 SQL로 인식했는가
  - EXECUTIONS — 그 SQL이 몇 번 실행되었는가
  - PARSE_CALLS — 파싱 요청이 몇 번 발생했는가
- 관찰은 두 각도로 나누어 진행한다
  - 2.5절 — 실행계획에 나타난 추정치와 실측치
  - 2.7절 — 공유 풀에 남은 SQL_ID와 실행 통계

2.5

리터럴 값에 따른 실측 비교

- HOSP_ID = 1 실행계획
- HOSP_ID = 500 실행계획
- E-Rows는 왜 다르고 Buffers는 왜 같은가

HOSP_ID = 1 실행계획

SQL_ID 0fqfut0m2jfxw — 대형병원·쏠림 구간

```
| Id | Operation         | Name           | E-Rows | A-Rows |   A-Time    | Buffers |
| 2  |  TABLE ACCESS FULL | MEDICAL_CLAIMS |   2795 |   3058 |00:00:00.01 |    3090 |

-- E-Rows  : 옵티마이저가 Parse 단계에서 계산한 추정 행 수
-- A-Rows  : 실제로 처리된 행 수 (Execute 이후에 생기는 값)
-- A-Time  : 이 단계에 실제로 걸린 시간
-- Buffers : 이 단계에서 읽은 블록 수 (논리적 I/O)
```

HOSP_ID = 1 실행계획 읽는 법

- 접근 경로 — TABLE ACCESS FULL (MEDICAL_CLAIMS)
  - HOSP_ID에 인덱스가 없어 옵티마이저가 고를 수 있는 경로가 이것뿐이다
- 추정과 실제 — E-Rows 2,795 / A-Rows 3,058
  - 추정치가 실제 행 수에 상당히 근접했다
  - Parse 단계에서 계산한 값이 Execute 결과와 거의 맞았다는 뜻이다
- 실측치 — A-Time 00:00:00.01, Buffers 3,090
  - Buffers는 이 단계에서 읽은 블록 수, 즉 논리적 I/O 지표다
- 이 SQL의 SQL_ID는 0fqfut0m2jfxw — 다음 슬라이드의 값과 비교한다
  - SQL_ID를 지금부터 기억해 두면 2.7절 관찰이 훨씬 쉬워진다

HOSP_ID = 500 실행계획

SQL_ID 71xyxd4t42wvj — 일반 병원

```
| Id | Operation         | Name           | E-Rows | A-Rows |   A-Time    | Buffers |
| 2  |  TABLE ACCESS FULL | MEDICAL_CLAIMS |    280 |    177 |00:00:00.01 |    3090 |

-- 문장 구조는 앞 슬라이드와 완전히 같다
-- 달라진 것은 hosp_id 뒤의 리터럴 값 하나뿐이다
-- 그런데도 SQL_ID가 다르고 E-Rows도 다르게 계산되었다
```

두 실행계획 비교 — 무엇이 다르고 무엇이 같은가

- 다른 것 — E-Rows
  - HOSP_ID=1은 2,795, HOSP_ID=500은 280으로 추정되었다(표본 히스토그램이라 수집마다 ±10% 흔들린다)
  - 실제 행 수 3,058과 177에 각각 가깝게 나왔다
- 같은 것 — 오퍼레이션과 Buffers
  - 둘 다 TABLE ACCESS FULL로 같은 접근 경로를 선택했다
  - Buffers는 두 조회 모두 3,090으로 동일하다
- 같은 문장 구조인데 추정치만 달라졌다는 점이 이 절의 관찰 포인트다
  - 왜 이렇게 정확한 추정이 가능했는가 — 다음 슬라이드
  - 왜 Buffers는 똑같았는가 — 그 다음 슬라이드

E-Rows가 값마다 다르게 나온 이유 — 통계정보

- 두 SQL은 별개의 SQL_ID로 각각 하드 파싱되었다
  - 파싱이 각각 일어났으므로 추정도 각각 따로 계산된다
- 추정의 근거는 HOSP_ID 컬럼의 통계정보다
  - USER_TAB_COL_STATISTICS에서 HISTOGRAM=HYBRID, NUM_BUCKETS=254로 확인된다
  - 균등 분포를 가정하지 않고 값별 분포를 구간(버킷)으로 나눠 저장한다
- 그래서 쏠린 값과 일반적인 값의 빈도를 구분해 추정할 수 있었다
  - 히스토그램의 내부 원리는 ch08에서 다룬다
- 뒤집어 보면 — 같은 텍스트여야 같은 계획이 나온다는 전제가 확인된다
  - 리터럴이 다르면 텍스트가 다르므로 Cost·E-Rows까지 달라질 수 있다

Buffers가 같았던 이유 — 선택지가 하나뿐

- HOSP_ID에는 인덱스가 없다
  - 옵티마이저가 고를 수 있는 접근 경로는 TABLE ACCESS FULL 하나뿐이다
- Full Scan은 조건에 맞는 행 수와 무관하게 테이블 전체를 읽는다
  - 어떤 값을 찾든 결국 테이블 전체를 다 읽어야 한다
  - 그래서 두 조회 모두 Buffers 3,090으로 같았다
- 판단 기준 — 추정이 정확해도 대안 경로가 없으면 결과는 바뀌지 않는다
  - 추정치는 여러 방법 중 하나를 고르는 데 쓰이는 값이다
  - 고를 것이 하나뿐이면 추정의 정확도는 선택에 영향을 주지 못한다
- 인덱스를 도입하면 이 상황이 어떻게 달라지는지는 ch06에서 확인한다

2.6~2.7

SQL_ID로 확인하는 파싱 재사용

- 이 장에 Bad SQL 사례가 없는 이유
- A. 리터럴 SQL의 SQL_ID·실행 통계
- B. 바인드 변수 SQL의 SQL_ID·실행 통계

이 장에 Bad SQL 사례가 없는 이유

- 이번 장의 주제는 SQL 자체의 문제가 아니다
  - 문장을 잘못 써서 느려진 사례를 분석하는 장이 아니다
- 주제는 SQL 처리 방식의 차이다
  - 같은 문장이라도 어떻게 넘기느냐에 따라 파싱 비용이 달라진다
  - 그래서 개선 대상이 문장이 아니라 호출 방식에 있다
- 교안도 2.6절 대신 2.7절에서 파싱 관점의 관찰 결과를 다룬다
  - SQL_ID·EXECUTIONS·PARSE_CALLS를 확인해 재사용 여부를 판단한다
- 관찰 질문 — 두 번 실행했을 때 SQL이 몇 개로 인식되는가
  - 이 질문 하나로 리터럴과 바인드 변수의 차이가 전부 드러난다

A. 리터럴 SQL의 SQL_ID·실행 통계

2.7절 Tuning — 값만 다른 두 문장을 각각 한 번씩 실행한 결과

```
SQL_ID          EXECUTIONS  PARSE_CALLS  SQL_TEXT
71xyxd4t42wvj   1           1            ...WHERE hosp_id = 500
0fqfut0m2jfxw   2           2            ...WHERE hosp_id = 1

-- 행이 두 개 나왔다는 사실 자체가 이 관찰의 결과다
-- 같은 로직의 SQL이 공유 풀에 두 개로 따로 등록되었다
-- 두 SQL_ID는 2.5절 실행계획에서 본 값과 그대로 일치한다
```

리터럴 SQL 관찰 결과 읽는 법

- 서로 다른 SQL_ID가 두 개 나왔다
  - 71xyxd4t42wvj와 0fqfut0m2jfxw — 각각 별도로 파싱·최적화되었다
- 각 SQL의 EXECUTIONS와 PARSE_CALLS가 모두 1이다
  - 한 번 실행했고 그 한 번을 위해 파싱했다는 뜻이다
  - 재사용이 일어난 흔적이 전혀 없다
- 이 패턴이 반복되면 어떻게 되는가
  - 병원 ID별로 계속 다른 값을 쓰면 SQL_ID가 계속 쌓인다
  - 공유 풀에 사본이 누적되고 그때마다 하드 파싱이 반복된다
  - 오래된 SQL이 밀려나면 다시 필요할 때 또 하드 파싱해야 한다

B. 바인드 변수 SQL의 SQL_ID·실행 통계

2.7절 Tuning — :b_hosp에 1과 500을 대입해 두 번 실행한 결과

```
SQL_ID          EXECUTIONS  PARSE_CALLS  SQL_TEXT
78qznuqcawm4c   2           2            ...WHERE hosp_id = :b_hosp

-- 서로 다른 값으로 두 번 실행했지만 행은 하나뿐이다
-- SQL_ID가 하나로 유지되었다는 뜻이다
-- EXECUTIONS=2 — 같은 실행계획을 공유 풀에서 재사용했다
```

바인드 변수 관찰 결과 읽는 법

- SQL_ID가 78qznuqcawm4c 하나로 유지되었다
  - :b_hosp에 서로 다른 값을 대입했지만 SQL 텍스트는 그대로였기 때문이다
- EXECUTIONS=2 — 같은 SQL이 두 번 실행되었다는 기록이다
  - 같은 실행계획을 공유 풀에서 재사용했음을 보여 준다
  - 값의 가짓수가 늘어도 SQL_ID는 여전히 하나로 유지된다
- 결론 — 매번 값이 바뀌는 조건으로 반복 조회하는 SQL이라면
  - 리터럴로 값을 직접 넣지 말고 바인드 변수를 쓴다
  - 공유 풀 부담과 하드 파싱 횟수를 함께 줄이는 길이다
- 단, 이 결론에는 조건이 붙는다 — 2.8절 마지막에서 다시 다룬다

2.8

파싱 관점 종합 비교

- 실행 방식별 SQL_ID 개수와 파싱 부담
- 바인드 변수가 항상 정답은 아니다
- 요약과 실습 안내

파싱 관점 종합 비교

2.8절 Benchmark — 2회 실행 기준

```
+---------------------------------------+-------------+-----------------------------------+
| 실행 방식                               | SQL_ID 개수 | 파싱 부담                          |
+---------------------------------------+-------------+-----------------------------------+
| 리터럴 SQL (hosp_id=1, hosp_id=500)     | 2개         | 매번 하드 파싱                      |
| 바인드 변수 SQL (:b_hosp에 1, 500 대입) | 1개         | 최초 1회만 하드 파싱, 이후 재사용 |
+---------------------------------------+-------------+-----------------------------------+

-- 리터럴 : 값의 가짓수만큼 SQL_ID가 늘어난다
-- 바인드 : 값이 달라져도 SQL_ID는 하나로 유지된다
-- 비교 지표는 시간이 아니라 SQL_ID 개수와 파싱 방식이다
```

바인드 변수가 항상 정답은 아니다

- 바인드 변수는 SQL_ID를 하나로 고정한다 — 실행계획도 하나만 캐시된다
  - 최초 하드 파싱 시점에 대입된 값을 기준으로 만들어진 계획이 남는다
  - 이후 다른 값에도 그 계획이 그대로 재사용된다 (바인드 변수 Peeking)
- 히스토그램이 있는 컬럼에서는 이것이 부작용이 될 수 있다
  - 쏠린 값과 일반적인 값은 원래 최적의 실행 방법이 다를 수 있다
  - 한 번 정해진 계획이 모든 값에 적용되는 결과가 된다
- 종합 판단 — 기본은 바인드 변수, 주의 대상은 분포가 크게 갈리는 컬럼
  - 무조건 바인드 변수도, 무조건 리터럴도 정답이 아니다

> **참고:**
> 바인드 변수 Peeking과 그 대응 방법(적응형 커서 공유)은 ch08에서 다룬다.

요약 (Summary)

- SQL 처리는 Parse → Bind → Execute → Fetch 순서로 진행된다
- 실행계획은 Parse 단계에서 만들어지고 이후 단계는 그것을 그대로 따라간다
- Oracle은 SQL 텍스트를 해시한 SQL_ID로 공유 풀 재사용 여부를 판단한다
- 리터럴 값이 다르면 텍스트가 달라져 별도의 SQL_ID로 각각 하드 파싱된다
- 바인드 변수를 쓰면 값이 달라도 같은 SQL_ID를 유지해 계획을 재사용한다
- 히스토그램이 있는 컬럼은 리터럴 값에 따라 E-Rows가 다르게 나올 수 있다
- E-Rows가 달라도 대안 경로가 없으면 Buffers는 같다 — 두 조회 모두 3,090
- 바인드 변수도 만능은 아니다 — Peeking 부작용은 ch08에서 다룬다

실습 안내

- 실습문제 10문항 — ch02_문제.md
  - 1번 Parse/Bind/Execute/Fetch 각 단계의 역할과 실행계획 생성 지점
  - 2·3번 리터럴과 바인드 변수의 SQL_ID 차이, 대규모 트래픽 시나리오 확장
  - 4·5번 히스토그램을 근거로 한 E-Rows 추정과 같은 계획 가정의 검증
  - 6번 바인드 변수 Peeking 부작용 추론
  - 7번 EXPLAIN PLAN이 네 단계 중 어디까지 수행하는가
  - 8·9번 Cost가 파싱 시점에 확정되는 이유, Buffers가 같았던 이유
  - 10번 공유 풀 누적 문제의 메모리·파싱 부하 관점 종합 서술
- 실습 전 확인 — 이번 장은 DDL 변경이 없다
  - 인덱스·통계를 임의로 바꾸면 교안의 실행계획이 재현되지 않는다

Oracle SQL 튜닝 과정 · ch03

실행계획 기초

실행계획 트리 · 오퍼레이션 판독 · Cost와 Rows · access vs filter · 카디널리티 추정 오류 | Oracle Database 19c · 진료비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - 실행계획을 옵티마이저가 만든 처리 절차의 트리로 설명한다
  - EXPLAIN PLAN과 DBMS_XPLAN.DISPLAY_CURSOR를 목적에 맞게 구분해 사용한다
  - Id·Operation·Name·Rows·Bytes·Cost 컬럼의 의미를 각각 설명한다
  - Cost가 시간이 아니라 무차원 상대 지표임을 근거를 들어 설명한다
  - Predicate Information의 access와 filter를 구분해 인덱스 활용 여부를 판별한다
  - 들여쓰기 깊이를 근거로 실행 순서를 Id 순서와 구분해 읽는다
  - E-Rows와 A-Rows의 차이로 카디널리티 추정 오류를 진단한다
  - 추정 오류가 조인 순서·조인 방식으로 전파되는 경로를 설명한다
  - 같은 PK 인덱스가 조건 성격에 따라 다른 오퍼레이션으로 갈리는 이유를 설명한다
  - UNIQUE·RANGE·FULL·FAST FULL SCAN 네 가지를 서로 구분한다

이 장의 구성 (Agenda)

- 3.1 Why — 실행계획을 확인해야만 원인을 특정할 수 있다
- 3.2 Concept — 실행계획이란 무엇인가 · 오퍼레이션 단위
- 3.3 Oracle Internals — CBO와 통계정보
- 3.4 Example — 첫 실행계획 뽑아 보기
- 3.5 Execution Plan — 컬럼별 판독 기준 · access vs filter
- 3.6 Bad SQL — 카디널리티 추정 오류 사례
- 3.7 Tuning 예고 — 이 차시에서 남겨 두는 것
- 3.8 Benchmark — 주요 Operation 6종 비교
- 요약 (Review) · 실습 안내

3.1

실행계획을 왜 읽어야 하는가

- 체감으로는 원인을 특정할 수 없다
- SQL 문장만으로는 처리 방식을 알 수 없다
- 본 차시의 범위 — 튜닝이 아니라 판독

체감으로는 원인을 특정할 수 없다

- SQL이 느린 이유는 실행계획을 확인해야 파악할 수 있다
  - '느린 것 같다'는 체감만으로는 원인 특정이 불가능하다
  - Full Scan 때문인지, 잘못된 조인 순서 때문인지, 통계정보 부정확 때문인지
  - 이 셋은 조치 방향이 전혀 다르므로 반드시 구분해야 한다
- 원인을 모르면 조치도 근거 없이 고르게 된다
  - 인덱스를 만들어야 할지, SQL을 고쳐야 할지 판단할 재료가 없다
  - 근거 없이 고른 조치는 우연히 맞아도 재현·설명이 되지 않는다
- 실행계획은 그 판단에 쓸 수 있는 1차 자료다

SQL 문장만으로는 처리 방식을 알 수 없다

- 동일한 결과를 내는 SQL이라도 Oracle의 처리 방식은 문장만으로 판단 불가
  - Full Scan인지 Index Scan인지, 어떤 조인 방식인지는 문장에 적혀 있지 않다
  - SQL은 원하는 결과를 기술할 뿐 가져오는 방법을 지정하지 않는다
- 옵티마이저가 상황에 따라 매번 다른 방법을 선택할 수 있기 때문
  - 데이터 분포 · 통계정보 · 인덱스 존재 여부에 따라 선택이 달라진다
  - 어제 인덱스를 타던 SQL이 오늘 Full Scan으로 바뀔 수 있다
- 그래서 '이 SQL은 인덱스를 탈 것이다'는 예상이 아니라 확인의 대상이다

본 차시의 범위 — 튜닝이 아니라 판독

- 본 차시 범위는 튜닝이 아니라 실행계획 판독이다
  - 인덱스 설계·SQL 재작성 같은 실제 개선 조치는 다루지 않는다
- 이후 모든 챕터에서 "이 방법이 왜 더 나은지"를 실행계획으로 증명해야 한다
  - 판독 능력이 그 증명의 전제 조건이다
  - 증명 없는 개선은 개선이 아니라 변경일 뿐이다
- 본 차시는 인덱스를 신규 생성하지 않는다
  - 기존 PK 인덱스(HOSPITALS, PATIENTS, MEDICAL_CLAIMS 등)만으로 진행한다
  - 스키마를 손대지 않아야 교안의 실행계획이 그대로 재현된다

3.2

실행계획이란 무엇인가

- 실행계획의 정의 — 처리 절차의 트리 표현
- 노드 하나 = 물리적 작업 하나
- 한 문장이 여러 작업으로 분해된다

실행계획의 정의

- 실행계획 — 옵티마이저가 SQL을 처리할 절차를 트리 구조로 표현한 것
  - SQL은 무엇을 원하는지를 쓰고, 실행계획은 어떻게 가져올지를 답한다
  - 사람이 쓴 것이 아니라 옵티마이저가 만들어 낸 산출물이다
- 트리이므로 부모와 자식 관계가 있다
  - 자식 노드의 결과가 부모 노드로 전달되어 조합된다
  - 가장 바깥의 뿌리 노드가 최종 결과를 반환한다
- 실행계획을 읽는다는 것은 이 트리를 읽는다는 뜻이다
  - 표처럼 생겼지만 실제로는 들여쓰기로 표현된 트리다

노드 하나 = 물리적 작업 하나

- 트리의 각 노드(Operation)는 물리적 작업 단위 하나에 대응한다
  - TABLE ACCESS FULL — 테이블 전체를 처음부터 끝까지 읽는다
  - INDEX SCAN 계열 — 인덱스에서 특정 값을 찾는다
  - JOIN 계열 — 두 결과 집합을 합친다
- 오퍼레이션 이름은 그 단계에서 무슨 일이 일어났는지의 요약이다
  - 이름만 읽어도 읽은 대상과 읽는 방식을 대략 파악할 수 있다
- 오퍼레이션 이름은 외우는 대상이 아니라 해석하는 대상이다
  - 단어를 쪼개서 읽으면 처음 보는 이름도 뜻을 짐작할 수 있다

한 문장이 여러 작업으로 분해된다

- SQL 한 문장이 여러 개의 물리적 작업으로 분해되어 실행된다
  - 그 분해와 조합 순서가 트리 구조로 나타난다
- 한 줄짜리 SELECT도 여러 노드로 늘어날 수 있다
  - 인덱스에서 위치를 찾는 단계와 테이블에서 행을 가져오는 단계는 별개다
  - 정렬·집계·조인도 각각 독립된 노드로 잡힌다
- 노드 수가 많다고 나쁜 계획인 것은 아니다
  - 각 노드가 무엇을 얼마나 읽었는지가 판단의 기준이다
  - 노드가 둘뿐이어도 그 하나가 전체를 훑으면 무거운 계획이다

3.3

CBO와 통계정보

- CBO — 비용이 가장 낮은 후보를 고른다
- Cost 계산에 쓰이는 통계정보
- 통계와 실제가 어긋나는 세 가지 원인
- 어긋나면 무슨 일이 생기는가

CBO — 비용이 가장 낮은 후보를 고른다

- 옵티마이저(CBO, Cost-Based Optimizer)의 동작
  - 동일 SQL을 처리할 수 있는 여러 후보 실행 방법을 만든다
  - 그중 예상 비용(Cost)이 가장 낮은 것을 선택한다
- 선택은 실행 전 추정에 근거한다
  - 후보를 실제로 하나씩 돌려 보고 고르는 것이 아니다
  - 돌려 볼 수 없으므로 통계정보로 결과를 미리 계산한다
- 따라서 추정의 재료인 통계정보가 어긋나면 선택도 어긋난다
  - 옵티마이저가 틀렸다기보다 재료가 틀렸다고 보는 편이 정확하다

Cost 계산에 쓰이는 통계정보

- Cost 계산에 사용되는 통계정보
  - 테이블 행 수
  - 각 컬럼의 고유값 개수(NDV)
  - 데이터 분포 — 히스토그램 존재 여부
  - 인덱스 클러스터링 팩터
- 통계정보는 데이터 자체가 아니라 데이터의 요약이다
  - 요약이 낡거나 거칠면 추정이 빗나간다
  - 요약이므로 원본과 다를 수 있다는 점을 항상 전제해야 한다

통계와 실제가 어긋나는 세 가지 원인

- 통계 미갱신 — 오래된 통계
  - 데이터는 늘었는데 통계는 예전 그대로인 경우
- 컬럼 간 상관관계 미반영
  - 옵티마이저는 두 컬럼 조건을 독립으로 가정하고 선택도를 곱해서 추정한다
  - 3.6절 실측 사례가 바로 이 경우다
- 데이터 쏠림(skew)에 히스토그램이 없는 경우
  - 값이 한쪽으로 몰려 있는데 평균 분포로 잘못 가정한다
  - 자주 나오는 값과 드문 값을 같은 비율로 취급하게 된다

어긋나면 무슨 일이 생기는가

- 위 원인 중 하나라도 해당되면 최적이 아닌 계획이 선택될 수 있다
  - 이 현상을 카디널리티 추정 오류라고 부른다
  - 카디널리티는 각 단계가 만들어 낼 행 수를 뜻한다
- 추정이 틀리면 그 위에 쌓인 판단이 함께 틀어진다
  - 몇 건짜리 집합인지에 따라 유리한 조인 방식이 달라지기 때문이다
  - 조인 순서, 조인 방식, 작업 영역 할당량까지 영향을 받는다
- 상세 대응은 ch08(Optimizer와 통계정보)에서 다룬다
  - 이 장에서는 계획에서 그 징후를 찾아내는 데까지만 간다

3.4

첫 실행계획 뽑아 보기

- 판독 대상 SQL
- EXPLAIN PLAN과 DBMS_XPLAN 사용법
- 예상 계획과 실측 계획의 차이
- 뽑은 계획을 읽는 법

판독 대상 SQL

3.4절 Example — 이 장 전체에서 반복해서 쓰는 SQL 한 문장

```
SELECT claim_id, total_amt
FROM medical_claims
WHERE claim_type = '입원' AND total_amt > 500000;

-- 조건 컬럼 두 개(CLAIM_TYPE, TOTAL_AMT) 모두 인덱스가 없는 상태
-- MEDICAL_CLAIMS의 인덱스는 PK_MEDICAL_CLAIMS(CLAIM_ID) 하나뿐이다
-- 이 차시에서는 인덱스를 만들지 않고 계획만 확인한다
```

실행계획을 뽑는 두 가지 방법

EXPLAIN PLAN(예상) · DBMS_XPLAN.DISPLAY_CURSOR(실측)

```
-- 1) 실제로 실행하지 않고 옵티마이저의 예상 계획만 산출
EXPLAIN PLAN FOR
SELECT claim_id, total_amt
FROM medical_claims
WHERE claim_type = '입원' AND total_amt > 500000;

-- 2) 산출된 예상 계획을 조회
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());

-- 3) SQL을 실제로 실행한 뒤, 실측 통계까지 붙여서 조회
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

예상 계획과 실측 계획

- EXPLAIN PLAN FOR ... — 실제로 실행하지 않고 예상 계획만 산출한다
  - 이어서 SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY())로 결과를 조회한다
  - 얻는 값은 Rows, Bytes, Cost, Time — 전부 실행 전 추정치다
- DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST') — 실행 후 통계
  - 얻는 값은 Starts, A-Rows, A-Time, Buffers — 전부 실측치다
  - 이 방식은 SQL을 실제로 실행한 뒤에만 쓸 수 있다
- 두 도구를 함께 써야 예상과 실제를 대조할 수 있다

> **참고:**
> 이 챕터의 모든 실행계획·행수·Buffers는 진료비청구심사 실습 DB에서 직접 캡처한 실측 결과다.

첫 실행계획 — 실행 전 예상

EXPLAIN PLAN + DBMS_XPLAN.DISPLAY 결과

```
| Id | Operation         | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
| 0  | SELECT STATEMENT  |                | 77946 |  2207K|   857   (1)| 00:00:01 |
| 1  | TABLE ACCESS FULL | MEDICAL_CLAIMS | 77946 |  2207K|   857   (1)| 00:00:01 |

Predicate Information (identified by operation id):
   1 - filter("CLAIM_TYPE"='입원' AND "TOTAL_AMT">500000)
```

이 계획을 어떻게 읽는가

- 노드가 둘뿐인 가장 단순한 형태다
  - Id 1 TABLE ACCESS FULL — MEDICAL_CLAIMS 전체를 처음부터 끝까지 읽는다
  - Id 0 SELECT STATEMENT — 그 결과를 최종 반환한다
- Predicate Information이 filter로 표시된 이유
  - 두 조건 컬럼에 인덱스가 없어 찾아갈 방법이 없다
  - 전부 읽어 온 뒤 행마다 조건을 검사해 걸러낸다
- Rows 77946은 결과 건수가 아니라 옵티마이저의 예상치다
  - 실제 건수는 3.6절에서 실행 후 A-Rows로 확인한다

3.5

컬럼별 판독 기준

- Id · Operation · Name — 구조를 나타내는 컬럼
- Rows · Bytes — 실행 전 추정치
- Cost — 무차원 상대 지표
- Predicate Information — access와 filter
- 실행 순서 판독 원칙

실행계획 컬럼 요약표

추정치와 실측치를 같은 이름으로 부르지 않는 것이 판독의 출발점

컬럼	의미	성격
Id	트리 노드 번호 (실행 순서가 아님)	구조
Operation	그 단계가 수행하는 물리적 작업명	구조
Name	작업 대상 오브젝트 (테이블 또는 인덱스)	구조
Rows / Bytes	예상 결과 건수 · 예상 데이터 크기	실행 전 추정치
Cost (%CPU)	후보 계획 비교용 상대 지표 (무차원)	실행 전 추정치
Time	예상 소요 시간	실행 전 추정치
Starts	그 단계가 수행된 횟수	실행 후 실측치
E-Rows	실행 전 추정 행 수	실행 전 추정치
A-Rows	실제로 처리한 행 수	실행 후 실측치
A-Time	그 단계까지 누적된 실제 소요 시간	실행 후 실측치
Buffers	읽은 블록 수 (논리적 I/O)	실행 후 실측치

```
-- EXPLAIN PLAN은 위쪽 추정치만, DISPLAY_CURSOR는 실측치까지 보여 준다
```

Id · Operation · Name

- Id — 트리 노드 번호
  - 생성 순서일 뿐 실행 순서가 아니다
  - Id 번호 순서(0→1→2)대로 읽으면 실제 실행 순서와 반대로 읽게 된다
- Operation / Options — 해당 단계가 수행하는 물리적 작업명
  - 앞부분이 무엇을 읽었는지, 뒷부분이 어떻게 읽었는지를 나타낸다
- Name — 작업 대상 오브젝트명(테이블 또는 인덱스)
  - Name이 인덱스명이면 인덱스를 읽은 단계다
  - Name이 비어 있으면 조인·정렬·집계처럼 대상 오브젝트가 없는 단계다

Rows · Bytes — 실행 전 추정치

- Rows / Bytes — 옵티마이저가 예상한 결과 건수와 데이터 크기
  - 실행 전 추정치이며 통계정보를 기반으로 계산된다
  - 실제 결과 건수가 아니다
- EXPLAIN PLAN만 보고 몇 건 나온다고 말하면 안 된다
  - 실제 건수는 실행 후 A-Rows로 확인한다
  - 두 값이 얼마나 벌어졌는지가 곧 진단 재료가 된다
- 추정치와 실측치를 같은 이름으로 부르지 않는 습관이 판독의 출발점이다
  - 보고서를 쓸 때도 예상 몇 건, 실제 몇 건으로 나눠 적는다

Cost — 무차원 상대 지표

- Cost — 여러 실행 방법 후보를 비교하려고 옵티마이저가 내부적으로 쓰는 상대 지표
  - 초 단위 실행 시간이 아니다
  - CPU 연산량과 I/O 예상 횟수를 옵티마이저 자체 공식으로 환산한 무차원 수치다
- Cost가 낮다는 것은 옵티마이저가 그렇게 판단했다는 뜻이다
  - 실제로 빨랐다는 뜻이 아니다
  - 실측은 A-Time과 Buffers로 따로 확인해야 한다
- %CPU는 그 Cost 중 CPU가 차지하는 비율을 나타낸다

Cost 비교는 어디까지 유효한가

- 비교가 유효한 범위 — 같은 SQL의 대안 계획들 사이
  - 같은 재료로 만든 후보들이라 같은 잣대가 적용된다
  - 힌트를 바꿔 가며 같은 SQL의 계획을 비교할 때가 대표적이다
- 서로 다른 두 SQL의 Cost를 나란히 놓고 판단하면 틀린다
  - 이게 더 작으니까 더 빠르다는 판단은 성립하지 않는다
  - 읽는 대상도 다르고 결과 집합의 성격도 다르기 때문이다
- 실습문제 9번이 바로 이 함정을 다룬다 — Cost 857과 860의 비교

Predicate Information — access와 filter

- Predicate Information — 각 오퍼레이션 단계에서 실제로 적용된 조건
  - 앞의 번호가 그 조건이 적용된 Id를 가리킨다
- access — 인덱스나 조인 상대를 찾아가는 데 쓰인 조건
  - 이 조건에 해당하는 컬럼의 인덱스가 있어야 발생 가능하다
  - 읽을 범위 자체를 줄여 주므로 읽는 양이 실제로 줄어든다
- filter — 이미 읽어 온 행들 중에서 걸러내는 데만 쓰인 조건
  - 인덱스 유무와 무관하게 항상 발생 가능하다
  - 읽는 양은 그대로이고 결과 건수만 줄어든다

같은 조건이 access와 filter로 갈리는 이유

- 같은 조건이라도 컬럼에 인덱스가 없으면 access가 아니라 filter로만 나타난다
  - 조건식이 같다고 처리 방식이 같은 것은 아니다
  - 조건을 어떻게 썼느냐보다 그 컬럼에 인덱스가 있느냐가 먼저다
- 인덱스를 만들었는데 왜 안 쓰이는지를 진단할 때 가장 먼저 볼 곳
  - Predicate Information에서 그 조건이 access인지 filter인지 확인한다
  - filter로만 나온다면 인덱스가 쓰이지 않았다는 직접적인 증거다
- 실습문제 8번 — 같은 HOSP_ID 조건이 한쪽은 access, 다른 쪽은 filter인 사례

실행 순서 판독 원칙

- 트리에서 들여쓰기가 가장 깊은 노드부터 먼저 실행된다
- 같은 들여쓰기 레벨에서는 아래쪽이 먼저 실행된다
- 그 결과가 위쪽·바깥쪽 노드로 전달되어 조합된다
  - 부모 노드는 자식의 결과가 도착해야 자기 일을 시작할 수 있다
- Id 번호 순서대로 읽으면 실제 실행 순서와 반대로 읽게 되므로 주의한다
  - 계획을 볼 때 숫자보다 들여쓰기 깊이부터 눈으로 확인하는 습관을 들인다
- 3.8절 Q6 계획에서 이 원칙을 직접 적용해 본다
  - 여섯 단계짜리 계획의 실행 순서를 손으로 적어 보는 연습을 권한다

3.6

카디널리티 추정 오류 사례

- E-Rows와 A-Rows의 차이
- 왜 어긋났는가 — 독립 가정의 곱셈
- 상관관계는 자동으로 인식되지 않는다
- 추정 오류는 전파된다
- 실무 판독 원칙

실행 후 실측 계획

3.4절 SQL을 실제 실행 후 DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST')

```
| Id | Operation         | Name           | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT  |                |      1 |        |    411 |00:00:00.02 |    3117 |
| 1  | TABLE ACCESS FULL | MEDICAL_CLAIMS |      1 |  77946 |    411 |00:00:00.02 |    3117 |

-- E-Rows : 실행 전 옵티마이저가 통계정보로 추정한 건수
-- A-Rows : 실제로 실행해서 나온 건수
-- 같은 SQL, 같은 데이터인데 두 값이 크게 벌어져 있다
```

E-Rows와 A-Rows

- E-Rows — 실행 전 옵티마이저가 통계정보를 근거로 추정한 건수
- A-Rows — 실제로 실행해서 나온 건수
- 이 사례의 실측 결과
  - E-Rows 77,946건 vs A-Rows 411건 — 약 190배 차이
  - 같은 SQL, 같은 데이터인데 예상과 실제가 이만큼 벌어졌다
- 함께 읽어야 할 실측 컬럼
  - Starts 1 — 이 단계가 한 번만 수행되었다
  - Buffers 3,117 — 실제로 읽은 블록 수, 전체를 훑었다는 증거

왜 어긋났는가 — 독립 가정의 곱셈

- 옵티마이저의 계산 절차
  - CLAIM_TYPE='입원'의 선택도를 구한다
  - TOTAL_AMT>500000의 선택도를 구한다
  - 두 값을 곱해서 전체 조건의 선택도를 추정한다
- 이 곱셈은 두 조건이 서로 독립이라는 가정 위에 서 있다
  - 한쪽 조건을 만족하는지가 다른 쪽에 아무 영향을 주지 않는다는 전제다
- 그러나 실제 데이터에서 두 조건이 서로 무관하지 않을 수 있다
  - 입원 진료가 외래보다 평균 진료비가 높은 경향이 있다면 두 조건은 겹친다

상관관계는 자동으로 인식되지 않는다

- 두 조건이 겹치는 방향으로 상관되어 있으면
  - 실제로 두 조건을 동시에 만족하는 행이 곱셈 추정치보다 적어질 수 있다
  - 반대로 많아지는 경우도 있다 — 방향은 데이터에 따라 다르다
- 이런 상관관계를 옵티마이저가 자동으로 알아채지는 못한다
  - 확장 통계(extended statistics)를 별도로 만들어 주지 않는 한
  - 항상 독립 가정으로 계산한다
- 즉 이 오차는 버그가 아니라 설계상의 기본 동작이다
  - 사람이 알려 주지 않으면 옵티마이저가 알 방법이 없다

추정 오류는 전파된다

- 이 SQL이 더 큰 쿼리의 일부였다면
  - 서브쿼리이거나 조인의 한쪽 입력이었을 경우
- 잘못된 77,946이라는 예상치를 근거로 후속 판단이 결정된다
  - 조인 순서 — 어느 쪽을 먼저 읽을지
  - 조인 방식 — Nested Loops인지 Hash Join인지
  - 작업 영역 크기 — 얼마나 큰 메모리를 잡을지
- 추정 오류는 그 자리에서 끝나지 않고 후속 판단으로 전파된다
  - 이것이 카디널리티 추정 오류를 중요하게 다루는 이유다

실무 판독 원칙

- E-Rows와 A-Rows 차이가 크면 카디널리티 추정 오류를 의심한다
  - 수십 배 이상 벌어지면 일단 의심 대상으로 표시한다
- 확인 순서
  - 어느 단계에서 차이가 벌어졌는지 계획 위에서 지목한다
  - 그 단계의 조건 컬럼과 통계 상태를 확인한다
  - 통계 미갱신인지, 상관관계인지, 히스토그램 부재인지 갈래를 나눈다
- 상세 대응은 ch08(Optimizer와 통계정보)에서 다룬다

> **참고:**
> 본 차시의 목표는 추정 오류를 고치는 것이 아니라, 실행계획에서 그 징후를 발견하는 것이다.

3.7

이 차시에서 남겨 두는 것

- Full Scan이 나온 이유
- 개선은 어느 챕터의 범위인가
- 판독을 먼저 확립하는 순서

Full Scan의 이유와 지금 고치지 않는 이유

- 이 SQL이 Full Scan인 이유
  - CLAIM_TYPE, TOTAL_AMT 두 컬럼 모두 인덱스가 없다
  - 찾아갈 길이 없으므로 전부 읽는 것 외에 선택지가 없었다
- 인덱스를 만들면 해결되는지, 어떤 인덱스가 적합한지는 ch06의 범위
- 본 차시에서 다루지 않는 이유
  - 지금 인덱스부터 만들면 왜 그 인덱스를 골랐는지 판단할 기준이 없다
  - 선택도와 컬럼 순서 같은 기준은 아직 배우지 않은 상태다
- 판독을 먼저 확립한 뒤 설계로 넘어가는 순서를 따른다

3.8

주요 Operation 6종 비교

- Q1 비인덱스 조건 · Q2 PK 등치 · Q3 PK 범위
- Q4 인덱스 정렬 활용 · Q5 인덱스만으로 집계
- Q6 조인과 집계가 섞인 실전형
- DDL 변경 없이 기존 PK 인덱스만으로 관찰

Benchmark — 6종 비교표

3.8절 — DDL 변경 없이 기존 PK 인덱스만으로 관찰한 6개 패턴

```
+----+-----------------------------------------+-----------------------------------------+---------+
| #  | 쿼리                                    | 핵심 Operation                          | Buffers |
+----+-----------------------------------------+-----------------------------------------+---------+
| Q1 | claim_type=.. AND total_amt>.. (비인덱스) | TABLE ACCESS FULL                       |   3,117 |
| Q2 | claim_id = '...' (PK 등치)              | INDEX UNIQUE SCAN -> BY INDEX ROWID     |       4 |
| Q3 | pat_id BETWEEN 100 AND 130 (PK 범위)    | INDEX RANGE SCAN -> ...BATCHED          |       9 |
| Q4 | ORDER BY claim_id FETCH FIRST 5 ROWS    | WINDOW NOSORT STOPKEY + INDEX FULL SCAN |       4 |
| Q5 | COUNT(claim_id)                         | SORT AGGREGATE + INDEX FAST FULL SCAN   |   1,891 |
| Q6 | HOSPITALS+MEDICAL_CLAIMS 조인 + GROUP BY | HASH GROUP BY + HASH JOIN               |   3,093 |
+----+-----------------------------------------+-----------------------------------------+---------+

-- Q4는 INDEX(medical_claims PK_MEDICAL_CLAIMS) 힌트로 강제한 결과
```

Q1 — 비인덱스 컬럼 조건 패턴

- 조건 컬럼에 인덱스가 없으면 선택지는 하나뿐 — TABLE ACCESS FULL
  - 조건은 access가 아니라 filter로만 적용된다
  - 읽는 양은 줄지 않고 결과 건수만 줄어든다
- 실측 Buffers 3,117 — 테이블 전체를 훑은 결과다
  - E-Rows와 A-Rows가 190배 차이 난 것도 이 패턴에서 함께 관찰되었다
- 이 패턴이 나오면 확인할 것
  - 조건 컬럼에 인덱스가 있는가, 있다면 왜 쓰이지 않았는가
  - 결과 건수가 전체에서 차지하는 비율은 얼마나 되는가

Q2 — PK 등치 조건

MEDICAL_CLAIMS · WHERE claim_id = '20240615-0013440'

```
| Id | Operation                   | Name             | Starts | E-Rows | A-Rows | Buffers |
| 0  | SELECT STATEMENT            |                  |      1 |        |      1 |       4 |
| 1  |  TABLE ACCESS BY INDEX ROWID | MEDICAL_CLAIMS   |      1 |      1 |      1 |       4 |
| 2  |   INDEX UNIQUE SCAN         | PK_MEDICAL_CLAIMS |      1 |      1 |      1 |       3 |

Predicate Information:
   2 - access("CLAIM_ID"='20240615-0013440')
```

Q3 — PK 범위 조건

PATIENTS · WHERE pat_id BETWEEN 100 AND 130

```
| Id | Operation                           | Name        | Starts | E-Rows | A-Rows | Buffers |
| 0  | SELECT STATEMENT                    |             |      1 |        |     31 |       9 |
| 1  |  TABLE ACCESS BY INDEX ROWID BATCHED | PATIENTS    |      1 |     32 |     31 |       9 |
| 2  |   INDEX RANGE SCAN                  | PK_PATIENTS |      1 |     32 |     31 |       5 |

Predicate Information:
   2 - access("PAT_ID">=100 AND "PAT_ID"<=130)
```

UNIQUE SCAN과 RANGE SCAN, 그리고 BATCHED

- 같은 PK 인덱스인데 오퍼레이션 이름이 갈리는 이유
  - 등치 조건은 결과가 최대 1건임이 보장된다 → INDEX UNIQUE SCAN
  - 범위 조건은 여러 건일 수 있어 시작점부터 끝점까지 순회한다 → INDEX RANGE SCAN
- 인덱스만으로 끝나지 않고 테이블 접근이 한 단계 더 붙는 이유
  - 인덱스에는 키 값과 행의 주소(ROWID)만 있다
  - SELECT 절이 요구하는 나머지 컬럼은 테이블에서 가져와야 한다
- BATCHED — ROWID를 모아 정렬한 뒤 테이블 블록에 접근하는 최적화
  - Oracle 12c 이상에서 도입되었고 단건 조회에는 붙지 않는다

Q4 — 인덱스 정렬 순서 재활용

SELECT claim_id FROM medical_claims ORDER BY claim_id FETCH FIRST 5 ROWS ONLY

```
| Id | Operation              | Name              | Starts | E-Rows | A-Rows | Buffers |
| 0  | SELECT STATEMENT       |                   |      1 |        |      5 |       4 |
| 1  |  VIEW                  |                   |      1 |      5 |      5 |       4 |
| 2  |   WINDOW NOSORT STOPKEY|                   |      1 |      5 |      5 |       4 |
| 3  |    INDEX FULL SCAN     | PK_MEDICAL_CLAIMS |      1 |      5 |      5 |       4 |

Predicate Information:
   1 - filter("from$_subquery$_002"."rowlimit_$$_rownumber"<=5)
   2 - filter(ROW_NUMBER() OVER ( ORDER BY "CLAIM_ID")<=5)
```

SORT가 사라진 이유 — WINDOW NOSORT STOPKEY

- PK 인덱스는 CLAIM_ID 값 순서대로 이미 정렬되어 저장되어 있다
  - ORDER BY가 원하는 순서와 인덱스에 저장된 순서가 정확히 같다
  - 따로 모아서 정렬할 필요 없이 인덱스를 순서대로 읽기만 하면 된다
- 이름을 세 조각으로 쪼개서 읽는다
  - NOSORT — 정렬 순서를 재활용해 정렬 단계를 생략했다
  - STOPKEY — 필요한 5건을 확보하는 즉시 인덱스 읽기를 멈춘다
  - WINDOW — 내부적으로 ROW_NUMBER() OVER (ORDER BY ...) 조건으로 변환되었다
- 이 예제는 힌트로 강제한 것이다 — 힌트 없이는 다른 계획이 선택될 수 있다

Q5 — 인덱스만으로 집계

SELECT COUNT(claim_id) FROM medical_claims

```
| Id | Operation              | Name               | Starts | E-Rows | A-Rows | Buffers |
| 0  | SELECT STATEMENT       |                    |      1 |        |      1 |    1891 |
| 1  |  SORT AGGREGATE        |                    |      1 |      1 |      1 |    1891 |
| 2  |   INDEX FAST FULL SCAN | PK_MEDICAL_CLAIMS  |      1 |   300K |   300K |    1891 |

-- Name 칸에 테이블이 아니라 인덱스만 등장한다
-- MEDICAL_CLAIMS 테이블 블록은 한 번도 읽지 않았다
```

INDEX FULL SCAN과 INDEX FAST FULL SCAN

- 테이블을 전혀 읽지 않을 수 있었던 이유
  - CLAIM_ID는 PRIMARY KEY라 NOT NULL이 보장된다
  - PK 인덱스에는 NULL이 들어갈 수 없으므로 인덱스 항목 수가 곧 COUNT 결과다
- INDEX FULL SCAN — 정렬된 순서를 지켜야 할 때 쓰인다
  - B-Tree의 리프 블록을 논리적인 순서대로 따라 읽는다
- INDEX FAST FULL SCAN — 결과의 순서가 상관없을 때 쓰인다
  - 디스크에 저장된 물리적 순서대로 여러 블록을 한 번에 읽는다
  - 같은 테이블인데도 Q1의 Buffers 3,117과 달리 1,891로 끝났다

Q6 — 조인과 집계가 섞인 실전형

HOSPITALS(HOSP_ID<=10)와 MEDICAL_CLAIMS 조인 후 병원 유형별 집계

```
| Id | Operation                           | Name           | Rows  | Bytes | Cost(%CPU)|
| 0  | SELECT STATEMENT                    |                |     4 |    84 |   860  (1)|
| 1  |  HASH GROUP BY                      |                |     4 |    84 |   860  (1)|
| 2  |   HASH JOIN                         |                |   292 |  4672 |   858  (1)|
| 3  |    TABLE ACCESS BY INDEX ROWID BATCHED| HOSPITALS    |    10 |   120 |     3  (0)|
| 4  |     INDEX RANGE SCAN                | PK_HOSPITALS   |    10 |       |     2  (0)|
| 5  |    TABLE ACCESS FULL                | MEDICAL_CLAIMS | 29128 |   113K|   855  (1)|

Predicate Information:
   2 - access("H"."HOSP_ID"="M"."HOSP_ID")
   4 - access("H"."HOSP_ID"<=10)
   5 - filter("M"."HOSP_ID"<=10)
```

Q6 — 실행 순서와 HASH JOIN의 두 입력

- 실행 순서 — 들여쓰기가 깊은 것부터, 같은 레벨이면 아래쪽부터
  - Id 4 INDEX RANGE SCAN → Id 3 TABLE ACCESS BY INDEX ROWID BATCHED
  - Id 5 TABLE ACCESS FULL → Id 2 HASH JOIN → Id 1 HASH GROUP BY → Id 0
- HASH JOIN은 두 입력을 서로 다른 역할로 쓴다
  - 작은 쪽으로 해시 테이블을 메모리에 만든다 (Build)
  - 큰 쪽을 한 행씩 그 해시 테이블에 대조한다 (Probe)
  - HOSPITALS Rows 10 vs MEDICAL_CLAIMS Rows 29128 — 작은 쪽이 Build 입력
- 작은 테이블은 인덱스로, 큰 테이블은 인덱스가 없어 Full Scan으로 처리되었다

요약 (Summary)

- 실행계획은 옵티마이저가 SQL 처리 절차를 트리로 표현한 것이며, 노드 하나가 물리적 작업 하나다
- 실행 순서는 들여쓰기가 깊은 것부터, 같은 레벨이면 아래쪽부터다 — Id 번호 순서가 아니다
- Rows·Cost·Bytes는 실행 전 추정치, A-Rows·Buffers·A-Time은 실행 후 실측치다
- 두 값은 EXPLAIN PLAN과 DBMS_XPLAN.DISPLAY_CURSOR로 각각 확인한다
- Cost는 무차원 상대 지표로, 같은 SQL의 대안 계획끼리만 비교가 유효하다
- access는 찾아가는 조건, filter는 가져온 뒤 걸러내는 조건 — 인덱스 유무로 갈린다
- 동일 PK 인덱스도 조건 성격에 따라 UNIQUE·RANGE·FULL·FAST FULL SCAN으로 갈린다
- E-Rows와 A-Rows 차이가 크면 카디널리티 추정 오류를 의심한다 (ch08에서 대응)

실습 안내

- 실습문제 10문항 — ch03_문제.md
  - 1~4번 판독 기본기 — filter와 Full Scan, E-Rows와 A-Rows, UNIQUE와 RANGE
  - 5~8번 심화 판독 — WINDOW NOSORT STOPKEY, FAST FULL SCAN, 실행 순서, access와 filter
  - 9~10번 종합 적용 — Cost 비교의 한계, 데이터가 커질 때 위험해질 시나리오 예측
- 문항 구성은 판독 기본기 → 심화 판독 → 종합 적용 순으로 이어진다
- 실습 전 확인 — 이번 장은 인덱스를 신규 생성하지 않는다
  - 기존 PK 인덱스만으로 모든 문항을 풀 수 있다
  - 인덱스나 통계를 임의로 바꾸면 교안의 실행계획이 재현되지 않는다

Oracle SQL 튜닝 과정 · ch04

SQL 성능 측정

EXPLAIN PLAN vs DISPLAY_CURSOR · A-Rows / A-Time / Buffers / Reads · Logical I/O vs Physical I/O · 콜드 캐시와 웜 캐시

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - EXPLAIN PLAN(추정)과 DISPLAY_CURSOR(실측)의 차이를 설명한다
  - DBMS_XPLAN.DISPLAY_CURSOR(...'ALLSTATS LAST')로 실행 통계를 조회한다
  - E-Rows / A-Rows / A-Time / Buffers / Reads가 각각 무엇을 세는지 구분한다
  - Logical I/O와 Physical I/O의 차이를 정의한다
  - Buffers ≥ Reads라는 부등식이 항상 성립하는 이유를 설명한다
  - 같은 Buffers에서도 소요 시간이 크게 달라질 수 있는 이유를 실측으로 설명한다
  - 캐시 예열 효과를 배제한 측정 절차를 설계한다
  - 개발 환경의 반복 테스트 결과를 운영 성능으로 단정할 수 없는 이유를 반박한다
  - SQL 진단 시 최소한 확인해야 할 지표 목록을 스스로 정리한다

이 장의 구성 (Agenda)

- 14.1 Why — 무엇을 근거로 빨라졌다고 말할 것인가
- 14.2 Concept — 측정 도구와 실행 통계 지표
- 14.3 Oracle Internals — Buffers와 Reads는 서로 다른 것을 센다
- 14.4 Example — 버퍼 캐시를 비우고 같은 쿼리를 두 번
- 14.5 Execution Plan — 콜드 캐시 vs 웜 캐시 실측
- 14.6 Bad SQL — Buffers만 보고 튜닝했다면
- 14.7 Tuning · Benchmark — 실무에서 이 차이가 중요한 이유
- 14.8~14.9 실습 안내 · 요약 (Review) — 14개 챕터 총정리

4.1

무엇을 근거로 빨라졌다고 말할 것인가

- 열세 개 챕터가 계속 써 온 지표, Buffers
- 그런데 이미 여러 번 만난 'Buffers만으론 안 보이는 비용'
- 이 차시의 실험 설계와 안전 범위

열세 개 챕터가 남긴 하나의 질문

- 이 과정은 ch01부터 Buffers를 핵심 지표로 계속 사용해 왔다
  - 인덱스 설계, SARGable, 조인, 서브쿼리, 페이징 — 모든 개선의 성적표가 Buffers였다
  - Buffers는 데이터 규모·동시성과 무관하게 SQL 자체의 효율을 보여 주는 지표이기 때문
- 그런데 ch11·ch13에서 이미 "Buffers만으로는 안 보이는 비용"을 여러 번 확인했다
  - Buffers는 거의 같은데 A-Time은 확연히 달라지는 사례들
- 본 차시는 그중 가장 근본적인 것을 정면으로 다룬다
  - Buffers(논리적 I/O)와 실제 디스크 접근(물리적 I/O)의 차이
  - 이 차이를 통제된 실험으로 직접 확인하고, 과정에서 배운 측정 도구를 총정리한다

이미 여러 번 만난 'Buffers만으론 안 보이는 비용'

- ch01 — IN vs EXISTS
  - 실행계획이 완전히 동일한데 A-Time만 크게 달랐던 사례
  - 당시에는 "캐시 효과"라고만 정리하고 근본 원인은 뒤로 미뤄 두었다
- ch11 — SORT UNIQUE
  - Buffers 수치에 잡히지 않는 정렬·중복 제거 비용
- ch13 — WINDOW SORT PUSHED RANK
  - 역시 Buffers 밖에서 발생하는 비용
- 이 장에서 다루는 것은 그 목록 중 가장 근본적인 항목이다
  - 메모리에서 읽었는가, 디스크까지 갔는가 — 같은 Buffers라도 실제 비용이 다르다

이 차시의 실험 설계와 안전 범위

- 소재: 같은 쿼리를 캐시 상태만 바꿔 두 번 실행한다
  - 버퍼 캐시가 빈 상태(콜드)와 채워진 상태(웜)에서 각각 실행
  - Logical I/O(Buffers)와 Physical I/O(Reads)의 차이를 직접 확인
- 캐시를 비우는 작업은 SYSDBA 권한으로 수행한다
  - ALTER SYSTEM FLUSH BUFFER_CACHE — 인스턴스 레벨 작업
  - SQLT 계정의 객체·데이터에는 영향이 없고 DDL 변경도 없다
- 왜 굳이 강제로 비우는가
  - 평소에는 캐시 상태를 통제할 수 없어 두 조건을 나란히 비교할 수 없기 때문

참고:
ALTER SYSTEM FLUSH BUFFER_CACHE는 인스턴스 전체의 버퍼 캐시를 비우므로, 같은 DB를 쓰는 다른 세션의 응답 시간도 함께 나빠진다. 운영 DB에서는 실행하지 않는다.

4.2

측정 도구와 실행 통계 지표

- 추정과 실측 — 두 갈래 도구
- EXPLAIN PLAN / DBMS_XPLAN.DISPLAY
- DBMS_XPLAN.DISPLAY_CURSOR(...'ALLSTATS LAST')
- E-Rows · A-Rows · A-Time · Buffers · Reads

추정과 실측 — 도구는 두 갈래다

- EXPLAIN PLAN — 실행하지 않는다
  - SQL을 실제로 돌리지 않고 옵티마이저의 예상 계획만 산출한다
  - 결과는 Rows / Cost / Bytes 등 전부 추정치
  - DBMS_XPLAN.DISPLAY로 저장된 계획을 보기 좋게 출력한다
- DBMS_XPLAN.DISPLAY_CURSOR(...'ALLSTATS LAST') — 실행한다
  - SQL을 실제로 실행한 뒤 그 실행 통계를 조회한다
  - 결과는 E-Rows / A-Rows / A-Time / Buffers / Reads 등 실측값
- 이 구분은 ch02부터 계속 사용해 온 원칙이다
  - 두 도구는 대체 관계가 아니라 용도가 다른 도구다

추정 — EXPLAIN PLAN + DBMS_XPLAN.DISPLAY

실행하지 않고 옵티마이저의 예상 계획만 얻는다

```
-- 1) 실행하지 않고 옵티마이저의 예상 계획만 산출한다
EXPLAIN PLAN FOR
SELECT claim_id, total_amt
  FROM medical_claims
  WHERE claim_id BETWEEN '20240601-0000000' AND '20240602-0000000';

-- 2) 저장된 계획을 보기 좋게 출력한다
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- 얻는 것    : Rows / Bytes / Cost  -> 전부 옵티마이저의 추정치
-- 얻지 못함  : A-Rows / A-Time / Buffers / Reads (실행해야 나오는 값)
```

실측 — GATHER_PLAN_STATISTICS + DISPLAY_CURSOR

실제로 실행한 뒤 그 실행 통계를 조회한다

```
-- 1) 실행 통계를 수집하도록 힌트를 준다
SELECT /*+ GATHER_PLAN_STATISTICS */ claim_id, total_amt
  FROM medical_claims
  WHERE claim_id BETWEEN '20240601-0000000' AND '20240602-0000000';

-- 2) 방금 실행한 커서의 실제 통계를 조회한다
SELECT * FROM TABLE(
         DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));

-- ALLSTATS LAST : 마지막 한 번의 실행분에 대한 실측 통계
-- 얻는 것 : E-Rows / A-Rows / A-Time / Buffers / Reads
--           추정(E-Rows)과 실제(A-Rows)를 한 화면에서 대조할 수 있다
```

실행 통계 지표 사전

- E-Rows — 옵티마이저가 예상한 행 수(추정)
- A-Rows — 실제로 처리된 행 수(Rows Processed)
  - E-Rows와 A-Rows의 괴리가 크면 통계정보 문제를 의심한다 (ch03·ch08)
- A-Time — 실제로 걸린 전체 경과 시간(Elapsed Time)
  - 사용자가 체감하는 시간에 가장 가까운 지표
- Buffers — 버퍼 캐시에서 블록을 읽은 횟수(Consistent Gets)
  - Logical I/O. "블록을 몇 번 요청했는가"
- Reads — 디스크에서 실제로 블록을 읽어 온 횟수(Physical Reads)
  - Physical I/O. 값이 0이면 컬럼 자체가 표시되지 않는다

추정 계획으로는 볼 수 없는 것

- EXPLAIN PLAN이 알려 주지 못하는 것들
  - 실제로 몇 건이 나왔는지(A-Rows)
  - 실제로 얼마나 걸렸는지(A-Time)
  - 블록을 몇 번 요청했는지(Buffers)
  - 그중 디스크까지 몇 번 갔는지(Reads)
- 이 장의 콜드/웜 캐시 차이는 원리상 EXPLAIN PLAN으로 확인할 수 없다
  - 캐시 상태는 실행 시점의 런타임 조건이지 계획의 속성이 아니기 때문
  - 같은 SQL이라면 캐시가 비어 있든 차 있든 EXPLAIN PLAN 결과는 동일하다
- 결론: 튜닝 성과의 증거는 반드시 실측 통계로 제시한다

Logical I/O와 Physical I/O

- Logical I/O — 논리적 I/O
  - 버퍼 캐시(메모리)에서 블록을 읽은 횟수
  - 실행계획에는 Buffers(= Consistent Gets)로 표시된다
- Physical I/O — 물리적 I/O
  - 디스크에서 실제로 블록을 읽어 온 횟수
  - 실행계획에는 Reads로 표시된다
- 두 값의 관계
  - 요청한 블록이 이미 메모리에 있으면 Logical I/O만 발생한다
  - 메모리에 없으면 디스크까지 내려가므로 Physical I/O가 함께 발생한다

4.3

Buffers와 Reads는 서로 다른 것을 센다

- Buffers — 블록을 몇 번 요청했는가
- Reads — 그중 실제로 디스크까지 간 횟수
- Buffers ≥ Reads 라는 부등식
- 캐시 예열 효과의 근본 원인

Buffers — 블록을 몇 번 요청했는가

- Buffers(Consistent Gets)는 요청 횟수를 센다
  - "이 블록이 필요하다"고 요청한 횟수 그 자체
  - 그 블록이 메모리에 있었는지 디스크에 있었는지는 세지 않는다
- 그래서 Buffers는 캐시 상태와 무관하다
  - 같은 SQL이 같은 데이터를 읽는다면 요청해야 할 블록 수는 언제나 같다
  - 캐시가 비어 있든 가득 차 있든 요청 횟수는 변하지 않는다
- 이 성질 때문에 Buffers는 SQL 자체의 효율을 재는 좋은 지표가 된다
  - 측정 시점의 우연한 조건에 흔들리지 않는다
  - 이 과정 열세 개 챕터가 Buffers를 성적표로 삼아 온 이유가 이것이다

Reads — 그중 디스크까지 간 횟수

- Reads(Physical Reads)는 요청 중 실제로 디스크까지 가야 했던 횟수만 센다
  - 요청한 블록이 이미 버퍼 캐시에 있으면 디스크까지 갈 필요가 없다
  - 그런 요청은 Buffers에는 잡히지만 Reads에는 잡히지 않는다
- 따라서 Reads는 Buffers의 부분집합이다
  - Buffers ≥ Reads — 이 부등식은 항상 성립한다
  - Reads가 Buffers보다 클 수는 없다(전체보다 부분이 클 수 없으므로)
- Reads는 캐시 상태에 따라 크게 달라진다
  - 캐시가 비어 있으면 요청 대부분이 디스크로 내려간다
  - 캐시가 채워져 있으면 0에 가까워지고, 0이면 컬럼 자체가 사라진다

캐시 예열 효과 — 반복 실행의 함정

- 같은 쿼리를 반복 실행하면 무슨 일이 일어나는가
  - 첫 실행이 읽어 온 블록들이 버퍼 캐시에 남는다
  - 두 번째 실행은 그 블록들을 메모리에서 그대로 재사용한다
- 그 결과
  - Buffers는 그대로다 — 요청해야 할 블록 수는 변하지 않으므로
  - Reads는 0에 가까워진다 — 디스크까지 갈 일이 없어지므로
  - A-Time은 크게 줄어든다 — 디스크 접근이 사라졌으므로
- ch01에서 이미 관찰한 현상의 근본 원인이 바로 이것이다
  - 실행계획이 같은데 시간만 달랐던 이유는 문법이 아니라 실행 순서였다

강제로 비우지 않아도 콜드는 발생한다

- ALTER SYSTEM FLUSH BUFFER_CACHE는 SYSDBA 권한의 인스턴스 레벨 명령이다
  - SQLT 같은 일반 애플리케이션 계정은 실행할 수 없다
- 그런데도 실무에서 콜드 캐시 상황은 계속 발생한다
  - 버퍼 캐시는 서버 메모리 중 정해진 크기만큼만 할당된다 — 용량이 유한하다
  - 한 서버에서 수많은 세션과 쿼리가 그 캐시를 동시에 공유해서 쓴다
- 공간이 부족해지면 오래 쓰이지 않은 블록부터 밀려난다
  - LRU 계열 정책으로 새 블록을 위한 자리를 확보한다
  - 한동안 조회되지 않은 테이블, 다른 대량 조회에 밀린 블록은 자연히 캐시에서 빠진다
  - 다음 접근 때 다시 디스크에서 읽어야 하는 콜드 상태가 된다

4.4

버퍼 캐시를 비우고 같은 쿼리를 두 번

- 실험 절차 — 변수는 캐시 상태 하나뿐
- 측정 대상 쿼리와 캐시 비우기
- 재현 가능한 측정의 조건

실험 절차 — 변수는 캐시 상태 하나뿐

- 1단계 — SYSDBA 세션에서 버퍼 캐시를 강제로 비운다
  - 이 시점 이후 첫 접근은 전부 디스크에서 읽어야 하는 상태가 된다
- 2단계 — SQLT 세션에서 대상 쿼리를 실행하고 실행 통계를 확인한다
  - 이것이 콜드 캐시 측정값이다
- 3단계 — 같은 쿼리를 즉시 다시 실행하고 실행 통계를 확인한다
  - 2단계가 올려놓은 블록이 그대로 남아 있으므로 웜 캐시 측정값이 된다
- 고정한 것과 바꾼 것
  - 고정: 쿼리문, 대상 테이블, 데이터, 실행계획, 세션
  - 변경: 버퍼 캐시 상태 하나뿐

Example — 캐시 비우기와 측정 대상 쿼리

같은 쿼리를 연속으로 두 번 실행한다

```
-- (SYSDBA) 버퍼 캐시를 강제로 비움
--          이 세션 이후 첫 접근은 전부 디스크에서 읽어야 함
ALTER SYSTEM FLUSH BUFFER_CACHE;

-- (SQLT) 같은 쿼리를 연속으로 두 번 실행
SELECT claim_id, total_amt FROM medical_claims
WHERE claim_id BETWEEN '20240601-0000000' AND '20240602-0000000';

-- 첫 번째 실행 -> 콜드 캐시 : 버퍼 캐시가 빈 상태에서 시작
-- 두 번째 실행 -> 웜  캐시 : 첫 실행이 올려놓은 블록을 재사용
--
-- 두 실행 모두 DISPLAY_CURSOR(...ALLSTATS LAST)로 통계를 확인한다
```

재현 가능한 측정의 조건

- 측정값을 남길 때 함께 기록해야 하는 것들
  - 캐시 상태 — 콜드인가 웜인가, 몇 번째 실행인가
  - 실행계획 — 두 측정에서 계획이 동일했는가
  - 데이터 조건 — 조회 범위와 결과 건수(A-Rows)가 같았는가
  - 측정 도구 — 추정인가 실측인가
- 한 번의 측정으로 결론을 내리지 않는다
  - 같은 조건을 여러 번 재고, 값이 흔들리면 흔들리는 이유를 먼저 설명한다
  - 설명되지 않는 편차가 남아 있는 상태의 수치는 근거로 쓰지 않는다
- 결과는 지표별로 나란히 적어 비교한다
  - Buffers · Reads · A-Time을 한 표에 놓아야 차이의 원인이 보인다

4.5

콜드 캐시 vs 웜 캐시 실측

- 콜드 캐시 — 버퍼 캐시를 비운 직후 첫 실행
- 웜 캐시 — 동일 쿼리 즉시 재실행
- 세 지표가 각각 무엇을 말하는가

콜드 캐시 — 버퍼 캐시를 비운 직후 첫 실행

DBMS_XPLAN.DISPLAY_CURSOR(...ALLSTATS LAST) 실측

```
| Id | Operation                           | Name             | A-Rows |   A-Time   | Buffers | Reads |
| 0  | SELECT STATEMENT                    |                  |    804 |00:00:00.01 |     787 |   785 |
| 1  |  TABLE ACCESS BY INDEX ROWID BATCHED| MEDICAL_CLAIMS   |    804 |00:00:00.01 |     787 |   785 |
| 2  |   INDEX RANGE SCAN                  | PK_MEDICAL_CLAIMS|    804 |00:00:00.01 |      61 |    64 |

-- Reads 컬럼이 존재한다 = 디스크까지 내려간 요청이 있었다는 뜻
-- 요청한 블록 787 중 785 를 디스크에서 읽어와야 했다
-- A-Time 00:00:00.01 : 이 VM 은 SSD 위라 785 블록 물리 읽기가 10ms 안에 끝났다(회전 디스크면 수 초)
```

웜 캐시 — 동일 쿼리 즉시 재실행

쿼리도 계획도 그대로, 캐시 상태만 다르다

```
| Id | Operation                           | Name             | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT                    |                  |    804 |00:00:00.01 |     776 |
| 1  |  TABLE ACCESS BY INDEX ROWID BATCHED| MEDICAL_CLAIMS   |    804 |00:00:00.01 |     776 |
| 2  |   INDEX RANGE SCAN                  | PK_MEDICAL_CLAIMS|    804 |00:00:00.01 |      61 |

-- Reads 컬럼이 아예 나타나지 않는다 = 물리적 읽기가 0이었다는 뜻
-- 요청한 블록이 전부 메모리에 있어 디스크 접근이 전혀 없었다
-- Buffers 776 : 콜드 캐시(787)와 사실상 동일하다 — 차이 11 은 물리 읽기 때 더 요청한 헤더 블록
-- A-Rows 804   : 처리한 행 수도 동일하다
```

해석 — 세 지표가 각각 말하는 것

- Buffers는 787 → 776 으로 사실상 동일했다
  - 논리적으로 요청한 블록 수는 캐시 상태와 무관하게 같기 때문
  - SQL 자체의 효율은 두 실행에서 조금도 달라지지 않았다는 뜻
- Reads는 785에서 컬럼 자체가 사라지는 상태로 바뀌었다
  - 콜드에서는 요청한 블록 대부분을 디스크에서 읽어야 했다
  - 웜에서는 전부 메모리에 있어 디스크 접근이 전혀 없었다
- A-Time은 둘 다 0.01초 — SSD 라 차이가 안 보이지만 회전 디스크면 수백 배가 되는 자리
  - 물리 읽기 한 건의 비용은 저장 장치가 정한다 — 그래서 시간이 아니라 Reads 로 판정한다
  - 같은 SQL, 같은 계획, 같은 결과 건수인데 체감 속도만 달라졌다

Buffers와 Reads의 작은 차이는 어디서 왔나

- 콜드 캐시 실측에서 두 값이 정확히 같지는 않았다
  - Buffers 787 / Reads 785 — 요청 단위와 읽기 단위가 다르다(선읽기, 캐시된 루트·브랜치)
- 왜 완전히 같지 않은가
  - 이 쿼리는 데이터 블록만 읽는 것이 아니라 실행 준비 과정에서도 블록을 참조한다
  - 세그먼트 헤더, 언두(undo) 관련 블록 등 부가적인 블록들
  - 그중 일부는 콜드 상태에서도 이미 캐시에 남아 있었을 수 있다
- 세부 원인 특정보다 중요한 것
  - 극단적 콜드 상태여도 Buffers와 Reads가 완전히 같을 필요는 없다
  - Reads가 Buffers를 넘어설 수는 없다 — 부분집합이므로

4.6

Buffers만 보고 튜닝했다면

- 같은 Buffers, 전혀 다른 체감 속도
- "개발 환경에서는 항상 빠르던데요"
- 지표 하나로 결론을 내리는 습관의 위험

같은 Buffers, 전혀 다른 체감 속도

- 이 실험에서 Buffers는 콜드든 웜이든 780 안팎으로 사실상 동일했다
- 만약 Buffers만 보고 판단했다면
  - "이 쿼리는 어떤 상황에서도 똑같은 비용"이라고 결론지었을 것이다
  - 785 블록의 물리 읽기(느린 저장 장치에서는 수 초)를 완전히 놓쳤을 것이다
- 이것이 앞선 장들에서 반복 확인한 현상의 근본 원인이다
  - ch01 — IN vs EXISTS의 캐시 효과
  - ch11 — SORT UNIQUE
  - ch13 — WINDOW SORT PUSHED RANK
  - "Buffers만으론 안 보이는 비용"의 가장 근본적인 정체가 Logical/Physical I/O 차이였다

"개발 환경에서는 항상 빠르던데요"

- 흔한 보고 — "여러 번 테스트해 봤는데 항상 빠릅니다"
- 왜 이 보고가 운영 성능을 보장하지 못하는가
  - 반복 테스트 환경은 이미 같은 쿼리·같은 테이블에 여러 번 접근한 상태다
  - 해당 블록들이 버퍼 캐시에 남아 있는 웜 캐시 상태일 가능성이 높다
  - 즉 개발자가 본 "항상 빠름"은 웜 캐시만 반복 관찰한 결과일 수 있다
- 이번 실측이 보여 준 것
  - 같은 쿼리, 같은 계획인데 콜드는 3.46초, 웜은 0.01초였다
  - 운영에서 오랜만에 조회되는 데이터나 서버 재시작 직후에는 전혀 다른 체감이 나온다

4.7

실무에서 이 차이가 중요한 이유

- 운영 DB는 웜에 가깝다 — 그러나 항상은 아니다
- 콜드에 가까운 상황들
- Buffers를 낮추는 튜닝이 콜드에서 더 크게 보상받는 이유

운영 DB는 웜에 가깝다 — 그러나 항상은 아니다

- 운영 중인 DB의 일반적인 상태
  - 자주 조회되는 데이터는 대부분 버퍼 캐시에 남아 있다
  - 그래서 평소 트래픽은 웜 캐시에 가까운 조건에서 처리된다
- 그 결과 생기는 착시
  - 개발자가 같은 쿼리를 반복 테스트하면 항상 빠르게 느껴질 수 있다
  - 문제가 드러나는 것은 캐시가 듣지 않는 조건이 왔을 때다
- 그러므로 두 조건을 모두 재야 한다
  - 평소 조건(웜)에서의 응답 시간 — 사용자 체감의 기준선
  - 최악 조건(콜드)에서의 응답 시간 — 장애 시 감당해야 할 상한

콜드 캐시에 가까운 상황들

- 새벽 배치
  - 월말 정산 리포트가 MEDICAL_CLAIMS와 CLAIM_DETAILS를 대량으로 훑는 경우
  - 새벽에 한 번만 실행되며, 직전까지 캐시에 있던 블록과는 무관한 블록을 요청한다
- 오랜만에 조회하는 오래된 데이터
  - 과거 이력 조회, 감사 대응, 연말 통계 등
- DB 재시작 직후
  - 버퍼 캐시가 완전히 비어 있는 상태에서 트래픽을 받기 시작한다
- 버퍼 캐시보다 훨씬 큰 대용량 테이블
  - 읽는 족족 밀려나므로 반복 조회해도 캐시 적중을 기대하기 어렵다
- 공통점 — 대량·비반복 조회는 필연적으로 콜드에 가깝다

콜드에서 Buffers 절감이 더 크게 보상받는 이유

- Buffers는 요청 횟수이고, 콜드에서는 그 요청이 대부분 물리 I/O가 된다
  - 즉 콜드 상황에서 Buffers는 곧 디스크 접근 횟수의 상한선에 가까워진다
- Buffers가 10배 큰 쿼리라면
  - 콜드 상황에서 물리 I/O 요청도 대략 10배로 늘어날 수 있다
  - 응답 시간이 훨씬 더 크게 나빠질 수 있다
- 그래서 Buffers를 낮추는 튜닝은 콜드에서 이득이 훨씬 크다
  - 웜 캐시 상황에서의 이득보다 콜드 캐시 상황에서의 이득이 훨씬 크다
  - 인덱스 설계, 조인 최적화, N+1 회피 등 이 과정 전체의 기법이 여기에 해당한다
- 결론 — Buffers가 낮으면 최악의 경우에도 피해가 그만큼 제한된다

Benchmark — 콜드 캐시 vs 웜 캐시

같은 쿼리·같은 실행계획, 캐시 상태만 다르다

상태	Buffers	Reads	A-Time
콜드 캐시(첫 실행)	787	785	0.01초
웜 캐시(재실행)	776	0(표시 안 됨)	0.01초
배율	1배(동일)	785 → 0	SSD 라 동일

-- 세로로 읽으면 : Buffers는 안 변하고 Reads와 A-Time만 변했다
-- 가로로 읽으면 : 같은 SQL이 조건에 따라 전혀 다른 체감을 준다

성능 측정 체크리스트

- 측정 전 — 무엇을 재는지 정한다
  - 평소 조건(웜)인가 최악 조건(콜드)인가를 먼저 결정한다
  - 비교 대상 두 SQL의 결과가 동일한지 먼저 확인한다(A-Rows 대조)
- 측정 중 — 변수를 하나만 바꾼다
  - 쿼리, 데이터, 세션, 시간대 중 바꾼 것이 하나인지 점검한다
  - 실행 직후 DISPLAY_CURSOR로 통계를 조회한다(다른 SQL을 끼워 넣지 않는다)
- 측정 후 — 조건과 함께 기록한다
  - Buffers · Reads · A-Time을 한 표에 나란히 적는다
  - 실행계획 원문을 함께 남겨 다른 사람이 검증할 수 있게 한다
  - 숫자만 있고 조건이 없는 보고서는 근거로 쓰지 않는다

진단 시 최소한 확인할 지표

이 과정 전체에서 다룬 지표를 다섯 개로 압축하면

#	지표	어디서 보는가	무엇을 알려주는가
1	Rows / Cost	EXPLAIN PLAN (추정)	옵티마이저의 예상 – 실행 전 1차 점검
2	E-Rows vs A-Rows	ALLSTATS LAST (실측)	추정과 실제의 괴리 – 통계정보 문제 의심
3	Buffers	ALLSTATS LAST (실측)	논리적 I/O 총량 – SQL 자체의 효율성
4	Reads	ALLSTATS LAST (실측)	실제 디스크 접근 – 캐시 상태에 따른 체감차
5	A-Time	ALLSTATS LAST (실측)	최종 소요 시간 – 위 지표들의 종합 결과

```
-- 순서대로 보는 것이 핵심 : 추정 -> 정확도 -> 효율 -> 캐시 -> 결과
```

14개 챕터를 측정의 관점으로 다시 읽기

- 무엇을 고칠 것인가 — 진단과 원리
  - ch01~ch03 SQL 튜닝의 정의 · SQL 실행 원리 · 실행계획 기초
  - ch08 Optimizer와 통계정보 — 추정이 어긋나는 이유
- 어떻게 고칠 것인가 — 접근 경로와 문장 구조
  - ch05~ch07 Full Scan vs Index Scan · 인덱스 설계 · SARGable SQL
  - ch09~ch10 JOIN 실행 원리 · Subquery / EXISTS / IN
  - ch11~ch13 UNION / DISTINCT · SORT / GROUP BY / 집계 · 페이징 SQL 튜닝
  - ch14 실무에서 자주 발생하는 SQL 문제
- 무엇을 근거로 나아졌다고 말할 것인가 — 측정
  - ch04 이 장 — 도구와 지표, 그리고 측정 조건의 통제

요약 (Review)

- EXPLAIN PLAN은 실행 전 추정치, DISPLAY_CURSOR(ALLSTATS LAST)는 실행 후 실측이다
  - ch02부터 계속 지켜 온 구분이며, 성과 증명에는 반드시 실측을 쓴다
- Buffers(논리적 I/O)와 Reads(물리적 I/O)는 서로 다른 것을 측정한다
  - Buffers는 요청 횟수, Reads는 그중 디스크까지 간 횟수 — Buffers ≥ Reads
- 같은 Buffers라도 캐시 상태에 따라 소요 시간이 수백 배 차이 날 수 있다
  - 이번 실측은 780 안팎의 같은 Buffers에서 Reads 785 → 0 의 차이를 보였다(시간 차는 저장 장치가 정한다)
- 개발 환경의 반복 테스트는 웜 캐시 상태이기 쉽다
  - 콜드 캐시의 실제 비용을 과소평가하게 만들 수 있으므로 조건을 명시해 기록한다
- Buffers를 낮추는 튜닝은 캐시 상태와 무관하게 항상 유효하다
  - 이 과정 전체에서 다룬 기법들은 콜드 캐시 상황에서 그 효과가 더욱 커진다

실습 안내

- ch04_문제.md — 10문항, 전부 서술형
- 문항 지도
  - 1·2번 — Buffers와 A-Time이 각각 무엇을 재는가, Reads 컬럼 소멸의 의미
  - 3번 [최상] — Buffers와 Reads의 차이, 그리고 부등식이 성립하는 이유
  - 4·5번 — 개발 환경 반복 테스트의 한계, EXPLAIN PLAN으로 확인 불가능한 것
  - 6번 [최상] — 앞선 챕터 사례와 Logical/Physical I/O의 연결
  - 7·8번 — 콜드에서 Buffers 절감의 가치, 실습 스키마 기준 콜드 상황 예시
  - 9번 [최상] — SYSDBA 없이도 콜드 캐시가 발생하는 이유
  - 10번 [최상] — 14개 챕터를 관통하는 진단 지표 목록 정리
- 해설: ch04_해설.md — 문항 번호가 1:1로 대응한다

데이터 접근·인덱스·통계

- SQLT ch05 · ch06 · ch07 · ch08
- 인덱스가 있는데 왜 안 타는가 → 통계

Oracle SQL 튜닝 과정 · ch05

Full Scan vs Index Scan

선택도와 카디널리티 · 두 비용 곡선의 교차점 · 클러스터링 팩터 · Index-Only

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - 선택도(Selectivity)와 카디널리티(Cardinality)를 정의하고 서로 구분한다
  - Full Scan의 비용이 테이블 전체 블록 수에 비례한다는 것을 실측으로 설명한다
  - Index Range Scan의 비용이 결과 건수와 랜덤 접근에 비례함을 설명한다
  - 두 비용 곡선이 교차하는 지점(손익분기)의 의미를 설명한다
  - 작은 테이블에서 Full Scan이 오히려 저렴한 이유를 근거를 들어 말한다
  - 같은 인덱스라도 범위가 넓어지면 Full Scan으로 전환되는 현상을 관찰한다
  - Index-Only Access가 선택도의 경계를 어떻게 바꾸는지 설명한다
  - 클러스터링 팩터가 교차점을 움직이는 변수임을 이해한다
  - '선택도 몇 % 이하면 인덱스'라는 단일 규칙이 왜 위험한지 반박한다
  - Full Scan이 유리한 상황을 두 가지 이상의 조건으로 스스로 정의한다

이 장의 구성 (Agenda)

- 4.1 Why — 읽는 법은 익혔지만 판단 기준이 없다
- 4.2 Concept — 선택도 · 카디널리티 · 데이터 분포
- 4.3 Oracle Internals — 두 비용 곡선과 교차점
- 4.4 Example — 작은 테이블 vs 큰 테이블 Full Scan
- 4.5 Execution Plan — HOSPITALS와 CLAIM_DETAILS 실측 비교
- 4.6 Bad SQL — 선택도에 따른 접근 방식 전환 관찰
- 4.7~4.8 Tuning 예고 · Benchmark — 선택도별 접근 방식 요약
- 4.9~4.10 실습 안내 · 요약 (Review)

5.1

읽는 법에서 판단 기준으로

- ch03에서 다룬 것과 남겨 둔 것
- '인덱스가 있으면 무조건 빠르다'는 통념
- 이 차시의 범위와 관찰 소재

ch03에서 여기까지

- ch03에서 얻은 것 — 실행계획을 읽는 어휘
  - TABLE ACCESS FULL, INDEX RANGE SCAN 등 오퍼레이션의 의미
  - 들여쓰기로 표현된 실행 순서와 단계별 Buffers 확인 방법
- ch03에서 다루지 않은 것 — 어느 쪽이 유리한지의 판단
  - 계획을 읽을 수 있다는 것과 그 계획이 최선인지 아는 것은 다른 능력
  - 옵티마이저가 왜 그 접근 방식을 골랐는지는 아직 설명하지 못한다
- 이 장의 질문 — 언제 Full Scan이 오히려 빠른가
  - 판단의 축은 두 개다: 테이블의 크기, 그리고 조건의 선택도

'인덱스는 항상 옳다'는 통념

- 현장에서 가장 널리 퍼진 통념 중 하나
  - "느리면 인덱스를 만들어라" — 원인 진단 없이 처방부터 하는 습관
  - "인덱스를 안 타는 계획은 잘못된 계획이다" — 실행계획을 오독하는 원인
- 이 통념이 틀리는 두 가지 대표 상황
  - 테이블이 작아서 전부 읽어도 이미 충분히 저렴한 경우
  - 조건에 걸리는 행이 너무 많아 랜덤 접근이 누적되는 경우
- 통념 대신 필요한 것 — 비용 구조에 대한 이해
  - 두 접근 방식이 각각 무엇에 비례해 비싸지는지를 알면 예측이 가능해진다
  - 이 장은 그 구조를 실습 DB 실측으로 확인한다

이 차시의 범위와 관찰 소재

- 이번 차시는 인덱스를 신규 생성하지 않는다
  - 기존 PK만 사용한다 — 스키마를 바꾸지 않고 접근 방식만 관찰
  - 인덱스 설계 자체는 ch06의 주제
- 관찰 소재 ① 테이블 크기 축
  - 작은 테이블 HOSPITALS와 큰 테이블 CLAIM_DETAILS의 Full Scan 비용 비교
- 관찰 소재 ② 선택도 축
  - PK_MEDICAL_CLAIMS를 이용한 범위 조회에서 범위를 점점 넓혀 가며 관찰
  - 선택도가 커질 때 접근 방식이 바뀌는 지점을 눈으로 확인한다
- 두 축을 따로 본 뒤 마지막에 합쳐서 판단 기준을 만든다

5.2

선택도와 카디널리티

- Selectivity — 조건이 걸러 내는 비율
- Cardinality — 조건을 만족하는 실제 행 수
- 데이터 분포와 이번 챕터 사례의 선택도

선택도 (Selectivity)

- 정의 — 조건을 만족하는 행이 전체 중 차지하는 비율
  - 선택도 = 조건을 만족하는 행 수 ÷ 전체 행 수
  - 0에 가까울수록 조건이 강하게 걸러 낸다는 뜻
- 선택도가 낮다는 말의 의미
  - 조건을 만족하는 행이 적다 = 걸러 낸 뒤 남는 것이 얼마 없다
  - 이럴수록 인덱스로 필요한 행만 콕 집어 오는 방식이 유리해진다
- 선택도가 높다는 말의 의미
  - 조건을 만족하는 행이 많다 = 어차피 대부분을 읽어야 한다
  - 이럴수록 통째로 읽고 걸러 내는 Full Scan이 유리해진다

카디널리티 (Cardinality)

- 정의 — 조건을 만족하는 실제 행의 개수
  - 카디널리티 = 선택도 × 전체 행 수
  - 비율이 아니라 개수라는 점이 선택도와의 결정적 차이
- 왜 둘을 구분해야 하는가
  - 같은 선택도라도 전체 행 수가 다르면 실제 처리량이 완전히 달라진다
  - 옵티마이저의 비용 계산에 직접 들어가는 값은 개수 쪽이다
- 실행계획에서 만나는 자리
  - E-Rows — 옵티마이저가 통계정보로 추정한 카디널리티
  - A-Rows — 실제로 처리된 행 수, 즉 실측 카디널리티
  - 둘의 차이가 크면 통계정보나 조건 표현을 의심한다

데이터 분포가 선택도를 바꾼다

- 같은 컬럼, 같은 형태의 조건이라도 선택도는 값마다 다르다
  - 값이 고르게 퍼져 있는지, 특정 값에 몰려 있는지에 따라 달라진다
  - 평균만 보고 판단하면 몰려 있는 값에서 예측이 크게 빗나간다
- 옵티마이저가 분포를 파악하는 방법
  - 기본은 컬럼의 서로 다른 값 개수를 이용한 균등 분포 가정
  - 분포가 치우친 컬럼은 히스토그램 통계로 보정한다
- 실무에서 사고가 나는 지점
  - 개발 환경의 소량 데이터와 운영의 실제 분포가 다를 때
  - 데이터가 쌓이면서 분포가 변했는데 통계를 갱신하지 않았을 때

이번 챕터 사례의 선택도 계산

- MEDICAL_CLAIMS 전체 30만 건을 기준으로 계산한다
  - 선택도 = 조건을 만족하는 행 수 ÷ 300,000
- 4.6절에서 관찰할 세 구간
  - 838건 → 838 ÷ 300,000 ≈ 0.28%
  - 25,406건 → 25,406 ÷ 300,000 ≈ 8.5%
  - 148,028건 → 148,028 ÷ 300,000 ≈ 49.3%
- 숫자를 미리 계산해 두는 이유
  - 실행계획을 보기 전에 어느 쪽이 유리할지 스스로 예측해 보게 하기 위함
  - 예측과 실측이 어긋나는 지점이 이 장에서 가장 배울 것이 많은 구간이다

5.3

두 비용 곡선과 교차점

- Full Scan의 비용 — 테이블 크기에 비례
- Index Range Scan의 비용 — 결과 건수에 비례
- 교차점과 그 위치를 움직이는 변수들

Full Scan의 비용 구조

- 비용은 테이블 전체 블록 수에 비례한다
  - 테이블에 저장된 블록을 처음부터 끝까지 전부 읽는다
  - 조건 검사는 블록을 다 읽어 온 다음에 이루어진다
- 따라서 결과 건수는 비용에 거의 영향을 주지 않는다
  - 조건에 1건이 걸리든 90만 건이 걸리든 읽는 블록 수는 같다
  - 비용이 사실상 고정적이라는 뜻 — 그래프로는 수평에 가까운 선
- 대신 한 번의 읽기 단위가 크다
  - 연속된 블록을 여러 개씩 묶어 읽으므로 블록당 단가가 낮다
  - 이 성질 때문에 대량 처리에서는 오히려 유리해진다

Index Range Scan의 비용 구조

- 비용은 조건을 만족하는 행의 개수에 비례한다
  - 인덱스에서 조건에 맞는 구간을 훑어 ROWID 목록을 얻는다
  - 그 ROWID로 테이블 블록을 하나씩 찾아가 실제 값을 읽는다
- 두 번째 단계가 비용의 핵심 — 랜덤 접근
  - 흩어져 있는 위치를 건건이 찾아가는 방식이라 단가가 높다
  - 결과가 많아질수록 이 접근이 누적되어 비용이 우상향한다
- 결과가 적을 때는 압도적으로 저렴하다
  - 필요 없는 블록을 아예 건드리지 않기 때문
  - 반대로 결과가 많아지면 Full Scan보다 비싸질 수 있다

두 곡선의 교차점 — 손익분기

- 두 비용 곡선을 한 그래프에 겹쳐 놓으면 교차점이 생긴다
  - Full Scan — 결과 건수와 무관하게 거의 수평
  - Index Range Scan — 결과 건수에 따라 우상향
- 교차점의 왼쪽과 오른쪽
  - 왼쪽(선택도가 낮은 구간) — 인덱스가 유리하다
  - 오른쪽(선택도가 높은 구간) — Full Scan이 유리하다
- 옵티마이저가 하는 일이 바로 이 교차점의 추정이다
  - 통계정보로 카디널리티를 추정해 양쪽 비용을 계산하고 싼 쪽을 고른다
  - 인덱스를 안 타는 계획은 대개 이 계산의 결과이지 오류가 아니다

클러스터링 팩터 — 교차점을 움직이는 변수

- 정의 — 인덱스의 정렬 순서와 실제 테이블 저장 순서가 얼마나 일치하는가
  - 잘 맞으면 인덱스 순서대로 읽을 때 같은 블록을 연달아 만난다
  - 어긋나면 행 하나마다 다른 블록으로 흩어져 랜덤 접근이 늘어난다
- 교차점에 미치는 영향
  - 일치도가 높으면 인덱스 쪽 곡선이 완만해져 교차점이 오른쪽으로 이동
  - 일치도가 낮으면 곡선이 가팔라져 교차점이 왼쪽으로 이동
- 같은 테이블이라도 인덱스마다 값이 다르다
  - 데이터가 들어온 순서와 인덱스 컬럼의 관계에 따라 결정되기 때문
  - 이 값이 나쁘면 선택도가 낮아도 인덱스가 기대만큼 빠르지 않다

작은 테이블에서 Full Scan이 유리한 이유

- 작은 테이블은 애초에 전체 블록 수가 적다
  - 전부 읽어도 비용의 절대값 자체가 크지 않다
- 반면 인덱스를 거치면 단계가 늘어난다
  - 인덱스 블록 접근 → 조건에 맞는 ROWID 확보 → 테이블 블록 접근
  - 최소 두 단계이며, 각 단계마다 블록 접근이 발생한다
- 결과적으로 오버헤드가 이득을 넘어설 수 있다
  - 조건에 1건만 걸리더라도 그냥 다 읽는 편이 더 쌀 수 있다
  - 조건에 많은 행이 걸리면 인덱스 쪽이 불리해지는 폭은 더 커진다
- 결론 — 작은 테이블에는 인덱스를 무조건 추가할 이유가 없다

5.4

작은 테이블 vs 큰 테이블

- 두 개의 Full Scan 쿼리
- 테이블 크기만 바꾼 통제된 비교
- 조건에 맞는 인덱스가 없는 상황

예제 쿼리 — 두 개의 Full Scan

조건에 맞는 인덱스가 없어 Full Scan만 가능한 두 상황을 비교한다

```
-- A. 작은 테이블(HOSPITALS, 1,000건)
SELECT COUNT(*) FROM hospitals WHERE hosp_type = '의원';

-- B. 큰 테이블(CLAIM_DETAILS, 약 90만 건, 비인덱스 컬럼)
SELECT COUNT(*) FROM claim_details WHERE claim_id = '20240615-0013440';

-- 두 쿼리 모두 조건 컬럼에 쓸 수 있는 인덱스가 없다
-- 따라서 접근 방식은 양쪽 다 TABLE ACCESS FULL로 고정된다
-- 달라지는 변수는 오직 하나 – 테이블 전체의 크기
```

비교 설계 — 무엇을 통제했는가

- 통제한 것 — 접근 방식
  - 두 쿼리 모두 조건에 맞는 인덱스가 없어 Full Scan만 가능한 상황
  - 접근 방식이 같아야 크기의 효과만 남는다
- 바꾼 것 — 테이블의 크기
  - HOSPITALS는 1,000건, CLAIM_DETAILS는 약 90만 건
- 일부러 반대로 놓은 것 — 결과 건수
  - 작은 테이블 쪽이 오히려 조건에 800건이나 걸린다
  - 큰 테이블 쪽은 조건에 3건만 걸린다
- 직관대로라면 큰 테이블 쪽이 훨씬 저렴해야 한다
  - 실측 결과가 그 직관을 뒤집는지 다음 슬라이드에서 확인한다

5.5

실행계획 실측 비교

- A. HOSPITALS — 800건이 걸리는 Full Scan
- B. CLAIM_DETAILS — 3건이 걸리는 Full Scan
- Buffers가 말해 주는 것

A. HOSPITALS 실행계획

1,000건짜리 테이블에서 조건에 800건이 걸린 경우

```
| Id | Operation         | Name      | E-Rows | A-Rows |   A-Time    | Buffers |
| 2  |  TABLE ACCESS FULL | HOSPITALS |    250 |    800 |00:00:00.04 |      15 |

-- A-Rows 800 — 조건에 걸린 실제 행 수
-- E-Rows 250 — 옵티마이저가 통계로 추정한 값, 실제보다 작게 잡혔다
-- Buffers 15 — 테이블 전체를 읽고도 접근한 블록 수가 이 정도에 그친다
-- 테이블 자체가 작기 때문에 전부 읽는 비용의 절대값이 크지 않다
```

B. CLAIM_DETAILS 실행계획

약 90만 건짜리 테이블에서 조건에 단 3건이 걸린 경우

```
| Id | Operation         | Name          | E-Rows | A-Rows |   A-Time    | Buffers |
| 2  |  TABLE ACCESS FULL | CLAIM_DETAILS |      3 |      3 |00:00:00.05 |    6621 |

-- A-Rows 3 — 조건에 걸린 실제 행은 단 3건뿐이다
-- E-Rows 3 — 이번에는 추정과 실제가 정확히 일치했다
-- Buffers 6621 — 결과가 3건인데도 접근한 블록 수는 이만큼이다
-- 조건에 맞는 인덱스가 없어 90만 건이 담긴 블록을 전부 읽어야 했다
```

두 실행계획을 나란히 읽기

- 오퍼레이션은 동일하다 — 양쪽 다 TABLE ACCESS FULL
  - 접근 방식이 같으므로 차이의 원인은 접근 방식이 아니다
- 결과 건수는 작은 테이블 쪽이 훨씬 많다
  - HOSPITALS는 800건, CLAIM_DETAILS는 3건
- 그런데 Buffers는 정반대다
  - HOSPITALS는 15, CLAIM_DETAILS는 6,621
  - 결과가 훨씬 적은 쪽이 블록 접근은 오히려 압도적으로 많다
- 따라서 Buffers를 결정한 변수는 결과 건수가 아니다
  - 남는 변수는 하나 — 테이블 전체의 크기
  - A-Time만 보면 차이가 작아 보이지만 자원 소비는 전혀 다르다

결론 — Full Scan 비용을 결정하는 것

- Full Scan의 비용은 결과 건수가 아니라 테이블 크기로 결정된다
  - CLAIM_DETAILS의 3건짜리 결과에 Buffers 6,621이 든 것이 그 증거
  - HOSPITALS는 800건이 걸렸는데도 Buffers 15에 그쳤다
- 조건은 읽는 양을 줄여 주지 않는다
  - 조건 검사는 블록을 다 읽어 온 뒤에 적용되는 필터일 뿐이다
- 실무에서 이 원칙을 쓰는 방법
  - 결과가 몇 건인지가 아니라 어느 테이블을 통째로 읽는지를 먼저 본다
  - 큰 테이블의 Full Scan이 계획에 보이면 그것만으로 점검 대상이 된다
  - 반대로 작은 테이블의 Full Scan은 대개 문제 삼을 필요가 없다

5.6

선택도에 따른 접근 방식 전환

- PK_MEDICAL_CLAIMS 범위 조회
- 범위를 넓혀 가며 관찰한 실측 표
- Index-Only가 만드는 예외

관찰용 쿼리 — 범위 조회

PK_MEDICAL_CLAIMS(CLAIM_ID 기준)를 이용해 범위를 점점 넓혀 가며 관찰한다

```
SELECT SUM(total_amt) FROM medical_claims
WHERE claim_id BETWEEN '20240101-0000000' AND :end_id;

-- :end_id만 바꿔 가며 대상 범위를 약 1일 → 1개월 → 6개월 → 11개월로 넓힌다
-- 인덱스는 새로 만들지 않는다 — 기존 PK_MEDICAL_CLAIMS만 사용
-- SELECT 절의 TOTAL_AMT는 PK 인덱스에 들어 있지 않은 컬럼이다
-- 따라서 인덱스로 찾은 행마다 테이블까지 가서 값을 읽어야 한다
```

실측 표 — 범위별 접근 방식

MEDICAL_CLAIMS 전체 30만 건 중 대상 건수와 선택도, 그때의 접근 방식

범위	대상 건수	선택도	Operation	Buffers
약 1일	838건	0.3%	TABLE ACCESS BY INDEX ROWID BATCHED + INDEX RANGE SCAN	731
약 1개월	25,406건	8.5%	TABLE ACCESS FULL	3,090
약 6개월	148,028건	49%	TABLE ACCESS FULL	3,090
약 11개월	274,000여건	91%	TABLE ACCESS FULL	3,090

-- 838건에서는 인덱스를 썼고 25,406건부터는 Full Scan으로 전환됐다
-- 전환 이후로는 대상이 몇 배로 늘어도 Buffers가 3,090으로 고정이다

표 읽기 ① 전환이 일어난 구간

- 838건(0.3%) — 인덱스를 사용했다
  - TABLE ACCESS BY INDEX ROWID BATCHED + INDEX RANGE SCAN
  - Buffers 731 — 필요한 블록만 골라 읽었다
- 25,406건(8.5%) — Full Scan으로 전환됐다
  - Buffers 3,090 — 테이블 전체를 읽는 비용으로 바뀌었다
- 두 조회에서 사용 가능한 인덱스는 동일했다
  - 인덱스가 사라진 것이 아니라 옵티마이저가 쓰지 않기로 판단한 것
  - 판단의 근거는 통계정보로 계산한 양쪽 비용의 비교
- 이 사이 어딘가에 교차점이 있다
  - 정확한 경계는 데이터와 통계에 따라 달라진다

표 읽기 ② Buffers가 고정되는 이유

- 25,406건, 148,028건, 274,000여건 — 세 경우 모두 Buffers 3,090
  - 대상 건수는 10배 넘게 차이 나는데 블록 접근은 완전히 같다
- 4.5절에서 확인한 원칙이 그대로 재현된 것이다
  - Full Scan은 테이블의 모든 블록을 읽는 작업이다
  - 조건에 몇 건이 걸리는지는 다 읽은 뒤의 문제일 뿐이다
- 두 축이 여기서 만난다
  - 4.5절은 테이블 크기가 다를 때의 비교였다
  - 4.6절은 같은 테이블에서 선택도만 바뀔 때의 비교다
  - 서로 다른 실험이 같은 원칙을 가리키므로 결론의 신뢰도가 올라간다

Index-Only — 예외가 생기는 지점

- 같은 범위 조건인데 결과가 달라지는 경우가 있다
  - COUNT(*)만 필요한 경우 — 테이블 컬럼 값을 읽을 필요가 없다
  - 이때는 148,028건(49%)에서도 인덱스만으로 처리되어 Buffers 910
- 왜 이런 차이가 생기는가
  - 인덱스에 이미 있는 정보만으로 답이 나오면 테이블에 갈 이유가 없다
  - 비용의 큰 몫을 차지하던 랜덤 접근 단계가 통째로 사라진다
- 결과적으로 교차점이 오른쪽으로 크게 밀린다
  - 선택도가 높아도 인덱스가 계속 유리할 수 있다는 뜻

참고:
COUNT(*)처럼 테이블 컬럼 접근이 필요 없는 쿼리는 838건은 물론 148,028건(49%)에서도 인덱스만으로 처리되어 Buffers가 910에 그쳤다.

SELECT 절이 경계를 옮긴다

- 같은 범위, 같은 인덱스, 다른 SELECT 절
  - COUNT(*) — 49%에서도 인덱스 유지, Buffers 910
  - SUM(total_amt) — 이미 8.5%에서 Full Scan으로 전환, Buffers 3,090
- 차이의 원인은 컬럼이 인덱스 안에 있는지 여부
  - PK_MEDICAL_CLAIMS에는 CLAIM_ID만 들어 있고 TOTAL_AMT는 없다
  - COUNT(*)는 행의 존재만 확인하면 되므로 인덱스로 완결된다
  - SUM(total_amt)은 값을 읽으러 테이블까지 가야 한다
- 따라서 조회 컬럼이 바뀌면 최적 접근 방식의 경계도 바뀐다
  - 선택도 하나만 보고 접근 방식을 예측하면 틀릴 수 있다

5.7~5.8

종합 — 판단 기준 만들기

- 선택도별 접근 방식 요약표
- 절대적인 경계 숫자가 없는 이유
- 원칙 적용 훈련과 다음 장 예고

Benchmark — 선택도별 접근 방식 요약

앞의 두 실측을 하나의 표로 정리한 것 — 규칙이 아니라 관찰의 요약이다

```
| 선택도 | 접근 방식 경향 | 이번 챕터 근거 |
|---|---|---|
| 매우 낮음(0.3% 수준) | Index Range Scan 유리 | 838건, Buffers 731 |
| 중간(8% 이상) | Full Scan으로 전환 | 25,406건부터 Buffers 3,090 |
| Index-Only 처리 가능 | 49%에서도 인덱스 유지 | COUNT(*)류, Buffers 910 |
| 테이블 자체가 작음 | 선택도 무관하게 Full Scan | HOSPITALS, Buffers 15 |

-- 위 두 행은 선택도 축, 아래 두 행은 그 축을 벗어나는 변수다
-- Index-Only 여부와 테이블 크기는 선택도와 독립적으로 작용한다
```

'몇 %부터'라는 숫자가 없는 이유

- 교차점의 위치는 상황마다 다르다
  - 테이블과 인덱스의 크기 — 전체 블록 수가 다르면 기준선이 달라진다
  - 클러스터링 팩터 — 랜덤 접근이 얼마나 흩어지는지에 따라 곡선이 달라진다
  - Index-Only 여부 — 테이블 접근 단계가 있는지 없는지
- 같은 테이블, 같은 인덱스에서도 달라진다
  - SELECT 절이 바뀌면 경계가 함께 움직인다 — 4.6절에서 확인
- 그래서 옵티마이저는 규칙이 아니라 계산으로 판단한다
  - 매번 통계정보를 근거로 양쪽 비용을 산출해 비교한다
  - 우리가 할 일은 그 계산이 맞는지 실행계획으로 검증하는 것

Full Scan이 유리한 상황 — 스스로 정의하기

- 조건 1 — 테이블 크기 관점
  - 전체 블록 수가 작아 통째로 읽는 비용의 절대값이 크지 않을 때
  - 인덱스를 거치는 추가 단계의 오버헤드가 그 이득보다 클 때
- 조건 2 — 선택도 관점
  - 조건을 만족하는 행의 비율이 높아 어차피 대부분을 읽어야 할 때
  - 랜덤 접근이 누적되는 비용이 순차 읽기 비용을 넘어설 때
- 예외 조건 — 필요한 컬럼이 인덱스 안에 있는가
  - 조건 2를 만족해도 Index-Only면 여전히 인덱스가 유리할 수 있다
  - 세 변수는 독립적이지 않고 함께 고려해야 한다

> **참고:**
> 조건 2를 만족하더라도 쿼리가 인덱스에 포함된 컬럼만 필요로 한다면 여전히 인덱스가 유리할 수 있다 — 두 조건은 독립적으로 작용하지 않는다.

원칙 적용 훈련 — REVIEW_LOG 사례

- 상황 — REVIEW_LOG에서 REVIEWER_ID = 'SYSTEM'인 행을 찾는다
  - REVIEW_LOG는 50만 건 — HOSPITALS보다 훨씬 크고 CLAIM_DETAILS와 비슷한 규모
  - 해당 값이 전체의 약 40%로 추정된다
  - 해당 컬럼에 인덱스는 없는 상태다
- 이 장의 원칙을 적용한 예측
  - 크기 축 — 작은 테이블이 아니므로 크기만으로는 Full Scan을 정당화할 수 없다
  - 선택도 축 — 약 40%는 이번 챕터에서 전환이 관찰된 구간을 크게 웃돈다
- 결론 — TABLE ACCESS FULL로 처리될 것으로 예상하는 것이 합리적이다
  - 설령 인덱스가 있었더라도 같은 판단이 나올 가능성이 크다

ch06 예고 — 인덱스 설계로

- 이번 장에서 다룬 것 — 이미 있는 인덱스를 쓸지 말지의 판단
  - 인덱스를 새로 만들지 않고 접근 방식의 선택만 관찰했다
- 다음 장에서 다룰 것 — 인덱스를 어떻게 만들 것인가
  - 선택도를 인덱스 설계에 어떻게 반영하는지
  - 컬럼 순서와 결합 인덱스의 판단 기준
- 이번 장의 개념이 다음 장의 전제가 된다
  - 선택도를 모르면 어떤 컬럼을 인덱스 선두에 둘지 판단할 수 없다
  - Index-Only의 이점을 모르면 커버링 설계를 떠올릴 수 없다
  - 클러스터링 팩터를 모르면 인덱스가 기대만큼 안 빠른 이유를 설명할 수 없다

요약 (Summary)

- Full Scan 비용은 결과 건수가 아니라 테이블 크기에 비례한다
- Index Range Scan 비용은 결과 건수와 랜덤 접근에 비례한다
- 두 비용 곡선이 교차하는 지점이 있고 옵티마이저는 그것을 계산으로 추정한다
- 선택도가 낮으면 인덱스, 높으면 Full Scan이 유리한 경향이 있다
- 인덱스가 있어도 옵티마이저가 쓰지 않기로 판단할 수 있다
- Index-Only 쿼리는 선택도가 높아도 인덱스가 계속 유리할 수 있다
- 작은 테이블은 인덱스 유무와 무관하게 Full Scan이 저렴한 경우가 많다
- 정확한 교차점은 데이터·인덱스·쿼리 특성마다 다르므로 실행계획으로 확인한다

실습 안내

- 실습문제 10문항 — ch05_문제.md
  - 1번 결과가 적은데 Buffers가 큰 이유 — Full Scan 비용 구조
  - 2번 인덱스가 있어도 쓰지 않는 경우의 설명
  - 3번 건수가 달라도 Buffers가 동일한 이유
  - 4·6번 COUNT(*)와 SUM(total_amt)의 차이, 선택도 단일 규칙에 대한 반박
  - 5번 선택도와 카디널리티의 정의 및 사례별 계산
  - 7번 이미 저렴한 작은 테이블에 인덱스를 추가하는 것의 손익
  - 8번 교차점을 하나의 숫자로 단정할 수 없는 이유
  - 9·10번 REVIEW_LOG 사례 예측과 Full Scan 유리 조건의 정의
- 실습 전 확인 — 이번 장은 인덱스를 신규 생성하지 않는다
  - 인덱스나 통계를 임의로 바꾸면 교안의 실행계획이 재현되지 않는다

Oracle SQL 튜닝 과정 · ch06

인덱스 설계

B-Tree 인덱스 · 복합 인덱스 컬럼 순서 · 선두 컬럼 · RANGE SCAN vs FAST FULL SCAN | Oracle Database 19c · 진료비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - B-Tree 인덱스가 컬럼 값을 정렬해 저장하는 구조라는 점을 설명한다
  - 단일 컬럼 인덱스와 복합 인덱스의 차이를 구분한다
  - 복합 인덱스의 선두 컬럼이 무엇인지 정의하고 그 역할을 설명한다
  - 선두 컬럼이 조건에 포함될 때 INDEX RANGE SCAN이 가능한 이유를 설명한다
  - 후행 컬럼만 조건일 때 INDEX FAST FULL SCAN으로 전환되는 이유를 설명한다
  - 같은 두 컬럼이라도 순서에 따라 Buffers가 뒤바뀌는 실측 결과를 읽는다
  - INDEX FAST FULL SCAN이 TABLE ACCESS FULL보다 저렴한 근거를 든다
  - 인덱스가 조회를 돕는 대신 DML 비용을 늘리는 트레이드오프를 설명한다
  - 조회 패턴을 근거로 복합 인덱스의 컬럼 순서를 결정하는 절차를 정리한다

이 장의 구성 (Agenda)

- 5.1 Why — ch05가 남긴 질문 · 잘못된 인덱스의 대가
- 5.2 Concept — B-Tree · 단일 컬럼 인덱스 · 복합 인덱스 · 선두 컬럼
- 5.3 Oracle Internals — 컬럼 순서가 중요한 이유 · 인덱스의 비용
- 5.4 Example — 인덱스 A (HOSP_ID, RECEIPT_DATE)와 세 가지 조건
- 5.5 Execution Plan — 인덱스 A 실측과 해석
- 5.6 Bad SQL — 컬럼 순서를 반대로 한 인덱스 B 실측
- 5.7 Tuning — 이번 스키마의 조회 패턴에 적용하기
- 5.8 Benchmark — 종합 비교표와 순서 결정 절차
- 5.9~5.10 실습 안내 · 요약 (Review)

6.1

인덱스 설계를 왜 배우는가

- ch05가 남긴 질문
- 잘못 설계된 인덱스의 대가
- 이번 차시의 목표와 진행 방식

ch05가 남긴 질문

- ch05에서 확인한 것 — 인덱스가 유리한 상황이 따로 있다
  - 선택도가 낮아 소수의 행만 걸러 내는 조건에서 인덱스가 효과적이었다
  - 반대로 넓은 범위를 읽어야 하면 통째로 읽는 편이 나을 수 있었다
- 그러나 아직 답하지 않은 질문이 남아 있다
  - 인덱스를 만들기로 했다면 어떤 컬럼을 고를 것인가
  - 여러 컬럼을 묶는다면 어떤 순서로 배치할 것인가
- 이번 차시의 목표는 그 순서가 실행계획에 미치는 영향을 실측으로 확인하는 것
  - 개념 설명이 아니라 같은 두 컬럼을 순서만 바꿔 만든 두 인덱스를 직접 비교한다

잘못 설계된 인덱스의 대가

- 잘못 설계된 인덱스는 있어도 안 쓰인다
  - 조회 조건이 선두 컬럼과 맞지 않으면 옵티마이저가 그 인덱스를 고르지 않거나
  - 고르더라도 전체를 훑는 값비싼 방식으로 처리하게 된다
- 쓰이지 않는 인덱스도 비용은 그대로 낸다
  - 저장 공간을 계속 차지하고, DML이 일어날 때마다 함께 갱신된다
  - 조회에는 도움을 못 주면서 입력·수정 성능만 갉아먹는 최악의 조합이 된다
- 그래서 인덱스 설계는 '만들까 말까'가 아니라 '어떻게 만들까'의 문제다
  - 개수를 늘려 해결하기 전에 컬럼과 순서를 먼저 맞추는 것이 순서다

이번 차시의 진행 방식

- 소재 — MEDICAL_CLAIMS의 두 컬럼으로 인덱스를 만들어 비교한다
  - 인덱스 A : (HOSP_ID, RECEIPT_DATE) 순서
  - 인덱스 B : (RECEIPT_DATE, HOSP_ID) — 컬럼 순서만 반대로 한 것
- 동일한 세 가지 조회 조건을 두 인덱스에서 각각 실행해 실행계획을 비교한다
  - 두 컬럼을 모두 조건으로 거는 경우
  - 앞쪽 컬럼 하나만 조건으로 거는 경우
  - 뒤쪽 컬럼 하나만 조건으로 거는 경우
- 본 차시는 실습용 인덱스를 생성·삭제한다 (IX_CH05_ 접두어)

6.2

인덱스의 기본 구조와 용어

- B-Tree 인덱스란
- 단일 컬럼 인덱스와 복합 인덱스
- 선두 컬럼의 정의
- 정렬 구조 개념도

B-Tree 인덱스란

- 컬럼 값을 정렬된 트리 구조로 저장하는 인덱스
  - 덕분에 특정 값이나 특정 범위를 빠르게 찾아갈 수 있다
  - Oracle에서 별도 지정 없이 CREATE INDEX를 하면 만들어지는 기본 형태다
- 핵심 성질은 '정렬되어 있다'는 것
  - 정렬되어 있으므로 찾는 값이 어디쯤 있는지 바로 짚어 갈 수 있다
  - 정렬 기준과 맞지 않는 조건이면 이 장점을 쓸 수 없다 — 5.3절의 주제
- 인덱스는 컬럼 값과 행의 위치(ROWID)만 담는다
  - 테이블처럼 모든 컬럼을 담지 않으므로 구조물 자체가 테이블보다 작다

단일 컬럼 인덱스와 복합 인덱스

- 단일 컬럼 인덱스 — 컬럼 하나만으로 구성한 인덱스
  - 그 컬럼을 조건으로 쓰는 조회에만 대응한다
- 복합 인덱스 — 여러 컬럼을 정해진 순서로 묶어 하나로 구성한 인덱스
  - 묶는 순서가 인덱스의 성격을 결정한다
  - (HOSP_ID, RECEIPT_DATE)와 (RECEIPT_DATE, HOSP_ID)는 서로 다른 인덱스다
- 같은 컬럼 집합이라도 순서가 다르면 대응하는 조회 패턴이 달라진다
  - 이것이 이 장에서 실측으로 확인할 핵심 명제다
  - 컬럼을 무엇으로 고를지보다 어떤 순서로 둘지가 더 자주 실수가 나는 지점이다

선두 컬럼 (Leading Column)

- 복합 인덱스에서 맨 앞에 오는 컬럼
  - 인덱스가 정렬되는 1차 기준이 되는 컬럼이다
  - (HOSP_ID, RECEIPT_DATE)의 선두 컬럼은 HOSP_ID다
- 선두 컬럼은 컬럼 이름에 붙는 고정 속성이 아니다
  - 인덱스 정의에서 몇 번째에 오는가로 결정된다
  - 순서를 바꾸면 같은 컬럼이 선두가 되기도 하고 후행이 되기도 한다
- 선두 컬럼이 아닌 나머지 컬럼을 이 장에서는 후행 컬럼이라 부른다
  - 후행 컬럼은 같은 선두 컬럼 값 안에서만 정렬되어 있다

복합 인덱스의 정렬 구조 — 개념도

선두 컬럼이 먼저 정렬되고, 후행 컬럼은 같은 선두 값 안에서만 정렬된다

```
-- 복합 인덱스 (선두 컬럼, 후행 컬럼) 이 저장되는 순서

   [ 선두 컬럼 = A ]  ->  그 구간 안에서 후행 컬럼 오름차순
   [ 선두 컬럼 = B ]  ->  그 구간 안에서 후행 컬럼 오름차순
   [ 선두 컬럼 = C ]  ->  그 구간 안에서 후행 컬럼 오름차순

-- 선두 컬럼이 조건에 있으면
--    [B] 구간의 시작 위치로 바로 점프할 수 있다   ->  INDEX RANGE SCAN

-- 후행 컬럼만 조건에 있으면
--    찾는 값이 A / B / C 구간에 흩어져 있다
--    시작 위치를 짚을 수 없어 인덱스 전체를 훑어야 한다

-- 전화번호부 비유
--    성   = 선두 컬럼  ->  성을 알면 해당 페이지로 바로 넘긴다
--    이름 = 후행 컬럼  ->  이름만 알면 첫 장부터 넘겨야 한다
```

6.3

컬럼 순서가 중요한 이유

- 복합 인덱스의 정렬 규칙
- 선두 컬럼이 조건에 있을 때
- 선두 컬럼이 빠졌을 때
- 인덱스의 장점과 비용

복합 인덱스의 정렬 규칙

- 선두 컬럼 값을 기준으로 먼저 정렬된다
  - 인덱스 전체가 선두 컬럼 값의 오름차순으로 늘어서 있다
- 그 안에서 두 번째 컬럼 값이 정렬된다
  - 같은 선두 컬럼 값을 가진 구간 내부에서만 후행 컬럼이 정렬되어 있다
  - 구간이 달라지면 후행 컬럼의 정렬은 다시 처음부터 시작한다
- 전화번호부와 같은 구조
  - '성'으로 먼저 정렬되고 그 안에서 '이름'으로 정렬되는 것과 동일하다
  - 따라서 후행 컬럼 값 하나는 인덱스 여러 구간에 흩어져 존재한다

선두 컬럼이 조건에 있을 때

- 조건에 선두 컬럼이 포함되어 있으면
  - 인덱스에서 해당 값(또는 범위)의 시작 위치와 끝 위치를 바로 찾아갈 수 있다
  - 그 구간만 순서대로 읽으면 되므로 읽는 양이 결과 건수에 비례한다
- 실행계획에는 INDEX RANGE SCAN으로 나타난다
  - Range, 즉 정렬된 구간 하나를 훑는다는 뜻의 이름이다
- 선두 컬럼과 후행 컬럼을 함께 조건으로 걸면 범위가 더 좁아진다
  - 선두 컬럼으로 구간을 잡고 그 안에서 후행 컬럼 조건까지 인덱스에서 적용한다
  - 이 경우가 복합 인덱스를 가장 온전히 활용하는 형태다

선두 컬럼이 빠졌을 때

- 조건에 선두 컬럼이 없고 후행 컬럼만 있으면
  - 인덱스가 그 컬럼 기준으로는 정렬되어 있지 않다
  - 따라서 시작 위치를 바로 찾아갈 수 없고 구간을 잡을 수 없다
- 옵티마이저가 선택할 수 있는 대안은 두 가지다
  - 인덱스 전체를 순서 없이 다 훑는 INDEX FAST FULL SCAN — 인덱스가 작을 때
  - 아예 그 인덱스를 쓰지 않고 다른 접근 경로를 고르는 것
- 어느 쪽이든 '인덱스를 만들었으니 빠를 것'이라는 기대는 성립하지 않는다
  - 인덱스는 존재만으로 성능을 보장하지 않는다 — 조건과 순서가 맞아야 한다

인덱스의 장점과 비용

- 장점 — 조건에 맞는 소수의 행을 빠르게 찾아낸다
  - 선택도가 낮을 때 효과적이라는 점은 ch05에서 확인했다
- 비용 ① 저장 공간
  - 인덱스는 별도의 구조물이므로 디스크 공간을 추가로 차지한다
- 비용 ② DML 부담
  - 원본 테이블에 INSERT·UPDATE·DELETE가 일어날 때마다 인덱스도 함께 갱신된다
  - 조회를 위해 만든 구조물이 입력·수정 경로에 작업을 하나 더 얹는 셈이다

> **참고:**
> 이 차시에서 DML 비용은 실측하지 않고 원리로만 다룬다 — 교안 03절 기준.

인덱스를 너무 많이 만들면

- 테이블 하나에 인덱스가 여러 개면
  - DML 한 번에 그 인덱스 개수만큼 추가 갱신이 발생한다
  - 인덱스를 늘릴수록 입력·수정 경로가 그만큼 무거워진다
- 조회 성능과 입력·수정 성능은 트레이드오프 관계에 있다
  - 조회를 위해 만든 인덱스가 입력·수정 성능을 갉아먹는 구조다
  - 어느 쪽을 얼마나 양보할지는 시스템의 성격에 따라 달라진다
- 그래서 '순서를 못 정하겠으면 두 개 만들면 된다'는 손쉬운 결론을 경계한다
  - 개수를 늘리기 전에 컬럼 순서로 해결되는지를 먼저 따져야 한다

> **참고:**
> 인덱스 개수를 늘리는 결정은 이 테이블의 DML 빈도와 시점을 확인한 뒤에 내린다.

6.4

실습 소재와 세 가지 테스트 조건

- 소재 설계 — 무엇을 고정하고 무엇을 바꾸는가
- 인덱스 A 생성과 세 조건 SQL
- 세 조건이 각각 검증하는 것

소재 설계 — 무엇을 고정하고 무엇을 바꾸는가

- 대상 테이블 — MEDICAL_CLAIMS
  - 실무 조회에서 병원과 접수일자가 함께 쓰이는 대표적인 조합이다
- 고정하는 것
  - 인덱스를 구성하는 컬럼 집합 — HOSP_ID와 RECEIPT_DATE 두 개
  - 테스트에 사용하는 세 가지 조회 조건
- 바꾸는 것 — 오직 컬럼 순서 하나
  - 인덱스 A는 (HOSP_ID, RECEIPT_DATE), 인덱스 B는 (RECEIPT_DATE, HOSP_ID)
  - 변수가 하나뿐이므로 결과 차이의 원인을 순서로 특정할 수 있다

인덱스 A 생성과 세 가지 조건

교안 04절 — 인덱스 A를 만들고 동일한 세 조건으로 테스트한다

```
-- 인덱스 A: (HOSP_ID, RECEIPT_DATE) 순서
CREATE INDEX ix_hosp_receipt ON medical_claims(hosp_id, receipt_date);

-- 세 가지 조건으로 테스트
SELECT COUNT(*) FROM medical_claims
 WHERE hosp_id = 1 AND receipt_date = DATE'2024-06-15';   -- 둘 다

SELECT COUNT(*) FROM medical_claims
 WHERE hosp_id = 1;                                       -- 선두만

SELECT COUNT(*) FROM medical_claims
 WHERE receipt_date = DATE'2024-06-15';                   -- 후행만

-- COUNT(*) 로 만든 이유: 테이블 접근 여부가 아니라
-- 인덱스를 어떤 오퍼레이션으로 읽었는지에 집중하기 위해서다
```

세 조건이 각각 검증하는 것

- 조건 ① 두 컬럼을 모두 사용 — 선두 컬럼 포함
  - 복합 인덱스를 가장 온전히 쓰는 형태가 실제로 가장 저렴한지 확인한다
- 조건 ② 앞쪽 컬럼 하나만 사용
  - 인덱스 A에서는 선두 컬럼 매치 상황이 된다
  - 인덱스 B에서는 같은 조건이 후행 컬럼 상황으로 바뀐다
- 조건 ③ 뒤쪽 컬럼 하나만 사용
  - 인덱스 A에서는 후행 컬럼만 매치되는 상황이 된다
  - 인덱스 B에서는 같은 조건이 선두 컬럼 매치로 바뀐다
- ②와 ③이 인덱스 A·B에서 정확히 뒤집히는지가 이 실험의 관전 포인트

6.5

인덱스 A 실행계획 실측

- 세 조건의 실측 결과표
- 해석 ① 선두 컬럼이 포함된 경우
- 해석 ② 후행 컬럼만 조건인 경우
- Fast Full Scan과 Full Table Scan의 비교

인덱스 A (HOSP_ID, RECEIPT_DATE) 실측

교안 05절 Execution Plan — 동일한 세 조건의 실행계획 실측 결과

```
+---------------------------------+----------------------+---------+---------+
| 조건                            | Operation            | Buffers | A-Rows |
+---------------------------------+----------------------+---------+---------+
| HOSP_ID=1 AND RECEIPT_DATE=..  | INDEX RANGE SCAN     |       3 |     11 |
| HOSP_ID=1          (선두 컬럼) | INDEX RANGE SCAN     |      12 |  3,058 |
| RECEIPT_DATE=..    (후행 컬럼) | INDEX FAST FULL SCAN |     953 |    789 |
+---------------------------------+----------------------+---------+---------+

-- 인덱스 A = ix_hosp_receipt (HOSP_ID, RECEIPT_DATE)
-- 선두 컬럼이 조건에 포함된 위 두 행만 INDEX RANGE SCAN 으로 처리됐다
-- 세 번째 행만 오퍼레이션이 다르고 Buffers 가 크게 뛰었다
```

해석 ① 선두 컬럼이 포함된 두 경우

- 두 컬럼을 모두 조건으로 건 경우 — INDEX RANGE SCAN, Buffers 3
  - 선두 컬럼으로 구간을 잡고 그 안에서 후행 컬럼 조건까지 인덱스에서 적용했다
  - 결과가 11건으로 매우 적고, 읽은 블록도 그만큼 적다
- 선두 컬럼만 조건으로 건 경우 — INDEX RANGE SCAN, Buffers 12
  - 구간의 시작과 끝을 짚을 수 있으므로 그 구간만 순서대로 읽었다
  - 결과가 3,058건으로 늘었지만 읽은 블록은 12에 그쳤다
- 공통점 — 조건에 선두 컬럼이 있으면 읽는 양이 결과 건수에 비례한다
  - 필요한 만큼만 읽는 것이 인덱스를 제대로 쓰는 모습이다

해석 ② 후행 컬럼만 조건일 때

- RECEIPT_DATE만 조건 — INDEX FAST FULL SCAN, Buffers 953
  - 인덱스가 RECEIPT_DATE 기준으로는 정렬되어 있지 않다
  - 시작 위치를 짚을 수 없어 인덱스 전체를 순서 없이 훑었다
- 결과는 789건인데 Buffers는 953까지 치솟았다
  - 읽은 양이 결과 건수가 아니라 인덱스 전체 크기에 비례했다
  - 조건에 맞지 않는 행의 엔트리까지 전부 읽고 걸러 낸 것이다
- 같은 인덱스인데 조건에 따라 오퍼레이션이 달라진 이유가 바로 정렬 기준이다
  - 인덱스에 값이 없어서가 아니라, 값이 흩어져 있어서 생긴 결과다

Fast Full Scan이 Full Table Scan보다 저렴한 이유

- 관찰 — Buffers 953은 비싸 보이지만 Full Table Scan보다는 적다
  - ch05 기준 TABLE ACCESS FULL의 Buffers는 3,090이었다
- 이유는 인덱스가 테이블보다 작은 구조물이기 때문이다
  - 인덱스는 해당 컬럼 값과 ROWID만 저장한다
  - 테이블은 모든 컬럼의 전체 데이터를 저장한다
  - 같은 행 수라도 인덱스 엔트리 하나가 테이블 행 전체보다 훨씬 작다
- 정리 — 제대로 된 Range Scan만큼 저렴하지는 않지만
  - 인덱스를 훑는 편이 테이블 전체를 읽는 것보다는 여전히 유리하다

6.6

컬럼 순서를 반대로 하면

- 인덱스 B 생성과 재실행
- 인덱스 B 실측 결과표
- 정확히 뒤바뀐 결과
- 두 컬럼을 모두 쓸 때는 왜 덜 중요한가

인덱스 B 생성 — 컬럼 순서만 반대로

교안 06절 — 인덱스 A를 삭제하고 순서를 뒤집은 인덱스 B로 동일한 세 조건을 재실행

```
-- 인덱스 A 를 삭제하고
DROP INDEX ix_hosp_receipt;

-- 컬럼 순서만 반대로 한 인덱스 B 를 만든다
CREATE INDEX ix_receipt_hosp ON medical_claims(receipt_date, hosp_id);

-- 조회 조건은 하나도 바꾸지 않고 그대로 재실행한다
SELECT COUNT(*) FROM medical_claims
 WHERE hosp_id = 1 AND receipt_date = DATE'2024-06-15';   -- 둘 다

SELECT COUNT(*) FROM medical_claims
 WHERE hosp_id = 1;                        -- 이번엔 후행 컬럼이 됐다

SELECT COUNT(*) FROM medical_claims
 WHERE receipt_date = DATE'2024-06-15';    -- 이번엔 선두 컬럼이 됐다
```

인덱스 B (RECEIPT_DATE, HOSP_ID) 실측

교안 06절 Bad SQL — 동일한 세 조건, 인덱스 컬럼 순서만 반대

```
+---------------------------------+----------------------+---------+---------+
| 조건                            | Operation            | Buffers | A-Rows  |
+---------------------------------+----------------------+---------+---------+
| HOSP_ID=1 AND RECEIPT_DATE=..   | INDEX RANGE SCAN     |       3 |      11 |
| HOSP_ID=1          (후행 컬럼) | INDEX FAST FULL SCAN |     954 |   3,058 |
| RECEIPT_DATE=..    (선두 컬럼) | INDEX RANGE SCAN     |       5 |     789 |
+---------------------------------+----------------------+---------+---------+

-- 인덱스 B = ix_receipt_hosp (RECEIPT_DATE, HOSP_ID)
-- 첫 행은 인덱스 A 와 사실상 동일하다 (Buffers 3, A-Rows 11)
-- 두 번째와 세 번째 행의 오퍼레이션이 인덱스 A 와 정확히 맞바뀌었다
```

정확히 뒤바뀐 결과

- HOSP_ID만 조건일 때
  - 인덱스 A에서는 선두 컬럼이라 저렴했다 — Buffers 12
  - 인덱스 B에서는 후행 컬럼이 되어 비싸졌다 — Buffers 954
- RECEIPT_DATE만 조건일 때
  - 인덱스 A에서는 후행 컬럼이라 비쌌다 — Buffers 953
  - 인덱스 B에서는 선두 컬럼이 되어 저렴해졌다 — Buffers 5
- 선두 컬럼은 컬럼 이름에 고정된 속성이 아니라는 증거
  - 인덱스 정의에서 몇 번째에 오는가가 그 컬럼의 운명을 결정한다
  - SQL은 한 글자도 바뀌지 않았고 데이터도 그대로다

두 컬럼을 모두 쓸 때는 왜 덜 중요한가

- 두 조건을 모두 건 첫 번째 경우는 A와 B가 사실상 같았다 — Buffers 3
  - 어느 쪽이 선두든 두 컬럼 다 조건에 있으면 인덱스를 온전히 활용할 수 있다
  - 선두 컬럼으로 구간을 잡고 그 안에서 나머지 조건까지 인덱스에서 적용한다
- 다만 '완전히 무관하다'는 뜻은 아니다
  - 두 컬럼의 선택도가 크게 다르면 중간 단계에서 좁혀지는 후보 행 수가 달라진다
  - 이번 사례는 결과가 11건으로 매우 적어 차이가 드러나지 않았을 뿐이다
- 정리 — 순서가 중요해지는 것은 '컬럼 하나만 단독으로 쓰일 때'다
  - 항상 두 컬럼을 함께 쓰는 조회라면 순서 결정의 부담이 작아진다

결론 — 선두 컬럼은 조회 패턴이 결정한다

- 어떤 컬럼을 선두에 둘지는 인덱스 자체의 성질로 정해지지 않는다
  - '이 인덱스를 어떤 조건으로 조회할 것인가'가 유일한 기준이다
- 두 컬럼을 항상 같이 조건으로 쓴다면
  - 순서 결정의 중요도가 상대적으로 낮아진다
- 컬럼 하나만 단독으로 자주 조회한다면
  - 그 컬럼이 반드시 선두에 있어야 한다
  - 단독 조회가 잦은 컬럼을 후행에 두면 이번 실측의 954 같은 상황이 된다
- 즉 인덱스 설계는 데이터 구조 설계가 아니라 조회 패턴 설계에 가깝다

6.7~6.8

설계 적용과 종합 비교

- 이번 스키마에 적용한다면
- 두 조회 패턴이 모두 잦다면
- 종합 비교표
- 컬럼 순서 결정 절차

이번 스키마에 적용한다면

- MEDICAL_CLAIMS를 실무에서 조회하는 패턴을 가정해 본다
- '특정 병원의 특정 기간 청구 조회'가 잦다면
  - (HOSP_ID, RECEIPT_DATE)가 적합하다
  - 병원 단독 조회도 선두 컬럼 매치로 저렴하게 처리된다
- '기간 전체 청구 현황 조회(병원 무관)'가 더 잦다면
  - (RECEIPT_DATE, HOSP_ID)가 적합하다
  - 기간 단독 조회가 선두 컬럼 매치가 되어 저렴해진다
- 판단의 근거는 실행 빈도와 응답 속도 요구다
  - 화면 목록과 쿼리 목록을 먼저 정리하는 것이 설계의 출발점이다

두 조회 패턴이 모두 잦다면

- 인덱스 하나로 두 패턴을 모두 완벽히 커버할 수는 없다
  - 어느 쪽을 선두에 두든 나머지 한쪽은 후행 컬럼 상황이 된다
- 선택지 ① 인덱스를 두 개 만든다
  - 두 조회 패턴 모두 각자의 전용 인덱스로 최적 처리된다
  - 대신 인덱스 개수만큼 DML 비용이 늘어난다 — 5.3절의 트레이드오프
- 선택지 ② 하나만 만들고 다른 조회가 다소 느린 것을 감수한다
  - 인덱스 개수는 유지되지만 한쪽 조회 패턴을 희생하게 된다
- 저울질의 기준 — 두 패턴의 상대적 중요도와 이 테이블의 DML 빈도
  - DML이 드물거나 비피크에 몰려 있다면 인덱스 추가를 감수할 만하다

종합 비교표

교안 08절 Benchmark — 인덱스 유무와 컬럼 순서에 따른 Buffers 비교

인덱스	조건	Buffers	비고
없음 (Full Scan)	두 컬럼 모두	3,090	ch03 베이스라인
A (HOSP_ID, RECEIPT_DATE)	HOSP_ID 만	12	선두 컬럼 매치
A (HOSP_ID, RECEIPT_DATE)	RECEIPT_DATE 만	953	후행 컬럼만 매치
B (RECEIPT_DATE, HOSP_ID)	HOSP_ID 만	954	이번엔 후행 컬럼
B (RECEIPT_DATE, HOSP_ID)	RECEIPT_DATE 만	5	이번엔 선두 컬럼

```
-- 같은 두 컬럼, 같은 조건인데 순서만 바꾸면
--    12 <-> 954 ,  953 <-> 5  로 정확히 뒤바뀐다
```

종합 비교표가 말하는 것

- 관찰 ① 순서를 바꾸면 비용이 정확히 맞바뀐다
  - HOSP_ID 단독 조건은 12에서 954로, RECEIPT_DATE 단독 조건은 953에서 5로
  - 인덱스 정의 한 줄이 두 조회 패턴의 운명을 동시에 뒤집었다
- 관찰 ② 후행 컬럼만 매치돼도 인덱스 없는 것보다는 낫다
  - 953·954는 베이스라인 3,090보다 여전히 적다
  - 다만 이 사실이 순서를 아무렇게나 정해도 된다는 뜻은 아니다
- 결론 — 선두 컬럼 선택이 복합 인덱스 설계에서 가장 먼저 결정할 사항이다
  - 컬럼을 무엇으로 고를지는 그다음 문제다

컬럼 순서 결정 절차

- 1단계 — 조회 패턴을 나열한다
  - 이 테이블에 대한 주요 조회들이 어떤 컬럼을 조건으로 쓰는지 정리한다
  - 각 패턴이 얼마나 자주 실행되는지, 응답 속도 요구가 어떤지도 함께 적는다
- 2단계 — 선두 컬럼 후보를 고른다
  - 가장 빈번하거나 가장 중요한 조회에서 단독으로도 자주 쓰이는 컬럼을 고른다
  - '단독으로도'가 핵심 조건이다 — 항상 함께 쓰이는 컬럼은 후보가 아니다
- 3단계 — 나머지 컬럼을 배치하고 남은 패턴을 검토한다
  - 두 컬럼을 함께 쓰는 조회의 빈도를 기준으로 후행 컬럼을 배치한다
  - 커버되지 않는 패턴은 추가 인덱스가 필요한지 트레이드오프와 함께 판단한다

실습 정리 — 원상복구 스크립트

차시 종료 시 실습 인덱스를 삭제하고 통계를 재수집한다 (교안 도입부 안내)

```
-- 이 차시에서 만든 실습용 인덱스를 모두 삭제한다
DROP INDEX ix_hosp_receipt;
DROP INDEX ix_receipt_hosp;

-- 남아 있는 인덱스를 확인한다
SELECT index_name, uniqueness
  FROM user_indexes
 WHERE table_name = 'MEDICAL_CLAIMS';

-- PK_MEDICAL_CLAIMS 외의 인덱스가 남아 있으면 안 된다

-- 통계를 다시 수집해 원래 상태로 되돌린다
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'MEDICAL_CLAIMS');

-- 정리하지 않으면 이후 챕터의 실행계획이 재현되지 않는다
```

요약 (Summary)

- 복합 인덱스는 선두 컬럼을 기준으로 먼저 정렬되고, 그 안에서 후행 컬럼이 정렬된다
- 조건에 선두 컬럼이 포함되어야 INDEX RANGE SCAN으로 저렴하게 처리된다
- 후행 컬럼만으로는 INDEX FAST FULL SCAN 또는 미사용으로 전환되어 비용이 커진다
- 선두 컬럼은 컬럼의 성질이 아니라 인덱스 정의에서의 위치로 결정된다
- 순서만 뒤집으면 Buffers가 12↔954, 953↔5로 정확히 맞바뀐다
- 두 컬럼을 모두 조건으로 쓰면 순서의 영향이 작아진다 (Buffers 3)
- Fast Full Scan도 인덱스가 테이블보다 작아 Full Table Scan(3,090)보다는 저렴하다
- 인덱스는 조회를 돕지만 DML 비용을 늘린다 — 많이 만드는 것이 능사가 아니다

실습 안내

- 실습문제 10문항 — ch06_문제.md
  - 1·8번 같은 인덱스인데 조건에 따라 오퍼레이션이 달라지는 이유
  - 2번 순서를 뒤집었을 때의 결과로 선두 컬럼의 정의를 설명
  - 3번 두 컬럼을 모두 쓸 때 순서의 영향이 작아지는 이유
  - 4번 Fast Full Scan이 Full Table Scan보다 저렴한 근거
  - 5·6번 실무 조회 빈도 가정과 인덱스를 두 개 만드는 비용
  - 7번 전화번호부 비유를 실측 결과와 대응시키기
  - 9·10번 단일 컬럼 추가와 순서 변경의 비교, 순서 결정 절차 정리
- 실습 전후 확인 — 인덱스는 IX_CH05_ 접두어로 만들고 종료 시 반드시 삭제한다
  - 삭제 후 통계를 재수집해야 이후 챕터의 실행계획이 재현된다

Oracle SQL 튜닝 과정 · ch07

SARGable SQL

SARGable 조건의 정의 · 컬럼 가공과 인덱스 · 묵시적 형변환의 방향 · Predicate Information 진단 | Oracle Database 19c · 진료비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - SARGable 조건을 '컬럼을 가공하지 않고 원본 값 그대로 비교하는 조건'으로 정의한다
  - 컬럼에 함수를 씌우면 일반 인덱스를 못 타게 되는 이유를 인덱스의 정렬 구조로 설명한다
  - 묵시적 형변환이 컬럼과 리터럴 중 어느 쪽에서 일어나는지 타입 우선순위로 판단한다
  - Predicate Information에서 컬럼이 함수로 감싸였는지 확인해 인덱스 미사용을 진단한다
  - 같은 조건을 문자열 리터럴로 쓸 때와 숫자 리터럴로 쓸 때의 실행계획 차이를 설명한다
  - 묵시적 형변환과 명시적 함수 호출이 옵티마이저에게 왜 같은 문제인지 설명한다
  - TO_CHAR로 월을 비교하는 조건을 범위 조건(`>=` / `<`)으로 재작성한다
  - LIKE 와일드카드 위치, 부정 조건, IS NULL이 일반 인덱스와 안 맞는 이유를 설명한다
  - 조건절만 보고 SARGable 여부를 판정하는 체크리스트를 실제 코드 리뷰에 적용한다

이 장의 구성 (Agenda)

- 6.1 Why — 인덱스가 있어도 못 타는 조건절이 있다
- 6.2 Concept — SARGable의 정의와 위반의 갈래
- 6.3 Oracle Internals — 묵시적 형변환의 방향
- 6.4 Example — BIRTH_DATE 세 가지 조건절 비교
- 6.5 Execution Plan — 실측 비교와 해석
- 6.6 Bad SQL — DATE 컬럼에서 같은 문제 재현
- 6.7 Tuning — 그 밖의 SARGable 위반 패턴
- 6.8~6.10 Benchmark 종합 · 실습 안내 · 요약 (Review)

7.1

인덱스가 있어도 못 타는 조건절

- ch06에서 남겨 둔 질문
- SARGable이라는 개념
- 이 차시의 목표와 범위

ch06에서 남겨 둔 질문

- ch06에서 확인한 것 — '인덱스를 만들면 빨라진다'
  - 같은 SQL을 인덱스 유무만 바꿔 비교했고, 인덱스가 있는 쪽이 유리했다
  - 그 실험에서는 조건절 작성 방식이 변수가 아니었다
- 그러나 다루지 않은 사실이 하나 있다 — 인덱스가 있어도 조건절을 잘못 쓰면 아예 못 탈 수 있다
  - 인덱스가 존재하는 것과 옵티마이저가 그 인덱스를 선택하는 것은 서로 다른 사건이다
  - 둘 사이에 조건절을 어떻게 썼는가라는 변수가 하나 더 놓여 있다
- 실무에서 이 차이가 사고로 이어지는 전형적인 경로
  - 인덱스를 만들었으니 됐다고 판단하고 실행계획을 확인하지 않는다
  - 느리다는 신고가 들어오면 인덱스를 하나 더 만드는 쪽으로 대응한다
  - 조건절은 그대로이므로 새 인덱스도 같은 이유로 쓰이지 않는다

SARGable이란 무엇인가

- SARGable = Search ARGument-able
  - 인덱스를 탐색하는 데 그대로 쓸 수 있는 형태의 조건을 뜻한다
  - 이름 그대로 그 조건이 검색 인수로 쓰일 자격이 있는가를 묻는 말이다
- 판정 기준은 하나로 압축된다 — 컬럼을 가공했는가
  - 컬럼을 가공하지 않고 원래 값 그대로 비교하는 조건이 SARGable하다
  - 컬럼에 함수·연산이 적용되면 그 순간 SARGable하지 않게 된다
- 가공의 주체가 사람인지 Oracle인지는 상관없다
  - 개발자가 직접 쓴 함수든, 타입이 안 맞아 자동으로 생긴 변환이든 결과는 같다
  - 눈에 보이지 않는 가공이 더 위험하다 — 6.3절에서 그 사례를 다룬다
- 이번 차시의 목표 — 인덱스가 있어도 못 타는 대표 패턴을 실측으로 확인한다
  - 대표 패턴은 명시적 함수 사용과 묵시적 형변환 두 가지다

이 차시의 목표와 범위

- 다루는 것 — 조건절 작성 방식이 인덱스 사용 여부를 가르는 지점
  - 명시적 함수 사용(TO_NUMBER, SUBSTR, TO_CHAR 등)
  - 묵시적 형변환 — 컬럼과 리터럴의 데이터 타입이 어긋나는 경우
  - 실측 소재는 PATIENTS.BIRTH_DATE(VARCHAR2 YYYYMMDD)와 MEDICAL_CLAIMS.RECEIPT_DATE(DATE)
- 개념으로만 짚고 넘어가는 것 — 6.7절
  - 앞에 %가 붙는 LIKE, 부정 조건, IS NULL — 실측 없이 원리만 정리한다
- 이 장에서 다루지 않는 것
  - 함수 기반 인덱스, 전문 검색 인덱스 등 별도 기법 — 본 과정 범위 밖
  - 인덱스 컬럼 순서 설계와 결합 인덱스 전략 — 별도 주제

참고:
본 차시는 실습용 인덱스를 생성·삭제한다. 종료 시 DROP INDEX로 정리해 PATIENTS·MEDICAL_CLAIMS가 PK만 남은 상태로 복구되었는지 확인한다.

7.2

SARGable의 정의와 위반의 갈래

- 함수와 인덱스 — 정렬 기준이 어긋난다
- 묵시적 형변환이라는 개념
- SARGable 위반의 갈래 정리

함수와 인덱스 — 정렬 기준이 어긋난다

- B-Tree 인덱스는 컬럼의 원본 값을 기준으로 정렬해 저장한다
  - 정렬돼 있기 때문에 특정 값이나 범위의 시작 지점을 곧바로 찾아갈 수 있다
  - 인덱스가 빠른 이유는 자료가 작아서가 아니라 순서를 알고 있기 때문이다
- 컬럼에 함수를 씌우면 비교 대상이 원본 값이 아니게 된다
  - TO_NUMBER(col), SUBSTR(col,..), TO_CHAR(col,..) 등이 모두 여기에 해당한다
  - 인덱스는 원본 값 순서로 정렬돼 있는데 비교는 변환된 값으로 이뤄진다
  - 변환된 값의 순서를 인덱스는 모르므로 그 인덱스로 바로 찾아갈 수 없다
- 그래서 옵티마이저는 그 인덱스를 접근 경로 후보에서 제외한다
  - 남는 선택지는 대개 테이블 전체를 읽으며 한 행씩 변환해 걸러내는 방식이다
  - 결과는 정확하지만 읽은 블록 수는 조건의 선택도와 무관하게 커진다

묵시적 형변환이라는 개념

- 조건절에서 컬럼과 리터럴의 데이터 타입이 다르면 그대로 비교할 수 없다
  - 문자와 숫자는 비교 규칙 자체가 다르므로 한쪽을 다른 쪽 타입으로 맞춰야 한다
- Oracle은 이때 둘 중 하나를 자동으로 변환해 비교한다 — 이것이 묵시적 형변환이다
  - 개발자가 변환 함수를 한 글자도 쓰지 않아도 내부적으로 발생한다
  - SQL 문장만 봐서는 변환이 일어났는지 알 수 없다는 점이 이 문제의 핵심이다
- 어느 쪽이 변환되는지가 인덱스 사용 여부를 가른다
  - 리터럴이 변환되면 컬럼은 원본 그대로 남으므로 인덱스를 탈 수 있다
  - 컬럼이 변환되면 컬럼에 함수를 씌운 것과 완전히 같은 효과가 된다
- 변환 방향은 개발자가 고르는 것이 아니라 Oracle의 규칙이 정한다
  - 그 규칙이 무엇인지는 6.3절에서 다룬다

SARGable 위반의 갈래 정리

- 갈래 1 — 명시적 함수·연산으로 컬럼을 가공
  - TO_NUMBER(SUBSTR(col,..)), TO_CHAR(col,..), col*2, col||'' 등
  - 눈에 보이므로 코드 리뷰로 잡을 수 있다
- 갈래 2 — 타입 불일치로 인한 묵시적 형변환
  - VARCHAR2 컬럼을 숫자 리터럴과 비교하는 등 타입을 어긋나게 쓴 경우
  - 눈에 보이지 않으므로 실행계획으로만 잡을 수 있다
- 갈래 3 — 인덱스의 정렬 특성과 맞지 않는 조건
  - 앞에 %가 붙는 LIKE, 부정 조건(!=, NOT IN), IS NULL
  - 컬럼을 가공하지는 않았지만 정렬 순서를 이용할 수 없는 형태다
- 본 차시는 갈래 1·2를 실측으로 확인하고, 갈래 3은 6.7절에서 개념으로 정리한다

7.3

묵시적 형변환의 방향

- 데이터 타입 우선순위
- 컬럼이 변환되면 벌어지는 일
- Predicate Information으로 진단하기

데이터 타입 우선순위 — 누가 변환되는가

- Oracle은 타입이 다른 두 값을 비교할 때 정해진 규칙으로 한쪽을 변환한다
  - 규칙을 단순화하면 NUMBER가 VARCHAR2보다 우선순위가 높다
  - 우선순위가 높은 쪽에 맞추므로 문자 쪽이 숫자로 변환된다
- VARCHAR2 컬럼을 NUMBER 리터럴과 비교하면 — WHERE varchar_col <= 19901231
  - 리터럴이 아니라 컬럼 쪽을 TO_NUMBER()로 감싸서 비교한다
  - 사람이 보기엔 컬럼을 건드린 적이 없지만 내부적으로는 함수가 씌워진 것과 같다
- 이 방향이 최악인 이유
  - 변환 대상이 리터럴이면 값 하나만 한 번 바꾸면 끝난다
  - 변환 대상이 컬럼이면 모든 행의 값을 하나씩 바꿔야 하고 인덱스도 무력화된다
- 즉 리터럴에 작은따옴표를 빼먹었을 뿐인데 인덱스를 못 타게 만들 수 있다

컬럼이 변환되면 벌어지는 일

- 조건절이 사실상 다음과 같이 바뀐 상태로 처리된다
  - WHERE birth_date <= 19901231 → WHERE TO_NUMBER(birth_date) <= 19901231
  - 개발자가 쓴 SQL 텍스트에는 TO_NUMBER가 어디에도 없다
- 결과적으로 명시적 함수 사용과 동일한 상황이 된다
  - 인덱스는 원본 문자열 순서로 정렬돼 있는데 비교는 숫자 값으로 이뤄진다
  - 옵티마이저는 그 인덱스로 찾아갈 수 없다고 판단하고 후보에서 제외한다
- 추가로 따라오는 위험 — 정확성 문제
  - 숫자로 바꿀 수 없는 값이 한 건이라도 섞이면 변환 자체가 실패할 수 있다
  - 평소 잘 돌던 SQL이 데이터가 하나 들어온 순간 오류를 내는 형태로 나타난다
- 성능 문제로만 보이지만 잠재적으로는 안정성 문제이기도 하다

Predicate Information으로 진단하기

- 묵시적 형변환은 SQL 텍스트로는 보이지 않는다 — 실행계획으로 진단한다
  - 실행계획 아래쪽의 Predicate Information 영역을 본다
  - 조건이 옵티마이저에게 어떤 형태로 전달됐는지가 그대로 적혀 있다
- 판정 방법은 단순하다 — 컬럼 이름이 함수에 감싸여 있는지 본다
  - filter("BIRTH_DATE"<='19901231') — 컬럼이 원본 그대로다
  - filter(TO_NUMBER("BIRTH_DATE")<=19901231) — 컬럼이 감싸여 있다
- access와 filter의 구분도 함께 읽는다
  - access는 인덱스에서 그 조건으로 찾아간 것을 뜻한다
  - filter는 읽어 온 행을 그 조건으로 걸러낸 것을 뜻한다

참고:
실행계획은 DBMS_XPLAN.DISPLAY_CURSOR에 ALLSTATS LAST 포맷을 주면 Predicate Information까지 함께 출력된다.

7.4

BIRTH_DATE 세 가지 조건절 비교

- 실측 소재 — PATIENTS.BIRTH_DATE
- Q1 vs Q2 — 작은따옴표 하나의 차이
- Q3 — 명시적 함수로 컬럼 가공

실측 소재 — PATIENTS.BIRTH_DATE

- BIRTH_DATE는 VARCHAR2 타입이며 YYYYMMDD 형식 문자열을 담는다
  - 예: '19901231' — 여덟 자리 문자열
  - DATE 타입이 아니라 문자열이라는 점이 이번 실험의 출발점이다
- 왜 이런 설계를 하는가
  - YYYYMMDD 문자열은 사전식 정렬 순서가 실제 날짜 순서와 일치한다
  - 그래서 문자열 범위 비교가 날짜 범위 비교와 같은 결과를 낸다
  - 이 성질 덕분에 함수 없이도 기간 조건을 SARGable하게 쓸 수 있다
- 이 컬럼에 인덱스를 만들고 세 가지 조건절로 같은 의도를 표현해 본다
  - Q1 — 문자열 리터럴 그대로 비교
  - Q2 — 따옴표를 뺀 숫자 리터럴과 비교
  - Q3 — SUBSTR과 TO_NUMBER로 컬럼을 직접 가공

Q1 vs Q2 — 작은따옴표 하나의 차이

6.4절 Example — 같은 의도, 리터럴 표기만 다른 두 조건절

```
-- 실습용 인덱스 생성 (차시 종료 시 DROP INDEX로 정리)
CREATE INDEX ix_birth ON patients(birth_date);

-- Q1. SARGable : 문자열 리터럴 그대로 비교  (BIRTH_DATE는 VARCHAR2)
SELECT COUNT(*) FROM patients WHERE birth_date <= '19901231';

-- Q2. 묵시적 형변환 : 따옴표 없는 숫자 리터럴과 비교
SELECT COUNT(*) FROM patients WHERE birth_date <= 19901231;

-- 두 SQL의 문자상 차이는 작은따옴표 두 개뿐이다
-- 결과 건수도 둘 다 37,425건으로 완전히 같다
-- 그런데 실행계획은 서로 다르다  -> 6.5절에서 확인
```

Q3 — 명시적 함수로 컬럼 가공

6.4절 Example — 개발자가 직접 함수를 씌운 경우

```
-- Q3. 명시적 함수 : 컬럼을 직접 가공
SELECT COUNT(*) FROM patients
WHERE TO_NUMBER(SUBSTR(birth_date,1,4)) <= 1990;

-- SUBSTR로 앞 네 자리를 잘라내고 TO_NUMBER로 숫자로 바꾼 뒤 비교한다
-- 개발자가 함수를 직접 썼으므로 컬럼 가공이 눈에 보인다

-- Q2와 Q3의 공통점 — 조건절에 birth_date가 원본 그대로 있지 않다
--   Q2 : TO_NUMBER("BIRTH_DATE")                <- Oracle이 자동으로 씌움
--   Q3 : TO_NUMBER(SUBSTR("BIRTH_DATE",1,4))    <- 개발자가 직접 씌움

-- 옵티마이저는 이 둘을 구분하지 않는다  -> 6.5절에서 확인
```

7.5

실행계획 — 세 조건절 실측 비교

- 세 쿼리의 실행계획 실측
- Q1은 왜 인덱스를 탔는가
- Q2와 Q3의 계획이 같은 이유

세 조건절의 실행계획 실측

6.5절 Execution Plan — 같은 컬럼, 같은 인덱스, 조건절 표기만 다름

```
+-----------------------------+-----------------------+---------+
| 쿼리                         | Operation             | Buffers |
+-----------------------------+-----------------------+---------+
| Q1 (문자열 리터럴)           | INDEX FAST FULL SCAN  |     148 |
| Q2 (숫자 리터럴, 형변환)     | TABLE ACCESS FULL     |     442 |
| Q3 (명시적 함수)             | TABLE ACCESS FULL     |     442 |
+-----------------------------+-----------------------+---------+

-- Predicate Information
--   Q1 : filter("BIRTH_DATE"<='19901231')
--   Q2 : filter(TO_NUMBER("BIRTH_DATE")<=19901231)
--   Q3 : filter(TO_NUMBER(SUBSTR("BIRTH_DATE",1,4))<=1990)

-- Q2와 Q3은 Plan hash value까지 완전히 동일했다
```

Q1은 왜 인덱스를 탔는가

- Q1의 Predicate Information — filter("BIRTH_DATE"<='19901231')
  - 컬럼이 가공되지 않은 원본 그대로 조건에 나타난다
  - BIRTH_DATE는 VARCHAR2이고 리터럴도 문자열이라 타입이 일치한다
  - 타입이 맞으므로 묵시적 형변환이 일어날 이유가 없다
- 그 결과 옵티마이저는 인덱스를 접근 경로 후보로 쓸 수 있었다
  - 실측 결과는 INDEX FAST FULL SCAN, Buffers 148
  - 조건 자체의 선택도가 낮아 인덱스를 통째로 읽는 방식이 선택됐다
- 그래도 테이블 전체를 읽는 것보다는 적게 읽는다
  - 인덱스에는 대상 컬럼과 ROWID만 들어 있어 블록당 담기는 항목 수가 많다
  - COUNT(*)에 필요한 값이 인덱스 안에 다 있으므로 테이블을 볼 필요가 없다

Q2 해석 — TO_NUMBER는 누가 넣었는가

- Q2의 Predicate Information — filter(TO_NUMBER("BIRTH_DATE")<=19901231)
  - 개발자는 SQL 어디에도 TO_NUMBER를 쓰지 않았다
  - 그런데 실행계획에는 컬럼이 TO_NUMBER로 감싸인 형태로 나타난다
- 이것이 Oracle이 리터럴이 아니라 컬럼 쪽을 변환했다는 직접적인 증거다
  - BIRTH_DATE는 VARCHAR2, 19901231은 따옴표가 없으므로 NUMBER 리터럴이다
  - NUMBER가 우선순위가 높으므로 문자 쪽인 컬럼이 변환 대상이 됐다
- 변환된 순간 인덱스의 정렬 순서를 활용할 수 없게 됐다
  - 실측 결과는 TABLE ACCESS FULL, Buffers 442
  - 인덱스는 그대로 존재하지만 접근 경로로 선택되지 않았다
- 진단 절차의 결론 — Predicate Information에 컬럼이 감싸여 있으면 의심한다

Q2와 Q3의 계획이 같은 이유

- 두 쿼리의 실행계획은 Plan hash value까지 완전히 동일했다
  - Q2는 개발자가 함수를 쓰지 않았고, Q3은 함수를 두 개나 썼다
  - 작성 방식이 전혀 다른데도 옵티마이저의 결론은 같았다
- 옵티마이저가 보는 것은 단 하나뿐이다
  - 누가 왜 이 함수를 넣었는가가 아니라 '인덱스 컬럼이 가공된 상태로 조건에 나타나는가'
  - 가공됐다면 원본 인덱스의 정렬 순서를 쓸 수 없다고 판단하고 동일하게 처리한다
- 여기서 나오는 실무적 결론
  - 실수로 생긴 문제와 의도적으로 만든 문제를 DB는 구분해 주지 않는다
  - 따라서 코드 리뷰 기준도 함수를 썼는가가 아니라 컬럼이 가공됐는가여야 한다
- 조건절을 읽을 때는 컬럼 이름이 괄호 안에 들어가 있는지부터 본다

결과가 같으면 무시해도 되는가

- Q1과 Q2의 결과 건수는 둘 다 37,425건으로 동일했다
  - 그래서 결과가 같으니 성능 차이는 무시해도 된다는 주장이 나올 수 있다
- 반박 1 — 지금 이미 차이가 나 있다
  - Buffers 148과 442로 약 3배 차이가 이미 관측됐다
  - PATIENTS는 5만 건짜리 비교적 작은 테이블이라 체감이 크지 않을 뿐이다
- 반박 2 — 데이터가 커지면 격차 자체가 벌어진다
  - TABLE ACCESS FULL은 테이블 크기에 비례해 읽는 양이 계속 커진다
  - INDEX FAST FULL SCAN이나 RANGE SCAN은 훨씬 완만하게 증가한다
  - 지금은 3배지만 데이터가 커질수록 그 배율 자체가 더 커질 수 있다
- 결론 — '무시해도 된다'는 판단은 현재 데이터 규모에서만 유효하다

7.6

DATE 컬럼에서 같은 문제 재현

- 실측 소재 — MEDICAL_CLAIMS.RECEIPT_DATE
- Q4 vs Q5 — 범위 조건과 TO_CHAR 가공
- 월 단위 조회를 SARGable하게 쓰기

실측 소재 — MEDICAL_CLAIMS.RECEIPT_DATE

- RECEIPT_DATE는 DATE 타입 컬럼이다
  - 앞 절의 BIRTH_DATE와 달리 문자열이 아니라 진짜 날짜 타입이다
  - 이 컬럼에 인덱스를 만들고 같은 실험을 반복한다
- 업무 요구는 하나다 — 특정 월의 청구 데이터를 조회한다
  - Q4 — 범위 조건으로 원본 값 그대로 비교한다
  - Q5 — TO_CHAR로 월 단위 문자열을 만들어 비교한다
- 두 조건은 논리적으로 같은 결과를 가리킨다
  - 실측 결과 건수도 두 쿼리 모두 24,655건으로 완전히 같다
  - 따라서 성능 차이가 나온다면 원인은 결과가 아니라 표현 방식이다
- 앞 절과 같은 실험 설계다 — 변수는 조건절 표현 하나뿐이다

Q4 vs Q5 — 범위 조건과 TO_CHAR 가공

6.6절 Bad SQL — 같은 결과를 내는 두 가지 월 단위 조회

```
CREATE INDEX ix_receipt ON medical_claims(receipt_date);

-- Q4. SARGable : 범위 조건으로 원본 값 그대로 비교
SELECT COUNT(*) FROM medical_claims
WHERE receipt_date >= DATE'2024-06-01'
  AND receipt_date <  DATE'2024-07-01';

-- Q5. 위반 : 컬럼을 TO_CHAR로 가공해 월 단위 문자열과 비교
SELECT COUNT(*) FROM medical_claims
WHERE TO_CHAR(receipt_date,'YYYYMM') = '202406';

-- 두 조건이 뜻하는 기간은 같다 – 결과 건수도 둘 다 24,655건
-- Q4는 컬럼이 원본 그대로, Q5는 컬럼이 TO_CHAR로 감싸여 있다
```

Q4 vs Q5 실행계획 실측

6.6절 Execution Plan — 결과는 같고 읽은 블록 수만 다르다

```
+---------------------+--------------------+---------+
| 쿼리                | Operation          | Buffers |
+---------------------+--------------------+---------+
| Q4 (범위 조건)      | INDEX RANGE SCAN   |      68 |
| Q5 (TO_CHAR 가공)   | TABLE ACCESS FULL  |   3,090 |
+---------------------+--------------------+---------+

-- Predicate Information
--   Q4 : access("RECEIPT_DATE">=... AND "RECEIPT_DATE"<...)
--   Q5 : filter(TO_CHAR(...,'YYYYMM')='202406')

-- Q4는 access, Q5는 filter
--   access : 인덱스에서 그 조건으로 '찾아갔다'  -> 읽는 양 자체가 줄어든다
--   filter : 읽어 온 행을 그 조건으로 '걸러냈다' -> 읽는 양은 줄지 않는다
```

결과는 같은데 Buffers는 45배

- 두 쿼리의 결과 건수는 24,655건으로 완전히 같다
  - 그런데 Buffers는 68과 3,090으로 45배 이상 차이가 난다
- Q5가 느린 이유 — 컬럼을 변환한 뒤 비교하기 때문
  - 인덱스가 원본 DATE 값 순서로 정렬돼 있다는 사실을 활용할 수 없다
  - 모든 행을 일일이 문자열로 변환해서 비교해야 하므로 전체를 읽어야 한다
- Q4가 빠른 이유 — 컬럼을 전혀 가공하지 않기 때문
  - 원본 값 그대로 범위를 비교하므로 인덱스에서 시작·끝 위치를 바로 찾아간다
  - 그 구간에 해당하는 항목만 읽으면 되므로 읽는 양이 조건 범위에 비례한다
- 결론 — 최종 결과 집합이 같더라도 옵티마이저가 인식하는 형태가 전혀 다르다
  - 논리적 동등성과 SARGable 여부는 별개의 문제다

월 단위 조회를 SARGable하게 쓰기

- '이번 달 데이터를 가져오자'는 요구는 어디에나 있다
  - 흔한 구현 — TO_CHAR(receipt_date,'YYYYMM') = TO_CHAR(SYSDATE,'YYYYMM')
  - 읽기는 쉽지만 컬럼을 가공하므로 Q5와 똑같은 문제가 발생한다
- 재작성 원칙 — 가공은 컬럼이 아니라 기준값 쪽에서 한다
  - receipt_date >= TRUNC(SYSDATE,'MM')
  - AND receipt_date < ADD_MONTHS(TRUNC(SYSDATE,'MM'), 1)
  - SYSDATE 쪽 계산은 실행 전에 상수처럼 한 번 결정되고 끝난다
  - receipt_date 컬럼 자체는 전혀 가공되지 않으므로 인덱스를 그대로 탄다
- 이 패턴이 기간 조회의 표준형이다
  - 시작은 이상(>=), 끝은 다음 구간 시작 미만(<)으로 쓴다
  - 날짜형에서 이하(<=)를 쓰면 시분초가 붙은 값이 누락될 수 있다

SARGable 재작성 패턴 모음

6.6~6.7절 Tuning — 실습 해설에 나온 재작성 사례 정리

```
-- 1) 이번 달 데이터 조회
--    위반 : WHERE TO_CHAR(receipt_date,'YYYYMM') = TO_CHAR(SYSDATE,'YYYYMM')
--    개선
WHERE receipt_date >= TRUNC(SYSDATE,'MM')
AND   receipt_date <  ADD_MONTHS(TRUNC(SYSDATE,'MM'), 1)

-- 2) 1990년대생 환자 조회  (BIRTH_DATE는 VARCHAR2 YYYYMMDD)
WHERE birth_date BETWEEN '19900101' AND '19991231'            -- 방법 1
WHERE birth_date >= '19900101' AND birth_date < '20000101'    -- 방법 2

-- 3) 코드 리뷰 사례  (HOSP_ID는 NUMBER 컬럼)
--    위반 : WHERE hosp_id || '' = '100'   <- 컬럼을 문자열로 가공
--    개선 : WHERE hosp_id = 100           <- 타입 일치, 가공 없음

-- 공통 원칙 : 가공은 컬럼이 아니라 리터럴·상수 쪽에서 한다
```

7.7

그 밖의 SARGable 위반 패턴

- LIKE와 와일드카드의 위치
- 부정 조건과 인덱스
- IS NULL과 B-Tree 인덱스

LIKE와 와일드카드의 위치

- LIKE '%검색어%' — 앞에 %가 붙는 패턴
  - 인덱스는 값을 왼쪽부터 정렬해서 저장한다
  - 중간부터 일치하는 값은 정렬 순서상 어디에 있는지 특정할 수 없다
  - 결국 전체를 훑어야 하므로 일반 B-Tree 인덱스로는 지원할 수 없다
  - 전문 검색 인덱스 등 별도 기법이 필요하며 본 과정 범위 밖이다
- LIKE '검색어%' — 뒤에만 %가 붙는 패턴
  - '이 문자열로 시작하는 값'을 찾는 것이므로 시작 지점을 바로 찾아갈 수 있다
  - 정렬된 순서에서 연속된 구간에 모여 있으므로 RANGE SCAN으로 처리 가능하다
- LIKE '%검색어' — 앞에만 %가 붙는 패턴
  - 끝부분이 일치해야 하므로 앞쪽 정렬 순서로는 시작 지점을 특정할 수 없다
- 판정 기준 — 와일드카드가 앞에 오는가만 보면 된다

부정 조건과 인덱스

- 부정 조건 — !=, NOT IN, NOT LIKE
  - 인덱스는 특정 값과 같은 것 또는 특정 범위 안에 있는 것을 찾는 데 최적화돼 있다
  - 연속된 한 구간을 찾아가는 구조라는 뜻이다
- '이 값이 아닌 모든 것'은 연속된 구간으로 표현하기 어렵다
  - 전체에서 한 구간만 빼는 형태이므로 나머지 전부가 대상이 된다
  - 읽어야 할 대상이 거의 전체라면 인덱스를 경유하는 이점이 사라진다
  - 그래서 대개 Full Scan으로 처리된다
- 실무에서의 대응 방향
  - 가능하면 긍정 조건으로 바꿔 쓸 수 있는지 먼저 검토한다
  - 값의 종류가 적다면 원하는 값들을 IN으로 나열하는 편이 나을 수 있다
  - 조건 자체를 바꿀 수 없다면 다른 조건으로 대상을 좁힐 수 있는지 본다

IS NULL과 B-Tree 인덱스

- 일반 B-Tree 인덱스는 인덱스 구성 컬럼 값이 모두 NULL인 행을 저장하지 않는다
  - NULL은 값이 아니라 값이 없는 상태이므로 정렬 위치를 정할 수 없기 때문이다
  - 저장되지 않았으니 인덱스 안에 찾을 대상 자체가 없다
- 그래서 IS NULL 조건은 그 인덱스를 쓸 수 없다
  - 인덱스가 있어도 NULL인 행은 인덱스만으로 찾아낼 방법이 없다
- 예외 — 복합 인덱스의 일부 컬럼만 NULL인 경우
  - 구성 컬럼 중 하나라도 값이 있으면 그 행은 인덱스에 저장된다
  - 따라서 복합 인덱스에서는 상황에 따라 활용 가능한 경우가 있다
- 설계 단계에서 함께 생각할 것
  - 자주 조회되는 컬럼이라면 NULL 대신 기본값을 쓰는 설계를 검토할 수 있다
  - 다만 NULL과 기본값은 의미가 다르므로 업무 정의부터 확인해야 한다

7.8

Benchmark 종합과 판정 체크리스트

- 두 컬럼 실측 종합
- SARGable 판정 체크리스트
- 요약과 실습 안내

Benchmark — 두 컬럼 실측 종합

6.8절 Benchmark — 타입도 위반 방식도 다르지만 결론은 같다

```
+-----------------------------+--------------------+------------------------+-----------------+
| 컬럼                        | SARGable 조건      | 위반 조건              | Buffers 차이    |
+-----------------------------+--------------------+------------------------+-----------------+
| PATIENTS.BIRTH_DATE         | <= '19901231'      | <= 19901231 (형변환)   | 148 vs 442      |
| MEDICAL_CLAIMS.RECEIPT_DATE | 범위 (>= / <)      | TO_CHAR(..)='202406'   | 68 vs 3,090     |
+-----------------------------+--------------------+------------------------+-----------------+

-- BIRTH_DATE   : 148 vs 442    -> 약 3배
-- RECEIPT_DATE : 68  vs 3,090  -> 45배 이상

-- 배율보다 중요한 사실 : 결과 집합은 같은데 읽은 블록 수만 달라졌다
-- VARCHAR2든 DATE든, 묵시적이든 명시적이든 결론은 하나로 수렴한다
```

SARGable 판정 체크리스트

- 조건절을 볼 때 이 순서로 확인한다
  - 1. 컬럼 이름이 함수나 연산의 괄호 안에 들어가 있는가
  - 2. 컬럼과 비교 대상 리터럴의 데이터 타입이 서로 같은가
  - 3. 문자 컬럼에 따옴표 없는 숫자 리터럴을 쓰지 않았는가
  - 4. 바인드 변수의 선언 타입이 컬럼 타입과 일치하는가
  - 5. 기간 조건을 TO_CHAR 비교가 아니라 범위 비교로 썼는가
  - 6. LIKE 패턴의 앞에 와일드카드가 붙어 있지 않은가
  - 7. 부정 조건이나 IS NULL로 대상을 지정하고 있지 않은가
- 확실하지 않으면 실행계획의 Predicate Information을 확인한다
  - 컬럼이 함수로 감싸여 있으면 그 조건은 SARGable하지 않다
  - access가 아니라 filter로만 잡혀 있으면 읽는 양이 줄지 않았다는 뜻이다

요약 (Review)

- 컬럼에 함수를 씌우면 명시적이든 묵시적이든 일반 인덱스를 탈 수 없게 된다
- 인덱스는 가공되지 않은 원본 값 순서로 정렬돼 있다 — 이 한 문장이 모든 결론의 근거다
- 타입이 다른 리터럴과 비교하면 Oracle이 컬럼 쪽을 자동 변환할 수 있다
- VARCHAR2 컬럼과 NUMBER 리터럴을 비교하면 컬럼이 TO_NUMBER로 감싸인다
- Predicate Information에서 컬럼이 함수로 감싸여 있는지 보면 진단할 수 있다
- 묵시적 형변환과 명시적 함수는 옵티마이저에게 완전히 같은 문제다 — 계획까지 동일했다
- '월 단위로 가져오자'는 요구는 TO_CHAR 비교가 아니라 범위 조건으로 써야 한다
- LIKE 앞쪽 와일드카드, 부정 조건, IS NULL도 일반 인덱스와 궁합이 나쁜 패턴이다
- 가공은 컬럼이 아니라 리터럴·상수 쪽에서 한다 — 이 장의 재작성 원칙

실습 안내

- 실습문제 10문항 — ch07_문제.md
  - 1~2번 따옴표 유무가 만든 실행계획 차이와 묵시적 형변환의 발생 원리
  - 3번 묵시적 형변환과 명시적 함수가 같은 계획이 된 이유
  - 4~5번 TO_CHAR 비교와 범위 조건의 Buffers 차이, 이번 달 조회 재작성
  - 6번 결과가 같으니 무시해도 된다는 주장에 대한 반박
  - 7~8번 뒤쪽 와일드카드 LIKE의 가능성, 부정 조건과 IS NULL의 원리
  - 9~10번 코드 리뷰 판정과 1990년대생 조회 조건 직접 작성
- 실습 전 확인 — 이번 장은 인덱스를 생성하고 삭제한다
  - 실습에서 만든 인덱스는 종료 시 반드시 DROP INDEX로 정리한다
  - PATIENTS와 MEDICAL_CLAIMS가 PK만 남은 상태로 복구됐는지 확인한다

Oracle SQL 튜닝 과정 · ch08

Optimizer와 통계정보

카디널리티 추정 · 히스토그램과 데이터 편중 · 통계 수집 시점 · 바인드 피킹

학습 목표 (Objectives)

- CBO가 통계정보를 근거로 비용을 계산해 실행 방법을 고르는 구조를 설명할 수 있다
- 히스토그램이 없는 컬럼의 등치 조건 E-Rows가 어떻게 계산되는지 말할 수 있다
  - 전체 행 수 ÷ NDV — 모든 값이 같은 빈도라는 균등 분포 가정
- 같은 컬럼에서 히스토그램 유무만 통제해 추정치 변화를 관측하고 해석할 수 있다
  - METHOD_OPT의 SIZE 1로 제거, SIZE AUTO로 복원
- USER_TAB_COL_STATISTICS로 히스토그램 종류와 버킷 수를 직접 확인할 수 있다
- 카디널리티 추정 오류가 조인 방식 선택까지 망가뜨리는 경로를 설명할 수 있다
- 히스토그램으로는 풀리지 않는 문제를 구분할 수 있다
  - 바인드 변수 Peeking으로 계획이 고정되는 경우
  - 두 컬럼의 조합 조건 — 컬럼 간 상관관계

이 장의 구성 (Agenda)

- 13.1 Why — 히스토그램은 없애 봐야 안다
- 13.2 CBO와 통계정보의 구성
- 13.3 히스토그램이 없으면 무엇으로 추정하는가
- 13.4 베이스라인 — 히스토그램이 있는 상태의 관측
- 13.5 히스토그램 제거 실험
- 13.6 복원과 SIZE AUTO
- 13.7 히스토그램의 한계와 운영 판단
  - 13.2와 13.3은 칠판 설명 구간, 13.4부터 13.6까지가 실측 구간
  - 13.7은 실습문제 7·9·10번으로 이어지는 확장 논의

8.1

Why — 히스토그램은 없애 봐야 안다

- ch02에서 남은 질문 — 히스토그램이 없었다면 어땠을까
- 같은 컬럼, 같은 데이터에서 통계만 통제하는 실험 설계
- 소재는 MEDICAL_CLAIMS.HOSP_ID의 실제 스큐

ch02가 남긴 질문

- ch02에서 확인한 것
  - HOSP_ID에 히스토그램이 있어서 리터럴 값마다 다른 E-Rows가 나온다
  - 값에 따라 추정치가 달라진다는 현상 자체는 이미 봤다
- 그런데 확인하지 못한 것
  - 히스토그램이 "없었다면" 어땠을지는 실제로 없애 보지 않으면 알 수 없다
  - 설명으로 들은 인과와 관찰로 확인한 인과는 다른 상태다
- 본 차시의 목표
  - 같은 컬럼·같은 데이터로 히스토그램 유무를 직접 통제해 본다
  - 카디널리티 추정에 미치는 영향을 눈으로 확인한다

실험 설계 — 제거하고, 관측하고, 복원한다

- 세 단계로 통제한다
  - 1단계 현재 상태(히스토그램 있음)에서 베이스라인 관측
  - 2단계 METHOD_OPT의 SIZE 1로 히스토그램을 제거하고 재관측
  - 3단계 SIZE AUTO로 복원하고 원래대로 돌아오는지 확인
- 왜 복원 단계까지 필요한가
  - 있음과 없음만 비교하면 다른 요인이 끼어들었을 가능성을 배제할 수 없다
  - 되돌렸을 때 되돌아와야 인과가 성립한다
- 실습 규칙
  - 통계정보를 일시적으로 변경하는 차시이므로 종료 시 반드시 원상복구
  - 복원 확인 기준은 HISTOGRAM과 NUM_BUCKETS 값이다

소재 — 진료비 청구 데이터의 실제 편중

- 이 스키마는 왜 좋은 소재인가
  - MEDICAL_CLAIMS.HOSP_ID에는 실제 스큐가 있다
  - 1~50번 병원에 청구가 50% 집중되어 있다
- 현실의 데이터는 대체로 균등하지 않다
  - 대형 병원 소수가 청구 건수의 큰 비중을 차지하는 것은 자연스러운 분포다
  - 심사 상태, 청구 유형처럼 값이 한쪽으로 몰리는 컬럼도 마찬가지다
- 그래서 관측 대상 값을 두 개로 잡는다
  - hosp_id = 1 — 쏠린 병원(스큐)
  - hosp_id = 500 — 일반 병원

8.2

CBO와 통계정보의 구성

- CBO — 통계를 근거로 비용을 계산해 실행 방법을 고른다
- Table Statistics / Column Statistics / Histogram
- Cardinality와 Selectivity의 정의

CBO — Cost-Based Optimizer

- 옵티마이저는 무엇을 하는가
  - 같은 SQL을 실행하는 여러 방법 중 하나를 고른다
  - 고르는 기준은 취향이 아니라 계산된 비용이다
- 비용은 어디서 오는가
  - 통계정보를 근거로 계산한다 — 통계가 틀리면 비용도 틀린다
  - 옵티마이저는 데이터를 직접 세어 보지 않는다
- 그래서 생기는 구조적 취약점
  - 옵티마이저의 판단은 통계라는 지도의 정확도에 종속된다
  - 지도가 낡거나 거칠면, 논리가 옳아도 결론이 틀린다

통계정보에는 무엇이 담기나

- Table Statistics
  - 테이블의 행 수, 블록 수, 평균 행 길이 등 테이블 전체의 규모 정보
- Column Statistics
  - 컬럼별 고유값 개수(NDV), 최소값·최대값, NULL 개수 등
  - NDV는 값의 종류가 몇 가지인지를 뜻할 뿐 각 값이 몇 건인지는 담지 않는다
- Histogram
  - 컬럼 값의 분포를 구간(버킷)별로 기록한 통계
  - 값이 균등하게 분포하지 않는 컬럼에서 정확한 추정을 가능하게 한다
- 세 층위의 관계
  - 테이블 통계는 규모, 컬럼 통계는 윤곽, 히스토그램은 분포의 세부

히스토그램이란 무엇인가

- 정의
  - 컬럼 값의 분포를 구간(버킷) 단위로 기록해 둔 통계
  - 자주 나오는 값과 드문 값을 구분해서 기억한다
- 왜 버킷인가
  - 값이 수천 종류라면 값마다 빈도를 모두 저장할 수 없다
  - 구간으로 나눠 요약하면 적은 저장 공간으로 분포의 모양을 담을 수 있다
- 버킷 수가 뜻하는 것
  - 구간이 많을수록 실제 분포를 더 세밀하게 반영한다
  - NUM_BUCKETS가 1이면 사실상 구간을 나누지 않은 것 — 히스토그램이 없는 상태
- 전제 조건
  - 히스토그램이 필요한 컬럼은 값이 한쪽으로 쏠린 컬럼이다

Cardinality와 Selectivity

- Cardinality — 카디널리티
  - 조건을 만족하는 행의 개수
  - 실행계획의 E-Rows가 옵티마이저가 추정한 카디널리티다
- Selectivity — 선택도
  - 조건을 만족하는 행의 비율
  - 카디널리티 = 전체 행 수 × 선택도
- 왜 이 값이 모든 판단의 출발점인가
  - 접근 방식, 조인 방식, 조인 순서가 모두 이 추정치 위에서 결정된다
  - 출발점이 틀리면 그 뒤의 논리가 옳아도 결론이 틀린다
- 정확도를 읽는 법
  - E-Rows와 A-Rows를 나란히 놓고 비율로 본다

8.3

히스토그램이 없으면 무엇으로 추정하는가

- 균등 분포 가정 — 전체 행 수 ÷ NDV
- 어떤 값을 조회하든 같은 E-Rows가 나오는 이유
- USER_TAB_COL_STATISTICS로 현재 상태 확인하기

균등 분포 가정 — 전체 행 수 ÷ NDV

- 히스토그램이 없는 컬럼의 등치 조건
  - 컬럼 = 값 형태의 조건에 대한 E-Rows는 대략 전체 행 수 ÷ NDV로 계산된다
- 이 식이 성립하려면 필요한 가정
  - 모든 값이 똑같은 빈도로 나타난다 — 균등 분포 가정
  - 옵티마이저는 분포를 모르므로 가장 무난한 가정을 택하는 것이다
- 이 가정의 결과
  - 값이 무엇이든 계산식이 같으므로 E-Rows도 항상 같아진다
  - 즉 추정치가 조회하는 값에 반응하지 않게 된다
- 판정 기준
  - 서로 다른 값에 같은 E-Rows가 나온다면 균등 가정이 적용된 것이다

균등 가정은 언제 맞고 언제 틀리는가

- 가정이 잘 맞는 경우
  - 값이 실제로 고르게 퍼져 있는 컬럼
  - 이런 컬럼에는 히스토그램이 있으나 없으나 추정 결과가 비슷하다
- 가정이 크게 틀리는 경우
  - 소수의 값에 데이터가 크게 쏠린 컬럼
  - 쏠린 값은 과소추정되고, 드문 값은 과대추정된다
- 오차의 방향이 값마다 반대라는 점이 중요하다
  - 평균으로 맞추는 추정이므로 양쪽 끝에서 동시에 틀린다
  - 한 컬럼 안에서 어떤 조회는 너무 크게, 어떤 조회는 너무 작게 잡힌다
- 이 스키마의 HOSP_ID는 후자에 해당한다

현재 상태를 어디서 확인하나

- USER_TAB_COL_STATISTICS
  - 컬럼별 통계 상태를 직접 확인할 수 있는 딕셔너리 뷰
- 이 장에서 볼 두 개의 컬럼
  - HISTOGRAM — 히스토그램의 종류 또는 NONE
  - NUM_BUCKETS — 분포를 몇 개 구간으로 나눠 기록했는지
- 읽는 법
  - HISTOGRAM이 NONE이고 NUM_BUCKETS가 1이면 히스토그램이 없는 상태
  - 값이 들어 있으면 그 종류와 구간 수만큼 분포가 기록된 상태
- 실습에서의 용도
  - 제거와 복원이 실제로 반영됐는지 판정하는 근거
  - 추정치의 변화와 통계 상태의 변화를 짝지어 확인한다

8.4

베이스라인 — 히스토그램이 있는 상태

- 관측 대상 두 개 — 쏠린 병원과 일반 병원
- 현재 통계 상태 확인 — HISTOGRAM과 NUM_BUCKETS
- E-Rows와 A-Rows를 나란히 놓고 정확도 판정

베이스라인 관측 — 조회와 통계 확인

04. Example — 같은 컬럼에서 값만 바꿔 두 번 관측한다

```
-- 쏠린 병원(스큐)
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = 1;

-- 일반 병원
SELECT COUNT(*) FROM medical_claims WHERE hosp_id = 500;

-- 현재 통계 상태 확인
SELECT column_name, histogram, num_buckets
  FROM user_tab_col_statistics
 WHERE table_name = 'MEDICAL_CLAIMS'
   AND column_name = 'HOSP_ID';

COLUMN_NAME   HISTOGRAM   NUM_BUCKETS
-----------   ---------   -----------
HOSP_ID       HYBRID              254
```

베이스라인 결과 — 히스토그램이 있을 때

05. Execution Plan — 두 조건의 추정치와 실제값 비교

조건	E-Rows	A-Rows	추정 정확도
hosp_id=1 (스큐)	2,795	3,058	근접 (비율 약 0.91배)
hosp_id=500 (일반)	280	177	근접 (비율 약 1.58배)

베이스라인 해석 — 무엇을 확인했나

- 두 값이 서로 다른 E-Rows를 받았다
  - 추정치가 조회하는 값에 반응하고 있다는 뜻
  - 균등 분포 가정이 적용되고 있지 않다는 증거이기도 하다
- 둘 다 실제값에 상당히 근접했다
  - 쏠린 값도, 일반 값도 실제와 같은 자릿수 범위 안에 들어왔다
  - 히스토그램이 스큐를 정확히 반영하고 있다는 뜻
- 통계 상태와 짝지어 읽는다
  - HISTOGRAM은 HYBRID, NUM_BUCKETS는 254
  - 분포가 254개 구간으로 기록되어 있는 상태
- 이 상태를 기준점으로 삼고 다음 절에서 통계만 바꾼다

8.5

히스토그램 제거 실험

- METHOD_OPT의 SIZE 1 — 버킷 하나는 히스토그램 없음
- 두 조건의 E-Rows가 완전히 같아지는 장면
- 과소추정이 조인 판단까지 끌고 가는 경로

히스토그램 강제 제거

06. Bad SQL — METHOD_OPT으로 버킷을 하나만 만들게 한다

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'MEDICAL_CLAIMS',
    METHOD_OPT => 'FOR COLUMNS HOSP_ID SIZE 1');  -- 버킷 1개 = 히스토그램 없음
END;
/

-- USER_TAB_COL_STATISTICS 재확인
COLUMN_NAME   HISTOGRAM   NUM_BUCKETS
-----------   ---------   -----------
HOSP_ID       NONE                  1

-- 데이터는 그대로, SQL도 그대로, 통계만 바뀌었다
```

제거 후 결과 — E-Rows가 하나로 고정된다

06. Bad SQL — 같은 두 조건을 다시 관측한 결과

조건	E-Rows	A-Rows	추정 정확도
hosp_id=1 (스큐)	300	3,058	약 10.2배 과소추정
hosp_id=500 (일반)	300	177	약 1.7배 과대추정

300이라는 숫자의 출처

- 계산식은 앞 절에서 이미 배웠다
  - MEDICAL_CLAIMS의 전체 행 수는 300,000
  - HOSP_ID의 NDV는 1,000
  - 300,000 ÷ 1,000 = 300
- 균등 분포 가정이 숫자로 확인된 것이다
  - 어떤 HOSP_ID 값을 조회하든 이 계산식은 같은 결과를 낸다
  - 그래서 두 조건의 E-Rows가 정확히 같아졌다
- 오차의 방향은 값마다 반대였다
  - 쏠린 값은 과소추정, 일반 값은 과대추정
  - 한 컬럼 안에서 양쪽으로 동시에 틀린다

과소추정이 위험한 진짜 이유

- 단독 실행만 보면 큰 문제가 없어 보인다
  - 조회 결과 자체는 여전히 정확하게 나온다
  - 틀린 것은 결과가 아니라 옵티마이저의 예상이다
- 그런데 이 조건이 조인의 일부라면
  - 옵티마이저는 이 결과 집합이 매우 작다고 오판한다
  - 작은 쪽을 드라이빙으로 삼는 조인 방식이 유리해 보이게 된다
- ch09과의 연결
  - 실제로는 3,058건인데 300건으로 보고 NL Join을 선택한다면
  - Outer 행 수가 예상보다 많아 Inner 반복 탐색도 그만큼 늘어난다
- 그래서 카디널리티 추정 오류는 국소적 오차로 끝나지 않는다

> **참고:**
> 이 사례는 실습문제 3번과 직접 연결된다. ch09의 조인 실행 원리를 함께 놓고 설명하면, 통계 문제와 조인 문제가 별개가 아니라 하나의 사슬이라는 점이 드러난다.

8.6

복원과 SIZE AUTO

- FOR ALL COLUMNS SIZE AUTO — 자동 판단으로 원복
- 복원 판정 기준은 숫자의 일치가 아니다
- 통계 수집은 표본 추출 기반이라는 사실

히스토그램 복원

07. Tuning — 자동 판단에 맡겨 원래 상태로 되돌린다

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'MEDICAL_CLAIMS',
    METHOD_OPT => 'FOR ALL COLUMNS SIZE AUTO');  -- 자동 판단으로 원복
END;
/

-- USER_TAB_COL_STATISTICS 재확인
COLUMN_NAME   HISTOGRAM   NUM_BUCKETS
-----------   ---------   -----------
HOSP_ID       HYBRID              254

-- 원상복구 확인 완료
```

복원 후 결과

07. Tuning — 스큐를 반영한 추정으로 돌아왔는가

조건	E-Rows	A-Rows	추정 정확도
hosp_id=1 (스큐, 복원 후)	2,958	3,058	근접 (비율 약 0.97배)

숫자가 정확히 같지 않아도 복원됐다고 보는 근거

- 관측된 사실
  - 복원 후 hosp_id=1의 E-Rows는 2,958 — 원래 베이스라인은 2,795
  - 두 값이 완전히 같지는 않다
- 왜 같지 않은가
  - 통계 수집은 전체 데이터를 다 읽는 것이 아니라 표본을 추출해 추정하는 방식이 기본
  - 같은 데이터라도 표본이 매번 조금씩 달라질 수 있다
- 그럼 무엇을 근거로 판정하나
  - HISTOGRAM과 NUM_BUCKETS가 원래와 같은 종류의 통계로 돌아왔는가
  - E-Rows가 다시 값에 따라 달라지는가 — 균등 가정으로 되돌아가지 않았는가
  - 추정치가 실제값 근처로 돌아왔는가

참고:
복원 확인은 말이 아니라 값으로 한다. HISTOGRAM과 NUM_BUCKETS 두 컬럼을 캡처해 남겨 두는 것을 이 차시의 종료 절차로 삼는다.

세 상태 종합 비교

08. Benchmark — 통계만 바꿔 가며 관측한 결과

상태	hosp_id=1 E-Rows	hosp_id=500 E-Rows	두 값이 다른가?
히스토그램 있음 (원본)	2,795	280	다름 (스큐 반영)
히스토그램 제거 (SIZE 1)	300	300	동일 (균등 가정)
히스토그램 복원 (AUTO)	2,958	158	다시 다름

METHOD_OPT — 히스토그램 생성 제어

- METHOD_OPT은 통계 수집 시 히스토그램 생성 방식을 지정하는 파라미터
  - FOR COLUMNS 컬럼명 SIZE 1 — 해당 컬럼의 히스토그램을 없앤다
  - FOR ALL COLUMNS SIZE AUTO — Oracle이 컬럼별로 필요 여부를 판단한다
- 지정하지 않으면 어떻게 되나
  - METHOD_OPT을 생략하면 기본값인 FOR ALL COLUMNS SIZE AUTO가 적용된다
  - 이 스키마의 생성 스크립트가 METHOD_OPT을 명시하지 않았는데도 HOSP_ID에 히스토그램이 생긴 이유
- SIZE AUTO가 판단하는 기준
  - 컬럼의 데이터 분포를 살펴 스큐가 있다고 판단되는 컬럼에만 만든다
  - 값의 종류가 많고 일부에 크게 쏠린 컬럼일수록 생성될 가능성이 높다

8.7

히스토그램의 한계와 운영 판단

- 히스토그램도 공짜가 아니다 — 수집 비용과 파싱 비용
- 바인드 변수 Peeking — 계획이 고정되면 이점이 무력화된다
- 컬럼 간 상관관계 — 히스토그램으로는 풀리지 않는 문제

히스토그램도 공짜가 아니다

- 만들고 유지하는 데 비용이 든다
  - 통계 수집 시간이 더 걸린다
  - 저장 공간을 차지한다
  - 옵티마이저가 실행계획을 세울 때 참고할 데이터도 더 많아진다
- 모든 컬럼에 정밀한 히스토그램을 만들면
  - 컬럼이 많은 테이블에서는 통계 수집 자체가 느려진다
  - 매 파싱 시 참고할 정보가 늘어 파싱 비용이 늘어날 수 있다
- 값이 실제로 균등한 컬럼에는 사실상 낭비다
  - 있으나 없으나 추정 결과가 비슷하기 때문
- 그래서 Oracle의 기본 정책이 SIZE AUTO다
  - 정말 필요한 컬럼에만 만들도록 설계된 것

바인드 변수 Peeking과의 상충

- ch02에서 확인한 것
  - 바인드 변수를 쓰면 최초 하드 파싱 시점의 값 하나를 기준으로 계획이 만들어진다
  - 그 계획이 이후 다른 값에도 재사용된다 — 바인드 변수 Peeking
- 히스토그램의 이점이 어디서 막히나
  - 옵티마이저는 이번에 대입된 그 값 하나에 대해서만 히스토그램을 참조한다
  - 첫 실행이 hosp_id=1이었는데 다음이 hosp_id=500이어도 처음 계획을 재사용한다
  - 히스토그램이 아무리 정확해도 계획이 고정되면 이점이 무력화된다
- 정리하면
  - 리터럴로 매번 새로 파싱되는 경우 — 히스토그램이 최대한 활용된다
  - 바인드 변수로 계획이 고정되는 경우 — 효과가 제한적일 수 있다

히스토그램으로는 풀리지 않는 문제

- ch03 Q1의 오차를 다시 본다
  - CLAIM_TYPE과 TOTAL_AMT 두 컬럼의 조합 조건에서 발생한 추정 오차
- 히스토그램이 기록하는 것
  - 컬럼 하나의 값 분포
  - 두 컬럼이 동시에 어떤 값을 가지는 경향, 즉 상관관계는 기록하지 않는다
- 따라서
  - 두 컬럼 각각에 히스토그램이 있었더라도 옵티마이저는 두 조건을 독립으로 가정한다
  - 선택도를 곱하는 계산은 그대로 유지되고 오차도 그대로 남는다
- 원인의 이름이 다르다
  - 이 오차는 히스토그램 부재가 아니라 컬럼 간 상관관계를 반영하는 통계의 부재다
  - 단일 컬럼 분포 문제와 다중 컬럼 상관관계 문제는 서로 다른 종류다

통계 이상을 의심하는 진단 절차

- 1단계 E-Rows와 A-Rows의 괴리를 확인한다
  - 자릿수 수준으로 벌어져 있는 단계가 어디인지 찾는다
- 2단계 조건 값을 바꿔 가며 E-Rows가 반응하는지 본다
  - 값이 달라도 추정치가 고정이면 균등 가정이 적용되고 있는 것
- 3단계 USER_TAB_COL_STATISTICS로 통계 상태를 확인한다
  - HISTOGRAM과 NUM_BUCKETS를 함께 본다
- 4단계 조건의 형태를 확인한다
  - 단일 컬럼 등치 조건인가, 여러 컬럼의 조합 조건인가
  - 리터럴인가 바인드 변수인가
- 5단계 통계 수집 시점과 데이터 변경 시점을 대조한다
  - 대량 적재나 대량 갱신 이후 통계가 갱신됐는지 확인한다

통계 수집 시점이라는 축

- 통계는 수집 시점의 스냅숏이다
  - 옵티마이저는 지금의 데이터가 아니라 마지막으로 수집된 요약본을 본다
- 그래서 생기는 어긋남
  - 대량 적재나 대량 삭제 직후에는 통계와 실제가 크게 벌어질 수 있다
  - 데이터 변화 속도가 수집 주기보다 빠르면 어긋남이 상시화된다
- 반대 방향의 사고도 있다
  - 통계를 재수집한 순간 계획이 바뀌어 어제까지 잘 돌던 SQL이 느려지기도 한다
  - 코드도 데이터도 그대로인데 계획만 바뀌었다면 통계를 먼저 의심한다
- 운영에서 필요한 것
  - 통계를 언제 수집했는지 기록하고, 변경 전후의 계획을 남겨 두는 것

요약 (Summary)

- 히스토그램이 없는 컬럼의 등치 조건 E-Rows는 전체 행 수 ÷ NDV로 계산된다
- 이 계산은 균등 분포 가정 위에 서 있어 어떤 값을 조회하든 같은 값이 나온다
- 히스토그램이 있으면 값별 빈도를 반영해 스큐를 정확히 추정할 수 있다
- METHOD_OPT으로 히스토그램 생성 여부를 제어하고, SIZE AUTO는 Oracle이 스스로 판단한다
- 통계 상태는 USER_TAB_COL_STATISTICS의 HISTOGRAM과 NUM_BUCKETS로 확인한다
- 카디널리티 추정이 크게 틀리면 조인 방식 등 후속 판단까지 잘못될 위험이 있다
- 통계 수집은 표본 추출 기반이므로 재수집 때마다 수치가 미세하게 다를 수 있다
- 히스토그램은 단일 컬럼 분포만 다룬다 — 상관관계와 바인드 고정 문제는 별개다

실습 안내

- 실습문제 10문항 — ch08_문제.md
  - 1·2번 300의 계산 근거와 히스토그램이 차이를 만드는 원리
  - 3번 과소추정이 조인 쿼리로 확장될 때의 위험 — ch09 연계
  - 4번 NUM_BUCKETS가 1에서 254로 바뀐 것의 의미
  - 5번 다른 컬럼에도 SIZE AUTO가 히스토그램을 만들지 예측하기
  - 6번 수치가 완전히 같지 않아도 복원됐다고 판단하는 근거
  - 7번 모든 컬럼에 정밀한 히스토그램이 항상 좋지는 않은 이유
  - 8번 METHOD_OPT을 명시하지 않았는데 히스토그램이 생긴 이유
  - 9·10번 바인드 변수 Peeking과의 상충, ch03 Q1 사례 재검토
- 실습 종료 규칙 — 반드시 FOR ALL COLUMNS SIZE AUTO로 원상복구
  - 복원 확인은 HISTOGRAM과 NUM_BUCKETS 값으로 한다

조인·서브쿼리·집합·정렬

- SQLT ch09 · ch10 · ch11 · ch12
-

Oracle SQL 튜닝 과정 · ch09

JOIN 실행 원리

Nested Loops · Hash Join · Sort Merge · Driving Table · 힌트는 지시일 뿐 보장이 아니다 | Oracle Database 19c · 진료비청구심사 실습 스키마

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - 조인을 '두 집합에서 조건에 맞는 행을 짝지어 합치는 연산'으로 정의한다
  - Nested Loops Join의 처리 흐름을 Outer·Inner 반복 구조로 단계별로 설명한다
  - Hash Join의 처리 흐름을 Build·Probe 두 단계로 나누어 설명한다
  - Sort Merge Join의 정렬·병합 구조와 다른 두 방식과의 차이를 설명한다
  - Driving Table을 정의하고 실행계획의 Starts 컬럼으로 판별한다
  - NL Join의 비용이 'Outer 행 수 × Inner 1회 탐색 비용'임을 설명한다
  - Inner 조인 컬럼의 인덱스 유무가 NL 비용을 좌우하는 이유를 설명한다
  - 힌트가 적용되지 않을 수 있음을 알고 실행계획으로 검증하는 절차를 수행한다
  - 실행계획에 나타난 내부 뷰를 쿼리 변환의 흔적으로 읽는다
  - 인덱스 유무·힌트 유무에 따른 Buffers 변화를 근거로 조인 방식을 판단한다

이 장의 구성 (Agenda)

- 7.1 Why — 조인을 왜 따로 배우는가
- 7.2 Concept — JOIN의 기본 원리 · NL · Hash · Sort Merge · Driving Table
- 7.3 Oracle Internals — 조인 비용의 구조와 선택 기준
- 7.4 Example — HOSPITALS 와 MEDICAL_CLAIMS 예제 SQL
- 7.5 Execution Plan — 인덱스 없는 상태에서 힌트 강제 실측
- 7.6 Bad SQL — 힌트가 의도대로 안 먹힌 사례
- 7.7 Tuning — 인덱스를 만들면 힌트 없이도 NL이 선택됨
- 7.8 Benchmark — 다섯 번의 실행 종합 비교
- 7.9~7.10 실습 안내 · 요약 (Review)

9.1

조인을 왜 따로 배우는가

- 단일 테이블 접근에서 조인으로
- 조인 방식이 만드는 비용 차이
- 이 차시의 목표와 실습 소재

단일 테이블 접근에서 조인으로

- 지금까지는 단일 테이블 접근만 다뤘다
  - Full Scan / Index Scan — 한 테이블을 어떻게 읽을 것인가의 문제
  - 그러나 실무 SQL 대부분은 여러 테이블을 조인한다
- 조인에는 '어떻게 읽을지'에 더해 '어떻게 짝지을지'가 추가된다
  - 같은 결과라도 짝짓는 방식에 따라 비용이 크게 달라질 수 있다
  - 접근 경로 선택보다 조인 방식 선택이 더 큰 차이를 만드는 경우가 많다
- 조인 방식은 옵티마이저가 고르지만, 근거를 읽을 줄 알아야 한다
  - 잘못 골랐을 때 무엇을 바꿔야 하는지 판단하려면 원리를 알아야 한다
  - 이 장에서는 힌트로 방식을 강제해 보며 그 판단 과정을 관찰한다

이 차시의 목표와 실습 소재

- 목표 1 — 세 가지 조인 방식의 동작 원리를 단계로 설명할 수 있게 한다
- 목표 2 — 힌트로 방식을 강제했을 때 실제로 적용되는지 실행계획으로 확인한다
- 실습 소재
  - HOSPITALS(1,000건) 중 HOSP_ID <= 5 로 좁힌 5건
  - MEDICAL_CLAIMS(30만 건)와 HOSP_ID 로 조인
  - 인덱스 유무 · 힌트 유무를 바꿔 가며 반복 실측
- 이 차시는 실습용 인덱스를 만들고 지운다
  - IX_CH07_ 접두어로 만들고 종료 시 반드시 삭제한다
  - 정리 후 MEDICAL_CLAIMS 에 PK_MEDICAL_CLAIMS 만 남는지 확인한다

9.2

조인의 기본 원리와 세 가지 방식

- JOIN의 정의와 결정해야 할 두 가지
- Nested Loops · Hash · Sort Merge
- Driving Table — 조인을 시작하는 쪽

JOIN의 기본 원리

- 조인은 두 집합에서 조인 조건에 맞는 행끼리 짝지어 하나로 합치는 연산
  - 대상은 테이블일 수도 있고 앞 단계의 결과 집합일 수도 있다
  - 세 테이블 이상의 조인도 결국 두 집합씩 차례로 처리한다
- 조인에서 결정해야 할 것은 두 가지다
  - 어느 쪽을 먼저 읽을 것인가 — 조인 순서와 Driving Table
  - 짝을 어떤 방법으로 찾을 것인가 — 조인 방식
- Oracle이 쓰는 대표적인 조인 방식은 세 가지다
  - Nested Loops Join — 반복 탐색
  - Hash Join — 해시 테이블을 만들어 한 번 훑기
  - Sort Merge Join — 정렬한 뒤 병합

Nested Loops Join의 동작 흐름

- 정의 — Outer(드라이빙)의 각 행마다 Inner(드리븐)에서 맞는 행을 찾아 반복 결합
- 처리 흐름
  - ① Outer 테이블을 조건으로 걸러 결과 행을 하나씩 꺼낸다
  - ② 꺼낸 행의 조인 컬럼 값으로 Inner 테이블을 탐색한다
  - ③ 매칭된 행을 결합해 상위 오퍼레이션으로 넘긴다
  - ④ Outer 행이 남아 있으면 ②로 돌아가 반복한다
- 실행계획에서 확인하는 법
  - 조인 오퍼레이션 이름이 NESTED LOOPS 로 나타난다
  - Inner 쪽 단계의 Starts 값이 Outer 결과 행 수와 같아진다
- 결합된 행을 하나씩 바로 내보내는 구조라 첫 행이 빨리 나온다

Hash Join의 동작 흐름

- 정의 — 작은 쪽으로 해시 테이블을 메모리에 만든 뒤 큰 쪽을 한 번 훑어 매칭
- 처리 흐름
  - ① Build — 작은 쪽을 읽어 조인 컬럼을 해시해 메모리에 적재한다
  - ② Probe — 큰 쪽을 한 번 읽으며 같은 해시 함수로 대응 위치를 찾는다
  - ③ 같은 버킷 안에서 조인 컬럼 값이 실제로 같은 행만 결합한다
- 특징
  - 양쪽을 각각 한 번씩만 읽으므로 반복 탐색이 없다
  - 해시로 위치를 찾는 방식이라 등가 조인에만 쓸 수 있다
  - 해시 테이블이 메모리에 다 들어가지 않으면 디스크로 분할해 처리한다
- 실행계획에서는 HASH JOIN 으로 나타난다

Sort Merge Join의 동작 흐름

- 정의 — 양쪽을 조인 컬럼 기준으로 정렬한 뒤 한 번씩 훑어 병합
- 처리 흐름
  - ① 양쪽 집합을 각각 조인 컬럼으로 정렬한다 (이미 정렬돼 있으면 생략)
  - ② 정렬된 두 스트림을 앞에서부터 나란히 비교한다
  - ③ 값이 같으면 결합하고, 작은 쪽 스트림을 한 칸 전진시킨다
- 특징
  - 정렬 비용이 크지만, 정렬이 끝나면 각각 한 번씩만 읽는다
  - 부등호 조인에도 쓸 수 있다는 점이 Hash Join과 다르다
  - 인덱스로 이미 정렬된 순서를 얻으면 정렬 단계를 줄일 수 있다
- 이번 차시의 실측 대상은 NL과 Hash 두 가지다 — Sort Merge는 원리만 짚는다

Driving Table — 조인을 시작하는 쪽

- 정의 — 조인을 시작하는 기준이 되는 테이블
  - NL에서는 Outer 쪽, Hash Join에서는 Build 쪽에 해당한다
- 왜 중요한가
  - NL에서는 Driving 쪽 행 수가 그대로 Inner 반복 횟수가 된다
  - Hash Join에서는 Build 쪽 크기가 해시 테이블 메모리 사용량을 좌우한다
- 실행계획에서 찾는 법
  - 조인 오퍼레이션 바로 아래 두 자식 중 먼저 나오는 쪽이 Driving
  - Starts 컬럼을 보면 Driving 쪽은 작고 반복되는 쪽은 그보다 크다
- 일반적으로 먼저 걸러져 행 수가 적어진 쪽이 Driving으로 적합하다

9.3

조인 비용의 구조와 선택 기준

- NL Join 비용 — 곱셈 구조
- Hash Join 비용 — 덧셈 구조
- 힌트는 지시일 뿐 보장이 아니다

NL Join의 비용 구조

- 비용 ≈ Outer 쪽 행 수 × (Inner 쪽에서 한 번 찾는 비용)
  - 곱셈 구조라 Outer 행 수가 늘면 비용이 그에 비례해 늘어난다
- Inner 쪽 조인 컬럼에 인덱스가 있어야 '한 번 찾는 비용'이 저렴하다
  - 인덱스가 있으면 한 번의 탐색이 인덱스 몇 블록 읽기로 끝난다
  - 인덱스가 없으면 Outer 행 하나마다 Inner 테이블을 통째로 훑어야 한다
- 인덱스 없는 NL이 치명적인 이유
  - 테이블 전체 스캔이 Outer 행 수만큼 반복되는 구조가 된다
  - Outer가 조금만 커져도 감당하기 어려운 비용이 된다
- 그래서 NL은 '작은 Outer + 인덱스 있는 Inner'라는 조건이 붙는 기법이다

Hash Join의 비용 구조

- 비용 ≈ 두 테이블을 각각 한 번씩 읽는 비용 (Build 한 번 + Probe 한 번)
  - 곱셈이 아니라 덧셈 구조다
- Outer 쪽에 인덱스가 없어도 상대적으로 안정적이다
  - 반복 탐색이 없으므로 인덱스가 없다는 사실이 곧바로 재앙이 되지 않는다
  - 대신 Build 쪽을 담을 작업 영역 메모리가 필요하다
- 대량 데이터를 한꺼번에 처리하는 조인에 잘 맞는다
  - 전체 결과를 다 만들어야 하는 집계 쿼리에 유리하다
- 다만 첫 행이 나오려면 Build가 끝나야 한다
  - 앞쪽 몇 건만 빨리 보여 주는 화면 조회에는 불리할 수 있다

세 방식 비교와 선택 기준

- Nested Loops
  - 한쪽이 매우 작고 다른 쪽 조인 컬럼에 인덱스가 있을 때 유리
  - 첫 행을 빨리 내보내야 하는 온라인 조회에 적합
- Hash Join
  - 양쪽이 다 크거나 인덱스가 없을 때 유리 — 등가 조인 전용
  - 작업 영역 메모리를 쓰는 대신 반복 탐색이 없다
- Sort Merge Join
  - 정렬 순서를 이미 확보했거나 비등가 조인이 필요할 때 후보
- 절대적 규칙이 아니다
  - 최종 판단은 옵티마이저가 통계정보로 비용을 계산해서 내린다
  - 우리가 할 일은 옵티마이저가 좋은 판단을 하도록 조건을 갖추는 것

힌트는 지시일 뿐 보장이 아니다

- 힌트는 옵티마이저에게 전달하는 '지시'다
  - 문법이 맞아도 적용이 거부되면 아무 오류 없이 조용히 무시된다
- 옵티마이저는 실행 전에 쿼리를 내부적으로 변환한다
  - 뷰 병합, 서브쿼리 언네스팅, GROUP BY 위치 조정 등
  - 변환 후에는 힌트가 가리키던 별칭 조합이 최종 조인과 어긋날 수 있다
- 그래서 힌트를 걸고 나면 반드시 실행계획을 다시 확인해야 한다
  - 의도한 오퍼레이션 이름이 실제로 나타났는지 확인한다
- 7.6절에서 이 현상을 실측 사례로 확인한다

> **참고:**
> 이 과정의 모든 실행계획·Buffers·A-Time은 진료비청구심사 실습 DB에서 직접 캡처한 실측값이다.

9.4

실습 예제 SQL

- HOSPITALS 와 MEDICAL_CLAIMS 의 조인
- 조인 조건과 필터 조건
- GROUP BY가 남기는 복선

예제 SQL

병원 유형별 청구 건수 — HOSPITALS 와 MEDICAL_CLAIMS 의 등가 조인

```
SELECT h.hosp_type, COUNT(*)
FROM   hospitals h, medical_claims m
WHERE  h.hosp_id = m.hosp_id
AND    h.hosp_id <= 5
GROUP  BY h.hosp_type;

-- HOSPITALS(1,000건) 중 HOSP_ID <= 5 조건으로 5건만 남는다
-- MEDICAL_CLAIMS(30만 건)와 HOSP_ID 로 조인한다
-- 이 한 문장을 인덱스 유무 · 힌트 유무만 바꿔 가며 반복 실행한다
```

예제 쿼리 읽기

- 조인 조건 — h.hosp_id = m.hosp_id
  - 등가 조인이므로 NL · Hash · Sort Merge 세 방식이 모두 후보가 된다
- 필터 조건 — h.hosp_id <= 5
  - HOSPITALS 1,000건 중 5건만 남는다 — 한쪽이 매우 작아지는 상황
  - 교과서적으로는 NL의 Outer로 삼기 좋은 조건이다
- 집계 — GROUP BY h.hosp_type
  - 병원 유형별로 청구 건수를 센다
  - 이 GROUP BY가 뒤에서 힌트 무력화의 원인이 된다
- 조건만 바꿔 반복 실행한 결과를 7.5절부터 순서대로 확인한다

9.5

실행계획 — 인덱스 없는 상태

- 실측 설계 — 무엇을 바꿔 가며 볼 것인가
- Q1 베이스라인 — 힌트 없음
- Q3 — USE_HASH 강제

실측 설계 — 무엇을 바꿔 가며 볼 것인가

- 인덱스 없는 상태 (MEDICAL_CLAIMS 에 PK 만 있음)
  - Q1 — 힌트 없음 (베이스라인)
  - Q2 — USE_NL(h m) 강제
  - Q3 — USE_HASH(h m) 강제
- MEDICAL_CLAIMS(HOSP_ID) 에 인덱스를 만든 뒤
  - Q4 — USE_NL(h m) 강제
  - Q5 — 힌트 없음
- 비교 축은 두 개다
  - 힌트 유무·종류가 실행계획을 바꾸는가
  - 인덱스 유무가 실행계획을 바꾸는가
- SQL 본문과 데이터는 모든 실행에서 동일하다

Q1 실행계획 — 힌트 없음 (인덱스 없음)

베이스라인 — 옵티마이저의 자체 판단

```
| Id | Operation                              | Name           | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT                       |                |      2 |00:00:00.02 |    3093 |
| 1  |  HASH GROUP BY                         |                |      2 |00:00:00.02 |    3093 |
| 2  |   HASH JOIN                            |                |  15029 |00:00:00.02 |    3093 |
| 3  |    TABLE ACCESS BY INDEX ROWID BATCHED | HOSPITALS      |      5 |00:00:00.01 |       3 |
| 4  |     INDEX RANGE SCAN                   | PK_HOSPITALS   |      5 |00:00:00.01 |       2 |
| 5  |    TABLE ACCESS FULL                   | MEDICAL_CLAIMS |  15029 |00:00:00.02 |    3090 |

-- Plan hash value 619494044
```

Q1 실행계획 해석

- 조인 오퍼레이션은 HASH JOIN (Id 2)
  - 힌트를 주지 않았는데 옵티마이저가 스스로 Hash Join을 선택했다
  - 한쪽이 5건인데도 NL을 고르지 않았다는 점이 중요하다
- HOSPITALS 쪽 (Id 3~4)
  - PK_HOSPITALS 를 INDEX RANGE SCAN 해서 5건으로 좁혔다
  - Buffers 3 — 사실상 비용이 들지 않았다
- MEDICAL_CLAIMS 쪽 (Id 5)
  - TABLE ACCESS FULL — 조인 컬럼에 인덱스가 없으니 전체를 훑는 수밖에 없다
  - Buffers 3090 — 전체 3093 중 거의 전부를 차지한다
- 비용의 위치를 지목하면 개선 지점도 함께 드러난다

Q3 — USE_HASH 를 강제했더니

- Q3 은 USE_HASH(h m) 로 Hash Join을 명시적으로 지시한 것
- 결과 — Q1 과 완전히 동일한 계획
  - Plan hash value 619494044 까지 같다
  - 오퍼레이션 구성과 Buffers 모두 차이가 없다
- 해석
  - 옵티마이저가 원래도 Hash Join을 선택하고 있었다는 뜻이다
  - 힌트는 '이미 가고 있던 길'을 한 번 더 지시한 것에 지나지 않았다
  - 이 쿼리와 이 데이터 상황에서는 Hash Join이 실제로 비용상 최선이었다
- Plan hash value 가 같다는 것은 실행계획 구조가 같다는 뜻이다
  - 두 계획을 비교할 때 가장 빠르게 확인할 수 있는 지표다

9.6

Bad SQL — 힌트가 안 먹힌 사례

- Q2 — USE_NL 강제 시도
- 실행계획에 나타난 내부 뷰
- 힌트 검증 절차

Q2 — USE_NL 을 걸었지만

- 의도 — 인덱스가 없는 상태에서 NL을 강제하면 어떻게 되는지 보려 했다
  - 이론상 MEDICAL_CLAIMS 전체 스캔이 Outer 행 수만큼 반복될 수 있는 시도
  - 즉 의도적으로 나쁜 계획을 만들어 비용을 확인하려는 실험이었다
- 지시 — USE_NL(h m)
  - h 를 Outer, m 을 Inner 로 하는 Nested Loops 를 요구한 것
- 기대했던 실행계획
  - 조인 오퍼레이션 자리에 NESTED LOOPS 가 나타나야 한다
  - Buffers 는 Q1 보다 크게 늘어나야 한다
- 실제 결과는 다음 슬라이드에서 확인한다

Q2 실행계획 — USE_NL 강제 (인덱스 없음)

힌트를 걸었는데 조인 오퍼레이션이 바뀌지 않았다

```
| Id | Operation                           | Name           | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT                    |                |      2 |00:00:00.03 |    3093 |
| 1  |  HASH GROUP BY                      |                |      2 |00:00:00.03 |    3093 |
| 2  |   HASH JOIN                         |                |  15029 |00:00:00.03 |    3093 |
| 3  |    VIEW                             | VW_GBF_7       |      5 |00:00:00.01 |       3 |
| 4  |     TABLE ACCESS BY INDEX ROWID BATCHED| HOSPITALS      |      5 |00:00:00.01 |       3 |
| 5  |     INDEX RANGE SCAN                | PK_HOSPITALS   |      5 |00:00:00.01 |       2 |
| 6  |    TABLE ACCESS FULL                | MEDICAL_CLAIMS | 300K   |00:00:00.01 |    3090 |

-- USE_NL 을 명시했지만 Id 2 는 여전히 HASH JOIN 이다
```

Q2 실행계획 해석

- 조인 오퍼레이션은 여전히 HASH JOIN (Id 2)
  - USE_NL 을 명시했는데도 NESTED LOOPS 가 나타나지 않았다
- Buffers 도 Q1 과 동일하게 3,093
  - 힌트가 실행계획을 전혀 바꾸지 못했다는 뜻이다
- 달라진 점은 하나 — Id 3 에 VIEW VW_GBF_7 이 끼어 있다
  - HOSPITALS 접근 단계가 내부 뷰로 한 번 감싸였다
  - 이 뷰가 힌트 무력화의 단서다
- 힌트를 걸었는데 오류도 경고도 없었다는 점에 주목한다
  - 적용되지 않은 힌트는 아무 흔적 없이 조용히 사라진다

Q1 vs Q2 대조

힌트만 바꿨을 때 무엇이 같고 무엇이 달라졌는가

항목	Q1 (힌트 없음)	Q2 (USE_NL 강제)
Id 2 조인 오퍼레이션	HASH JOIN	HASH JOIN
HOSPITALS 를 감싸는 뷰	없음	VW_GBF_7
MEDICAL_CLAIMS 접근	TABLE ACCESS FULL	TABLE ACCESS FULL
Buffers (Id 0)	3093	3093
A-Time (Id 0)	00:00:00.02	00:00:00.03

```
-- 조인 방식과 Buffers 는 그대로 — 힌트는 계획을 바꾸지 못했다
-- 유일한 변화는 내부 뷰 VW_GBF_7 의 등장이다
```

왜 힌트가 무시됐나 — VW_GBF_7 의 정체

- VW_GBF_7 은 옵티마이저가 쿼리를 변환하며 만든 내부 뷰의 이름이다
  - GROUP BY 를 조인보다 먼저 처리하도록 쿼리를 바꾸는 과정에서 생성됐다
  - 그 과정에서 HOSPITALS 쪽 집합이 이 뷰로 감싸였다
- 힌트가 가리키는 대상이 어긋났다
  - USE_NL(h m) 은 원래 SQL 에 쓰인 별칭 h, m 을 기준으로 지정한 것이다
  - 변환된 쿼리에서 h 는 더 이상 최종 조인의 직접 참여자가 아니다
  - 별칭 조합이 대응되지 않아 힌트가 적용되지 못한 것으로 보인다
- 결과 — 옵티마이저의 기본 선택인 Hash Join 이 그대로 유지됐다
  - 실행계획에 낯선 VW_ 이름이 보이면 쿼리 변환을 의심한다

실무 교훈 — 힌트 검증 절차

- 힌트를 추가했다는 사실만으로는 아무것도 보장되지 않는다
- 힌트 적용 후 반드시 확인할 것
  - ① 의도한 오퍼레이션 이름이 실행계획에 실제로 나타났는가
  - ② Buffers 와 A-Rows 가 기대한 방향으로 움직였는가
  - ③ 계획에 낯선 내부 뷰가 끼어들지 않았는가
- 이번 사례는 운이 좋았던 경우다
  - 힌트가 무시된 덕분에 반복 전체 스캔이라는 최악을 피했다
  - 힌트가 그대로 적용되는 쿼리 형태였다면 성능이 크게 나빠졌을 수 있다

> **참고:**
> 적용되지 않은 힌트는 오류를 내지 않는다 — 실행계획을 다시 조회하는 것 외에는 확인할 방법이 없다.

9.7

Tuning — 인덱스를 만들면

- 조인 컬럼 인덱스 생성과 재실행
- Q4 — NESTED LOOPS 의 등장
- Q5 — 힌트 없이도 NL 이 선택됨

인덱스 생성과 재실행

조인 컬럼에 인덱스를 만들고 같은 쿼리를 다시 실행한다

```
-- MEDICAL_CLAIMS 의 조인 컬럼에 실습용 인덱스 생성
CREATE INDEX ix_ch07_hospid ON medical_claims(hosp_id);

-- Q4. 인덱스가 있는 상태에서 USE_NL 강제
SELECT /*+ USE_NL(h m) */ h.hosp_type, COUNT(*)
FROM   hospitals h, medical_claims m
WHERE  h.hosp_id = m.hosp_id
AND    h.hosp_id <= 5
GROUP  BY h.hosp_type;

-- 실습 종료 시 반드시 정리한다
DROP INDEX ix_ch07_hospid;
```

Q4 실행계획 — USE_NL 강제 (인덱스 있음)

같은 힌트, 같은 SQL — 인덱스만 추가한 결과

```
| Id | Operation                             | Name           | Starts | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT                      |                |      1 |      2 |00:00:00.01 |      44 |
| 1  |  HASH GROUP BY                        |                |      1 |      2 |00:00:00.01 |      44 |
| 2  |   NESTED LOOPS                        |                |      1 |  15029 |00:00:00.01 |      44 |
| 3  |    TABLE ACCESS BY INDEX ROWID BATCHED | HOSPITALS      |      1 |      5 |00:00:00.01 |       3 |
| 4  |     INDEX RANGE SCAN                  | PK_HOSPITALS   |      1 |      5 |00:00:00.01 |       2 |
| 5  |    INDEX RANGE SCAN                   | IX_CH07_HOSPID |      5 |  15029 |00:00:00.01 |      41 |

-- Plan hash value 872004878
```

Q4 실행계획 해석

- 이번엔 NESTED LOOPS 가 실제로 나타났다 (Id 2)
  - 같은 힌트, 같은 SQL 인데 결과가 달라졌다 — 바뀐 것은 인덱스뿐이다
- Inner 접근 방식이 바뀌었다
  - Q2 의 TABLE ACCESS FULL 이 INDEX RANGE SCAN (IX_CH07_HOSPID) 으로 바뀌었다
  - 이 단계의 Buffers 는 41
- 전체 Buffers 는 3,093 에서 44 로
  - 교안 기준 약 70배 줄어들었다
  - 줄어든 몫의 대부분은 MEDICAL_CLAIMS 전체 스캔이 사라진 것이다
- 힌트가 아니라 인덱스가 계획을 바꿨다는 점이 이 장의 결론이다

Starts 가 말해 주는 것

- Starts 는 해당 오퍼레이션이 몇 번 시작됐는지, 즉 몇 번 반복 실행됐는지를 나타낸다
- Q4 에서 Starts 열을 세로로 읽어 보자
  - Id 3~4 HOSPITALS — Starts 1, 한 번만 접근해 5건으로 좁혔다
  - Id 5 IX_CH07_HOSPID — Starts 5, 그 5건 각각에 대해 반복 탐색했다
- NL Join 의 정의가 숫자로 드러난 장면이다
  - 'Outer 각 행마다 Inner 를 찾는다'가 실행계획 수치로 그대로 확인됐다
- Driving Table 판정 근거로도 쓴다
  - Starts 가 1 인 쪽이 Outer(HOSPITALS), 5 인 쪽이 Inner(MEDICAL_CLAIMS)
- Outer 행 수가 늘면 Starts 도 늘고 비용도 그만큼 따라 늘어난다

Q5 — 힌트 없이도 NL 이 선택됐다

- Q5 는 인덱스가 있는 상태에서 힌트를 빼고 실행한 것이다
- 결과 — Q4 와 완전히 동일한 계획
  - Plan hash value 872004878 까지 같다
  - 적응형 계획(adaptive plan)이라는 안내 문구까지 동일하게 나타났다
- 해석 — 우연이 아니라 필연에 가깝다
  - 옵티마이저는 후보들의 비용을 계산해 가장 싼 쪽을 고른다
  - 인덱스가 생기면서 NL 의 예상 비용이 Hash Join 보다 낮아진 것이다
- 이번엔 힌트가 필요조차 없었다
  - 조건을 갖춰 주면 옵티마이저가 스스로 같은 결론에 도달한다

9.8

Benchmark — 실행 결과 종합

- 상황별 조인 방식과 Buffers
- 결정적 변수는 무엇이었나
- 조인 방식 선택 체크리스트

벤치마크 — 실행 결과 종합

힌트를 바꾼 경우와 인덱스를 만든 경우

```
+----------------------------------+----------------------------+---------+
| 상황                              | 조인 방식                   | Buffers |
+----------------------------------+----------------------------+---------+
| 인덱스 없음, 힌트 없음 (Q1)       | HASH JOIN                  |   3,093 |
| 인덱스 없음, USE_NL 강제 (Q2)     | HASH JOIN (힌트 미적용)     |   3,093 |
| 인덱스 없음, USE_HASH 강제 (Q3)   | HASH JOIN                  |   3,093 |
| 인덱스 있음, USE_NL 강제 (Q4)     | NESTED LOOPS               |      44 |
| 인덱스 있음, 힌트 없음 (Q5)       | NESTED LOOPS (자동 선택)    |      44 |
+----------------------------------+----------------------------+---------+

-- 위쪽 : 힌트를 무엇으로 바꿔도 결과가 같다
-- 아래쪽 : 인덱스가 생기자 힌트 유무와 무관하게 같다
```

벤치마크 해석 — 결정적 변수는 인덱스였다

- 힌트만 바꾼 경우(Q1·Q2·Q3)는 Buffers 가 모두 3,093 으로 같았다
  - 인덱스가 없는 동안에는 무엇을 지시해도 결과가 달라지지 않았다
- 인덱스를 만든 경우(Q4·Q5)는 힌트 유무와 무관하게 44 였다
  - 인덱스가 생기자 힌트가 없어도 NL 이 선택됐다
- 정리하면
  - 인덱스 유무가 힌트보다 훨씬 결정적인 변수였다
  - 힌트로 방식을 바꾸려 하기 전에 조건이 갖춰졌는지부터 확인한다
- 주의 — 이 결론은 이번 쿼리 형태에서 관측된 것이다
  - 다른 쿼리 형태에서는 힌트가 그대로 적용될 수도 있다

조인 방식 선택 체크리스트

- 먼저 확인할 것
  - 조인 조건이 등가인가 — 아니면 Hash Join 은 후보에서 빠진다
  - 어느 쪽이 조건으로 먼저 좁혀지는가 — 그쪽이 Driving 후보다
  - 반대쪽 조인 컬럼에 인덱스가 있는가 — 없으면 NL 은 위험하다
- 방식별 선택 신호
  - 좁혀진 쪽이 아주 작고 반대쪽에 인덱스가 있다 → Nested Loops
  - 양쪽이 다 크거나 인덱스가 없다 → Hash Join
  - 정렬 순서를 이미 확보했거나 비등가 조인이다 → Sort Merge Join
- 마지막 단계는 언제나 같다 — 실행계획으로 확인한다
  - 체크리스트는 가설을 세우는 도구이지 결론을 내리는 도구가 아니다

요약 (Summary)

- 조인은 두 집합에서 조건에 맞는 행을 짝지어 합치는 연산이고, 방식에 따라 비용이 달라진다
- NL 은 Outer 각 행마다 Inner 를 찾는 방식이라 비용이 'Outer 행 수 × 1회 탐색 비용'이다
- NL 은 Inner 조인 컬럼에 인덱스가 있고 Outer 가 작을 때 유리하다
- Hash Join 은 Build 한 번 + Probe 한 번으로 양쪽을 한 번씩만 읽어 상대적으로 안정적이다
- Sort Merge Join 은 정렬 후 병합하는 구조이며 비등가 조인에도 쓸 수 있다
- Driving Table 은 조인을 시작하는 쪽이며 실행계획의 Starts 로 판별할 수 있다
- 힌트는 지시일 뿐 보장이 아니다 — 쿼리 변환이 얽히면 조용히 무시될 수 있다
- 인덱스가 없을 때는 힌트를 바꿔도 Buffers 가 3,093 으로 같았고, 인덱스를 만들자 44 로 줄었다

실습 안내

- 실습문제 10문항 — ch09_문제.md
  - 1·6번 Q1 과 Q3, Q4 와 Q5 의 계획이 같아진 이유 — 우연인가 필연인가
  - 2·3번 내부 뷰를 근거로 한 힌트 무력화 추론과 힌트 검증 절차
  - 4·5번 인덱스가 NL 비용에 결정적인 이유와 Starts 값의 의미
  - 7번 Outer 조건을 HOSP_ID<=500 으로 넓혔을 때의 Buffers 변화 예측
  - 8·9번 Hash Join 의 안정성, '느리면 무조건 USE_NL' 통념 반박
  - 10번 Q4·Q5 에서 Outer 와 Inner 지목 및 판단 근거 서술
- 실습 후 정리 — 반드시 확인할 것
  - DROP INDEX 로 IX_CH07_ 접두어 인덱스를 삭제한다
  - MEDICAL_CLAIMS 에 PK_MEDICAL_CLAIMS 만 남았는지 확인한다

Oracle SQL 튜닝 과정 · ch10

Subquery / EXISTS / IN

스칼라 서브쿼리 캐싱 · 인라인 뷰 · 상관 서브쿼리 · Unnesting과 View Merging

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - 스칼라 서브쿼리·인라인 뷰·상관 서브쿼리를 구분해 설명한다
  - 실행계획의 Starts 컬럼을 그 단계가 실제로 실행된 횟수로 읽는다
  - 스칼라 서브쿼리 캐싱의 원리를 설명하고 Starts 값으로 확인한다
  - 참조 값의 distinct 개수가 캐싱 효과를 좌우한다는 점을 설명한다
  - 'SELECT 절 서브쿼리는 N+1처럼 느리다'는 통념을 실측으로 검증한다
  - 상관 서브쿼리가 VW_SQ_1 형태의 뷰와 조인으로 변환되는 과정을 읽는다
  - 변환된 계획이 원본 테이블을 몇 번 읽는지 세어 비용 차이를 설명한다
  - 인라인 뷰의 바깥 WHERE가 HAVING과 동등하게 처리되는 현상을 설명한다
  - 뷰 병합이 언제 안전하지 않을 수 있는지 판단 기준을 세운다
  - 서브쿼리 성능을 문법 형태가 아니라 물리적 실행 구조로 판단한다

이 장의 구성 (Agenda)

- 8.1 Why — ch01 이후, 서브쿼리의 나머지 얼굴들
- 8.2 Concept — 스칼라 서브쿼리 · 인라인 뷰 · 상관 서브쿼리
- 8.3 Oracle Internals — 스칼라 서브쿼리 캐싱과 내부 변환
- 8.4~8.5 Example / Execution Plan — 스칼라 서브쿼리 vs JOIN 실측
- 8.6 Bad SQL — 상관 서브쿼리가 항상 저렴하지는 않다
- 8.7 Tuning — 인라인 뷰의 WHERE는 HAVING과 같아지는가
- 8.8~8.9 Benchmark / Practice — 실측 요약과 실습 안내
- 8.10 Review — 이 장의 결론

10.1

ch01 이후, 서브쿼리의 나머지 얼굴들

- ch01이 세운 원칙
- 서브쿼리는 한 가지가 아니다
- 이 차시가 새로 다루는 세 패턴

ch01이 세운 원칙

- ch01의 결론 — 결과가 같은 여러 SQL 중에서 실측으로 고른다
  - IN·EXISTS·JOIN 세 SQL은 결과가 완전히 같았지만 비용은 같지 않았다
  - 어느 문법이 빠르다는 통념이 아니라 실행계획과 Buffers가 판단 근거였다
- 그때 확인한 또 하나의 사실 — 옵티마이저는 SQL을 그대로 실행하지 않는다
  - IN과 EXISTS가 둘 다 HASH JOIN RIGHT SEMI라는 같은 형태로 변환됐다
  - 작성한 문법과 실제 실행 구조 사이에 변환 단계가 하나 끼어 있다
- 이 장은 그 원칙을 서브쿼리의 나머지 쓰임새로 확장한다
  - 문법이 달라도 같은 구조로 수렴하는 경우, 반대로 갈라지는 경우를 함께 본다
  - 판단 기준은 ch01과 동일하다 — 실측된 실행계획과 자원 소비량

서브쿼리는 한 가지가 아니다

- 서브쿼리를 어디에 놓느냐에 따라 역할이 완전히 달라진다
  - SELECT 절 — 이 행에 붙일 값 하나를 가져온다 (스칼라 서브쿼리)
  - FROM 절 — 중간 결과 집합을 만들어 테이블처럼 쓴다 (인라인 뷰)
  - WHERE 절 — 조건 판정에 쓴다 (IN·EXISTS·비교 연산자)
- ch01에서 다룬 것은 WHERE 절의 존재 여부 확인 한 가지였다
  - IN·EXISTS·JOIN+DISTINCT는 모두 있느냐 없느냐를 묻는 패턴이었다
- 이 차시는 나머지 세 패턴을 실측으로 다룬다
  - 스칼라 서브쿼리 — SELECT 절에 값 하나를 붙이는 것
  - 인라인 뷰 — FROM 절에 서브쿼리를 넣는 것
  - 상관 서브쿼리 — 바깥 쿼리의 값을 참조하는 것

이 차시의 범위와 전제

- 이번 차시는 DDL 변경이 없다
  - 기존 PK만 사용하며 인덱스를 새로 만들지 않는다
  - 인덱스나 통계를 임의로 바꾸면 교안의 실행계획이 재현되지 않는다
- 다루는 세 가지 패턴과 각각의 실측 사례
  - 스칼라 서브쿼리 — 청구에 병원명을 붙이는 Q1과 조인으로 붙이는 Q2
  - 상관 서브쿼리 — 자신이 속한 병원의 평균보다 큰 청구를 찾는 Q3
  - 인라인 뷰 — 집계 결과를 걸러내는 Q4와 HAVING으로 쓴 Q5
- 판단 지표는 ch01과 같다
  - 실행계획의 Operation 구조, Starts, A-Rows, Buffers를 함께 본다
  - 이번 장에서는 특히 Starts 컬럼이 결정적인 단서로 쓰인다

> **참고:**
> 이 과정의 모든 실행계획·Starts·Buffers 값은 진료비청구심사 실습 DB에서 직접 캡처한 실측값이다.

10.2

스칼라 서브쿼리 · 인라인 뷰 · 상관 서브쿼리

- 스칼라 서브쿼리란
- 인라인 뷰란
- 상관 서브쿼리란
- 서브쿼리와 JOIN의 관계

스칼라 서브쿼리 (Scalar Subquery)

- 정의 — SELECT 절에서 단일 값 하나를 반환하는 서브쿼리
  - 이 행에 관련된 다른 테이블의 값 하나를 붙여 오는 용도로 흔히 쓰인다
  - 이번 장의 Q1이 청구 행에 병원명을 하나씩 붙이는 전형적인 예다
- 문법상의 제약
  - 반드시 값 하나를 반환해야 한다 — 여러 행이 나오면 실행 오류가 난다
  - 조건에 맞는 행이 없으면 NULL이 반환된다 — 외부 조인과 성질이 비슷하다
- 읽는 사람 입장에서의 장점
  - FROM 절을 건드리지 않으므로 원본 쿼리의 골격이 흐트러지지 않는다
  - 값을 하나 더 붙이는 요구사항이 늘어도 SELECT 절만 수정하면 된다
- 성능에 대한 통념 — 행마다 실행되니 느릴 것이라는 예상
  - 이 예상이 왜 항상 맞지는 않은지가 8.3절과 8.5절의 주제다

인라인 뷰 (Inline View)

- 정의 — FROM 절에 괄호로 감싼 서브쿼리를 테이블처럼 사용하는 것
  - 중간 결과 집합을 하나 만들어 바깥 쿼리가 그것을 다시 조회한다
  - 이번 장의 Q4가 병원별 청구 건수를 만든 뒤 바깥에서 거르는 예다
- 주로 쓰이는 상황
  - 집계한 결과를 다시 필터링하거나 다른 테이블과 조인해야 할 때
  - 여러 단계의 가공을 단계별로 나눠 읽기 쉽게 쓰고 싶을 때
- 읽는 사람 입장에서의 장점
  - 가공 단계가 괄호로 분리되어 각 단계의 의도가 눈에 보인다
  - 뷰 객체를 새로 만들지 않고도 뷰와 같은 효과를 얻는다
- 성능에 대한 의문 — 중간 집합을 실제로 만들어 놓고 쓰는가
  - 옵티마이저가 이를 어떻게 처리하는지가 8.7절의 주제다

상관 서브쿼리 (Correlated Subquery)

- 정의 — 서브쿼리 내부에서 바깥 쿼리의 컬럼을 참조하는 서브쿼리
  - 바깥 쿼리의 행이 바뀌면 서브쿼리 결과도 달라질 수 있다
  - ch01에서 본 EXISTS 서브쿼리도 상관 서브쿼리의 한 예였다
- 비상관 서브쿼리와의 차이
  - 비상관 — 바깥과 무관하게 독립적으로 한 번 계산하면 되는 서브쿼리
  - 상관 — 바깥 값이 입력으로 들어가야 계산이 완성되는 서브쿼리
- 위치와 무관한 분류라는 점에 주의
  - SELECT 절에 있어도, WHERE 절에 있어도 상관 서브쿼리일 수 있다
  - 이번 장의 Q1은 스칼라 서브쿼리이면서 동시에 상관 서브쿼리다
- 문법이 시사하는 실행 그림 — 바깥 행마다 반복 실행
  - 이 그림이 실제 실행과 얼마나 다른지가 이 장의 관전 포인트다

서브쿼리와 JOIN의 관계 — 내부 변환

- 옵티마이저는 서브쿼리를 그 모양 그대로 실행하지 않는 경우가 많다
  - 많은 경우 조인 형태로 내부 변환한다 — 이를 unnesting이라 부른다
  - 중첩된 구조를 풀어 평평한 조인으로 만든다는 뜻이다
- ch01에서 이미 그 증거를 봤다
  - IN과 EXISTS가 둘 다 HASH JOIN RIGHT SEMI로 변환됐다
  - 서로 다른 문법이 같은 물리 구조로 수렴한 사례였다
- 변환이 남기는 흔적 — 계획에 등장하는 내부 뷰 이름
  - VW_NSO_1, VW_SQ_1 같은 이름은 옵티마이저가 만든 내부 뷰다
  - 원본 SQL에 없던 이름이 계획에 보이면 변환이 일어났다는 신호다
- 따라서 판단은 작성한 문법이 아니라 변환된 계획을 보고 해야 한다
  - 문법만 보고 비용을 예단하면 거의 항상 틀린다

10.3

Oracle Internals — 스칼라 서브쿼리 캐싱

- 문법과 실행의 괴리
- distinct 개수가 정하는 캐싱 효과
- N+1 통념 점검
- 집계 비교 상관 서브쿼리의 변환

스칼라 서브쿼리 캐싱 — 문법과 실행의 괴리

- 문법이 그리는 그림 — 행마다 한 번씩 실행
  - SELECT 절에 있으니 결과 행 수만큼 서브쿼리가 돌 것처럼 보인다
  - 이 그림 때문에 스칼라 서브쿼리는 N+1과 같다는 통념이 생겼다
- 실제 동작 — 같은 입력 값의 결과를 캐시해 재사용한다
  - 이미 계산한 입력 값이 다시 들어오면 서브쿼리를 실행하지 않는다
  - 캐시에서 값을 즉시 가져오므로 테이블 접근 자체가 발생하지 않는다
- 실행계획에서 확인하는 방법 — Starts 컬럼
  - Starts는 그 오퍼레이션이 실제로 시작된 횟수를 보여 준다
  - 결과 행 수보다 Starts가 훨씬 작다면 캐싱이 작동한 것이다
- 이 캐시는 한 SQL 실행 안에서만 유지된다
  - 버퍼 캐시나 세션을 넘나드는 결과 캐시와는 다른 개념이다

distinct 개수가 정하는 캐싱 효과

- 캐싱 효과의 크기를 정하는 것은 행 수가 아니라 참조 값의 종류 수다
  - 교안의 예 — 행이 15,029개여도 그 안의 HOSP_ID가 1~5의 5종류뿐이라면
  - 서브쿼리는 이론상 최대 5번만 실제로 실행되고 나머지는 캐시에서 온다
- 반대로 distinct 값이 많아지면 효과는 줄어든다
  - 값의 종류가 500이면 실제 실행 횟수도 이론상 최대 500까지 늘어난다
  - 캐시 항목이 값의 종류만큼 쌓이는 구조이기 때문이다
- 그래서 판단 기준은 단순하다
  - 참조 컬럼의 distinct 개수가 전체 행 수에 비해 작을수록 유리하다
  - 코드 테이블·분류 코드·상태 값처럼 종류가 적은 컬럼이 가장 잘 맞는다
- 반대로 거의 유일한 값을 참조하면 캐싱은 사실상 도움이 되지 않는다

'SELECT 절 서브쿼리는 느리다'는 통념 점검

- 통념의 논리 구조
  - 서브쿼리가 N번 반복된다 → 반복 비용이 누적된다 → 그래서 느리다
  - 이 논리는 N이 결과 행 수와 같다는 전제 위에 서 있다
- 캐싱이 그 전제를 무너뜨린다
  - 실제 반복 횟수는 행 수가 아니라 distinct 참조 값의 개수로 줄어든다
  - 종류가 적으면 반복 자체가 사실상 무의미할 정도로 적어진다
- 따라서 통념은 틀린 것이 아니라 조건이 빠진 것이다
  - 참조 값의 종류가 많을 때는 통념이 여전히 유효할 수 있다
  - 종류가 적을 때는 성립하지 않는다 — 8.5절에서 실측으로 확인한다
- 결론 — 통념을 버리는 대신 통념이 성립하는 조건을 기억한다

집계와 비교하는 상관 서브쿼리의 변환

- 자주 쓰이는 패턴 — 그룹 평균보다 큰 행 찾기
  - 바깥 행의 그룹 키로 그룹 평균을 구해 그 행의 값과 비교하는 형태
  - 문법만 보면 행마다 평균을 다시 계산하는 것처럼 보인다
- 옵티마이저의 처리 — 미리 계산한 뷰와의 조인으로 변환
  - 그룹별 집계를 한 번에 계산해 내부 뷰로 만든 뒤 원본과 조인한다
  - 이 역시 unnesting의 한 형태다
- 변환의 좋은 점
  - 행마다 반복 실행하는 최악의 시나리오는 피할 수 있다
- 변환이 남기는 문제
  - 집계를 위한 읽기와 원본을 위한 읽기가 각각 필요해질 수 있다
  - 같은 테이블을 여러 번 읽는 구조가 되면 변환 자체가 비용이 된다

10.4~10.5

스칼라 서브쿼리 vs JOIN — 실측 비교

- 실측 소재 — 청구에 병원명 붙이기
- Q1 스칼라 서브쿼리 / Q2 JOIN
- 두 실행계획 읽기
- Starts와 Buffers가 말해 주는 것

실측 소재 — 청구에 병원명 붙이기

- 요구사항 — 청구 목록에 병원 이름을 함께 보여 준다
  - MEDICAL_CLAIMS에서 청구 번호와 총액을 가져온다
  - 각 청구의 HOSPITALS 병원명을 한 컬럼으로 붙인다
  - 조건은 HOSP_ID가 5 이하인 청구로 한정한다
- 두 가지 작성 방법 — 결과는 완전히 같다
  - Q1 — SELECT 절에 스칼라 서브쿼리를 넣어 병원명을 가져온다
  - Q2 — FROM 절에 HOSPITALS를 넣고 조인으로 병원명을 가져온다
- 이 소재를 고른 이유
  - HOSP_ID가 5 이하이므로 참조 값의 종류가 5로 매우 적다
  - 캐싱 효과가 가장 잘 드러나는 조건이다
- 두 쿼리의 결과 행은 15,029건으로 동일하다

Q1. 스칼라 서브쿼리로 병원명 붙이기

SELECT 절의 서브쿼리가 바깥 행의 hosp_id를 참조한다 — 스칼라이면서 상관 서브쿼리

```
-- Q1. 스칼라 서브쿼리로 병원명 붙이기
SELECT c.claim_id, c.total_amt,
       (SELECT h.hosp_name FROM hospitals h WHERE h.hosp_id = c.hosp_id) hosp_name
FROM medical_claims c
WHERE c.hosp_id <= 5;

-- FROM 절에는 medical_claims 하나뿐이다 – 골격이 단순하게 유지된다
-- 괄호 안 서브쿼리는 바깥의 c.hosp_id를 입력으로 받는다
-- 문법상으로는 결과 행마다 한 번씩 실행되는 것처럼 보인다
-- 실제 실행 횟수는 실행계획의 Starts 컬럼에서 확인한다
```

Q2. JOIN으로 병원명 붙이기

결과는 Q1과 완전히 같다 — 15,029건. 문법 골격만 다르다

```
-- Q2. JOIN으로 병원명 붙이기
SELECT c.claim_id, c.total_amt, h.hosp_name
FROM medical_claims c, hospitals h
WHERE c.hosp_id = h.hosp_id AND c.hosp_id <= 5;

-- FROM 절에 두 테이블이 들어가고 조인 조건이 WHERE로 내려왔다
-- SELECT 절은 컬럼을 그대로 나열하기만 한다
-- 서브쿼리가 없으므로 반복 실행이라는 그림 자체가 없다
-- 코드 리뷰에서 Q1을 이 형태로 바꾸라고 권하는 경우가 많다
```

Q1 실행계획 — 스칼라 서브쿼리

결과 행은 15,029건. Id 1·2의 Starts 컬럼을 먼저 본다

```
| Id | Operation                    | Name           | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT             |                |      1 |        |  15029 |00:00:00.02 |    4081 |
| 1  |  TABLE ACCESS BY INDEX ROWID| HOSPITALS      |      5 |      1 |      5 |00:00:00.01 |      13 |
| 2  |   INDEX UNIQUE SCAN          | PK_HOSPITALS   |      5 |      1 |      5 |00:00:00.01 |       8 |
| 3  |  TABLE ACCESS FULL           | MEDICAL_CLAIMS |      1 |  15283 |  15029 |00:00:00.02 |    4081 |

-- Id 1, 2 : HOSPITALS 접근 — Starts가 5다
-- Id 3    : MEDICAL_CLAIMS 전체 스캔 — Starts는 1이다
-- 결과가 15,029건인데 HOSPITALS 접근은 5번뿐이다
```

Q1 실행계획 읽는 법

- Id 3 — MEDICAL_CLAIMS TABLE ACCESS FULL
  - Starts 1 — 원본 테이블을 한 번 전체 스캔한다
  - A-Rows 15029 — 조건을 통과한 청구 행이 결과 건수와 같다
  - E-Rows 15283 — 옵티마이저의 추정치로 실제와 큰 차이는 없다
- Id 1·2 — HOSPITALS 접근 (인덱스로 한 건씩 찾는 형태)
  - INDEX UNIQUE SCAN으로 PK_HOSPITALS를 타고 한 건을 찾는다
  - Starts 5 — 이 단계가 실제로 다섯 번만 실행됐다
  - A-Rows 5 — 다섯 번의 실행이 각각 한 건씩을 돌려줬다
- Buffers 배분 — 전체 4,081 가운데 HOSPITALS 몫은 13
  - 비용의 거의 전부가 MEDICAL_CLAIMS 전체 스캔에서 발생했다
  - 서브쿼리 쪽 비용은 전체에서 차지하는 비중이 매우 작다

Q2 실행계획 — JOIN

결과 행 15,029건으로 Q1과 동일. 구조는 HASH JOIN 한 덩어리다

```
| Id | Operation                           | Name           | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT                    |                |      1 |        |  15029 |00:00:00.02 |    4083 |
| 1  |  HASH JOIN                          |                |      1 |     76 |  15029 |00:00:00.01 |    4083 |
| 2  |   TABLE ACCESS BY INDEX ROWID BATCHED| HOSPITALS      |      1 |      5 |      5 |00:00:00.01 |       3 |
| 3  |    INDEX RANGE SCAN                 | PK_HOSPITALS   |      1 |      5 |      5 |00:00:00.01 |       2 |
| 4  |   TABLE ACCESS FULL                 | MEDICAL_CLAIMS |      1 |  15283 |  15029 |00:00:00.01 |    4080 |

-- Id 2, 3 : HOSPITALS를 한 번에 훑어 해시 테이블을 만든다 — Starts 1
-- Id 4    : MEDICAL_CLAIMS 전체 스캔 — Starts 1
-- 모든 단계의 Starts가 1이다 — 반복이라는 개념 자체가 없다
```

Q2 실행계획 읽는 법 — Q1과의 대비

- 구조의 차이 — 반복이 있는가 없는가
  - Q1은 HOSPITALS 접근이 다섯 번 반복된 구조였다
  - Q2는 모든 단계의 Starts가 1인 한 번짜리 해시 조인이다
- HOSPITALS를 읽는 방식도 다르다
  - Q1 — INDEX UNIQUE SCAN으로 한 건씩 찾아 다섯 번 반복
  - Q2 — INDEX RANGE SCAN으로 다섯 건을 한 번에 훑는다
- 그런데 비용은 거의 같다
  - Q1의 HOSPITALS 몫 Buffers 13, Q2는 3 — 절대량이 둘 다 작다
  - MEDICAL_CLAIMS 전체 스캔이 4,080 안팎으로 비용의 대부분을 차지한다
- 전체 Buffers는 Q1이 4,081, Q2가 4,083이다
  - 구조가 달라도 지배적인 비용 요인이 같으면 총량은 비슷해진다

Starts=5가 말해 주는 것

- 관측 사실 — 결과는 15,029건인데 HOSPITALS 접근은 5번뿐이다
  - 문법대로라면 15,029번 실행됐어야 할 자리다
  - 실제로는 HOSP_ID의 값 종류인 5만큼만 실행됐다
- 원인 — 8.3절의 스칼라 서브쿼리 캐싱
  - 같은 HOSP_ID가 다시 들어오면 이미 계산한 병원명을 캐시에서 꺼낸다
  - 값이 1부터 5까지 다섯 종류뿐이라 실제 실행도 다섯 번으로 끝난다
- 이 관측이 중요한 이유
  - 문법이 그리는 실행 그림과 실제 실행이 다르다는 직접 증거다
  - Starts 하나만 봐도 캐싱 작동 여부를 판별할 수 있다
- 점검 습관 — 결과 행 수와 Starts를 나란히 놓고 본다
  - 둘의 격차가 크면 캐싱이나 다른 내부 변환이 개입한 것이다

Buffers 4,081 vs 4,083 — 통념의 반증

- 실측 결과 — 두 방식의 Buffers가 거의 같다
  - Q1 스칼라 서브쿼리 4,081 / Q2 JOIN 4,083
  - 결과 행은 15,029건으로 완전히 동일한 상태에서의 비교다
- 통념이 예측한 것 — 스칼라 서브쿼리가 크게 불리할 것
  - 반복 실행이 누적되어 조인보다 훨씬 비쌀 것이라는 예측이었다
  - 실측은 그 예측을 지지하지 않는다
- 통념이 성립하지 않은 이유
  - 캐싱으로 실제 반복이 5회로 줄어 누적 비용이 생기지 않았다
  - 두 방식 모두 원본 테이블을 한 번만 읽는 구조로 수렴했다
- 올바른 결론 — 스칼라 서브쿼리가 항상 안전하다는 뜻은 아니다
  - 참조 값의 종류가 적다는 조건이 붙은 결론이다
  - 조건이 달라지면 결론도 달라질 수 있으므로 매번 실측한다

10.6

Bad SQL — 상관 서브쿼리가 항상 저렴하지는 않다

- 소재 — 자기 병원 평균보다 큰 청구
- VW_SQ_1 변환 읽기
- 두 번의 Full Scan
- Buffers 6,612가 나온 이유

소재 — 자기 병원 평균보다 큰 청구 찾기

- 요구사항 — 각 청구가 자신이 속한 병원의 평균 총액보다 큰지 판정한다
  - 비교 기준이 행마다 다르다 — 병원이 다르면 평균도 다르다
  - 조건은 앞 절과 같이 HOSP_ID가 5 이하로 한정한다
- 자연스러운 작성 방식 — WHERE 절의 상관 서브쿼리
  - 서브쿼리가 바깥 행의 HOSP_ID를 받아 그 병원의 평균을 구한다
  - 바깥 행의 총액과 그 평균을 비교한다
- 실무에서 매우 흔한 패턴
  - 평균 이상, 최댓값과 동일, 그룹 상위 등 여러 요구사항이 같은 모양이다
  - 문법이 요구사항을 거의 그대로 옮겨 놓아 읽기도 쉽다
- 관전 포인트 — 옵티마이저는 이 문법을 어떤 구조로 바꾸는가

Q3. 그룹 평균과 비교하는 상관 서브쿼리

서브쿼리가 바깥 행의 hosp_id를 참조해 그 병원의 평균 총액을 구한다

```
-- Q3. 자신이 속한 병원의 평균보다 총액이 큰 청구
SELECT c.claim_id, c.hosp_id, c.total_amt
FROM medical_claims c
WHERE c.hosp_id <= 5
AND   c.total_amt > (SELECT AVG(c2.total_amt) FROM medical_claims c2 WHERE c2.hosp_id = c.hosp_id);

-- 서브쿼리가 참조하는 테이블이 바깥과 같은 medical_claims다
-- 바깥 행의 c.hosp_id가 서브쿼리의 입력으로 들어간다
-- 문법상으로는 행마다 평균을 다시 계산하는 것처럼 보인다
-- 실제로는 옵티마이저가 이 구조를 다른 형태로 변환한다
```

Q3 실행계획 — 상관 서브쿼리

원본 SQL에 없던 VW_SQ_1이 등장한다 — 옵티마이저가 만든 내부 뷰

```
| Id | Operation            | Name           | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT     |                |      1 |        |   6462 |00:00:00.03 |    6612 |
| 1  |  HASH JOIN           |                |      1 |   7951 |   6462 |00:00:00.03 |    6612 |
| 2  |   VIEW               | VW_SQ_1        |      1 |     50 |      5 |00:00:00.02 |    3090 |
| 3  |    HASH GROUP BY     |                |      1 |     50 |      5 |00:00:00.02 |    3090 |
| 4  |     TABLE ACCESS FULL| MEDICAL_CLAIMS |      1 |  15283 |  15029 |00:00:00.01 |    3090 |
| 5  |   TABLE ACCESS FULL  | MEDICAL_CLAIMS |      1 |  15283 |  15029 |00:00:00.01 |    3518 |

-- Id 4 : MEDICAL_CLAIMS 전체 스캔 -> HASH GROUP BY -> VW_SQ_1
-- Id 5 : MEDICAL_CLAIMS 전체 스캔 (원본 행 확보)
-- 같은 테이블에 대한 TABLE ACCESS FULL이 두 번 나타난다
```

Q3 실행계획 읽는 법

- Id 4 → Id 3 → Id 2 — 기준값을 만드는 갈래
  - MEDICAL_CLAIMS를 전체 스캔해 15,029건을 읽는다
  - HASH GROUP BY로 HOSP_ID별 평균을 계산해 5건으로 줄인다
  - 그 결과를 VW_SQ_1이라는 내부 뷰로 감싼다 — Buffers 3,090
- Id 5 — 비교 대상 행을 가져오는 갈래
  - MEDICAL_CLAIMS를 다시 전체 스캔해 15,029건을 읽는다 — Buffers 3,518
- Id 1 — 두 갈래를 HASH JOIN으로 결합한다
  - HOSP_ID를 연결 고리로 각 행에 자기 병원 평균을 붙인다
  - 조건을 통과한 6,462건이 최종 결과가 된다
- 모든 단계의 Starts가 1이다 — 반복 실행은 일어나지 않았다
  - 옵티마이저가 상관 서브쿼리를 조인 구조로 풀어냈다는 뜻이다

두 번의 Full Scan은 각각 무엇을 하나

- Id 4의 Full Scan — 비교 기준을 만들기 위한 스캔
  - HOSP_ID별 AVG(total_amt)를 미리 한 번에 계산하려고 읽는다
  - 상관 서브쿼리를 그룹별 평균 미리 계산 형태로 변환한 결과다
  - 읽은 15,029건이 집계를 거쳐 5건으로 압축된다
- Id 5의 Full Scan — 비교 대상 행을 가져오기 위한 스캔
  - 원래 바깥 쿼리가 필요로 하던 청구 행 자체를 읽는다
  - 조건 HOSP_ID <= 5에 해당하는 15,029건을 확보한다
- 두 스캔은 목적이 서로 다르다
  - 하나는 기준값을 구하기 위한 것, 다른 하나는 대상을 얻기 위한 것
  - 그래서 하나로 합칠 수 없고 각각 읽어야 한다
- 결과적으로 같은 테이블을 두 번 읽는 구조가 만들어진다

Buffers 6,612의 분해

- 실측된 전체 Buffers는 6,612다
  - Id 4 계열(평균 계산용 스캔) — 3,090
  - Id 5(원본 조회용 스캔) — 3,518
  - 두 값을 더하면 약 6,608로 실측 6,612와 거의 일치한다
- 앞 절과의 비교
  - Q1 4,081 / Q2 4,083 — 두 방식 모두 원본을 한 번만 읽었다
  - Q3 6,612 — 원본을 두 번 읽는 구조라 그만큼 커졌다
- 해석 — 비용의 차이는 문법이 아니라 스캔 횟수에서 나왔다
  - 테이블 스캔이 비용의 대부분을 차지하는 상황이었다
  - 그 스캔이 두 번 필요한 구조는 한 번짜리보다 비쌀 수밖에 없다
- 점검 습관 — 계획에서 같은 테이블 이름이 몇 번 나오는지 센다
  - 가장 빠르고 확실한 비용 예측 단서 가운데 하나다

'옵티마이저가 알아서 해준다'의 한계

- 흔한 주장 — 상관 서브쿼리는 옵티마이저가 최적화하니 안심해도 된다
- Q3가 그 주장의 반례다
  - 옵티마이저는 실제로 최적화했다 — 행마다 반복 실행을 피했다
  - 그룹 집계를 미리 계산하는 영리한 형태로 변환했다
  - 그런데도 Q1·Q2보다 비쌌다
- 두 문장을 구분해야 한다
  - 옵티마이저가 알아서 최적화해 준다 — 대체로 맞다
  - 그 결과가 항상 가장 저렴하다 — 보장되지 않는다
- 이유 — 옵티마이저는 주어진 SQL 구조 안에서만 최선을 다한다
  - SQL 자체가 같은 테이블을 두 번 읽어야 하는 구조라면
  - 변환은 그 구조의 한계를 완전히 넘어서지 못한다
- 따라서 SQL을 쓸 때 데이터를 몇 번 읽게 되는지를 먼저 생각해야 한다

10.7

Tuning — 인라인 뷰의 WHERE는 HAVING과 같아지는가

- Q4 인라인 뷰 / Q5 HAVING
- 구조와 비용 비교
- 뷰 병합(View Merging)
- 병합이 안전하지 않은 경우

Q4 인라인 뷰 / Q5 HAVING

병원별 청구 건수가 1000건을 넘는 병원만 뽑는다 — 두 방식의 결과는 같다

```sql
-- Q4. 인라인 뷰 + 바깥 WHERE
SELECT * FROM (SELECT hosp_id, COUNT(*) cnt FROM medical_claims GROUP BY hosp_id) v
WHERE v.cnt > 1000;

-- Q5. HAVING으로 직접 작성
SELECT hosp_id, COUNT(*) cnt FROM medical_claims GROUP BY hosp_id HAVING COUNT(*) > 1000;

-- Q4 : 집계 결과 집합을 먼저 만들고 바깥에서 거르는 모양
-- Q5 : 집계와 필터가 한 문장 안에 붙어 있는 모양
-- 문법 골격은 다르지만 요구하는 결과는 완전히 같다
-- 관전 포인트 : 옵티마이저가 두 문장을 같은 구조로 만들 것인가
```

Q4와 Q5 — 구조와 비용 비교

오퍼레이션 트리는 다르지만 Buffers는 완전히 같다

```
+------------------+-----------------------------------------------------+---------+
| 쿼리             | Operation 구조                                      | Buffers |
+------------------+-----------------------------------------------------+---------+
| Q4 (인라인 뷰)   | FILTER -> HASH GROUP BY -> TABLE ACCESS FULL        |   3,090 |
| Q5 (HAVING)      | HASH GROUP BY(필터 내장) -> TABLE ACCESS FULL       |   3,090 |
+------------------+-----------------------------------------------------+---------+

-- 두 방식 모두 TABLE ACCESS FULL이 1회뿐이다
-- Q4에만 별도의 FILTER 단계가 하나 더 붙는다
-- 그럼에도 Buffers는 3,090으로 동일하다
```

뷰 병합 (View Merging)이란

- 옵티마이저의 또 다른 내부 변환
  - 인라인 뷰를 별도 단계로 두지 않고 바깥 쿼리와 한 덩어리로 합친다
  - unnesting이 서브쿼리를 조인으로 푸는 것이라면, 병합은 뷰의 경계를 지우는 것이다
- 병합이 주는 이점
  - 중간 결과 집합을 따로 만들어 두는 부담이 사라진다
  - 바깥 조건을 안쪽으로 밀어 넣어 미리 걸러 낼 여지가 생긴다
  - 조인 순서 선택의 폭이 넓어져 더 나은 계획을 고를 수 있다
- Q4에서 관측된 것
  - 테이블을 한 번만 읽고 Q5와 Buffers가 3,090으로 동일했다
  - 겉보기 SQL 스타일이 달라도 비용은 같을 수 있다는 사례다
- 따라서 인라인 뷰를 성능상의 이유로 무조건 피할 근거는 없다

FILTER 단계가 붙어도 비용이 같은 이유

- 관측 — Q4에는 FILTER가 있고 Q5에는 없다
  - Q4는 인라인 뷰의 골격을 살리되 바깥 WHERE를 별도 FILTER로 적용했다
  - Q5는 HAVING이 애초에 GROUP BY 처리와 한 단계로 묶여 표현됐다
- 오퍼레이션 개수는 다른데 Buffers는 같다
  - 실질적인 작업량이 동일하기 때문이다
  - 테이블을 한 번 스캔한다 → 그룹별로 집계한다 → 조건에 맞는 그룹만 남긴다
  - 두 방식 모두 이 세 가지 일을 똑같이 수행한다
- FILTER가 비용을 거의 늘리지 않는 이유
  - 이미 집계가 끝난 작은 결과 집합에 대해 적용되기 때문이다
  - 큰 테이블을 다시 읽는 일이 아니므로 Buffers에 흔적이 남지 않는다
- 교훈 — 오퍼레이션 개수와 비용은 비례하지 않는다
  - 어떤 단계가 얼마나 많은 데이터를 다루는지를 봐야 한다

병합이 항상 안전하지는 않다

- 인라인 뷰 WHERE와 HAVING이 같아진다는 결론의 적용 범위
  - 이번 사례는 단순 집계(COUNT)만 들어 있는 인라인 뷰였다
  - 모든 인라인 뷰에 같은 결론을 그대로 옮길 수는 없다
- 조심해야 하는 연산 — 결과 집합 전체를 기준으로 계산되는 것들
  - ROWNUM — 그 시점까지 만들어진 결과 집합을 기준으로 번호가 매겨진다
  - 분석 함수 RANK() OVER (...) 등 — 순위 계산에 순서와 시점이 관여한다
- 왜 위험한가
  - 병합으로 필터링 순서가 바뀌면 어떤 행이 뽑히는지 자체가 달라질 수 있다
  - 성능 문제가 아니라 결과가 달라지는 정합성 문제로 나타난다
- 실제 동작
  - 옵티마이저는 이런 경우 병합하지 않고 뷰를 별도 단계로 유지하기도 한다
  - 필요하면 개발자가 NO_MERGE 힌트로 병합을 명시적으로 막을 수 있다

> **참고:**
> 이번 사례는 단순 집계 인라인 뷰에서 얻은 결론이다. 인라인 뷰의 내용이 달라지면 결론도 달라질 수 있으므로 항상 실행계획으로 확인한다.

10.8~10.10

Benchmark · Practice · Review

- 세 사례 실측 요약
- 세 사례가 공통으로 말하는 것
- 실무 판단 체크리스트
- 이 장의 결론

8.8 Benchmark — 세 사례 실측 요약

결론이 서로 다르다는 점이 이 표의 핵심이다

```
+--------------------+------------------+----------------+-----------+-----------+
| 비교               | 방식 A           | 방식 B         | Buffers A | Buffers B |
+--------------------+------------------+----------------+-----------+-----------+
| 값 하나 붙이기     | 스칼라 서브쿼리  | JOIN           |     4,081 |     4,083 |
| 그룹 평균과 비교   | 상관 서브쿼리    | (JOIN 미측정)  |     6,612 |         - |
| 집계 결과 필터링   | 인라인 뷰+WHERE  | HAVING         |     3,090 |     3,090 |
+--------------------+------------------+----------------+-----------+-----------+

-- 값 하나 붙이기   : distinct 값이 적어 캐싱으로 거의 동일
-- 그룹 평균과 비교 : 테이블 2회 스캔되는 구조라 오히려 비쌈
-- 집계 결과 필터링 : 뷰 병합으로 완전히 동등
```

세 사례가 공통으로 말하는 것

- 세 사례의 결론은 모두 달랐다
  - 스칼라 서브쿼리 — 캐싱 덕분에 조인과 거의 같은 비용으로 수렴했다
  - 상관 서브쿼리 — 조인 구조로 변환됐지만 두 번 읽는 구조라 더 비쌌다
  - 인라인 뷰 — HAVING과 물리적으로 동등한 구조로 병합되어 완전히 같았다
- 그런데 세 사례를 관통하는 원칙은 하나다
  - 비용을 결정한 것은 SQL이 어떻게 생겼는가가 아니었다
  - 결과적으로 데이터를 몇 번 어떻게 읽게 되는가가 비용을 결정했다
- 문장으로 정리하면
  - 서브쿼리 성능은 문법 형태가 아니라 옵티마이저가 만들어 낸
  - 물리적 실행 구조에 달려 있다
- 그래서 판단 절차도 하나로 정해진다
  - 문법을 보고 예단하지 말고 변환된 실행계획을 열어 확인한다

실무 판단 체크리스트

- 스칼라 서브쿼리를 쓸 때 확인할 것
  - 참조 컬럼의 distinct 값 개수가 전체 행 수에 비해 충분히 작은가
  - 실행계획의 Starts가 결과 행 수보다 훨씬 작게 나오는가
  - 서브쿼리가 값 하나만 반환하는 것이 데이터로 보장되는가
- 상관 서브쿼리를 쓸 때 확인할 것
  - 계획에 같은 테이블 이름이 몇 번 등장하는지 센다
  - 내부 뷰 이름이 보이면 어떤 변환이 일어났는지 해석한다
  - 두 번 읽는 구조라면 한 번으로 줄일 방법이 있는지 검토한다
- 인라인 뷰를 쓸 때 확인할 것
  - 뷰가 병합됐는지 별도 단계로 남았는지 계획에서 확인한다
  - ROWNUM이나 분석 함수가 들어 있다면 결과까지 반드시 대조한다
- 공통 — 비교는 결과가 완전히 같은 상태에서만 성립한다

요약 (Summary)

- 서브쿼리는 위치에 따라 역할이 다르다 — 스칼라·인라인 뷰·상관 서브쿼리
- 옵티마이저는 서브쿼리를 조인 형태로 내부 변환한다 (unnesting)
- 스칼라 서브쿼리는 같은 입력 값의 결과를 캐시해 재사용한다
- Starts가 결과 행 수보다 훨씬 작으면 캐싱이 작동한 증거다
- 캐싱 효과는 참조 값의 distinct 개수가 적을수록 커진다
- 상관 서브쿼리는 VW_SQ_1 형태의 뷰와 조인으로 변환될 수 있다
- 그 변환이 원본 테이블을 두 번 읽게 만들면 오히려 비싸진다
- 인라인 뷰의 바깥 WHERE는 뷰 병합으로 HAVING과 동등해질 수 있다
- 오퍼레이션 개수와 비용은 비례하지 않는다 — 다루는 데이터 양을 본다
- 결론 — 서브쿼리 성능은 문법이 아니라 변환된 물리 구조가 결정한다

실습 안내

- 실습문제 10문항 — ch10_문제.md
  - 1~2번 Starts=5의 의미와 N+1 통념이 성립하지 않은 이유
  - 3번 참조 값이 500종류로 늘어날 때의 변화 예측 (캐싱 원리 적용)
  - 4~5번 두 번의 Full Scan이 하는 일과 Buffers 6,612의 근거
  - 6번 '옵티마이저가 알아서 해준다'는 주장에 대한 반박
  - 7~8번 인라인 뷰와 HAVING의 동등성, 트리가 달라도 비용이 같은 이유
  - 9번 ROWNUM·분석 함수가 있을 때 뷰 병합이 안전한지 원칙 적용
  - 10번 세 사례를 종합한 결론 문장 완성과 근거 서술
- 실습 전 확인 — 이번 장은 DDL 변경이 없다
  - 기존 PK만 사용하며 인덱스나 통계를 임의로 바꾸지 않는다
  - 스키마를 변경하면 교안의 실행계획이 재현되지 않는다

Oracle SQL 튜닝 과정 · ch11

UNION / UNION ALL / DISTINCT

중복 제거의 비용 · SORT UNIQUE · Buffers의 사각지대 · 옵티마이저의 DISTINCT 생략

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - UNION과 UNION ALL의 차이를 결과 집합과 실행계획 양쪽에서 설명할 수 있다
  - UNION이 내부적으로 UNION-ALL 뒤에 SORT UNIQUE 단계를 덧붙인다는 사실을 실행계획에서 확인할 수 있다
  - 중복 제거 비용이 Buffers가 아니라 A-Time과 작업 메모리 지표에 나타나는 이유를 설명할 수 있다
  - 두 결과 집합이 논리적으로 겹칠 수 있는지를 기준으로 UNION과 UNION ALL을 선택할 수 있다
  - PK·유니크 제약이 걸린 컬럼의 DISTINCT를 옵티마이저가 생략한다는 점을 실행계획으로 증명할 수 있다
  - 같은 컬럼명이라도 어느 테이블의 어떤 제약이냐에 따라 DISTINCT 생략 여부가 달라짐을 추론할 수 있다
  - 중복 제거 연산의 비용을 판단할 때 확인할 항목을 체크리스트로 정리할 수 있다

이 장의 구성 (Agenda)

- 9.1 Why — 습관적인 UNION 선택이 위험한 이유
- 9.2 Concept — UNION · UNION ALL · DISTINCT의 정의
- 9.3 Oracle Internals — 중복 제거 비용은 어디에 나타나는가
- 9.4 Example — UNION vs UNION ALL 실측 소재
- 9.5 Execution Plan — 두 계획의 실측 비교와 해석
- 9.6 Bad SQL — 오히려 완전히 사라지는 DISTINCT
- 9.7~9.8 Tuning 판단 기준과 Benchmark 종합
- 9.9~9.10 Practice · Review

11.1

습관적인 UNION 선택이 위험한 이유

- 현장에서 UNION과 UNION ALL이 선택되는 방식
- "일단 안전하게 UNION"이 감추는 비용
- 이 차시의 목표와 범위

현장에서 UNION은 어떻게 선택되는가

- UNION과 UNION ALL을 습관적으로 아무거나 쓰는 경우가 많다
  - 두 결과 집합에 중복이 있을 수 있는지를 따져보지 않는다
  - "일단 중복 제거되는 UNION을 쓰자"는 식의 선택이 관행처럼 굳어진다
- 이 선택이 위험한 이유는 두 방향 모두다
  - 겹칠 수 없는 집합에 UNION을 쓰면 필요 없는 중복 제거 비용을 매번 지불한다
  - 겹치는 집합에 UNION ALL을 쓰면 성능이 아니라 결과 자체가 틀어진다
- 판단의 근거는 문법 취향이 아니라 데이터의 성질이어야 한다
  - 두 집합이 논리적으로 겹칠 수 있는가 — 이것이 유일한 1차 판단 기준

이 차시의 목표와 범위

- 목표 1 — 중복 제거에 실제로 어떤 비용이 드는지 실측으로 확인한다
  - 개념적으로 "정렬이 필요하다"는 설명을 실행계획 지표로 옮겨 본다
- 목표 2 — 그 비용이 항상 발생하는 것은 아님을 확인한다
  - 옵티마이저가 유일성을 증명할 수 있으면 중복 제거 단계 자체가 사라진다
- 소재 — 두 가지 실측 시나리오
  - "입원 청구를 낸 환자" ∪ "의료급여 환자"를 UNION과 UNION ALL로 비교
  - 이미 유일함이 보장된 컬럼에 불필요한 DISTINCT를 붙였을 때의 실행계획
- 범위 — DDL 변경 없음. 기존 PK만 사용하며 모든 결과는 실습 DB 실측값

11.2

UNION · UNION ALL · DISTINCT

- UNION과 UNION ALL의 정의
- DISTINCT의 정의와 적용 범위
- 중복 제거에 비용이 드는 이유

UNION과 UNION ALL의 정의

- UNION — 두 결과 집합을 합치면서 중복된 행을 제거한다
  - 합집합의 수학적 정의에 가깝다. 같은 행이 여러 번 나와도 결과에는 한 번만 남는다
  - 중복을 제거하려면 어떤 행이 이미 나왔는지를 알아야 하므로 추가 작업이 필요하다
- UNION ALL — 두 결과 집합을 그대로 이어붙인다
  - 중복 제거가 없다. 앞 집합을 다 내보내고 뒤 집합을 이어서 내보내면 끝이다
  - 판단할 것이 없으므로 추가 작업도 없다
- 두 연산자는 성능 옵션이 아니라 서로 다른 결과를 내는 연산이다
  - 두 집합에 겹치는 행이 하나도 없을 때에만 두 결과가 같아진다

DISTINCT의 정의와 적용 범위

- DISTINCT — 결과 집합에서 중복된 행을 제거한다
  - 집합 연산자와 달리 단일 쿼리 하나에도 적용할 수 있다
  - 선택된 컬럼 조합 전체를 기준으로 같은 값을 하나로 줄인다
- UNION이 하는 일과 본질적으로 같은 작업이다
  - UNION = UNION ALL로 합친 결과에 중복 제거를 적용한 것
  - 따라서 UNION의 비용을 이해하면 DISTINCT의 비용도 같은 원리로 이해된다
- 실무에서 DISTINCT가 붙는 전형적인 자리
  - 조인 때문에 행이 늘어난 결과를 원래 건수로 되돌리려는 목적
  - 중복 여부를 확신하지 못해 "혹시 몰라서" 붙여 두는 습관적 사용

중복 제거에는 왜 비용이 드는가

- 중복을 제거하려면 같은 값을 찾아내야 한다
  - 한 행만 보고는 그 행이 중복인지 알 수 없다. 다른 행들과 비교해야 한다
- DB가 쓰는 대표적인 두 가지 방법
  - 정렬(SORT) — 전체 결과를 정렬해 같은 값을 이웃으로 모은 뒤 인접 중복을 걷어낸다
  - 해시(HASH) — 값을 해시로 묶어 같은 버킷에 모은 뒤 중복을 판정한다
- 두 방법 모두 결과 전체를 한 번은 모아 두어야 한다
  - 앞쪽 몇 건만 보고 결론 낼 수 없으므로 중간 결과를 쌓을 작업 공간이 필요하다
  - 즉 CPU 연산량과 작업 메모리를 함께 소비하는 연산이다
- 이 비용이 실행계획의 어느 지표에 나타나는지는 다음 절에서 확인한다

11.3

중복 제거 비용은 어디에 나타나는가

- UNION의 내부 구조 — UNION-ALL + SORT UNIQUE
- Buffers가 재는 것과 재지 못하는 것
- A-Time과 작업 메모리 지표 읽는 법
- 옵티마이저가 DISTINCT를 지울 수 있는 조건

UNION의 내부 구조

- UNION은 하나의 오퍼레이션이 아니라 두 단계로 실행된다
  - 1단계 UNION-ALL — 두 집합을 그대로 합친다
  - 2단계 SORT UNIQUE — 합쳐진 결과를 정렬해 중복을 제거한다
- 즉 UNION 실행계획은 UNION ALL 실행계획 위에 한 단계가 더 얹힌 모양이다
  - 아래쪽 테이블 접근 단계는 두 쿼리가 완전히 동일하다
  - 차이는 UNION-ALL 위에 SORT UNIQUE가 있느냐 없느냐 하나뿐이다
- 이 구조가 이번 장의 실측을 아주 깨끗한 비교로 만들어 준다
  - 변수가 딱 하나이므로 관측된 차이의 원인을 그 한 단계로 특정할 수 있다

Buffers가 재는 것과 재지 못하는 것

- Buffers는 블록을 몇 번 읽었는지를 세는 지표다
  - 버퍼 캐시 또는 디스크에서 블록을 가져온 횟수, 즉 논리적 I/O
  - "얼마나 계산했는가"가 아니라 "얼마나 읽었는가"만 센다
- SORT UNIQUE는 블록을 새로 읽지 않는다
  - 대상은 이미 앞 단계에서 메모리로 가져온 결과 집합이다
  - 테이블도 인덱스도 다시 건드리지 않으므로 Buffers가 늘어날 이유가 없다
- 따라서 Buffers만으로는 중복 제거 비용이 전혀 보이지 않는다
  - Buffers가 같다는 사실은 "I/O가 같다"는 뜻이지 "비용이 같다"는 뜻이 아니다

참고:
Buffers는 I/O 비용만 나타내는 지표이지 전체 비용의 전부가 아니다. 정렬 같은 CPU·메모리 연산은 Buffers에 반영되지 않으므로, 이 지표 하나만 보고 판단하면 CPU 부하가 큰 연산을 통째로 놓칠 수 있다.

A-Time과 작업 메모리 지표

- 중복 제거 비용은 두 종류의 지표에 흔적을 남긴다
  - A-Time — 그 단계가 실제로 소비한 시간. CPU 연산 시간이 여기에 반영된다
  - OMem · 1Mem · Used-Mem — 그 단계가 사용한 작업 메모리 관련 값
- 작업 메모리 관련 컬럼이 채워져 있다는 것 자체가 신호다
  - 테이블 접근 단계에는 이 값들이 비어 있다 — 모을 필요가 없기 때문
  - 값이 찍힌 단계는 중간 결과를 모아 두어야 하는 연산이라는 뜻이다
- 실행계획을 읽을 때의 습관
  - Buffers 열만 훑지 말고 A-Time과 메모리 열을 같은 줄에서 함께 읽는다
  - 어느 단계에서 시간이 늘었는지를 단계별 A-Time 차이로 좁혀 들어간다

옵티마이저가 DISTINCT를 지울 수 있는 조건

- DISTINCT도 원칙적으로는 SORT나 HASH 기반 중복 제거가 필요하다
- 그러나 옵티마이저가 중복이 없음을 증명할 수 있으면 이야기가 달라진다
  - 조회 대상 컬럼에 PK나 유니크 인덱스 같은 제약이 걸려 있는 경우
  - 제약은 DB가 보장하는 사실이므로 옵티마이저가 메타데이터로 확신할 수 있다
  - 중복이 나올 수 없다면 중복 제거 단계를 실행할 이유도 없다 — 통째로 생략
- 생략의 조건은 "실제로 중복이 없다"가 아니라 "증명할 수 있다"이다
  - 데이터상 우연히 중복이 없어도 제약이 없으면 옵티마이저는 확신할 수 없다
  - 조인을 거친 결과나 제약이 없는 컬럼이라면 자동 생략을 기대할 수 없다

11.4

UNION vs UNION ALL 실측 소재

- 두 집합의 정의와 겹침 가능성
- Q1. UNION 버전
- Q2. UNION ALL 버전

실측 소재 — 두 집합의 정의

- 집합 A — 입원 청구를 낸 환자
  - MEDICAL_CLAIMS에서 CLAIM_TYPE이 입원인 행의 PAT_ID
- 집합 B — 의료급여 환자
  - PATIENTS에서 INS_TYPE이 의료급여인 행의 PAT_ID
- 두 집합은 논리적으로 겹칠 수 있는가
  - 한 환자가 의료급여 대상이면서 입원 청구를 낸 경우를 막을 규칙이 없다
  - 즉 겹칠 수 있다 — 그렇다면 중복 제거 여부가 결과 건수를 바꾼다
- 비교 설계
  - 두 쿼리는 집합 연산자만 다르고 나머지는 완전히 동일하다
  - 바깥에서 COUNT(*)로 감싸 결과 건수를 바로 확인한다

Q1. UNION (중복 제거)

9.4절 Example — 두 집합을 UNION으로 합쳐 건수를 센다

```sql
-- Q1. UNION (중복 제거)
SELECT COUNT(*) FROM (
  SELECT pat_id FROM medical_claims WHERE claim_type = '입원'
  UNION
  SELECT pat_id FROM patients WHERE ins_type = '의료급여'
);
```

Q2. UNION ALL (중복 미제거)

9.4절 Example — 연산자만 UNION ALL로 바꾼 동일 쿼리

```
-- Q2. UNION ALL (중복 미제거)
SELECT COUNT(*) FROM (
  SELECT pat_id FROM medical_claims WHERE claim_type = '입원'
  UNION ALL
  SELECT pat_id FROM patients WHERE ins_type = '의료급여'
);
```

11.5

실행계획 실측 비교

- Q1. UNION 실행계획과 해석
- Q2. UNION ALL 실행계획과 해석
- 두 계획의 유일한 차이와 Buffers·A-Time
- 결과 건수 차이가 말해주는 것

Q1. UNION 실행계획

9.5절 Execution Plan — SORT UNIQUE 단계에만 작업 메모리 값이 찍혀 있다

```
| Id | Operation             | Name          | A-Rows |   A-Time    | Buffers | OMem  | 1Mem | Used-Mem  |
| 0  | SELECT STATEMENT      |               |      1 | 00:00:00.07 |    3532 |       |      |           |
| 1  |  SORT AGGREGATE       |               |      1 | 00:00:00.07 |    3532 |       |      |           |
| 2  |   VIEW                |               |  35985 | 00:00:00.07 |    3532 |       |      |           |
| 3  |    SORT UNIQUE        |               |  35985 | 00:00:00.06 |    3532 | 1824K | 791K | 1621K (0) |
| 4  |     UNION-ALL         |               |  64011 | 00:00:00.04 |    3532 |       |      |           |
| 5  |      TABLE ACCESS FULL| MEDICAL_CLAIMS|  59511 | 00:00:00.02 |    3090 |       |      |           |
| 6  |      TABLE ACCESS FULL| PATIENTS      |   4500 | 00:00:00.01 |     442 |       |      |           |
```

Q1 실행계획 해석

- 아래에서 위로 읽는다
  - MEDICAL_CLAIMS와 PATIENTS를 각각 전체 스캔한다
  - UNION-ALL이 두 결과를 그대로 합쳐 64,011건을 만든다
  - SORT UNIQUE가 정렬·중복 제거를 거쳐 35,985건으로 줄인다
  - VIEW를 거쳐 SORT AGGREGATE가 최종 건수를 센다
- 읽어야 할 지표들
  - 전체 Buffers는 3,532이고 두 테이블 접근이 각각 3,090과 442를 차지한다
  - SORT UNIQUE 단계의 Buffers는 그 아래 단계와 같은 3,532 — 늘지 않았다
  - 작업 메모리 관련 값 1824K · 791K · 1621K는 SORT UNIQUE 행에만 나타난다

Q2. UNION ALL 실행계획

9.5절 Execution Plan — SORT UNIQUE 단계가 통째로 빠져 있다

```
| Id | Operation              | Name          | A-Rows |   A-Time    | Buffers |
| 0  | SELECT STATEMENT       |               |      1 | 00:00:00.03 |    3532 |
| 1  |  SORT AGGREGATE        |               |      1 | 00:00:00.03 |    3532 |
| 2  |   VIEW                 |               |  64011 | 00:00:00.02 |    3532 |
| 3  |    UNION-ALL           |               |  64011 | 00:00:00.02 |    3532 |
| 4  |     TABLE ACCESS FULL  | MEDICAL_CLAIMS |  59511 | 00:00:00.01 |    3090 |
| 5  |     TABLE ACCESS FULL  | PATIENTS      |   4500 | 00:00:00.01 |     442 |
```

Q2 실행계획 해석

- Q1에서 한 단계가 빠졌을 뿐 나머지는 동일하다
  - 두 테이블 전체 스캔 → UNION-ALL → VIEW → SORT AGGREGATE
  - SORT UNIQUE가 없으므로 합친 64,011건이 그대로 위로 올라간다
- 지표 비교
  - Buffers는 3,532로 Q1과 완전히 같다 — 테이블 접근도 3,090과 442로 동일
  - A-Time은 0.03초로 Q1의 0.07초보다 짧다
  - 작업 메모리 관련 컬럼은 애초에 채워질 단계가 없다
- 변수가 하나뿐인 비교이므로 차이의 원인도 하나로 특정된다
  - 두 계획의 유일한 차이는 SORT UNIQUE 한 단계다

두 계획의 유일한 차이

- 같은 것
  - 접근 경로 — 두 테이블 모두 전체 스캔으로 동일
  - 단계별 Buffers — MEDICAL_CLAIMS 3,090, PATIENTS 442, 전체 3,532
  - UNION-ALL 단계에서 합쳐진 건수 64,011
- 다른 것
  - Q1에만 SORT UNIQUE 단계가 있다
  - 최종 결과 건수 — 35,985 대 64,011
  - A-Time — 0.07초 대 0.03초
- 결론적으로 관측된 모든 차이는 SORT UNIQUE 한 단계에서 비롯된다

Buffers가 같다는 사실이 말해주는 것

- 두 쿼리의 Buffers는 3,532로 완전히 동일했다
  - SORT UNIQUE 단계가 추가됐는데도 한 블록도 더 읽지 않았다
- 왜 늘지 않았는가
  - SORT UNIQUE의 대상은 UNION-ALL이 이미 메모리로 올려놓은 결과 집합이다
  - 테이블이나 인덱스 블록을 다시 읽을 필요가 없는 순수 CPU·메모리 연산이다
  - Buffers는 블록을 읽은 횟수만 세므로 이 연산은 지표에 잡히지 않는다
- 해석의 함정
  - "Buffers가 같다"에서 "비용이 같다"로 건너뛰면 안 된다
  - 지표가 보여주지 않는다는 것과 비용이 없다는 것은 전혀 다른 말이다

A-Time 차이가 말해주는 것

- A-Time은 Q1이 0.07초, Q2가 0.03초였다
  - 두 배 이상의 차이가 Buffers에는 전혀 나타나지 않았다
- 차이가 발생한 자리
  - Q1은 합쳐진 64,011건을 정렬·중복 제거해 35,985건으로 줄이는 일을 더 한다
  - 이 작업은 실제 CPU 시간을 소비하므로 A-Time에 그대로 반영된다
  - Q2에는 그 단계가 아예 없으므로 그만큼 시간이 짧다
- 지표를 함께 읽어야 하는 이유
  - Buffers는 I/O 비용, A-Time과 메모리 값은 CPU·메모리 비용을 보여준다
  - 같은 Buffers에서도 다른 A-Time이 나올 수 있다는 사실을 실측으로 확인한 셈

결과 건수 차이 — 28,026건의 의미

- 결과 건수
  - Q1 UNION = 35,985건 — 중복이 제거된 고유한 환자 수
  - Q2 UNION ALL = 64,011건 — 두 조회 결과를 그대로 이어붙인 건수
- 차이 28,026건이 뜻하는 것
  - 입원 청구를 냈으면서 동시에 의료급여 환자이기도 한 사람이 그만큼 있었다
  - UNION ALL 결과에서는 이 사람들이 두 번씩 세어진 상태다
- 건수 차이 자체가 유용한 정보다
  - 두 집합에 실제로 중복이 있었는지를 보여주는 직접적인 증거가 된다
  - 차이가 0이라면 두 집합이 겹치지 않았다는 뜻이고 UNION ALL로 바꿔도 안전하다

성능이 아니라 정확성의 문제

- 이 시나리오에서 UNION ALL은 더 빠르지만 답이 틀린다
  - "환자가 몇 명인지" 세는 것이 목적이라면 중복된 환자가 두 번 카운트된다
  - 고유 환자 수는 35,985명인데 64,011이라는 과대 집계된 값을 얻게 된다
- 튜닝의 대전제
  - 대안은 "결과가 같다"는 전제 아래에서만 비교할 수 있다
  - 결과가 달라지는 변경은 튜닝이 아니라 사양 변경이다
- 실무에서 이 사고가 늦게 발견되는 이유
  - 쿼리가 에러 없이 정상 종료되므로 숫자만 조용히 부풀어 오른다
  - 집계 리포트는 검증 대상이 되기 어려워 한참 뒤에야 발견되는 경우가 많다

11.6

오히려 완전히 사라지는 DISTINCT

- Q3 · Q4 — DISTINCT를 붙인 쿼리와 안 붙인 쿼리
- 두 쿼리의 공통 실행계획
- 생략이 가능했던 근거와 그 한계

Q3 · Q4 — 유일한 컬럼에 붙인 DISTINCT

9.6절 Bad SQL — HOSP_ID는 HOSPITALS의 PK다

```sql
-- Q3. 이미 유일함이 보장된 컬럼에 DISTINCT
SELECT DISTINCT hosp_id FROM hospitals WHERE hosp_id <= 100;

-- Q4. DISTINCT 없이 동일 조회
SELECT hosp_id FROM hospitals WHERE hosp_id <= 100;
```

Q3 · Q4 공통 실행계획

9.6절 Execution Plan — Plan hash value가 완전히 동일하다 (1619867082)

```
| Id | Operation          | Name         | A-Rows |   A-Time    | Buffers |
| 0  | SELECT STATEMENT   |              |    100 | 00:00:00.01 |       9 |
| 1  |  INDEX RANGE SCAN  | PK_HOSPITALS |    100 | 00:00:00.01 |       9 |
```

계획 해석 — 중복 제거 오퍼레이션이 없다

- 관측 사실
  - DISTINCT를 붙인 Q3과 안 붙인 Q4의 실행계획이 완전히 동일하다
  - SORT UNIQUE도 HASH UNIQUE도 계획에 전혀 나타나지 않는다
  - 두 쿼리 모두 PK_HOSPITALS 인덱스 범위 스캔 한 단계로 끝난다
- 지표
  - Buffers는 두 쿼리 모두 9, A-Time도 모두 0.01초
  - 중복 제거를 위한 어떤 추가 비용도 관측되지 않는다
- 즉 DISTINCT 요청 자체가 아무 비용 없이 통째로 생략됐다
  - 실행 중에 건너뛴 것이 아니라 계획을 세울 때부터 없던 일이 됐다

생략의 근거 — PK 제약과 증명 가능성

- HOSP_ID는 HOSPITALS 테이블의 기본 키다
  - PK는 테이블 안에서 값이 절대 중복될 수 없음을 DB가 보장하는 제약이다
  - 옵티마이저는 이 제약을 메타데이터로 알고 있다
- 추론의 흐름
  - 이 컬럼을 그대로 조회하면 애초에 중복이 나올 수 없다
  - 중복이 없으면 DISTINCT가 요구하는 작업도 할 이유가 없다
  - 따라서 그 단계를 계획에서 제거해도 결과가 달라지지 않는다
- 핵심은 "증명할 수 있는가"이지 "실제로 중복이 없는가"가 아니다

참고:
제약조건은 성능 기능이 아니라 정합성 기능으로만 이해되는 경우가 많다. 그러나 옵티마이저가 신뢰하고 활용할 수 있는 사실의 출처가 곧 제약조건이므로, 제약을 제대로 선언하는 것 자체가 튜닝의 일부다.

"불필요한 DISTINCT는 항상 낭비"가 틀린 이유

- 이 명제는 하나의 전제 위에서만 성립한다
  - 전제 — 옵티마이저가 중복 제거를 항상 곧이곧대로 실행한다
  - 그러나 Q3과 Q4의 실측이 그 전제를 반증한다
- 실측이 보여준 것
  - 유일성을 증명할 수 있는 상황에서는 DISTINCT가 비용 없이 사라진다
  - Q3과 Q4의 계획이 완전히 같으므로 "낭비"라고 할 근거 자체가 없다
- 다만 반대 방향의 오해도 경계해야 한다
  - "DISTINCT를 아무 데나 붙여도 무해하다"는 뜻은 결코 아니다
  - 옵티마이저가 유일성을 증명할 수 없는 경우에는 실제 비용이 그대로 발생한다

반례 — 같은 컬럼명, 다른 테이블

- 만약 같은 패턴을 MEDICAL_CLAIMS에 적용한다면
  - MEDICAL_CLAIMS의 HOSP_ID는 PK가 아니라 FK 성격의 컬럼이다
  - 한 병원에 여러 청구 건이 연결될 수 있으므로 값이 얼마든지 중복된다
- 옵티마이저의 판단은 달라진다
  - "이 컬럼은 중복이 없다"고 증명할 근거가 없다
  - 따라서 DISTINCT를 실제로 수행해야 하고 중복 제거 오퍼레이션이 계획에 나타난다
- 얻는 교훈
  - 같은 컬럼명이라도 어느 테이블의 어떤 제약을 가진 컬럼이냐가 결론을 바꾼다
  - DISTINCT를 볼 때는 컬럼 이름이 아니라 그 컬럼의 제약을 확인해야 한다

11.7~11.8

판단 기준과 Benchmark 종합

- UNION과 UNION ALL의 선택 기준
- 겹칠 수 없는 조건의 예시
- DISTINCT를 빼도 되는가
- 네 쿼리 종합 벤치마크와 체크리스트

UNION과 UNION ALL의 선택 기준

- 1차 질문 — 두 결과 집합이 논리적으로 겹칠 수 있는가
  - 겹칠 수 있다면 UNION이 맞다. 이번 실습이 그런 경우였다
  - 입원 청구를 낸 환자와 의료급여 환자는 겹칠 수 있는 집합이었다
- 겹칠 수 없는 구조라면 UNION ALL로 바꾼다
  - 서로 배타적인 조건으로 나뉜 집합이라면 중복이 생길 수 없다
  - 불필요한 SORT UNIQUE 비용을 안전하게 아낄 수 있다
- 판단은 데이터의 현재 상태가 아니라 구조에서 나와야 한다
  - 지금 우연히 겹치지 않는 것과 구조적으로 겹칠 수 없는 것은 다르다
  - 확신이 서지 않으면 두 버전의 건수를 세어 차이가 있는지 확인한다

겹칠 수 없는 조건의 예시

- 예시 — 입원 청구와 외래 청구
  - CLAIM_TYPE이 입원인 청구와 외래인 청구를 각각 조회한다
  - 한 청구 건의 CLAIM_TYPE은 동시에 두 값을 가질 수 없다
  - 따라서 두 조회 결과의 CLAIM_ID 집합은 절대 겹칠 수 없다
- 이 경우 UNION을 UNION ALL로 바꿔도 안전하다
  - 중복이 생길 수 없으므로 결과가 완전히 동일하다
  - SORT UNIQUE의 CPU 비용을 그대로 절약할 수 있다
- 앞의 소재와 무엇이 달랐는가
  - 앞 소재는 서로 다른 테이블의 서로 다른 속성으로 나눈 집합이었다
  - 이 예시는 같은 컬럼의 값으로 나눈 집합이라 배타성이 구조에서 보장된다

DISTINCT를 빼도 되는가

- 조회 대상이 PK나 유니크 인덱스 컬럼이라면
  - DISTINCT를 빼도 결과가 같다
  - 옵티마이저가 어차피 생략해줄 가능성이 높으므로 성능 차이도 없다
  - 다만 읽는 사람에게 혼란을 주므로 정리해 두는 편이 낫다
- 조인 이후의 컬럼처럼 중복 가능성이 실제로 있는 경우
  - DISTINCT가 결과의 정확성을 위해 반드시 필요하다
  - 이때 DISTINCT를 지우는 것은 튜닝이 아니라 결과를 망가뜨리는 변경이다
- 조인 때문에 생긴 중복이라면 근본 해결은 다른 곳에 있다
  - 왜 조인 결과에 행이 늘어났는지, 즉 조인 조건과 카디널리티를 먼저 본다
  - DISTINCT로 덮는 것은 증상 처리이고 조인 구조를 고치는 것이 원인 처리다

종합 벤치마크

9.8절 Benchmark — 네 쿼리 실측 요약

비교	Buffers	A-Time	비고
UNION (Q1)	3,532	0.07초	SORT UNIQUE 추가, 결과 35,985건
UNION ALL (Q2)	3,532	0.03초	SORT UNIQUE 없음, 결과 64,011건(중복 포함)
DISTINCT, PK 컬럼 (Q3)	9	0.01초	SORT UNIQUE 완전히 생략됨
DISTINCT 없음, 동일 조회 (Q4)	9	0.01초	Q3과 완전히 동일한 계획

중복 제거 비용 판단 체크리스트

- ① 두 결과가 논리적으로 겹칠 수 있는가
  - 겹칠 수 없다면 중복 제거 자체가 필요 없다 — UNION ALL 또는 DISTINCT 제거가 안전하다
  - 판단 근거는 데이터의 현재 상태가 아니라 구조적 배타성이어야 한다
- ② 옵티마이저가 유일성을 증명할 수 있는가
  - PK나 유니크 제약이 걸린 컬럼이라면 붙여 두어도 비용 없이 생략된다
  - 제약이 없는 컬럼이나 조인을 거친 결과라면 기대할 수 없다
- ③ 중복 제거가 실제로 필요하다면 그 비용을 어디서 확인할 것인가
  - Buffers가 아니라 A-Time과 작업 메모리 사용량으로 확인한다
  - Buffers만 보고 "비용이 없다"고 오판하지 않도록 주의한다

요약 (Summary)

- UNION은 UNION-ALL로 합친 뒤 SORT UNIQUE로 중복을 제거하는 두 단계 연산이다
- SORT UNIQUE는 이미 메모리에 있는 데이터를 다루므로 Buffers를 늘리지 않는다
- 실측에서 Q1과 Q2의 Buffers는 3,532로 같았지만 A-Time은 0.07초와 0.03초로 갈렸다
- Buffers만으로 UNION과 UNION ALL의 비용을 비교하면 안 된다는 것이 이 장의 첫 결론이다
- 결과 건수 35,985와 64,011의 차이 28,026은 두 집합에 실제로 있던 중복의 규모다
- 겹칠 수 있는 집합에서 UNION ALL로 바꾸면 빨라지는 대신 답이 틀린다
- PK 컬럼에 붙인 DISTINCT는 옵티마이저가 비용 없이 통째로 생략했다 — Q3과 Q4의 계획이 동일
- 생략의 조건은 옵티마이저가 유일성을 증명할 수 있는 경우로 한정된다

실습 안내

- 실습문제 10문항 — ch11_문제.md
  - 1~3번 SORT UNIQUE가 있는데 Buffers가 안 늘어난 이유와 A-Time 차이의 원인
  - 4~5번 28,026건 차이의 의미와 UNION ALL을 쓰면 안 되는 이유
  - 6~7번 DISTINCT 생략의 근거와 "불필요한 DISTINCT는 낭비" 명제 반박
  - 8번 FK 컬럼에 같은 패턴을 적용했을 때의 결과 추론
  - 9번 이 스키마에서 겹칠 수 없는 두 집합 예시 만들기
  - 10번 중복 제거 비용 판단 시 확인할 항목 종합 정리
- 풀이 방법
  - 먼저 근거를 문장으로 적고 나서 실행계획으로 확인하는 순서를 지킨다
  - 해설은 ch11_해설.md에서 문항 번호와 1:1로 대응한다

Oracle SQL 튜닝 과정 · ch12

SORT / GROUP BY / 집계

HASH GROUP BY vs SORT GROUP BY · Predicate Pushdown · 정렬 생략의 두 조건 · 선택도와 SORT ORDER BY | 실습 DB 실측

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - ORDER BY의 SORT 연산이 언제 발생하고 언제 생략되는지 조건으로 설명한다
  - GROUP BY의 두 처리 방식 — HASH GROUP BY와 SORT GROUP BY를 구분한다
  - 정렬·해시 집계가 작업 영역(PGA)에서 수행된다는 점과 디스크 정렬의 의미를 설명한다
  - Predicate Pushdown의 정의와 안전 조건을 설명한다
  - 그룹 키에 대한 필터와 집계 결과에 대한 필터를 구분해 판별한다
  - HAVING 조건이 그룹화 이전으로 밀려나지 못하는 이유를 설명한다
  - 필터 위치를 바꾼 두 SQL이 동일한 실행계획이 된 원인을 실행계획으로 설명한다
  - 선택도에 따라 같은 컬럼의 정렬 생략 여부가 달라지는 이유를 설명한다
  - 정렬 생략의 두 조건을 적용해 실제 SQL의 SORT 발생 여부를 판정한다
  - 정렬 유무만으로 비용을 단정하지 않고 테이블 접근 방식까지 함께 확인한다

이 장의 구성 (Agenda)

- 10.1 정렬과 집계는 왜 중요한가
  - 리포트성 쿼리의 뼈대 / 검증할 두 가지 통념 / 이 차시의 소재와 범위
- 10.2 Concept — ORDER BY와 GROUP BY의 처리 방식
  - 정렬 연산의 정체 / 인덱스가 정렬을 대신하는 원리 / HASH vs SORT GROUP BY / 작업 영역
- 10.3 Oracle Internals — 필터 위치보다 중요한 것
  - Predicate Pushdown / 그룹 키 필터 / HAVING / ch10 사례와의 대조
- 10.4 Example — GROUP BY 필터 위치 실측
  - Q1·Q2 SQL / 공통 실행계획 / 동일한 계획이 된 원인 / 실무 교훈
- 10.5 Bad SQL — ORDER BY의 정렬 생략은 선택도에 달려 있다
  - 넓은 범위와 좁은 범위 비교 / 정렬이 없는 쪽이 더 비싼 역설
- 10.6 Tuning — 정렬 생략의 두 조건
  - 두 조건 / ch05와의 연결 / HASH GROUP BY의 생략 불가 / 체크리스트
- 10.7 Benchmark & Review

12.1

정렬과 집계는 왜 중요한가

- 리포트성 쿼리의 뼈대 — GROUP BY와 ORDER BY
- 이번 장이 검증할 두 가지 통념
- 이 차시의 소재와 범위

정렬과 집계는 어디에나 있다

- 리포트성 쿼리에는 GROUP BY와 ORDER BY가 거의 빠짐없이 들어간다
  - 병원별 청구 건수, 기간별 합계, 상위 N건 조회가 모두 이 두 연산 위에 있다
- 두 연산은 데이터를 모아서 처리한다 — 앞 단계 결과 전체가 필요하다
  - 한 행씩 흘려보내는 연산과 달리 중간 결과를 담아 둘 자리가 필요하다
  - 입력이 다 들어오기 전에는 첫 행조차 내보낼 수 없다
- 그래서 이 두 연산이 언제 비용을 유발하고 언제 생략되는지가 관건이다
  - 문법을 아는 것과 비용 구조를 아는 것은 다른 문제다
  - 이번 장은 그 조건을 실습 DB 실측으로 확인한다

이번 장이 검증할 두 가지 통념

- 통념 1 — '필터를 먼저 걸어야 집계가 빨라진다'
  - 집계 대상 행이 적을수록 유리하다는 직관에서 나온 조언
  - 그래서 인라인 뷰 안쪽으로 WHERE를 옮겨 쓰는 관행이 널리 퍼져 있다
- 통념 2 — '인덱스 컬럼으로 정렬하면 정렬이 생략된다'
  - 인덱스는 정렬돼 있으니 그 순서를 그대로 쓰면 된다는 직관
- 두 통념 모두 완전히 틀린 말은 아니다 — 조건이 빠져 있을 뿐이다
  - 통념은 대개 특정 상황에서 관측된 사실에서 조건만 떨어져 나간 형태다
  - 이번 장은 두 통념을 실측으로 검증하고 빠진 조건을 복원한다

이 차시의 소재와 범위

- 소재 1 — GROUP BY 대상을 줄이는 순서 (필터를 먼저 vs 나중에)
  - 같은 결과를 내는 두 SQL을 필터 위치만 바꿔 작성해 실행계획을 비교한다
- 소재 2 — ORDER BY 대상 컬럼에 따라 정렬(SORT)이 생략되는지 여부
  - 같은 두 컬럼을 정렬 기준으로 바꿔가며, 넓은 범위와 좁은 범위를 각각 확인한다
- 본 차시는 DDL 변경이 없다 — 기존 PK만 사용한다
  - 인덱스를 추가로 만들지 않고 접근 경로 변화만으로 결과가 갈린다
- 아래 예제는 전부 실습 DB 실측 결과다

12.2

Concept — ORDER BY와 GROUP BY

- ORDER BY — 정렬 연산의 정체
- 인덱스가 정렬을 대신할 수 있는 이유
- GROUP BY — HASH 방식과 SORT 방식
- 작업 영역(PGA)과 디스크 정렬

ORDER BY — 정렬 연산의 정체

- ORDER BY는 결과를 특정 순서로 정렬하라는 요구다
  - 요구를 만족시키는 방법은 두 가지 — 직접 정렬하거나, 이미 정렬된 것을 쓰거나
- 직접 정렬하면 실행계획에 SORT ORDER BY 오퍼레이션으로 나타난다
  - 정렬 대상 행을 모두 모아야 하므로 CPU와 메모리를 함께 소비한다
  - 정렬 대상 행 수가 늘면 비용은 그보다 빠르게 늘어난다
- 반대로 인덱스가 이미 그 순서로 정렬돼 있으면 별도의 SORT 연산 없이 그대로 반환할 수 있다
  - 이때 실행계획에는 SORT 오퍼레이션 자체가 나타나지 않는다

인덱스가 정렬을 대신할 수 있는 이유

- 인덱스는 키 값 순서로 정렬돼 저장되는 구조다
  - INDEX RANGE SCAN은 그 저장 순서대로 읽어 상위 단계에 넘긴다
  - 그래서 읽어 나오는 결과가 이미 인덱스 키 순서로 정렬돼 있다
- 정렬 요구가 그 순서와 일치하면 다시 정렬할 이유가 없다
  - 옵티마이저는 이 경우 SORT ORDER BY 단계를 계획에서 아예 제거한다
- 반대로 TABLE ACCESS FULL은 어떤 순서도 보장하지 않는다
  - 테이블에 저장된 물리적 순서로 읽을 뿐이며 이는 정렬 순서와 무관하다
  - 따라서 Full Scan이 선택되면 정렬 요구는 반드시 별도 SORT로 처리된다

GROUP BY — HASH 방식과 SORT 방식

- GROUP BY는 지정한 컬럼 값이 같은 행끼리 묶어 집계하는 연산이다
- 처리 방식은 크게 두 가지다
  - HASH GROUP BY — 그룹 키를 해시로 묶어 집계하는 해시 기반 방식
  - SORT GROUP BY — 정렬된 데이터를 순서대로 훑으며 같은 값끼리 묶는 정렬 기반 방식
- 이 스키마에서는 대부분 HASH GROUP BY가 선택된다
  - 이번 장의 모든 GROUP BY 쿼리에서 공통으로 HASH GROUP BY가 나타났다
- 어느 쪽이 선택될지는 옵티마이저가 비용으로 판단한다
  - 개발자가 문법으로 지정하는 대상이 아니다 — 실행계획으로 확인할 대상이다

HASH GROUP BY와 SORT GROUP BY

두 방식의 성격 비교 — 어느 쪽이 선택될지는 옵티마이저의 비용 판단

```
+------------------+------------------------------+--------------------------+
| 구분             | HASH GROUP BY                | SORT GROUP BY            |
+------------------+------------------------------+--------------------------+
| 그룹화 원리      | 그룹 키를 해시로 묶어 집계   | 정렬된 순서를 훑으며 묶음 |
| 결과 순서        | 보장 없음                    | 그룹 키 순서로 나옴       |
| 인덱스 정렬 활용 | 활용하지 않음                | 이미 정렬돼 있으면 유리   |
| 정렬 순서로 생략 | 생략되지 않음                | 해당 없음(정렬이 곧 처리) |
| 이번 장 관측     | 모든 GROUP BY 쿼리에서 관측 | 관측되지 않음             |
+------------------+------------------------------+--------------------------+

-- 두 방식 모두 그룹핑 작업 자체는 수행한다 — 없어지는 연산이 아니다
-- 선택은 옵티마이저의 비용 판단 영역이며 실행계획으로 확인한다
```

작업 영역(PGA)과 디스크 정렬

- 정렬과 해시 집계는 세션의 작업 영역(PGA)에서 수행된다
  - 실행계획의 OMem·1Mem·Used-Mem 컬럼으로 사용량을 확인할 수 있다 (ch01)
- 작업 영역 안에서 처리가 끝나면 메모리에서만 수행된다 — 가장 저렴한 경우다
- 대상 데이터가 작업 영역보다 크면 임시 테이블스페이스로 넘긴다
  - 중간 결과를 디스크에 쓰고 다시 읽어야 하므로 물리 I/O가 추가된다
  - 한 번 넘기면 되는 경우와 여러 번 나눠 처리해야 하는 경우로 나뉜다
- 근본 대책은 작업 영역을 키우는 것이 아니라 정렬·집계 대상을 줄이는 것이다
  - 대상을 줄이거나, 이번 장처럼 연산 자체를 생략시키는 방향이다

'집계 대상을 줄이자'는 조언의 실체

- 집계할 행의 수가 적을수록 GROUP BY의 CPU·메모리 부담이 줄어든다
  - 그래서 "필터를 먼저 걸어 집계 대상을 줄이자"는 조언이 흔하다
- 조언의 방향 자체는 맞다 — 문제는 그 일을 누가 하느냐다
  - 개발자가 SQL을 다시 써서 손으로 옮기는 방법
  - 옵티마이저가 조건을 알아서 앞 단계로 밀어넣는 방법
- 옵티마이저가 이미 해 주는 일을 손으로 다시 하면 SQL만 복잡해진다
  - 가독성은 나빠지고 성능은 그대로인 결과가 나올 수 있다
- 다음 절에서 옵티마이저가 어디까지 해 주는지 확인한다

12.3

Oracle Internals — 필터 위치보다 중요한 것

- Predicate Pushdown이란
- 그룹 키에 대한 필터는 왜 밀어넣을 수 있는가
- HAVING은 왜 밀어넣을 수 없는가
- ch10 인라인 뷰 사례와의 대조

Predicate Pushdown이란

- 옵티마이저가 필터 조건을 데이터 처리 파이프라인의 더 이른 단계로 옮기는 최적화다
  - 조건 밀어넣기라고 부른다
- 목적 — 뒤 단계에서 처리할 데이터량을 줄이는 것
  - 앞에서 걸러 내면 뒤 단계의 정렬·집계·조인 대상이 함께 줄어든다
- 안전 조건 — 그 조건을 먼저 적용해도 최종 결과가 달라지지 않아야 한다
  - 옵티마이저는 이 동치성을 증명할 수 있을 때만 밀어넣는다
  - 증명할 수 없으면 SQL에 어떻게 쓰여 있든 원래 위치에서 평가한다
- 즉 밀어넣기는 옵티마이저의 재량이 아니라 규칙에 따른 판단이다

그룹 키에 대한 필터는 밀어넣을 수 있다

- WHERE 조건이 GROUP BY 키 컬럼 자체에 걸려 있는 경우
  - 집계 결과가 아니라 그룹을 나누는 기준 컬럼에 대한 조건이라는 뜻이다
- 이 조건은 SQL의 어디에 적혀 있든 그룹화 이전으로 자동으로 밀어넣을 수 있다
  - 인라인 뷰 바깥에 써 있어도 마찬가지다
- 근거 — 어떤 행이 특정 그룹 키 값을 갖는지는 그룹화 전이나 후나 변하지 않는다
  - 먼저 걸러 낸 뒤 그룹을 나누나, 나눈 뒤 해당 그룹만 남기나 결과가 완전히 같다
  - 그룹을 나누는 기준 자체가 이미 좁혀지므로 결과가 달라지지 않는다
- 그룹 키에 대한 등치 조건은 이 안전 조건을 항상 만족한다

HAVING은 왜 밀어넣을 수 없는가

- HAVING처럼 집계 함수의 결과에 거는 조건은 성격이 다르다
  - COUNT(*), SUM(...) 같은 값은 그룹 안의 모든 행을 다 모아 봐야 알 수 있다
- 그래서 이 조건은 집계가 끝난 뒤에만 평가할 수 있다
  - 그룹화 이전으로 밀어넣는 것 자체가 원천적으로 불가능하다
  - 밀어넣으면 아직 계산되지 않은 값과 비교하는 셈이 되어 버린다
- 판별 기준은 한 문장으로 정리된다
  - '이 조건을 평가하는 데 집계 결과가 필요한가' — 필요하면 밀어넣을 수 없다
  - 원본 테이블에 이미 존재하는 값에 대한 조건이면 밀어넣을 수 있다

ch10 인라인 뷰 사례와의 대조

- ch10 — 인라인 뷰 바깥의 WHERE v.cnt > 1000
  - cnt는 집계 함수의 계산 결과 — 그룹화가 끝나야 값을 알 수 있다
  - 따라서 그룹화 이전으로 밀어넣을 수 없다
- ch12 — 인라인 뷰 바깥의 WHERE claim_type = '입원'
  - claim_type은 원본 테이블에 이미 존재하는 값이자 그룹 키 자체다
  - 그룹화와 무관하게 미리 알 수 있으므로 밀어넣기가 가능하다
- 둘 다 인라인 뷰 바깥에서 필터링하는 똑같은 모양이다
  - 모양이 같아도 처리 방식은 다르다 — 겉모양으로 판단할 수 없는 이유

> **참고:**
> ch10에서는 밀어넣기가 불가능했지만, 집계 자체를 다시 계산할 필요는 없어 비용은 같았다 — 밀어넣기 불가가 곧 손해를 뜻하지는 않는다.

12.4

Example — GROUP BY 필터 위치 실측

- 실측 소재 — 필터 위치만 바꾼 두 SQL
- Q1·Q2 공통 실행계획
- 왜 완전히 같은 계획이 되었나
- 실무 교훈

실측 소재 — 필터 위치만 바꾼 두 SQL

- 대상 테이블 — MEDICAL_CLAIMS
- Q1 — WHERE로 먼저 필터한 뒤 hosp_id로 집계
  - 집계 대상 컬럼이 아니라 그룹 키에 대한 필터라는 점이 중요하다
- Q2 — hosp_id와 claim_type으로 집계한 뒤 인라인 뷰 바깥에서 필터
  - 불필요하게 그룹핑 컬럼을 하나 더 추가한, 의도한 비효율 패턴이다
  - 겉보기에는 hosp_id × claim_type 조합만큼 더 많은 그룹을 만드는 것처럼 보인다
- 두 SQL의 결과는 같다 — 비교 대상은 실행계획과 Buffers다
  - 결과가 같은 상태에서 자원만 비교한다는 이 과정의 원칙 그대로다

Q1 — WHERE로 먼저 필터

10.4절 Example — 그룹 키에 대한 필터를 WHERE에 직접 작성

```
-- Q1. WHERE로 먼저 필터(집계 대상 컬럼 아님, 그룹 키에 대한 필터)
SELECT hosp_id, COUNT(*), SUM(total_amt)
FROM   medical_claims
WHERE  claim_type = '입원'
GROUP BY hosp_id;

-- 필터가 그룹 키(claim_type)에 대한 조건이다
-- 집계 함수의 결과에 거는 조건이 아니므로 그룹화 이전 평가가 가능하다
-- 결과 그룹은 hosp_id 단위로만 만들어진다
```

Q2 — 그룹핑 후 바깥에서 필터

10.4절 Example — 의도한 비효율 패턴, 그룹핑 컬럼을 하나 더 추가

```
-- Q2. 불필요하게 claim_type까지 그룹핑한 뒤 바깥에서 필터
SELECT hosp_id, cnt, total FROM (
  SELECT hosp_id, claim_type, COUNT(*) cnt, SUM(total_amt) total
  FROM   medical_claims
  GROUP BY hosp_id, claim_type
) WHERE claim_type = '입원';

-- 겉보기: hosp_id x claim_type 조합만큼 그룹을 만든 뒤 걸러 내는 형태
-- 인라인 뷰 바깥의 필터지만, 참조 대상은 여전히 그룹 키 컬럼이다
-- ch10의 WHERE v.cnt > 1000 과는 참조 대상이 다르다
```

Q1·Q2 공통 실행계획

Plan hash value 완전히 동일: 3971657403

```
Q1, Q2 공통 실행계획 (Plan hash value 완전히 동일: 3971657403)

| Id | Operation          | Name           | A-Rows |   A-Time   | Buffers |
| 0  | SELECT STATEMENT   |                |   1000 |00:00:00.03 |    3090 |
| 1  |  HASH GROUP BY     |                |   1000 |00:00:00.03 |    3090 |
| 2  |   TABLE ACCESS FULL| MEDICAL_CLAIMS|  59511 |00:00:00.03 |    3090 |

-- Q2를 일부러 비효율적으로 작성했는데도 실행계획이 Q1과 완전히 같다
-- claim_type 으로 그룹을 나누는 단계가 계획에 존재하지 않는다
-- HASH GROUP BY 위에 TABLE ACCESS FULL 하나뿐인 단순한 구조
```

실행계획 읽기 — 무엇이 같았나

- Plan hash value가 3971657403으로 완전히 동일하다
  - 계획이 비슷한 정도가 아니라 같은 계획으로 변환됐다는 뜻이다
- 오퍼레이션 구성도 동일하다 — HASH GROUP BY 위에 TABLE ACCESS FULL 하나
  - Q2에만 있어야 할 claim_type 그룹핑 단계가 계획에 남아 있지 않다
- 실측 수치도 동일하다
  - A-Rows 1000, A-Time 00:00:00.03, Buffers 3090
  - TABLE ACCESS FULL 단계의 A-Rows는 59511 — 필터를 통과한 행 수다
- 즉 SQL의 겉모양 차이가 실행 구조에는 전혀 반영되지 않았다

왜 완전히 같은 계획이 되었나

- claim_type = '입원'은 GROUP BY의 기준 컬럼 자체에 대한 조건이다
  - 집계 결과가 아니라 그룹을 나누는 기준 컬럼이라는 점이 결정적이다
- 옵티마이저가 이 조건을 그룹화 이전으로 자동으로 밀어넣었다
  - 실행계획에서는 TABLE ACCESS FULL 단계의 필터로 나타난다
- 밀어넣는 순간 claim_type은 하나의 값으로 고정된다
  - 고정된 컬럼으로는 그룹을 더 나눌 이유가 없다
  - 그래서 GROUP BY hosp_id만 남긴 것과 동일한 계획이 됐다
- 두 단계 변환이 연달아 일어난 결과다 — 밀어넣기, 그리고 불필요한 그룹 키 제거

실무 교훈 — 필터 위치 걱정의 실체

- "필터를 먼저 쓰지 않으면 손해"라는 걱정은 대체로 기우다
  - 단 그 필터가 그룹 키에 대한 것이라는 조건이 붙는다
  - 옵티마이저가 알아서 밀어넣어 주기 때문이다
- 중요한 것은 필터를 SQL의 어디에 썼느냐가 아니다
  - 그 필터가 그룹 키에 관한 것이냐, 집계 결과에 관한 것이냐가 처리 방식을 결정한다
- 그렇다고 확인 없이 아무렇게나 써도 된다는 뜻은 아니다
  - 밀어넣기가 성립하는 조건을 알고 쓰는 것과 모르고 쓰는 것은 다르다
  - 결론은 언제나 같다 — 실행계획으로 확인한다

12.5

Bad SQL — 정렬 생략은 선택도에 달려 있다

- 실험 설계 — 같은 두 컬럼, 다른 선택도
- 범위가 넓을 때 — PK인데도 정렬이 남는다
- 범위가 좁을 때 — 정렬이 사라진다
- 정렬이 없는 쪽이 더 비싼 역설

실험 설계 — 같은 두 컬럼, 다른 선택도

- 같은 두 컬럼을 정렬 기준으로 바꿔가며 비교한다
  - CLAIM_ID — PK 컬럼, 인덱스의 정렬 기준과 일치한다
  - TOTAL_AMT — 인덱스가 없는 컬럼이다
- 필터 범위는 두 가지로 나눈다
  - 넓을 때 — 약 1개월, 8.5% 선택도
  - 좁을 때 — 약 1일, 0.27% 선택도
- 8.5% 구간은 ch05에서 이미 Full Scan 전환 구간으로 확인된 범위다
  - 즉 접근 경로가 달라지도록 의도적으로 설계한 두 구간이다

범위가 넓을 때 — 약 1개월, 8.5% 선택도

10.5절 Bad SQL — 두 정렬 기준 모두 SORT가 발생

```
+------------------------------+-------------------------------------+---------+
| 정렬 기준                    | Operation                           | Buffers |
+------------------------------+-------------------------------------+---------+
| ORDER BY total_amt (비인덱스) | SORT ORDER BY + TABLE ACCESS FULL |   3,090 |
| ORDER BY claim_id (PK 컬럼)   | SORT ORDER BY + TABLE ACCESS FULL |   3,090 |
+------------------------------+-------------------------------------+---------+

-- 두 경우 모두 정렬이 생략되지 않았다
-- claim_id 가 PK라 해도 접근 경로가 TABLE ACCESS FULL 이면 순서 보장이 없다
-- ch05에서 확인된 Full Scan 전환 구간이라 인덱스 자체가 채택되지 않았다
```

넓은 범위 해석 — PK인데도 정렬이 남는 이유

- 정렬이 생략되려면 데이터를 이미 정렬된 순서로 가져와야 한다
  - 그러려면 정렬 순서를 보장하는 구조, 즉 인덱스를 통해 접근해야 한다
- 이 사례에서는 선택도가 8.5%로 이미 Full Scan이 유리한 구간이었다
  - ch05의 접근 방식 전환 원칙에 따라 옵티마이저가 TABLE ACCESS FULL을 선택했다
- Full Scan은 테이블에 저장된 물리적 순서로 읽는다
  - 물리적 순서는 정렬 순서와 무관하다 — 어떤 순서도 보장하지 않는다
  - 그 컬럼이 PK이든 아니든 이 접근 경로에서는 정렬 순서를 보장받지 못한다
- 그래서 두 경우 모두 별도의 SORT ORDER BY가 필요했다

범위가 좁을 때 — 약 1일, 0.27% 선택도

10.5절 Bad SQL — 인덱스 컬럼 정렬에서만 SORT가 생략

```
+-------------------------------+-----------------------------------+---------+
| 정렬 기준                     | Operation                         | Buffers |
+-------------------------------+-----------------------------------+---------+
| ORDER BY claim_id (PK 컬럼)   | INDEX RANGE SCAN만 (정렬 생략됨) |     791 |
| ORDER BY total_amt (비인덱스) | SORT ORDER BY + INDEX RANGE SCAN |     731 |
+-------------------------------+-----------------------------------+---------+

-- 좁은 선택도 덕분에 INDEX RANGE SCAN 이 채택됐다
-- claim_id 정렬은 인덱스가 이미 그 순서로 반환하므로 SORT 단계가 아예 없다
-- total_amt 는 인덱스에 없는 컬럼이라 여전히 정렬이 필요하다
-- 정렬을 생략한 쪽(791)이 정렬한 쪽(731)보다 Buffers 가 크다는 점에 주목
```

좁은 범위 해석 — 정렬이 사라진 이유

- 좁은 범위에서는 선택도가 매우 낮아 인덱스 접근이 유리해졌다
  - 옵티마이저가 PK_MEDICAL_CLAIMS를 통한 INDEX RANGE SCAN을 선택했다
  - ch05의 원칙이 그대로 적용된 결과다
- 인덱스는 CLAIM_ID 값 순서로 정렬되어 저장된다
  - INDEX RANGE SCAN으로 읽어오는 순서가 ORDER BY claim_id의 요구와 정확히 일치한다
  - 이미 원하는 순서로 나오므로 다시 정렬할 필요가 없어 SORT 단계가 생략됐다
- 반면 ORDER BY total_amt는 여전히 SORT ORDER BY가 필요하다
  - 인덱스에 없는 컬럼이므로 인덱스의 순서를 활용할 수 없다

정렬이 없는 쪽이 더 비싼 역설

- 정렬을 생략한 claim_id 쪽 Buffers는 791, 정렬이 발생한 total_amt 쪽은 731이다
  - 정렬이 있는 쪽이 오히려 Buffers가 작다
- 원인은 테이블 블록 접근 방식의 차이에 있다
  - TABLE ACCESS BY INDEX ROWID — 개별 ROWID로 하나씩 접근
  - TABLE ACCESS BY INDEX ROWID BATCHED — ROWID를 먼저 모아 가까운 것끼리 묶어 접근
  - BATCHED는 개별 접근보다 효율적일 수 있다 (ch03)
- 정렬 비용을 상쇄하고도 남을 만큼 테이블 접근이 효율적이었던 경우다
  - "정렬이 없으면 항상 더 싸다"는 단순한 가정은 항상 맞지는 않다

> **참고:**
> 결국 실측해서 직접 비교해야 한다 — 요인이 여럿일 때는 어느 하나만 보고 예측한 결론이 뒤집힐 수 있다.

12.6

Tuning — 정렬 생략의 두 조건

- 정렬 생략의 두 조건
- ch05 선택도 원칙과의 연결
- HASH GROUP BY는 생략되지 않는다
- GROUP BY + ORDER BY 조합 추론 / 체크리스트

정렬 생략의 두 조건

- '인덱스 컬럼으로 정렬하면 정렬이 생략된다'는 조건부 사실이다
  - 컬럼 하나만 보는 조건이라 부정확하다
- 정확한 조건은 두 가지가 함께 성립해야 한다
  - (1) 정렬 대상 컬럼이 인덱스의 정렬 기준과 일치해야 한다
  - (2) 그 인덱스가 실제로 이 쿼리의 접근 경로로 채택되어야 한다
- 둘 중 하나라도 빠지면 정렬은 그대로 발생한다
  - 넓은 범위 사례는 (1)은 만족했지만 (2)가 성립하지 않은 경우다
  - 좁은 범위의 total_amt는 (2)는 성립했지만 (1)이 빠진 경우다

ch05 선택도 원칙과의 연결

- ch05에서 다룬 원칙 — 선택도에 따라 접근 방식이 전환된다
  - 선택도가 충분히 낮으면 인덱스, 높으면 Full Scan이 유리해진다
- 그 원칙이 ORDER BY의 정렬 생략 여부에도 그대로 연결된다
  - 접근 경로가 바뀌면 정렬 순서를 물려받을 수 있는지도 함께 바뀐다
- 같은 컬럼이라도 쿼리의 선택도에 따라 정렬 생략 여부가 달라질 수 있다
  - 이번 실습의 claim_id가 정확히 그 사례다
- 그래서 "이 SQL은 정렬이 없다"는 말은 조건 없이는 성립하지 않는다
  - 데이터가 늘어 선택도가 달라지면 어제 없던 정렬이 오늘 생길 수 있다

HASH GROUP BY는 생략되지 않는다

- HASH GROUP BY는 이번 장의 모든 GROUP BY 쿼리에서 공통으로 나타났다
- HASH GROUP BY는 그룹 키 값을 해시로 묶어 집계하는 방식이다
  - 인덱스의 정렬 순서와 무관하게 동작한다
  - 정렬된 순서로 데이터를 읽어와도 HASH GROUP BY 자체는 생략되지 않는다
  - 그룹핑이라는 작업 자체가 필요 없어지는 것이 아니기 때문이다
- 정렬 순서를 활용해 그룹화 비용을 줄이려면 SORT GROUP BY가 선택돼야 한다
  - 이는 옵티마이저가 별도로 판단하는 영역이다
  - 이번 챕터의 실습에서는 SORT GROUP BY가 관찰되지 않았다

GROUP BY + ORDER BY 조합은 어떻게 될까

- 요구 — 병원별 청구 건수를 병원 ID 순으로 정렬해 보여 달라
  - GROUP BY hosp_id ORDER BY hosp_id 형태로 작성하게 된다
- HASH GROUP BY로 처리된다면
  - 해시 기반 처리 특성상 결과가 hosp_id 순서로 나온다는 보장이 없다
  - 이론적으로는 별도의 SORT ORDER BY가 추가로 필요할 가능성이 높다
- 옵티마이저가 두 절이 같은 컬럼임을 인지한다면
  - SORT GROUP BY를 선택해 그 정렬 결과를 ORDER BY 결과로 재사용할 수도 있다
- 어느 쪽이 선택될지는 데이터 규모·통계정보에 따른 비용 판단의 문제다
  - 단정하지 말고 실행계획으로 직접 확인한다

실무 체크리스트

- 집계 쿼리를 볼 때
  - 이 필터는 그룹 키에 대한 것인가, 집계 결과에 대한 것인가
  - 집계 결과에 대한 조건이라면 그룹화 이후에만 평가된다는 점을 감안했는가
- 정렬 쿼리를 볼 때
  - 정렬 대상 컬럼이 인덱스의 정렬 기준과 일치하는가
  - 그 인덱스가 이 쿼리의 접근 경로로 실제 채택됐는가
  - 실행계획에 SORT ORDER BY가 남아 있는가
- 비교할 때
  - 정렬 유무만이 아니라 테이블 접근 방식까지 함께 본다
  - 요인이 여럿이면 예측보다 실측을 우선한다

12.7

Benchmark & Review

- 네 시나리오 실측 요약
- 관측 사실과 흔한 오해 정리
- 이 장의 결론

Benchmark — 네 시나리오 실측 요약

10.7절 Benchmark — 이번 장 전체 실측 결과

```
+-----------------------------------------------+---------+-----------------------------------+
| 시나리오                                      | Buffers | SORT 발생 여부                    |
+-----------------------------------------------+---------+-----------------------------------+
| GROUP BY 키에 대한 필터 (Q1, Q2 – 위치 무관)  |   3,090 | HASH GROUP BY 발생 (정렬과 무관) |
| 넓은 범위 ORDER BY (8.5%, 두 컬럼 모두)       |   3,090 | SORT ORDER BY 발생                |
| 좁은 범위 ORDER BY, 인덱스 컬럼 (claim_id)    |     791 | SORT 생략됨                       |
| 좁은 범위 ORDER BY, 비인덱스 컬럼 (total_amt) |     731 | SORT ORDER BY 발생                |
+-----------------------------------------------+---------+-----------------------------------+

-- 1행: 필터 위치를 바꿔도 실행계획이 완전히 동일했다
-- 2행: 접근 경로가 Full Scan 이면 PK 컬럼 정렬도 생략되지 않는다
-- 3~4행: 같은 접근 경로에서도 정렬 기준 컬럼에 따라 결과가 갈린다
```

관측 사실과 흔한 오해 정리

- 관측 1 — GROUP BY 필터 위치를 바꿔도 실행계획이 완전히 같았다
- 관측 2 — 같은 컬럼도 선택도에 따라 정렬 생략 여부가 달라졌다
- 오해 1 — '필터를 먼저 쓰지 않으면 손해다'
  - 그 필터가 그룹 키에 대한 것이라면 대체로 기우다
- 오해 2 — '인덱스 컬럼으로 정렬하면 정렬이 생략된다'
  - 그 인덱스가 접근 경로로 채택되어야 비로소 성립한다
- 오해 3 — '정렬이 없으면 항상 더 싸다'
  - 좁은 범위 사례의 Buffers 791 대 731이 반례다
- 오해 4 — 'GROUP BY 결과는 그룹 키 순서로 나온다'
  - HASH GROUP BY는 결과 순서를 보장하지 않는다

요약 (Summary)

- WHERE 조건이 GROUP BY의 그룹 키 자체에 대한 것이면 옵티마이저가 그룹화 이전으로 밀어넣을 수 있다
- 그래서 필터를 SQL의 어디에 썼는지는 대체로 실행계획에 영향을 주지 않았다 — Plan hash value까지 동일
- HAVING처럼 집계 결과에 거는 조건은 집계가 끝나야 값을 알 수 있어 밀어넣을 수 없다
- 판별 기준 한 문장 — 이 조건을 평가하는 데 집계 결과가 필요한가
- ORDER BY 대상이 인덱스 컬럼이라는 것만으로는 정렬이 생략되지 않는다
- 정렬 생략의 두 조건 — 인덱스 정렬 기준과 일치할 것, 그 인덱스가 접근 경로로 채택될 것
- 같은 컬럼이라도 선택도에 따라 정렬 생략 여부가 달라진다 (ch05와 연결)
- HASH GROUP BY는 정렬 순서로 생략되지 않는다 — 방식 변경만 가능한 연산이다

실습 안내

- 실습문제 10문항 — ch12_문제.md
  - 1~2번 Q1·Q2의 계획이 동일해진 이유와 Predicate Pushdown 서술
  - 3번 ch10 인라인 뷰 사례와의 차이 — 집계 결과 필터와 그룹 키 필터
  - 4~5번 넓은 범위와 좁은 범위에서 정렬 생략 여부가 갈린 이유
  - 6번 정렬 생략의 정확한 조건 서술
  - 7번 Buffers 776과 717의 역전이 시사하는 바
  - 8번 HASH GROUP BY가 정렬 순서로 생략될 수 있는 연산인지
  - 9번 GROUP BY + ORDER BY 조합에 대한 원칙 적용 추론
  - 10번 SQL의 겉모양보다 먼저 확인할 것에 대한 결론
- 실습 전 확인 — 이번 장은 DDL 변경이 없다
  - 인덱스를 추가로 만들면 교안의 실행계획이 재현되지 않는다

실무 SQL 문제와 SQL 부문 평가

- SQLT ch13 · ch14 · 심화 30제 V · 최종실습시험 03·04
- 평가 40%. 종료 시 인덱스 원복 확인

Oracle SQL 튜닝 과정 · ch13

페이징 SQL 튜닝

부분범위 처리 · ROWNUM 중첩 뷰와 FETCH FIRST · WINDOW SORT PUSHED RANK vs NOSORT STOPKEY · Keyset Pagination

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - 부분범위 처리의 개념을 정의하고 전체범위 처리와 구분해 설명한다
  - OFFSET n ROWS FETCH NEXT m ROWS ONLY 구문의 동작을 설명한다
  - ROWNUM 중첩 인라인 뷰 방식의 페이징 SQL 구조를 읽고 작성한다
  - OFFSET 쿼리가 내부적으로 ROW_NUMBER()로 변환되는 과정을 설명한다
  - WINDOW SORT PUSHED RANK와 WINDOW NOSORT STOPKEY를 구분해 읽는다
  - OFFSET이 깊어질 때 Buffers는 그대로인데 A-Time이 늘어나는 이유를 말한다
  - 정렬 작업 메모리(OMem·1Mem·Used-Mem)를 성능 판단 근거로 함께 확인한다
  - Keyset Pagination의 SQL을 작성하고 실행계획으로 효과를 확인한다
  - INDEX RANGE SCAN DESCENDING이 가능한 구조적 이유를 설명한다
  - Keyset 방식의 제약을 UI 요구사항과 연결해 적용 여부를 판단한다

이 장의 구성 (Agenda)

- 11.1 Why — 뒷페이지로 갈수록 느려지는 화면
- 11.2 Concept — OFFSET/FETCH · ROWNUM · Keyset Pagination
- 11.3 Oracle Internals — OFFSET이 깊어지면 왜 느려지는가
- 11.4 Example — 청구 목록을 접수일 내림차순으로 페이징
- 11.5 Execution Plan — OFFSET 깊이에 따른 실측
- 11.6 Bad SQL — Keyset 방식과의 실행계획 비교
- 11.7 Tuning · Benchmark — Keyset의 제약과 절충안
- 11.8~11.9 실습 안내 · 요약 (Review)

13.1

뒷페이지로 갈수록 느려지는 화면

- 리스트·게시판 화면에서 흔한 성능 민원
- 1페이지는 빠른데 뒤로 갈수록 느려진다
- 이 차시의 목표와 관찰 소재

현장에서 반복되는 민원

- "1페이지는 빠른데 뒷페이지로 갈수록 느려진다"
  - 리스트·게시판·검색 결과 화면에서 매우 흔하게 발생하는 문제
  - 개발 단계에서는 대개 1~2페이지만 확인하므로 발견되지 않는다
- 민원이 애매하게 보고되는 이유
  - 같은 SQL, 같은 화면인데 바인드 값 하나만 다르므로 재현이 어렵다
  - "가끔 느리다"로 접수되어 서버 부하나 네트워크 탓으로 오진되기 쉽다
- 이 장은 이 현상을 실측으로 특정한다
  - 왜 깊은 페이지에서만 느려지는지를 실행계획 오퍼레이션 단위로 확인
  - OFFSET 깊이가 늘어도 비용이 거의 늘지 않는 대안까지 함께 확인

> **참고:**
> 이 장의 모든 실행계획·A-Time·Buffers·작업 메모리 값은 진료비청구심사 실습 DB에서 직접 캡처한 실측값이다.

이 차시의 목표와 관찰 소재

- 목표 ① 현상의 원인을 실측으로 확인한다
  - 깊은 페이지에서 비싸지는 비용이 정확히 어느 오퍼레이션에서 발생하는지
  - 그 비용이 실행계획의 어느 컬럼에 나타나고 어느 컬럼에는 안 나타나는지
- 목표 ② OFFSET 깊이와 무관한 대안을 확인한다
  - Keyset Pagination — 마지막으로 본 값을 기준으로 다음 페이지를 가져오는 방식
- 관찰 소재
  - 청구 목록을 접수일 내림차순으로 20건씩 페이징 조회
  - MEDICAL_CLAIMS 테이블, 정렬 기준 컬럼은 receipt_date
  - 같은 요구사항을 OFFSET 방식과 Keyset 방식 두 가지로 작성해 비교
- 이번 차시는 실습용 인덱스를 만들었다가 종료 시 원상복구한다

13.2

OFFSET/FETCH · ROWNUM · Keyset

- OFFSET n ROWS FETCH NEXT m ROWS ONLY
- ROWNUM 중첩 인라인 뷰 — 전통적 방법
- Keyset Pagination — 값으로 접근하는 방식

OFFSET / FETCH 구문

- OFFSET n ROWS FETCH NEXT m ROWS ONLY
  - 정렬된 결과에서 앞의 n건을 건너뛰고 그다음 m건을 가져온다
  - ORDER BY 뒤에 붙이며, 정렬 순서가 정해져야 "몇 번째"가 의미를 가진다
- 페이지 번호와의 매핑이 직관적이다
  - 한 페이지 m건이면 p페이지의 OFFSET은 (p-1) * m
  - 화면의 페이지 번호를 그대로 바인드 변수로 넘길 수 있다
- FETCH FIRST m ROWS ONLY — OFFSET 없이 상위 m건만
  - "최근 20건"처럼 첫 화면만 필요할 때 쓰는 형태
- 주의 — ORDER BY 없이 쓰면 어떤 m건이 나올지 보장되지 않는다
  - 정렬 기준이 없으면 실행계획이 바뀔 때 결과 집합도 달라질 수 있다

전통적 방법 — ROWNUM 중첩 인라인 뷰

OFFSET/FETCH가 없던 시절 Oracle에서 페이징을 구현하던 방법 (구조 참고용)

```
-- 인라인 뷰를 두 겹으로 감싸고 ROWNUM을 비교하는 전통적 패턴
SELECT claim_id, receipt_date
FROM   ( SELECT a.*, ROWNUM AS rn
         FROM   ( SELECT claim_id, receipt_date
                  FROM   medical_claims
                  ORDER  BY receipt_date DESC ) a
         WHERE  ROWNUM <= :끝행 )
WHERE  rn > :시작행;

-- 가장 안쪽 뷰: 정렬만 담당한다
-- 가운데 뷰: 정렬된 결과에 ROWNUM으로 번호를 매기고 끝행까지 자른다
-- 바깥 쿼리: 시작행 이전을 버린다

-- ROWNUM은 행이 확정되는 시점에 매겨지므로 ORDER BY보다 먼저 계산된다
-- 그래서 정렬을 안쪽 뷰로 한 번 감싸 두지 않으면 엉뚱한 결과가 나온다
```

ROWNUM 방식과 OFFSET/FETCH의 관계

- 표현은 다르지만 요구사항은 같다
  - 둘 다 "정렬된 결과에서 몇 번째부터 몇 번째까지"를 뽑는 구조
  - ROWNUM 중첩 뷰는 그 구조를 사람이 직접 손으로 짠 것에 가깝다
- OFFSET/FETCH는 그 구조를 문법으로 흡수한 것
  - 중첩 뷰를 직접 쓰지 않아도 되므로 SQL이 짧고 오작성 위험이 줄어든다
  - 내부적으로는 여전히 순번을 매기는 작업이 수행된다 — 11.3에서 확인
- 그래서 문법을 바꾼다고 성능이 저절로 좋아지지 않는다
  - "FETCH FIRST로 바꿨는데 왜 안 빨라지나"는 흔한 오해
  - 비용을 결정하는 것은 문법이 아니라 실행계획에 나타나는 오퍼레이션이다

Keyset Pagination — 값으로 접근한다

- "몇 번째부터"가 아니라 "마지막으로 본 값보다 작은 것부터"
  - 직전 페이지의 마지막 행 값을 커서처럼 들고 다니며 다음 페이지를 요청
  - 예: WHERE receipt_date < :마지막으로_본_날짜
- 순번을 셀 필요가 없어진다는 점이 핵심이다
  - 조건이 값 비교이므로 인덱스에서 그 지점을 바로 찾아 들어갈 수 있다
  - 페이지 깊이와 무관하게 시작점을 한 번에 특정할 수 있다
- 다른 이름으로도 불린다
  - 커서 기반 페이징 · Seek Method — 부르는 이름은 달라도 구조는 같다
- 대신 잃는 것이 있다 — 11.7에서 다룬다
  - 임의의 페이지 번호로 바로 점프하는 기능을 포기해야 한다

세 가지 접근을 한 화면에 정리

- ROWNUM 중첩 인라인 뷰
  - 접근 기준은 순번 · 표현은 수동 · 오작성 위험이 상대적으로 크다
- OFFSET n ROWS FETCH NEXT m ROWS ONLY
  - 접근 기준은 순번 · 표현은 문법으로 흡수 · 페이지 번호 UI에 그대로 대응
  - 19c에서 사용하는 표준 형태이며 이 장의 실측 대상이다
- Keyset Pagination
  - 접근 기준은 값 · 다음 페이지의 시작점을 조건으로 표현
  - 순차 이동 UI에 적합하고 임의 페이지 점프에는 부적합하다
- 앞의 두 방식은 접근 기준이 같으므로 비용 구조도 같은 계열이다
  - 진짜 갈림길은 문법이 아니라 "순번이냐 값이냐"에서 갈린다

13.3

OFFSET이 깊어지면 왜 느려지는가

- ROW_NUMBER() 변환과 순위 매김의 범위
- Buffers에 안 나타나는 CPU·메모리 비용
- WINDOW SORT PUSHED RANK와 NOSORT STOPKEY

OFFSET을 처리하려면 순번이 필요하다

- OFFSET n ROWS는 "정렬된 순서에서 n번째"라는 뜻이다
  - 따라서 Oracle은 각 행이 몇 번째인지를 알아야 조건을 판정할 수 있다
  - 내부적으로 ROW_NUMBER() OVER (ORDER BY ...) 형태로 변환된다
  - 변환 흔적은 실행계획의 Predicate Information에서 확인할 수 있다
- 순번은 앞에서부터 차례로만 매길 수 있다
  - 중간부터 세기 시작할 방법이 없으므로 원칙적으로 처음부터 세야 한다
  - OFFSET n에 FETCH m이면 최소한 n+m번째 행까지 순번이 필요하다
- 그래서 OFFSET이 커질수록 "순서를 매겨야 하는 범위"가 커진다
  - 건너뛰는 행이라고 해서 공짜로 건너뛰는 것이 아니다

그 비용은 어디에 나타나는가

- 읽어야 하는 테이블 블록 수는 OFFSET과 무관하다
  - 전체를 한 번 스캔하는 것은 OFFSET 값이 얼마든 동일하기 때문
  - 따라서 Buffers는 OFFSET 깊이에 따라 거의 변하지 않는다
- 늘어나는 것은 정렬·순위 매김 작업량이다
  - CPU 시간 — 실행계획의 A-Time에 나타난다
  - 작업 메모리 — OMem · 1Mem · Used-Mem에 나타난다
- ch11에서 확인한 함정이 여기서 그대로 재현된다
  - ch11는 UNION의 SORT UNIQUE, 이 장은 순위 매김 정렬
  - 둘 다 이미 메모리에 올린 데이터를 다루므로 추가 블록 접근이 없다
- 결론 — 정렬 계열 연산이 있으면 Buffers만으로 판단하지 않는다

WINDOW SORT PUSHED RANK

- 전체 결과에 순위를 매기는 오퍼레이션
  - ROW_NUMBER()로 변환된 순번 계산이 이 오퍼레이션에서 수행된다
  - PUSHED RANK — 순위 조건을 정렬 단계 안쪽으로 밀어 넣었다는 뜻
- OFFSET이 커질수록 이 오퍼레이션의 부담이 커진다
  - 순위를 매겨야 하는 행 수 자체가 OFFSET에 비례해 늘어나기 때문
  - 정렬된 상태를 유지해야 하므로 작업 메모리 사용량도 함께 커진다
- 이 오퍼레이션이 보이면 페이지 깊이를 반드시 확인한다
  - 1페이지 기준 실행계획만 보고 넘어가면 문제를 놓치게 된다
  - 실무에서는 가장 깊은 페이지를 기준으로도 한 번 측정해 보는 것이 안전하다

WINDOW NOSORT STOPKEY

- 이름 자체가 두 가지 이득을 말해 준다
  - NOSORT — 입력이 이미 정렬된 순서로 들어오므로 정렬하지 않는다
  - STOPKEY — 필요한 건수를 채우는 즉시 더 읽지 않고 멈춘다
- NOSORT가 가능한 조건
  - 정렬 기준 컬럼의 인덱스를 순서대로 따라 읽는 계획이 선택된 경우
  - 인덱스가 이미 정렬 순서를 제공하므로 별도 정렬 단계가 사라진다
- STOPKEY가 가능한 조건
  - 앞에서부터 필요한 만큼만 읽으면 결과가 확정되는 구조일 것
  - 전체를 다 봐야 순위가 확정되는 구조에서는 중단할 수 없다
- ch03의 Q4, ch12의 좁은 범위 사례와 같은 원리다

부분범위 처리 — 이 장의 축

- 전체범위 처리 — 결과 전체를 만들어 놓고 그중 일부를 준다
  - 전체를 읽고 정렬하고 순위를 매긴 뒤 해당 구간을 잘라 내는 방식
  - 앞의 몇 건만 필요해도 마지막 행까지의 작업이 선행되어야 한다
- 부분범위 처리 — 필요한 만큼만 만들고 멈춘다
  - 요청한 건수를 채우는 순간 남은 데이터는 건드리지 않는다
  - 실행계획에서는 STOPKEY 계열 오퍼레이션으로 드러난다
- 페이징은 본질적으로 부분범위 처리가 어울리는 요구사항이다
  - 화면에 20건만 보여 주는데 30만 건을 정렬할 이유가 없다
  - 그런데 OFFSET 방식은 순번 때문에 전체범위 처리로 끌려간다
- 이 장의 튜닝은 요구사항을 부분범위 처리로 되돌리는 작업이다

13.4

청구 목록 페이징 조회

- 실습용 인덱스 생성 — IX_RECEIPT
- OFFSET 방식 SQL
- Keyset 방식 SQL

관찰 소재 정리

- 요구사항 — 청구 목록을 접수일 내림차순으로 20건씩 페이징 조회
  - 대상 테이블은 MEDICAL_CLAIMS, 정렬 기준은 receipt_date
  - 조회 컬럼은 claim_id와 receipt_date 두 개
- 같은 요구사항을 두 방식으로 작성한다
  - OFFSET 방식 — 페이지 번호로 접근한다
  - Keyset 방식 — 마지막으로 본 값으로 접근한다
- 실습용 인덱스를 하나 만든다
  - receipt_date 단일 컬럼 인덱스 — 정렬 순서를 제공하기 위한 것
  - 두 방식 모두 같은 인덱스가 존재하는 상태에서 측정한다
- 차시 종료 시 실습 인덱스는 반드시 삭제한다

> **참고:**
> 실습용 인덱스는 IX_CH11_ 접두어로 만들고 차시 종료 시 DROP INDEX로 정리한다. MEDICAL_CLAIMS에는 PK_MEDICAL_CLAIMS만 남아야 한다.

인덱스 생성

정렬 기준 컬럼에 인덱스를 만들어 두고 두 방식을 같은 조건에서 비교한다

```
-- 접수일 단일 컬럼 인덱스
CREATE INDEX ix_receipt ON medical_claims(receipt_date);

-- 이 인덱스의 역할은 두 가지다
--   1) receipt_date 조건으로 진입 지점을 찾을 수 있게 한다 (access)
--   2) receipt_date 순서를 그대로 제공한다 (정렬 생략의 전제)

-- 인덱스가 있다는 사실만으로 옵티마이저가 반드시 쓰는 것은 아니다
-- 쓸 이유가 있어야 쓴다 — 그 이유가 무엇인지가 이 장의 관찰 대상

-- 차시 종료 시 정리
-- DROP INDEX ix_receipt;
```

OFFSET 방식 — 페이지 번호로 접근

정렬된 결과에서 앞의 n건을 건너뛰고 그다음 20건을 가져온다

```
-- OFFSET 방식: 페이지 번호로 접근
SELECT claim_id, receipt_date FROM medical_claims
ORDER BY receipt_date DESC
OFFSET :페이지시작 ROWS FETCH NEXT 20 ROWS ONLY;

-- 바인드 변수 :페이지시작 하나로 모든 페이지를 표현할 수 있다
-- 1페이지면 0, 2페이지면 20, p페이지면 (p-1)*20

-- WHERE 절이 전혀 없다는 점에 주목할 것
-- 특정 값으로 인덱스를 찾아 들어갈 조건(access predicate)이 없다
-- 조건이라고는 "몇 번째부터 몇 번째까지"라는 순번 조건뿐이다

-- 화면 개발 입장에서는 가장 편한 형태 – 페이지 번호를 그대로 넘기면 된다
```

Keyset 방식 — 마지막으로 본 값으로 접근

직전 페이지의 마지막 접수일을 기준으로 그다음 20건을 가져온다

```
-- Keyset 방식: 마지막으로 본 값으로 접근
SELECT claim_id, receipt_date FROM medical_claims
WHERE receipt_date < :마지막으로_본_날짜
ORDER BY receipt_date DESC
FETCH FIRST 20 ROWS ONLY;

-- 앞의 SQL과 비교했을 때 달라진 것은 두 가지뿐이다
--   1) WHERE 절이 생겼다 – 값으로 진입 지점을 지정한다
--   2) OFFSET이 사라졌다 – 순번을 셀 필요가 없어졌다

-- 애플리케이션은 화면에 뿌린 마지막 행의 receipt_date를 들고 있다가
-- "다음" 요청 시 그 값을 그대로 바인드 변수로 넘긴다

-- 페이지 번호가 아니라 커서를 넘기는 구조라는 점이 핵심이다
```

두 SQL을 나란히 읽기

- 공통점 — 결과는 동등하다
  - 같은 테이블, 같은 정렬 기준, 같은 20건
  - 같은 인덱스가 존재하는 상태에서 실행한다
- 차이 ① 조건의 성격
  - OFFSET 방식 — 순번 조건만 있고 값 조건이 없다
  - Keyset 방식 — receipt_date 값 비교 조건이 있다
- 차이 ② 다음 페이지를 표현하는 방법
  - OFFSET 방식 — 페이지 번호를 늘린다
  - Keyset 방식 — 직전 페이지의 마지막 값을 넘긴다
- 이 두 차이가 실행계획에서 어떤 차이로 나타나는지가 다음 절의 주제다

13.5

OFFSET 깊이에 따른 실측

- OFFSET 0 · 200,000 · 280,000 비교
- Buffers는 그대로, A-Time과 작업 메모리는 급증
- Buffers만 보면 놓치는 비용

실측 표 — OFFSET 깊이별 비용

같은 SQL의 바인드 값만 바꿔 세 번 실행한 결과 (실습 DB 실측)

OFFSET	Buffers	A-Time	정렬 작업 메모리(Used-Mem)
0 (1페이지)	3,090	0.02초	2K
200,000 (약 1만 페이지)	3,090	0.14초	10M
280,000 (약 1만4천 페이지)	3,090	0.18초	14M

```
-- Buffers는 세 경우 모두 3,090으로 동일하다
-- A-Time은 0.02초에서 0.18초로 9배 늘었다
-- 정렬 작업 메모리는 2K에서 14M로 급증했다

-- 계획에는 TABLE ACCESS FULL과 WINDOW SORT PUSHED RANK가 나타났다
```

표 읽기 ① Buffers가 같은 이유

- 세 경우 모두 Buffers 3,090으로 동일하다
  - OFFSET 값이 얼마든 읽은 블록 수가 변하지 않았다는 뜻
- 이유는 접근 방식에 있다 — TABLE ACCESS FULL
  - Full Scan은 조건과 무관하게 테이블의 모든 블록을 읽는다
  - 이 쿼리에는 WHERE 조건이 없어 걸러 낼 것도 없다
  - 전체를 한 번은 다 읽어야 순위를 매길 수 있으므로 OFFSET과 무관하다
- 그래서 Buffers는 이 문제를 진단하는 데 쓸모가 없다
  - I/O 비용은 정말로 늘지 않았다 — 지표가 틀린 것이 아니다
  - 다만 이 사례의 비용은 I/O가 아닌 곳에서 발생하고 있을 뿐이다

표 읽기 ② A-Time과 메모리가 늘어난 이유

- A-Time 0.02초 → 0.18초, 작업 메모리 2K → 14M
  - OFFSET이 커질수록 두 값이 함께 커지는 경향이 뚜렷하다
- 원인은 WINDOW SORT PUSHED RANK 오퍼레이션이다
  - 전체 결과에 ROW_NUMBER()로 순번을 매기는 작업
  - OFFSET 0이면 20번째까지만 순번이 필요하다
  - OFFSET 280,000이면 280,020번째까지 순번을 매겨야 한다
- 순번을 매기려면 정렬된 상태를 유지해야 한다
  - 그만큼의 CPU 시간과 작업 메모리가 함께 필요해진다
  - 이 비용이 A-Time과 Used-Mem에 나타난 것이다
- 읽은 블록은 그대로인데 그 블록으로 하는 일이 늘어난 셈이다

오판의 위험 — Buffers만 봤다면

- 만약 Buffers만 보고 판단했다면 어떤 결론이 나왔을까
  - "페이지가 깊어져도 비용은 그대로다"
  - "이 페이징 쿼리는 튜닝할 것이 없다"
- 실제 사용자 경험은 정반대다
  - 깊은 페이지로 갈수록 응답 시간이 눈에 띄게 느려진다
  - 동시 사용자가 많으면 작업 메모리 압박으로 서버 전체에 영향을 준다
- 판단 절차로 정리하면
  - ① 실행계획에 정렬 계열 오퍼레이션이 있는지 먼저 확인한다
  - ② 있으면 Buffers와 함께 A-Time·Used-Mem을 반드시 본다
  - ③ 바인드 값을 바꿔 가며 최소 두 지점 이상에서 측정한다
- ch11의 교훈이 그대로 재현된 사례다

13.6

Keyset 방식과의 실행계획 비교

- WINDOW NOSORT STOPKEY의 등장
- INDEX RANGE SCAN DESCENDING
- OFFSET 쿼리는 왜 인덱스를 못 썼는가

Keyset 방식 실행계획

같은 정도의 깊이(약 280,000번째 부근)를 Keyset 방식으로 조회한 결과

```
| Id | Operation                     | Name            | A-Rows |   A-Time    | Buffers |
| 0  | SELECT STATEMENT              |                 |     20 |00:00:00.01 |      25 |
| 1  |  VIEW                         |                 |     20 |00:00:00.01 |      25 |
| 2  |   WINDOW NOSORT STOPKEY       |                 |     20 |00:00:00.01 |      25 |
| 3  |    TABLE ACCESS BY INDEX ROWID| MEDICAL_CLAIMS  |     20 |00:00:00.01 |      25 |
| 4  |     INDEX RANGE SCAN DESCENDING| IX_RECEIPT     |     20 |00:00:00.01 |       6 |

-- 앞 절의 OFFSET 280,000 방식: Buffers 3,090 / A-Time 0.18초
-- 이번 Keyset 방식:           Buffers    25 / A-Time 0.01초

-- 오퍼레이션 구성 자체가 다르다는 점에 주목할 것
-- TABLE ACCESS FULL + WINDOW SORT PUSHED RANK  →  전체범위 처리
-- INDEX RANGE SCAN + WINDOW NOSORT STOPKEY     →  부분범위 처리
```

실행계획 해석 — 왜 이렇게 저렴한가

- INDEX RANGE SCAN DESCENDING — 시작점을 바로 찾는다
  - receipt_date < :마지막_날짜 조건으로 인덱스의 해당 지점에 즉시 진입
  - 그 지점부터 내림차순으로 읽어 나가므로 앞을 셀 필요가 없다
- WINDOW NOSORT STOPKEY — 정렬도 없고 초과 작업도 없다
  - 인덱스가 이미 정렬 순서를 제공하므로 정렬 단계가 사라졌다(NOSORT)
  - 20건을 채우는 즉시 더 읽지 않고 멈췄다(STOPKEY)
- 숫자로 확인되는 결과
  - Buffers 25, A-Time 0.01초 — 인덱스 스캔 단계만 보면 Buffers 6
  - 모든 단계의 A-Rows가 20 — 초과 처리한 행이 없다
- "필요한 지점부터 필요한 만큼만"이 계획에 그대로 드러나 있다

DESCENDING이 가능한 구조적 이유

- DESCENDING — 인덱스를 값이 큰 것부터 작은 것 순서로 읽었다는 뜻
- 의문 — 인덱스는 오름차순으로 저장되는데 어떻게 가능한가
  - B-Tree 인덱스의 리프 블록은 양방향으로 연결되어 있다
  - 더블 링크드 리스트 구조이므로 반대 방향으로도 따라갈 수 있다
  - 체인을 역방향으로 읽으면 내림차순으로 읽는 것과 같은 효과가 난다
- 실무적 함의
  - ORDER BY ... DESC 하나 때문에 내림차순 인덱스를 따로 만들 필요가 없다
  - 단일 컬럼 정렬이라면 기존 오름차순 인덱스로 그대로 처리 가능하다
- 예외 — 여러 컬럼의 정렬 방향이 엇갈리는 경우
  - 한 컬럼은 오름차순, 다른 컬럼은 내림차순이면 역방향 읽기로 해결되지 않는다

OFFSET 쿼리는 왜 인덱스를 못 썼는가

- 같은 인덱스가 있었는데 OFFSET 방식은 TABLE ACCESS FULL을 선택했다
- 이유 ① 찾아 들어갈 조건이 없다
  - OFFSET 쿼리에는 WHERE 절이 전혀 없다
  - 특정 값으로 인덱스에 진입할 access predicate가 존재하지 않는다
  - 결국 전체 데이터에 순위를 매겨야 하므로 인덱스를 처음부터 끝까지 훑는 셈
- 이유 ② 인덱스만으로는 필요한 컬럼을 다 못 가져온다
  - 인덱스에는 receipt_date만 있고 claim_id는 테이블에서 가져와야 한다
  - 인덱스를 타면 테이블 접근이 추가로 발생한다
  - 그럴 바에는 Full Scan으로 한 번에 읽는 편이 낫다고 판단한 것으로 보인다
- 옵티마이저가 인덱스를 버린 것은 버그가 아니라 계산 결과다

13.7

Keyset의 제약과 실무 절충

- 임의 페이지 점프가 어려운 이유
- UI 유형별 적합성 판단
- 절충안과 종합 비교

Keyset 방식의 제약

- 임의의 페이지 번호로 바로 이동하기 어렵다
  - "정확히 500페이지로 가줘" 같은 요청을 처리할 방법이 마땅치 않다
- 왜 그런가
  - 다음 페이지를 만들려면 "지금 보고 있는 페이지의 마지막 값"이 있어야 한다
  - 500페이지의 시작 값을 알려면 그 앞의 데이터를 결국 세어 봐야 한다
  - 세는 순간 OFFSET 방식과 같은 비용 구조로 돌아간다
- 구조적으로 커서 정보에 의존하는 방식이다
  - 페이지 번호가 아니라 위치를 들고 다니는 구조
  - 그래서 순차적으로 다음·이전으로 넘어가는 이동에만 자연스럽다
- "더 빠른 방식"이 아니라 "다른 계약을 맺은 방식"으로 이해해야 한다

UI 유형별 적합성 판단

- Keyset이 잘 맞는 화면
  - 무한 스크롤 — 아래로 내릴 때마다 다음 묶음을 붙이는 방식
  - "더보기" 버튼 — 순차적으로만 앞으로 나아가는 구조
  - 피드형 타임라인 — 페이지 번호 개념 자체가 없는 화면
- Keyset을 그대로 적용하기 어려운 화면
  - 게시판형 — 페이지 번호를 직접 클릭해 임의로 이동하는 구조
  - 전체 페이지 수와 현재 페이지 번호를 표시해야 하는 화면
- 판단의 출발점은 DB가 아니라 화면 요구사항이다
  - 먼저 물어야 할 질문 — 사용자가 정말 임의 페이지로 점프하는가
  - 요구사항이 관행일 뿐이라면 화면 설계를 바꾸는 것이 가장 저렴한 튜닝이다

OFFSET을 유지해야 한다면 — 절충안

- 전제 — 페이지 번호 UI를 포기할 수 없는 상황
  - 이 경우 OFFSET 방식의 비용을 감수하거나 범위를 제한해야 한다
- 절충안 ① 페이지 깊이에 제한을 둔다
  - 앞쪽 N페이지까지만 번호 이동을 허용하고 그 이상은 순차 이동으로 유도
  - 검색 조건을 더 좁히도록 안내하는 방식과 함께 쓰면 자연스럽다
  - 실제로 아주 깊은 페이지까지 넘겨보는 사용자는 드물다
- 절충안 ② 두 방식을 혼합한다
  - 페이지 번호 UI는 유지하되 내부적으로 Keyset과 결합해 처리
  - 자주 접근하는 앞쪽 페이지는 캐싱하거나 별도 로직으로 처리
- 절충안 ③ 정렬 기준을 좁히는 WHERE 조건을 함께 요구한다
  - 기간 조건 등으로 대상 자체를 줄이면 순위 매김 범위도 함께 줄어든다

Benchmark — 종합 비교

두 방식을 깊이별로 나란히 놓은 실측 종합표

```
|  방식  |      깊이       | Buffers | A-Time |
|--------|-----------------|---------|--------|
| OFFSET |               0 |   3,090 | 0.02초 |
| OFFSET |         200,000 |   3,090 | 0.14초 |
| OFFSET |         280,000 |   3,090 | 0.18초 |
| Keyset | 약 280,000 상당 |      25 | 0.01초 |

-- OFFSET 방식은 깊이가 늘수록 A-Time이 늘지만 Buffers는 고정이다
-- Keyset 방식은 같은 깊이에서 Buffers 25, A-Time 0.01초에 그친다

-- 표를 읽는 두 방향
--   세로로 읽기: 한 방식 안에서 깊이가 미치는 영향
--   가로로 읽기: 같은 깊이에서 방식이 만드는 차이
```

요약 (Review)

- OFFSET이 깊어질수록 순위를 매겨야 하는 범위가 커져 CPU·메모리 비용이 커진다
- 이 비용은 Buffers에는 잘 안 나타나고 A-Time과 작업 메모리에 나타난다
- OFFSET 쿼리는 내부적으로 ROW_NUMBER()로 변환되며 WINDOW SORT PUSHED RANK로 처리된다
- Keyset Pagination은 인덱스로 다음 페이지의 시작점을 바로 찾아간다
- 그 결과 페이지 깊이와 무관하게 비용이 거의 일정하다
- 인덱스가 정렬 순서를 제공하면 WINDOW NOSORT STOPKEY가 선택될 수 있다
- INDEX RANGE SCAN DESCENDING은 리프 블록 양방향 연결 덕분에 가능하다
- Keyset은 순차 이동만 가능하므로 OFFSET을 완전히 대체하지 못할 수 있다
- 페이징 튜닝의 본질은 전체범위 처리를 부분범위 처리로 되돌리는 것이다

실습 안내

- 실습문제 10문항 — ch13_문제.md
  - 1번 OFFSET과 무관하게 Buffers가 같은 이유 — Full Scan의 성질
  - 2번 A-Time과 Used-Mem이 늘어난 원인 오퍼레이션의 특정
  - 3번 "Buffers가 안 늘었으니 문제없다"는 판단에 대한 반박
  - 4번 NOSORT STOPKEY와 RANGE SCAN DESCENDING의 결합 효과
  - 5번 값 조건과 ROW_NUMBER() 변환의 대조
  - 6번 Keyset의 제약과 UI 유형별 적합성
  - 7번 게시판형 화면에서의 절충안 제안 (자유 서술)
  - 8번 DESCENDING의 의미와 B-Tree 구조적 근거 추론
  - 9번 WHERE 조건 부재와 인덱스 미사용의 연결
  - 10번 ch11와 묶어 Buffers 단독 판단의 위험을 종합
- 실습 종료 시 IX_CH11_ 접두어 인덱스를 반드시 정리한다

Oracle SQL 튜닝 과정 · ch14

실무에서 자주 발생하는 SQL 문제

N+1 문제 · 반복 호출 패턴 · V$SQL 누적 통계 · 조인 1회 대체 | Oracle Database 19c · 진료 비청구심사 스키마 기준

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
  - N+1 패턴의 정의를 말하고 왜 그렇게 부르는지 설명한다
  - 실행계획 한 장만으로는 N+1이 드러나지 않는 이유를 설명한다
  - V$SQL의 EXECUTIONS와 BUFFER_GETS로 반복 호출을 진단한다
  - CLAIM_DETAILS 반복 조회가 매번 Full Scan이 되는 이유를 설명한다
  - N+1의 총 비용이 반복 횟수의 배수로 커지는 구조를 설명한다
  - 조인 1회 방식이 Full Scan을 한 번으로 고정시키는 원리를 설명한다
  - 실측 Buffers 비교로 두 방식의 격차를 근거를 들어 읽는다
  - 인덱스 추가가 완화책일 뿐 근본 해법이 아닌 이유를 설명한다
  - 배치 조회 등 애플리케이션 레벨의 대안을 제시한다
  - 개발 환경에서 N+1이 숨는 이유와 발견 방법을 정리한다

이 장의 구성 (Agenda)

- 12.1 Why — SQL 밖에 있는 성능 문제
  - 한 문장의 실행계획 분석으로 닿지 않는 영역
- 12.2 Concept — N+1 문제와 그 밖의 흔한 유형
  - N+1의 정의, SELECT * · 불필요한 JOIN · 중복 조회 등
- 12.3 Oracle Internals — 비용이 왜 배수로 커지는가
  - 인덱스 없는 컬럼 반복 조회, Full Scan의 곱셈
- 12.4 Example — N+1 재현과 조인 1회 대체
  - PL/SQL 루프로 재현, 아우터 조인 + GROUP BY로 대체
- 12.5 Execution Plan — 실측 비교
  - V$SQL 누적 통계와 조인 1회 실행계획
- 12.6 Bad SQL — N+1이 특히 위험한 이유
  - 목록 건수에 비례해 악화되는 확장성 문제
- 12.7~12.8 Tuning & Benchmark
  - 근본 해법과 완화책의 구분, 실측 요약

14.1

Why — SQL 밖에 있는 성능 문제

- 지금까지의 관점과 그 한계
- 애플리케이션 호출 패턴이라는 사각지대
- 이번 챕터가 다루는 소재

지금까지의 관점 — 한 문장의 실행계획

- ch01부터 ch13까지는 'SQL 한 문장'을 분석 단위로 삼았다
  - 실행계획을 뽑고, 접근 경로와 조인 방식을 읽고, 자원 소비량을 비교했다
  - 분석의 시작과 끝이 언제나 하나의 SQL 텍스트였다
- 이 방법은 강력하지만 전제가 하나 깔려 있다
  - 느린 원인이 '그 문장 안'에 있을 것이라는 전제
  - 그래서 문장을 고치면 문제가 풀린다는 결론으로 이어졌다
- 그런데 실무 성능 문제의 상당수는 이 전제를 벗어난다
  - SQL 자체는 문제없는데, 그 SQL을 애플리케이션 코드가 반복 호출하는 방식에서 생김
  - 문장을 아무리 들여다봐도 원인이 보이지 않는 부류의 문제
- 분석 단위를 '문장'에서 '요청 한 건이 만들어 내는 SQL 전체'로 넓혀야 한다

애플리케이션 호출 패턴이라는 사각지대

- DB가 보는 것과 개발자가 보는 것이 어긋난다
  - DB는 '이 SQL 한 건'을 성실히 처리했을 뿐 — 각각은 정상 처리
  - 개발자는 화면 한 번 여는 데 걸린 시간을 본다 — 그 안에 SQL이 몇 건인지는 안 보임
- 그 사이를 메우는 정보가 '누적 통계'다
  - 한 번의 실행 비용이 아니라, 같은 SQL이 몇 번 실행되어 총 얼마를 썼는가
  - Oracle은 이 정보를 V$SQL 같은 동적 성능 뷰에 이미 쌓아 두고 있다
- ORM·프레임워크가 이 사각지대를 넓혀 놓는 경우가 많다
  - 객체 하나를 참조하면 SQL이 한 건 나가는 구조라 호출 지점이 코드에 드러나지 않음
  - 반복문 안에서 무심코 접근한 속성 하나가 SQL 수십 건이 되기도 한다
- 이번 챕터는 SQL 밖의 문제, 즉 반복 호출 패턴을 실측으로 확인한다

이번 챕터가 다루는 소재

- 소재: '청구 상세 조회 API'
  - 청구 20건에 대해 CLAIM_DETAILS를 건별로 반복 조회하는 N+1 패턴
  - 이 패턴을 PL/SQL 루프로 실제 재현하고, 조인 1회로 대체했을 때와 비교한다
- 실습 조건
  - 본 차시는 DDL 변경 없음 — 기존 PK만 사용한다
  - 즉 인덱스를 새로 만들어 상황을 바꾸지 않고, 있는 그대로의 비용을 측정한다
- 왜 PL/SQL로 재현하는가
  - 애플리케이션 코드를 준비하지 않고도 DB 안에서 반복 호출 구조를 그대로 만들 수 있다
  - 같은 세션·같은 DB에서 두 방식을 비교하므로 조건 차이가 끼어들 여지가 적다
- 아래 예제는 전부 실습 DB 실측 결과다

> **참고:**
> 이 장의 모든 실행계획·Buffers·누적 통계는 진료비청구심사 실습 DB에서 직접 캡처한 실측값이다

14.2

Concept — N+1 문제와 흔한 유형들

- N+1 문제의 정의
- 왜 'N+1'이라 부르는가
- 실무에서 자주 나타나는 다른 문제 유형
- 그중 N+1을 골라 다루는 이유

N+1 문제의 정의

- N+1 문제란
  - 목록 1건을 조회(1번)한 뒤, 그 목록의 각 항목마다 관련 데이터를 추가로 조회(N번)하는 패턴
  - 총 쿼리 실행 횟수가 N+1번이 된다
- 구조를 두 조각으로 나눠 보면
  - 1 — 목록을 가져오는 조회 (예: 병원 1번의 청구 목록)
  - N — 목록의 각 행마다 따로 나가는 조회 (예: 청구별 상세내역 건수)
- 핵심은 '나쁜 SQL이 하나 있다'가 아니라 '평범한 SQL이 여러 번 나간다'는 것
  - 각 조회는 그 자체로 단순하고 흠잡을 데 없어 보인다
  - 문제는 문장의 품질이 아니라 문장이 실행되는 횟수에 있다
- 그래서 이 문제는 실행계획이 아니라 실행 횟수를 봐야 보인다

왜 'N+1'이라 부르는가 — 이름이 곧 설명

- 이름의 1과 N이 각각 무엇인지 분리해 보자
  - 1은 목록을 얻는 최초 조회 — 이 조회는 한 번만 나간다
  - N은 목록의 건수 — 목록이 몇 건이냐에 따라 추가 조회 횟수가 정해진다
- 그래서 N은 상수가 아니라 변수다
  - 목록이 20건이면 스무 번, 100건이면 백 번의 추가 조회가 나간다
  - 코드는 한 줄도 바뀌지 않았는데 데이터에 따라 부하가 달라진다
- 이 성질이 N+1을 다른 성능 문제와 구분 짓는다
  - 다른 문제는 대체로 '한 번의 비용'이 문제 — 고치면 그 비용이 줄어든다
  - N+1은 '횟수'가 문제 — 데이터가 늘면 같은 코드가 더 나빠진다
- 그래서 개발 시점에는 멀쩡하고 운영에서 터지는 유형에 속한다

실무에서 자주 나타나는 다른 문제 유형

- SELECT * — 불필요한 컬럼까지 가져옴
  - 필요 없는 컬럼 때문에 전송량과 메모리가 늘고, 커버링 인덱스 기회를 놓친다
- 불필요한 JOIN — 쓰지도 않는 테이블을 조인
  - 결과에 쓰이지 않는 테이블이 조인에 남아 있으면 그만큼 접근 비용이 더해진다
- 같은 데이터를 여러 번 중복 조회
  - 한 요청 안에서 이미 읽은 값을 다시 읽는 구조 — N+1과 뿌리가 같다
- 과도한 페이지 크기
  - 한 화면에 필요 없는 양까지 한 번에 가져오면 정렬·전송 비용이 함께 커진다
- 데이터 분포를 고려하지 않은 SQL
  - ch05·ch08과 연결되는 주제 — 같은 조건도 분포에 따라 유불리가 뒤집힌다
- 이번 챕터는 그중 가장 흔하고 파급력이 큰 N+1을 실측으로 다룬다

그중 N+1을 골라 다루는 이유

- 가장 흔하다
  - 목록 화면과 상세 정보의 조합은 거의 모든 업무 시스템에 존재한다
  - 따라서 이 패턴을 만들 기회도 그만큼 많다
- 파급력이 가장 크다
  - 다른 유형은 대체로 한 번의 비용을 키우지만, N+1은 비용에 횟수를 곱한다
  - 곱셈은 덧셈보다 훨씬 빨리 커진다
- 발견하기가 가장 어렵다
  - 실행계획 한 장만 보면 아무 문제가 없어 보인다
  - 느린 SQL을 찾는 통상적인 방법으로는 걸러지지 않는다 — 각 실행은 빠를 수 있다
- 그래서 진단 방법을 따로 배워야 하는 유형이다
  - 이 장의 절반은 '어떻게 고치는가'가 아니라 '어떻게 찾는가'에 쓰인다

14.3

Oracle Internals — 비용이 왜 배수로 커지는가

- CLAIM_DETAILS의 인덱스 상황
- 반복이 곱셈이 되는 구조
- 조인은 왜 한 번으로 끝나는가
- 고정비용과 변동비용

CLAIM_DETAILS의 인덱스 상황

- CLAIM_DETAILS는 CLAIM_ID에 인덱스가 없다
  - 이 테이블의 PK는 DETAIL_ID뿐이다
  - 즉 상세를 '청구별로' 찾아갈 수 있는 진입로가 없는 상태
- 그래서 WHERE claim_id = :값 으로 조회하면
  - 매번 TABLE ACCESS FULL이 발생한다
  - ch05에서 이미 확인한 사례와 같은 상황이다
- 여기서 중요한 것은 '한 건을 찾는데 전부를 읽는다'는 비대칭
  - 결과는 몇 건 안 되지만, 그 몇 건을 찾기 위해 테이블 전체를 훑어야 한다
  - 찾는 값이 어디 있는지 모르니 끝까지 읽어 보는 수밖에 없다
- 이 비대칭이 반복될 때 무슨 일이 벌어지는가 — 다음 슬라이드

반복이 곱셈이 되는 구조

- 이 조회를 20번 반복하면 TABLE ACCESS FULL 자체가 20번 반복된다
  - 한 번의 Full Scan 비용이 이미 상당한데(약 6,600 Buffers), 그것이 20번
  - 단순 곱셈으로 커진다 — 더해지는 것이 아니라 곱해진다
- 왜 매번 처음부터 다시 읽는가
  - 각 실행은 서로 독립적인 SQL 실행이다 — 앞 실행이 무엇을 읽었는지 이어받지 않는다
  - 첫 번째 반복에서 테이블 전체를 읽었어도, 두 번째 반복은 다시 처음부터 시작한다
- 즉 같은 데이터를 20번 읽는다
  - 20번의 스캔이 사실상 동일한 작업의 반복 — 새로 얻는 정보는 매번 극히 일부
  - 낭비의 정체가 바로 이 중복 읽기다
- 총 비용 ≈ 반복 횟수 × 단건 Full Scan 비용

조인은 왜 한 번으로 끝나는가

- 조인으로 한 번에 처리하면 CLAIM_DETAILS를 단 한 번만 Full Scan한다
  - 그 한 번의 스캔 결과 안에서 20건 모두를 동시에 매칭시킬 수 있다
  - Hash Join의 특성 — ch09에서 다룬 내용
- Hash Join이 하는 일을 한 줄로 요약하면
  - 작은 쪽으로 해시 테이블을 만들어 두고, 큰 쪽을 한 번 훑으며 던져 맞춘다
  - 큰 쪽을 여러 번 읽을 이유가 없다 — 지나가면서 짝을 다 찾기 때문
- 그 결과
  - Full Scan 비용이 조회 건수와 무관하게 '한 번'으로 고정된다
  - 찾을 청구가 20건이든 200건이든 CLAIM_DETAILS를 읽는 횟수는 그대로 한 번
- 같은 데이터를 읽되, 몇 번 읽느냐가 두 방식을 가른다

고정비용과 변동비용으로 정리하면

- 조인 1회 방식의 비용 구조
  - CLAIM_DETAILS Full Scan 1회 — 목록 건수와 무관한 고정비용
  - 여기에 조인·집계 오버헤드가 조금 붙는다
  - 목록이 늘어도 늘어나는 부분은 목록 쪽 접근 비용뿐
- N+1 방식의 비용 구조
  - 단건 Full Scan 비용 × 반복 횟수 — 전부가 변동비용
  - 고정된 부분이 사실상 없다 — 목록 건수가 곧 배수가 된다
- 그래서 두 방식의 비율은 대략 반복 횟수에 수렴한다
  - 이 예측을 12.5절 실측으로 확인하게 된다
- 판단 기준: 목록이 커질 때 비용이 어떻게 움직이는지를 먼저 그려 보라
  - 코드를 고치기 전에 구조부터 분류하면 어디를 손댈지가 정해진다

14.4

Example — N+1 재현과 조인 1회 대체

- PL/SQL 루프로 N+1 재현
- 재현 코드 읽기
- 조인 1회로 대체
- 두 코드의 구조 비교

N+1 패턴 재현 (PL/SQL)

12.4절 Example — 청구 20건에 대해 상세내역을 건별로 반복 조회

```
-- N+1 패턴(PL/SQL로 재현): 청구 20건에 대해 상세내역을 건별로 반복 조회
DECLARE
  v_cnt NUMBER;
BEGIN
  FOR r IN (SELECT claim_id FROM medical_claims
            WHERE hosp_id = 1 AND ROWNUM <= 20) LOOP

    SELECT COUNT(*) INTO v_cnt
      FROM claim_details
     WHERE claim_id = r.claim_id;

  END LOOP;
END;
/

-- 커서 FOR 루프가 목록 조회 1번 (= N+1의 '1')
-- 루프 본문의 SELECT COUNT(*)가 항목별 조회 N번 (= N+1의 'N')
-- r.claim_id는 PL/SQL이 자동으로 바인드 변수(:B1)로 넘긴다
```

재현 코드 읽기 — 세 가지 관전 포인트

- ROWNUM <= 20 — 목록을 스무 건으로 제한
  - 재현 규모를 고정해 두어야 두 방식을 같은 조건에서 비교할 수 있다
  - 이 20이 곧 N+1의 N이 된다
- 루프 본문의 SELECT COUNT(*) — 반복되는 조회
  - CLAIM_ID로 CLAIM_DETAILS를 찾는다 — 인덱스가 없으므로 매번 Full Scan
  - 이 한 문장이 20번 실행된다
- r.claim_id — 바인드 변수로 넘어간다
  - PL/SQL은 변수를 자동으로 바인드 변수로 처리한다(실측에서 :B1로 나타남)
  - 덕분에 20번 실행이 같은 SQL_ID를 공유해 V$SQL에서 누적으로 관측된다
  - 만약 리터럴을 매번 다르게 넣었다면 하드 파싱 반복이라는 문제가 더 붙는다 — ch02
- 결과 자체는 조인 버전과 동일하다 — 다른 것은 비용뿐

조인 1회로 대체

12.4절 Example — 같은 결과를 한 문장으로

```
-- 조인 1회로 대체
SELECT c.claim_id, COUNT(d.detail_id) cnt
FROM (SELECT claim_id FROM medical_claims
      WHERE hosp_id = 1 AND ROWNUM <= 20) c,
     claim_details d
WHERE c.claim_id = d.claim_id(+)
GROUP BY c.claim_id;

-- 인라인 뷰 c : N+1에서 '목록 조회 1번'에 해당하던 부분
-- (+) 아우터 조인 : 상세내역이 없는 청구도 결과에서 빠지지 않게 한다
--                   (COUNT는 0으로 잡힌다)
-- GROUP BY : 루프를 돌며 건별로 세던 일을 집계 한 번으로 대체

-- 반복 실행이 사라지고 SQL 문장 하나가 되었다는 점이 핵심
```

두 코드의 구조 비교

- 같은 것
  - 대상 — 병원 1번의 청구 20건
  - 결과 — 청구별 상세내역 건수
  - 읽는 데이터의 범위도 사실상 같다
- 다른 것 — 오직 하나, 실행 횟수
  - N+1: 목록 조회 1번 + 상세 조회 20번
  - 조인: 문장 1번
- 구조 전환의 요령
  - 루프의 반복 대상을 인라인 뷰로 옮긴다 — 루프 조건이 곧 뷰의 WHERE가 된다
  - 루프 본문의 조회를 조인으로 붙인다 — 결과 누락을 막으려면 아우터 조인
  - 루프 안에서 세던 값을 GROUP BY 집계로 옮긴다
- 이 세 단계는 대부분의 N+1에 그대로 적용되는 변환 절차다

14.5

Execution Plan — 실측 비교

- 측정 방법 — 왜 V$SQL인가
- N+1 패턴의 누적 통계
- 조인 1회의 실행계획
- 두 방식의 비용 대비

측정 방법 — 왜 V$SQL을 보는가

- N+1 패턴은 바인드 변수를 써서(:B1) 20번 반복 실행한 뒤 V$SQL에서 누적 비용을 확인한다
  - 실행계획은 '한 번의 실행'을 설명하는 도구다
  - 반복 호출의 총량은 실행계획에 나타나지 않는다
- V$SQL이 채워 주는 정보
  - EXECUTIONS — 그 SQL이 몇 번 실행되었는가
  - BUFFER_GETS — 그 SQL이 누적으로 소비한 논리적 I/O
  - ELAPSED_TIME — 누적 소요 시간
- 두 값을 나누면 1회 평균이 나온다
  - BUFFER_GETS / EXECUTIONS = 1회 평균 Buffers
  - 이 평균이 단건 Full Scan 비용과 같다면, 매 실행이 같은 일을 반복했다는 뜻
- 조인 1회 버전은 문장 하나이므로 통상적인 실행계획으로 확인한다

N+1 패턴 — V$SQL 누적 통계 (실측)

12.5절 Execution Plan — 바인드 변수(:B1)로 20번 반복 실행 후 누적 비용

```
SQL_ID          EXECUTIONS  BUFFER_GETS  AVG_BUFFERS  ELAPSED_TIME(μs)
3w1a060hkngfr   20          132,420      6,621        305,359

SQL_TEXT: SELECT COUNT(*) FROM CLAIM_DETAILS WHERE CLAIM_ID = :B1


-- EXECUTIONS  : 같은 SQL_ID가 20번 실행되었다
-- BUFFER_GETS : 20번 실행이 누적으로 소비한 논리적 I/O
-- AVG_BUFFERS : BUFFER_GETS / EXECUTIONS = 132,420 / 20
-- SQL_TEXT    : 바인드 변수(:B1) 덕분에 20번이 하나의 SQL_ID로 묶였다
```

V$SQL 결과 해석

- EXECUTIONS = 20 — 반복이 실제로 일어났음을 확인
  - PL/SQL 루프가 스무 번 돌았고, 그때마다 같은 SQL이 나갔다
- AVG_BUFFERS = 6,621 — 1회 평균 비용
  - 132,420 ÷ 20 = 6,621
  - 이 값이 ch05에서 확인한 CLAIM_DETAILS 단건 조회 Full Scan 비용(약 6,621)과 거의 같다
  - 매 실행이 사실상 동일한 Full Scan을 반복했다는 증거
- BUFFER_GETS = 132,420 — 총 비용
  - 12.3절에서 예측한 '반복 횟수 × 단건 Full Scan 비용' 구조가 그대로 확인된다
- ELAPSED_TIME = 374,094 μs — 누적 소요 시간
  - 시간 값은 캐시 상태에 흔들리므로 판단의 주된 근거는 Buffers로 둔다(ch01 원칙)
- 결론: 이 SQL은 느린 것이 아니라 자주 불린 것이다

조인 1회 — 실행계획 (실측)

12.5절 Execution Plan — 같은 결과를 한 문장으로 처리했을 때

```
| Id | Operation           | Name           | A-Rows |   A-Time    | Buffers |
| 0  | SELECT STATEMENT    |                |     20 |00:00:00.04 |    6645 |
| 1  |  HASH GROUP BY      |                |     20 |00:00:00.04 |    6645 |
| 2  |   HASH JOIN OUTER   |                |     65 |00:00:00.04 |    6645 |
| 3  |    VIEW             |                |     20 |00:00:00.01 |      23 |
| 4  |     COUNT STOPKEY   |                |     20 |00:00:00.01 |      23 |
| 5  |      TABLE ACCESS FULL| MEDICAL_CLAIMS|     20 |00:00:00.01 |      23 |
| 6  |    TABLE ACCESS FULL| CLAIM_DETAILS  |    899K|00:00:00.03 |    6622 |


-- Id 6 : CLAIM_DETAILS에 대한 TABLE ACCESS FULL이 단 한 번만 나타난다
-- Id 4 : ROWNUM <= 20 조건이 COUNT STOPKEY로 처리되어 조기 종료
-- Id 2 : Hash Join으로 20건 전부를 한 번의 스캔 안에서 매칭
```

조인 실행계획 해석 — 두 테이블의 비용 차이

- MEDICAL_CLAIMS — Buffers 23
  - ROWNUM <= 20 조건이 COUNT STOPKEY로 처리된다
  - 조건을 만족하는 20건을 찾는 즉시 스캔을 멈춘다 — 그래서 23에 그쳤다
  - 전체를 읽지 않고 필요한 만큼만 읽고 끝낸 경우
- CLAIM_DETAILS — Buffers 6,622
  - 조기 종료 조건이 없다 — 조인 대상을 빠짐없이 찾으려면 전체를 다 읽어야 한다
  - A-Rows 899K — 실제로 읽어 낸 행 수가 그대로 드러난다
- 총 Buffers 6,645 = 6,622 + 23 (+ 상위 단계의 추가 비용 없음)
  - HASH JOIN OUTER와 HASH GROUP BY는 이미 읽은 데이터를 메모리에서 가공할 뿐
  - 상위 단계로 올라가도 Buffers가 늘지 않는 이유가 여기 있다
- A-Time 00:00:00.04 — 문장 전체가 한 번에 끝났다

두 방식의 비용 대비

- N+1 패턴: 총 132,420 Buffers
  - 20번 실행 누적, 1회 평균 6,621
- 조인 1회: 총 6,645 Buffers
  - CLAIM_DETAILS Full Scan 단 1회 + 목록 접근 23
- 비율: 132,420 ÷ 6,645 ≈ 19.9배
  - 정확히 반복 횟수(20)에 가까운 배수로 커졌다
  - CLAIM_DETAILS Full Scan 비용(약 6,622)이 그대로 20번 곱해진 결과이기 때문
  - 12.3절에서 세운 원리가 실측으로 그대로 확인된 것
- 여기서 얻는 판단 기준
  - 비율이 반복 횟수에 수렴한다면, 그 반복은 매번 같은 일을 되풀이하고 있다는 뜻
  - 반대로 비율이 반복 횟수보다 훨씬 작다면 반복마다 읽는 범위가 작다는 뜻이다
- 이 예측 가능성이 N+1 진단을 쉽게 만든다 — 배수를 미리 계산해 볼 수 있다

14.6

Bad SQL — N+1이 특히 위험한 이유

- 20건이 100건이 되면
- 개발 환경에서 문제가 숨는 이유
- 확장성 문제로서의 N+1

20건이 100건이 되면

- 이번 실습은 겨우 20건 기준이었다
  - 실무 화면에서는 '청구 목록 100건'을 한 번에 보여 주는 일이 흔하다
- 목록 100건에 N+1을 적용하면
  - 100번의 반복이 된다
  - Buffers는 단순 계산으로도 약 662,200(6,622×100)에 달할 것으로 예상된다
- 같은 조건에서 조인 1회 방식은
  - 여전히 약 6,600~7,000 수준에 머무를 것
  - CLAIM_DETAILS를 읽는 횟수가 그대로 한 번이기 때문
  - 늘어나는 것은 목록 쪽 접근 비용뿐이고, 그 비용은 애초에 작다
- N+1의 위험한 특징: 목록 건수가 늘어날수록 배수로 나빠진다
  - 코드는 그대로인데 데이터가 늘면서 성능이 무너지는 구조

개발 환경에서 문제가 숨는 이유

- 개발 환경에서 테스트 데이터가 적을 때는 문제가 안 보인다
  - 예를 들어 목록이 5건이라면 반복도 5번뿐이다
  - 체감할 만한 지연이 생기지 않아 그냥 통과한다
- 운영 환경에서 데이터가 많아지면 급격히 느려진다
  - 예를 들어 100건, 1000건으로 늘어나는 순간 배수가 그대로 반영된다
  - 전형적인 패턴이 바로 이것이다
- 왜 테스트로 걸러지지 않는가
  - 기능 테스트는 결과의 정확성을 보지 성능을 보지 않는다 — N+1은 결과가 정확하다
  - 성능 테스트도 데이터 규모가 작으면 통과해 버린다
  - 코드 리뷰에서도 각 조각이 자연스러워 보여 지적하기 어렵다
- 결론: 운영 규모를 가정한 데이터로 테스트해야 발견된다

확장성 문제로서의 N+1

- 성능 문제는 크게 두 부류로 나뉜다
  - 지금 느린 문제 — 언제 측정해도 같은 수준으로 느리다, 발견이 쉽다
  - 커지면 느려지는 문제 — 지금은 멀쩡하다, 발견이 어렵다
- N+1은 후자의 전형이다
  - 데이터가 적을 때는 숨어 있다가 데이터가 늘면서 드러난다
  - 그래서 확장성(scalability) 문제로 분류된다
- 확장성 문제를 다루는 방법이 따로 있다
  - 측정을 한 번이 아니라 규모를 바꿔 가며 두세 번 한다
  - 절대값이 아니라 기울기를 본다 — 규모가 배로 늘 때 비용이 몇 배가 되는가
  - 기울기가 1보다 크면 그 코드는 언젠가 반드시 문제를 일으킨다
- N+1은 기울기가 정확히 1인(선형) 구조 — 조인 1회는 기울기가 거의 0이다

14.7~14.8

Tuning & Benchmark

- 근본 해법 — 조인 1회로 대체
- 배치 조회라는 절충안
- 완화책 — 인덱스 추가와 그 한계
- 진단 도구와 실측 요약

근본 해법 — 반복 조회를 조인 1회로

- 가장 직접적인 해법: 12.4절처럼 반복 조회를 조인 1회로 대체한다
  - 실행 횟수가 21번에서 1번으로 줄어든다
  - 반복되던 Full Scan 비용이 '한 번'으로 고정된다
- 왜 이것이 근본 해법인가
  - 증상이 아니라 구조를 바꾸기 때문 — 반복 자체를 없앤다
  - 목록 건수가 늘어나도 비용이 배수로 커지지 않는다
  - 즉 확장성 문제의 성질 자체가 사라진다
- 적용할 때 확인할 것
  - 결과 집합이 이전과 동일한가 — 아우터 조인 여부를 반드시 점검
  - 한 문장이 지나치게 커지지 않는가 — 읽기 어려운 SQL은 다음 사람에게 부담이 된다
  - 필요한 컬럼만 가져오는가 — SELECT * 문제를 새로 만들지 않도록
- 튜닝은 바꾸는 일이 아니라 바꾼 뒤 확인하는 일까지 포함한다

절충안 — 배치 조회 (IN 리스트)

12.7절 Tuning — 조인이 당장 어려울 때의 현실적인 대안

```
-- 1단계: 목록에서 claim_id만 먼저 모은다 (애플리케이션 쪽에서 수집)
SELECT claim_id
  FROM medical_claims
 WHERE hosp_id = 1
   AND ROWNUM <= 20;

-- 2단계: 모은 값을 한 번의 쿼리로 묶어서 조회
SELECT claim_id, COUNT(*) cnt
  FROM claim_details
 WHERE claim_id IN (:id1, :id2, :id3, ... , :id20)
 GROUP BY claim_id;

-- 효과 : 반복 실행 횟수가 목록 건수만큼에서 1번으로 줄어든다
--        CLAIM_DETAILS를 한 번만(또는 훨씬 적게) 스캔하게 된다
-- 주의 : IN 리스트 길이가 변하면 SQL 텍스트가 바뀌어 별도 커서가 된다
--        묶음 크기를 고정해 두면 커서 재사용에 유리하다
```

완화책 — 인덱스 추가와 그 한계

- 애플리케이션 구조상 조인이 당장 어렵다면
  - 최소한 반복되는 조회의 WHERE 절 컬럼(CLAIM_ID)에 인덱스를 추가한다
  - 그것만으로도 개별 조회 1회의 비용을 6,621에서 훨씬 작은 값(인덱스 탐색 수준)으로 낮출 수 있다
- 다만 이것은 '완화책'이지 근본 해법이 아니다
  - 반복 호출 구조 자체를 없애지 못한다 — 반복 횟수는 전혀 줄지 않는다
  - '1회 비용 × 반복 횟수' 구조는 그대로 남고, 1회 비용만 작아진 것
- 반복 횟수가 계속 늘어나면 인덱스를 타도 결국 배수로 커지는 것은 마찬가지다
  - ch05 원칙: 결과가 몇 건이든 Full Scan은 고정비용이지만
  - 인덱스 탐색은 반복될수록 그 나름대로 누적된다
- 판단 기준: 완화책은 시간을 벌어 줄 뿐 구조 문제를 닫아 주지 않는다
  - 인덱스를 넣었다면 반드시 구조 개선 과제를 함께 남겨 두라

> **참고:**
> 본 차시는 DDL 변경 없이 측정했다 — 인덱스 추가는 개념상의 완화책으로만 다룬다

진단 — 반복 호출을 찾아내는 조회 패턴

12.7절 Tuning — 실행계획이 아니라 누적 통계를 본다

```
-- 실행 횟수가 유독 많은 SQL을 먼저 훑는다
SELECT sql_id, executions, buffer_gets, elapsed_time, sql_text
  FROM v$sql
 ORDER BY executions DESC;

-- 누적 논리 I/O가 큰 SQL을 훑는다 (1회는 싸지만 총량이 큰 SQL이 걸린다)
SELECT sql_id, executions, buffer_gets,
       buffer_gets / NULLIF(executions, 0) AS avg_buffers, sql_text
  FROM v$sql
 ORDER BY buffer_gets DESC;

-- 읽는 요령
-- executions 가 크고 avg_buffers 도 작지 않다  -> N+1 강력한 신호
-- executions 가 작고 avg_buffers 가 크다        -> 단일 SQL 튜닝 대상
-- 1회 실행 시간으로만 정렬하면 N+1은 상위에 올라오지 않는다
```

진단 절차 정리 — 코드 리뷰와 모니터링

- DB 세션·시스템 통계
  - V$SQL의 EXECUTIONS가 비정상적으로 높은 SQL을 주기적으로 조회해 찾아낸다
- APM(Application Performance Monitoring) 도구
  - 하나의 API 요청 안에서 동일한 패턴의 SQL이 여러 번 호출되는 것을 트랜잭션 단위로 보여 준다
  - 요청 하나를 펼쳤을 때 같은 모양의 막대가 줄지어 있으면 N+1이다
- ORM 프레임워크의 쿼리 로그
  - 많은 ORM이 'N+1 감지' 경고 기능을 제공한다
  - 개발 모드에서 실행된 모든 SQL을 로그로 남겨 눈으로 반복을 확인할 수도 있다
- 부하 테스트
  - 데이터 건수를 늘려 가며 응답 시간이 선형으로(또는 그 이상으로) 나빠지는지 확인한다
- 코드 리뷰에서 볼 지점
  - 반복문 안에서 DB 접근이 일어나는가 — 이 한 가지만 봐도 대부분 걸러진다

Benchmark — 실측 요약

12.8절 Benchmark — 두 방식의 실측 비교

```
+----------------------+-----------+-------------+--------------------------------+
| 방식                  | 실행 횟수 | 총 Buffers  | 비고                            |
+----------------------+-----------+-------------+--------------------------------+
| N+1 (개별 반복 조회)  | 20        | 132,420     | 매번 CLAIM_DETAILS Full Scan   |
| 조인 1회              | 1         | 6,645       | CLAIM_DETAILS Full Scan 단 1회 |
+----------------------+-----------+-------------+--------------------------------+


  비율 : 132,420 / 6,645 = 약 19.9 배

  N+1  1회 평균 Buffers : 6,621     (132,420 / 20)
  N+1  누적 소요 시간   : 374,094 us

  -- 반복 횟수(20)에 가까운 배수 — 03절 원리 그대로
```

요약 (Summary)

- N+1 문제는 SQL 문장 자체가 아니라 '그 SQL이 몇 번 반복 호출되는가'에서 비롯된다
- 그래서 실행계획만 봐서는 보이지 않고, V$SQL의 누적 통계로 확인해야 한다
  - EXECUTIONS(실행 횟수)와 BUFFER_GETS(누적 비용), 그리고 둘을 나눈 1회 평균
- 반복 조회 대상 컬럼에 인덱스가 없으면 각 반복이 Full Scan이 되어 피해가 특히 커진다
  - 실측: 20번 반복 132,420 Buffers, 1회 평균 6,621
- N+1을 조인 1회로 바꾸면 반복되던 비용이 '한 번'으로 고정된다
  - 실측: 조인 1회 6,645 Buffers — 비율 약 19.9배, 반복 횟수 20에 수렴
  - 목록 건수가 늘어나도 비용이 배수로 커지지 않는다
- 인덱스 추가는 1회 비용을 낮추는 완화책일 뿐 반복 횟수를 줄이지 못한다
- 개발 환경의 적은 테스트 데이터로는 N+1이 잘 드러나지 않는다
  - 실제 운영 규모를 가정한 테스트가 중요하다

실습 안내

- 실습문제 10문항: ch14_문제.md
  - 1번 — 평균 Buffers 계산과 단건 Full Scan 비용이 일치하는 이유
  - 2번 — 조인 1회 계획에서 CLAIM_DETAILS Full Scan이 몇 번 나타나는가
  - 3번 — 비율이 20에 가까운 이유와 목록 200건일 때의 예측
  - 4번 — 목록 5건·100건일 때 예상 Buffers와 개발 환경의 함정
  - 5번 — 실행계획만으로 안 보이는 이유와 대신 확인할 V$SQL 컬럼
  - 6번 — 인덱스 추가 후에도 남는 문제
  - 7번 — 바인드 변수 대신 리터럴을 썼다면 생겼을 추가 문제(ch02)
  - 8번 — 조인 외의 애플리케이션 레벨 대안(배치 조회)
  - 9번 — MEDICAL_CLAIMS와 CLAIM_DETAILS의 접근 비용 차이
  - 10번 — 코드 리뷰에서 N+1을 발견하는 도구와 방법
- 풀이 요령: 숫자를 외우지 말고 '반복 횟수 × 1회 비용' 구조로 설명하라

인스턴스 튜닝 개요와 기반

- 브리지 01 · TUN ch15 · ch16 · ch17 · ch18 · 환경 점검
- SQL 한 문장에서 시스템으로

Oracle Database 19c DB 튜닝 과정

브리지 01 — SQL 한 문장에서 시스템으로

Buffers 와 A-Rows 로 재던 것을 DB Time 과 대기 클래스로 다시 잰다 · "내 SQL 이 아니라 시스템이 문제" 를 판별하는 세 가지 징후 · 열흘의 진단 절차 한 장

학습 목표 (Objectives)

- 앞서 쓴 측정 단위(Buffers, A-Rows, Elapsed)가 인스턴스 층위에서 어떤 단위(DB Time, 대기 클래스)로 바뀌는지 설명한다.
- 같은 느린 화면을 두고 SQL 층위의 질문과 인스턴스 층위의 질문이 어떻게 다른지 구분한다.
- "내 SQL 이 아니라 시스템이 문제" 인지 판별하는 세 가지 징후를 V$ 뷰로 확인한다.

지금까지와 앞으로 — 층위가 바뀐다

같은 데이터베이스, 다른 해상도

	SQL 부문	인스턴스 부문
대상	SQL 한 문장	인스턴스 전체·세션·시스템
질문	이 SQL 은 왜 느리고 어떻게 고치는가	어느 SQL 이, 어떤 대기 때문에, DB Time 의 몇 % 를 쓰는가
도구	DBMS_XPLAN, 실행 통계, 인덱스, 힌트	V$ 시간 모델, AWR/ADDM/ASH, 대기 이벤트, 어드바이저
측정 단위	Buffers · Reads · A-Rows/E-Rows · Elapsed	DB Time · DB CPU · 대기 클래스별 시간 · 델타
종착점	실행계획이 바뀌고 Buffers 가 줄었다	시스템 전체 DB Time 이 줄었고 상위 대기가 바뀌었다

같은 화면, 두 개의 질문

"청구 조회 화면이 느리다" 는 신고를 받았을 때

- SQL 층위의 질문 — 그 화면이 실행하는 SQL 의 실행계획은 무엇이고, Buffers 는 얼마이며, 인덱스를 타는가? → Day 1~4 의 방법으로 답한다.
- 인스턴스 층위의 질문 — 그 시각 인스턴스의 DB Time 은 평소 대비 얼마나 늘었고, 늘어난 시간은 CPU 인가 대기인가, 대기라면 어떤 클래스인가? → Day 6~8 의 방법으로 답한다.
- 두 질문의 답이 어긋날 때가 진짜 문제다 — SQL 은 그대로인데 느려졌다면 원인은 SQL 밖에 있다.
- 그래서 순서가 중요하다 — 인스턴스 층위에서 범위를 좁힌 뒤 SQL 층위로 내려간다. 거꾸로 하면 멀쩡한 SQL 을 고치느라 하루를 쓴다.

"내 SQL 이 아니라 시스템이 문제" — 세 가지 징후

이 셋 중 하나라도 보이면 SQL 을 고치기 전에 멈춘다

- ① 실행계획과 Buffers 는 그대로인데 Elapsed 만 늘었다 — 논리 읽기 한 번당 시간이 늘어난 것이므로 I/O·CPU·대기 쪽 문제다 (ch27, ch34).
- ② 내 SQL 의 Elapsed 합이 인스턴스 DB Time 의 몇 % 도 안 된다 — 내 SQL 을 0 으로 만들어도 시스템은 그대로다. 상위 SQL 부터 찾아야 한다 (ch22 SQL 섹션).
- ③ 특정 시간대에만 느리다 — SQL 이 시간대를 가릴 리 없다. 그 시간대에 도는 배치·자동 작업·잠금 경합을 본다 (ch25 ASH, ch30 자동 작업).
- 공통 조치 — 조치 전 상태를 먼저 캡처한다. 스냅샷 없이 고친 튜닝은 효과를 증명할 수 없다 (ch17 실습 05).

징후 ②를 숫자로 — 내 SQL 은 DB Time 의 몇 % 인가

V$SQL 의 elapsed_time 과 V$SYS_TIME_MODEL 의 DB time 을 같은 단위(초)로 놓고 비교한다

```
-- 1) 앞에서 쓰던 그 SQL 의 SQL_ID 를 찾는다
SYS@orcl> SELECT sql_id, plan_hash_value, executions,
  2         ROUND(elapsed_time/1e6, 2) AS elapsed_s,
  3         buffer_gets
  4    FROM v$sql
  5   WHERE sql_text LIKE 'SELECT /* BR01 */%';

-- 2) 인스턴스가 기동 후 지금까지 쓴 DB Time (누적, 초)
SYS@orcl> SELECT ROUND(value/1e6, 2) AS db_time_s
  2    FROM v$sys_time_model
  3   WHERE stat_name = 'DB time';

-- 3) 비중 = elapsed_s / db_time_s * 100
--    누적값끼리의 비교이므로 방향만 본다. 구간 델타로 다시 재는 법은 ch19 실습 05·06.
--    실행 결과는 실습 환경에서 캡처한다.
```

열흘의 진단 절차 한 장

- ① DB Time 을 잰다 → CPU 인가 대기인가 → 어느 대기 클래스인가 (ch19)
- ② 구간 보고서로 상위 대기·상위 SQL 을 지목한다 (ch22 AWR · ch24 ADDM · ch25 ASH)
- ③ 대기 이벤트에서 원인 세그먼트·세션·SQL 로 좁힌다 (ch26·ch27)
- ④ 지목된 SQL 의 실행계획을 읽고 고친다 → ch 1~14 로 돌아온다 (브리지 02)
- ⑤ 손으로 고친 결과를 어드바이저 권고와 대조한다 (ch30, 브리지 03)
- ⑥ 조치 전후를 기간 비교로 증명한다 (ch23) — SQL 층위의 Buffers 비교와 짝을 이룬다
- ⑦ SQL 로 해결되지 않는 나머지는 메모리·I/O 구성으로 간다 (ch29~ch34)

인스턴스 부문에서 특히 경계할 세 가지

SQL 부문에는 없던 함정

- 히트율 신앙 — 버퍼 캐시 히트율 99% 인데 응답시간이 나쁜 상황을 ch31에서 직접 만든다. 비율은 병목을 숨긴다.
- 권고 맹신 — ADDM·어드바이저의 권고를 그대로 적용하면 안 되는 사례를 ch24에서 판별한다. 권고는 가설이지 결론이 아니다.
- 근거 없는 조치 — 조치 전 상태를 캡처하지 않은 튜닝은 효과를 증명할 수 없다. ch17 실습 05 에서 절차로 각인한다.
- SQL 부문의 습관을 그대로 가져온다 — 실행계획을 읽기 전에 SQL 을 고치지 않았듯, 대기 이벤트를 읽기 전에 파라미터를 바꾸지 않는다.

라이선스 — 한 문장만 먼저

자세한 내용은 바로 이어지는 ch15 라이선스 절

- 앞에서 쓴 DBMS_XPLAN·V$SQL·V$SQL_PLAN 은 Enterprise Edition 기본 기능이다 — 추가 라이선스 없음.
- AWR·ADDM·ASH·DBA_HIST_* 는 **Diagnostics Pack**, Day 9 의 SQL Tuning/Access Advisor·SPA 는 **Tuning Pack** 이 필요하다.
- 교육 환경에서 무심코 쓰던 것을 실무에서 그대로 쓰면 라이선스 위반이 된다 — 각 장 도입부에서 다시 짚는다.
- 미보유 환경에서도 같은 것을 보는 대안 경로(Statspack, V$ 직접 조회)를 모든 해당 장에 함께 싣는다.

요약 (Summary)

- 층위가 바뀐다 — SQL 한 문장(Buffers·A-Rows)에서 인스턴스 전체(DB Time·대기 클래스)로.
- 순서가 중요하다 — 인스턴스 층위에서 좁힌 뒤 SQL 층위로 내려간다.
- 세 가지 징후 — 계획·Buffers 는 그대로인데 Elapsed 만 증가 / 내 SQL 이 DB Time 의 몇 % 도 안 됨 / 특정 시간대에만 느림.
- 열흘의 지도 — 재고(ch19) → 지목(ch22·ch24·ch25) → 좁히고(ch26·ch27) → 고치고(Ch 1~4) → 대조하고(ch30) → 증명한다(ch23).
- 다음 — TUN ch15 성능 튜닝 개요: 응답시간 vs 처리량, 도구 지도, 라이선스.

Oracle Database 19c 성능 튜닝 실무 과정

성능 튜닝 개요

Performance Tuning Overview — 무엇을 튜닝이라 부르는가

응답시간 vs 처리량 · DB Time · 도구 지도(V$·AWR·ADDM·ASH·Advisor) · 관리 팩 라이선스

ch15

학습 목표 (Objectives)

- 응답시간과 처리량을 구분하고, 성능 목표를 정성 표현이 아니라 수치로 정의한다.
- 튜닝이 필요한 시점을 사전 예방적 모니터링과 병목 제거로 판단한다.
- 증상과 원인을 혼동하지 않고, 대기 이벤트에서 원인으로 좁혀 들어간다.
- DB Time이 이 과정의 공통 화폐임을 이해하고 실측으로 확인한다.
- 성능 도구 지도(V$·AWR·ADDM·ASH·Advisor)를 그리고 상황별로 고른다.
- 관리 팩 라이선스(Diagnostics/Tuning Pack)의 경계와 확인 방법을 안다.
- 라이선스가 없을 때의 대안 경로(V$ 직접 조회·Statspack)를 안다.
- 히트율 신앙·권고 맹신·근거 없는 조치의 세 함정을 경계한다.

이 장의 구성 (Agenda)

- 15.1 응답시간과 처리량 — 성능을 무엇으로 재는가
- 15.2 튜닝이 필요한 시점 — 언제 손대는가
- 15.3 DB Time — 이 과정의 공통 화폐
- 15.4 성능 도구 지도 — 무엇을 언제 쓰는가
- 15.5 관리 팩 라이선스 — 무심코 쓰면 위반이다
- 15.6 튜닝 목표를 수치로 정의하는 법
- 15.7 세 가지 경계 — 이 과정의 태도
- ※ ADMIN 31장의 AWR·ADDM 개념 소개를 판독·실무 적용으로 확장한다(non-CDB orcl 19.3).

15.1

응답시간과 처리량

- 성능을 재는 두 축
- 응답시간=서비스+대기 / 처리량=단위시간당 작업
- 둘은 종종 상충한다
- V$SYSMETRIC 실측

응답시간(Response Time)

한 요청이 시작되어 끝날 때까지

- 응답시간 = 서비스 시간(CPU) + 대기 시간(자원을 기다린 시간)
- 사용자 체감 성능은 대부분 응답시간으로 나타난다
- 인스턴스 튜닝은 대개 대기 시간을 줄이는 데 집중한다
- 목표는 평균이 아니라 백분위수(95/99퍼센타일)로 표현한다

처리량(Throughput)

단위 시간당 완료한 작업의 양

- 초당 트랜잭션 수(TPS)·초당 실행 수·시간당 처리 건수로 표현
- 배치·데이터 웨어하우스에서 핵심 지표가 된다
- V$SYSMETRIC의 User Transaction Per Sec·Executions Per Sec
- '한 건이 빠른가'가 아니라 '정해진 시간에 얼마나 많이'가 관건

두 지표는 종종 상충한다

워크로드 성격에 따라 우선순위

- 배열 크기를 키우면 처리량↑ 응답시간↑(개별 요청은 느려질 수 있음)
- 특정 세션에 자원을 몰면 응답시간↓ 전체 처리량↓
- OLTP는 응답시간 우선, 배치·DW는 처리량 우선
- 목표는 워크로드별로 우선순위를 정해 세운다

V$SYSMETRIC 실측 — 무부하 시 0

응답시간·처리량 대표 메트릭 (최근 60초)

```
SYS@orcl> SELECT metric_name, ROUND(value,2) value, metric_unit
  2       FROM v$sysmetric WHERE intsize_csec BETWEEN 5900 AND 6100
  3         AND metric_name IN ('Average Active Sessions',
  4         'Executions Per Sec','Response Time Per Txn', ...);

METRIC_NAME                    VALUE METRIC_UNIT
------------------------------ ----- ----------------------
Average Active Sessions            0 Active Sessions
Database Time Per Sec              0 CentiSeconds Per Second
Executions Per Sec              0.36 Executes Per Second
Response Time Per Txn              0 CentiSeconds Per Txn
User Transaction Per Sec           0 Transactions Per Second
```

15.2

튜닝이 필요한 시점

- 사전 예방적 모니터링
- 병목 제거
- 증상과 원인의 구분
- 확장성과 용량 계획

사전 예방적 모니터링

Proactive Monitoring

- 정해진 주기로 통계를 관찰해 동작·자원 사용 변화를 확인
- 대개 구성 변경으로 이어지지 않는다
- 성능 저하가 없는데 근거 없이 만지는 것은 위험한 도박
- 그런 조작은 사후 대응의 규율(캡처→조치→재측정)을 따라야 한다

병목 제거

Bottleneck Elimination

- 성능 문제는 특정 자원의 과다 사용에서 온다 = 병목
- 가장 큰 병목 '하나'를 찾아 그 영향을 줄인다
- 여러 개를 동시에 건드리면 인과를 잃는다
- 고칠 곳은 셋: 애플리케이션 / 오라클 구성 / 호스트 하드웨어

증상과 원인을 혼동하지 않기

The Symptoms and the Problems

- 많은 성능 통계는 원인이 아니라 증상을 가리킨다
- 물리 I/O 느림 → 디스크? 아니면 나쁜 SQL의 과다 I/O?
- 래치 경합은 대개 애플리케이션 변경으로 해소
- 증상(대기)에서 원인(세그먼트·세션·SQL)으로 좁혀 들어간다

확장성과 용량 계획

튜닝의 배경

- 확장성: 자원을 늘린 만큼 처리 능력이 비례해 느는 성질
- 경합 때문에 자원 사용이 두 배를 넘는 경우가 많다
- 확장성을 떨어뜨리는 최대 원인은 애플리케이션 설계
- 설계에서 심어진 한계는 하드웨어로 넘지 못한다

15.3

DB Time — 공통 화폐

- 이질적 문제를 하나의 저울로
- DB Time = DB CPU + 대기
- 실측: CPU 약 17.5%
- Average Active Sessions

DB Time이란 무엇인가

AWR·ADDM이 성능을 재는 공통 화폐

- DB Time = 모든 활성 세션의 CPU 시간 + 대기 시간의 합
- 벽시계 시간이 아니라 세션 시간의 합(세션 10개×1초=10초)
- 성능 개선 = 같은 워크로드의 DB Time을 줄이는 것
- 이질적 문제도 'DB Time의 몇 %'로 환산해 비교·우선순위

DB Time 구성 실측

V$SYS_TIME_MODEL (누적, 마이크로초)

```
SYS@orcl> SELECT stat_name, value FROM v$sys_time_model
  2       WHERE stat_name IN ('DB time','DB CPU',
  3         'sql execute elapsed time','parse time elapsed')
  4       ORDER BY value DESC;

STAT_NAME                          VALUE
------------------------------ ----------
DB time                          49551722
sql execute elapsed time         12277777
DB CPU                            8662345
parse time elapsed               1935695
```

Average Active Sessions

DB Time에서 파생되는 동시성 지표

- AAS = 구간 DB Time ÷ 구간 벽시계 경과 시간
- '평균 몇 개 세션이 동시에 일하고 있었는가'
- 호스트 CPU 코어 수와 비교하면 붐빔을 직관적으로 파악
- 코어 4개인데 AAS 8이면 감당량의 두 배가 경쟁 중

15.4

성능 도구 지도

- V$ · AWR · ADDM · ASH · Advisor
- 경쟁이 아니라 층위가 다른 협력
- 상황에 따라 무엇을 쓰는가
- STATISTICS_LEVEL 전제

V$ 동적 성능 뷰

모든 도구가 참조하는 원천

- 인스턴스 기동 시 초기화되는 메모리 구조 = 지금 이 순간
- 라이선스 불필요, 실시간 진단에 강함
- 진단 경로: 세션 → 대기 → SQL → 세그먼트 (ch26)
- 약점: 과거를 못 본다 / 값은 누적이라 델타로 봐야 함

AWR (Automatic Workload Repository)

성능의 기억 장치 🔒 Diagnostics Pack

- 휘발성 V$ 통계를 스냅샷으로 떠 SYSAUX에 영구 저장
- 기본 60분 간격·8일 보존 / 두 스냅샷 구간을 요약
- 실측: 스냅샷 153개(snap_id 141~293, 08-14~08-25)
- '어제 오후 2시'를 사후 재구성 / Load Profile·Top 10 Events

ADDM (Automatic Database Diagnostic Monitor)

자가 진단 엔진 🔒 Diagnostics Pack

- AWR 데이터를 자동 분석해 병목 진단·개선안 제시
- DB Time을 하향식(top-down)으로 분해해 근본 원인 추적
- 결과 = Finding(무엇이) · Recommendation(어떻게) · Rationale(왜)
- 권고는 결론이 아니라 가설 — 영향도·근거 확인 후 채택(ch24)

ASH (Active Session History)

초 단위 순간 포착 🔒 Diagnostics Pack

- 활성 세션 상태를 기본 1초 주기로 샘플링(순환 버퍼)
- AWR=60분 거시 요약 ↔ ASH=초 단위 미시 포착
- AWR 평균에 묻힌 짧은 스파이크를 잡아낸다
- 진입점: ashrpt.sql, V$ACTIVE_SESSION_HISTORY

어드바이저 — 어떻게 고칠 것인가

DBA_ADVISOR_DEFINITIONS (실측)

```
SYS@orcl> SELECT advisor_name FROM dba_advisor_definitions
  2       ORDER BY advisor_name;

ADDM                          SQL Repair Advisor
Compression Advisor           SQL Tuning Advisor       🔒 Tuning Pack
SPM Evolve Advisor            SQL Workload Manager
SQL Access Advisor   🔒        Segment Advisor
SQL Performance Analyzer🔒    Statistics Advisor
                              Tune MView / Undo Advisor
```

도구 선택 — 상황에 따라

옳은 질문: 지금 내 문제의 성격은?

- 지금 이 순간 느리다 → V$ 뷰 (라이선스 불필요)
- 어제 그 시간에 무슨 일이 → AWR 🔒
- 무엇이 문제인지 모르겠다 → ADDM 🔒
- 짧게 튀었다 사라진 스파이크 → ASH 🔒
- 특정 SQL을 어떻게 → SQL Tuning Advisor 🔒 (결론 나면 SQL튜닝 과정)

15.5

관리 팩 라이선스

- Diagnostics / Tuning Pack
- CONTROL_MANAGEMENT_PACK_ACCESS
- DBA_FEATURE_USAGE_STATISTICS
- 라이선스 없이 보는 법

어떤 기능이 어느 팩에 속하는가

무심코 쓰면 라이선스 위반

- Diagnostics Pack: AWR·ADDM·ASH·DBA_HIST_*·V$ACTIVE_SESSION_HISTORY
- Tuning Pack: SQL Tuning/Access Advisor·SPA (Diagnostics Pack 선행)
- 팩 불필요: V$ 뷰·Statspack·Segment/Undo/Statistics Advisor
- LMS 감사에서 DBA_FEATURE_USAGE_STATISTICS가 근거가 된다

CONTROL_MANAGEMENT_PACK_ACCESS

어떤 팩을 쓸 수 있는지 선언

```
SYS@orcl> SELECT name, value FROM v$parameter
  2       WHERE name = 'control_management_pack_access';

NAME                           VALUE
------------------------------ -------------------
control_management_pack_access  DIAGNOSTIC+TUNING

-- 값: NONE / DIAGNOSTIC / DIAGNOSTIC+TUNING
-- 라이선스 없으면 NONE 으로 두어 수집을 막는 것이 안전
```

사용 이력 확인

DBA_FEATURE_USAGE_STATISTICS (실측)

```
SYS@orcl> SELECT name, detected_usages,
  2         TO_CHAR(last_usage_date,'YYYY-MM-DD') last_used
  3       FROM dba_feature_usage_statistics WHERE name IN (...);

NAME                           DETECTED_USAGES LAST_USED
------------------------------ --------------- ----------
ADDM                                         0
Automatic SQL Tuning Advisor                 2 2026-08-21
Automatic Workload Repository                0
SQL Tuning Advisor                           0
```

라이선스가 없으면 무엇으로 보는가

원천 데이터는 언제나 V$에 있다

- 구간 요약: AWR → Statspack 보고서 / V$SYSSTAT 델타
- DB Time·대기: ADDM → V$SYS_TIME_MODEL + V$SYSTEM_WAIT_CLASS
- 활성 세션 상태: ASH → V$SESSION+V$SESSION_WAIT 반복 샘플링
- 핵심: 누적값을 두 시점 델타로 계산 = 원리는 동일

15.6

목표를 수치로 정의

- 대상 선정 → 현재값 → 목표값 → 판정
- 상위 부하 SQL 실측
- 기준선(baseline)
- SQL 튜닝 과정과의 경계

상위 부하 SQL — 대상 선정

V$SQL 논리읽기(buffer_gets) 기준 (SQLT 계정)

```
SYS@orcl> SELECT sql_id, executions execs, buffer_gets,
  2         ROUND(buffer_gets/GREATEST(executions,1)) gets_per_exec,
  3         ROUND(elapsed_time/1e6,2) elapsed_sec, ...
  4       FROM v$sql WHERE parsing_schema_name='SQLT'
  5       ORDER BY buffer_gets DESC FETCH FIRST 3 ROWS ONLY;

SQL_ID         EXECS BUFFER_GETS GETS_PER_EXEC ELAPSED_SEC
------------- ------ ----------- ------------- -----------
3j9gw8s0v82pc    200      618000          3090        1.23
fus9168r58axj      3       19869          6623        0.87
axnpq1h1y89da      1        4586          4586        0.50
```

목표 수치화 4단계와 기준선

목표 없이 파라미터부터 바꾸지 않는다

- ① 대상 업무 선정 → ② 현재값 측정 → ③ 목표값 → ④ 판정 기준
- 목표는 응답시간·처리량 같은 측정 가능한 양으로만
- 기준선: 정상일 때 값을 모르면 '느려졌다'를 증명 못 한다
- 경계: SQL 한 문장 수정으로 결론 나면 SQL 튜닝 과정으로

15.7

세 가지 경계

- 히트율 신앙
- 권고 맹신
- 근거 없는 조치
- 뿌리는 하나 — 직접 측정·판정하지 않음

히트율 신앙을 경계한다

단일 지표 하나로 결론 내지 않는다

- 버퍼 캐시 히트율 99%가 건강을 뜻하지 않는다
- 비효율 SQL이 같은 블록을 반복 읽으면 히트율은 오히려 오른다
- 히트율이 높은데 응답시간이 나쁜 상황을 ch31에서 직접 재현
- 히트율은 목표가 아니라 하나의 참고 지표

권고 맹신을 경계한다

권고는 결론이 아니라 가설

- ADDM·어드바이저의 권고는 검토 후 채택할 가설
- 영향도 낮은 권고에 시간 쓰거나 맥락 무시하면 해가 된다
- 권고를 그대로 따르면 안 되는 사례를 ch24·ch30에서 판별
- '왜 이 권고인가(근거)'와 '내 상황에 맞는가'를 본 뒤 채택

근거 없는 조치를 경계한다

조치 전 캡처 없는 튜닝은 도박

- 조치 전 상태를 캡처하지 않으면 성공·실패를 증명 못 한다
- 모든 실습 트랜스크립트는 '조치 전 → 조치 → 조치 후' 3단 캡처
- 한 번에 하나만 바꾸고 같은 지표를 같은 방식으로 재측정
- 채택하지 않은 권고의 이유도 기록한다

퀴즈 (Quiz)

- Q1. 응답시간과 처리량의 차이를 각각 한 줄로 설명하고, 상충하는 예를 하나 들어라.
- Q2. 'DB Time = DB CPU + 대기'가 실측(49,551,722 / 8,662,345)에서 어떻게 확인되는가?
- Q3. '지금 느리다'와 '어제 오후 2시'에 각각 V$·AWR·ADDM·ASH 중 무엇을 먼저 쓰는가?
- Q4. AWR·ADDM·ASH의 관리 팩은? 라이선스 없을 때 대안 경로 두 가지는?
- Q5. CONTROL_MANAGEMENT_PACK_ACCESS=DIAGNOSTIC+TUNING이면 곧 라이선스 보유인가?
- Q6. 히트율 99%인데 응답시간이 나쁠 수 있는 이유는?

요약 (Summary)

- 성능은 응답시간·처리량 두 축으로 재고, 목표는 수치로 정의해야 판정 가능하다.
- 튜닝은 사전 예방적 모니터링과 병목 제거. 근거 없이 만지는 것은 도박이다.
- 증상과 원인을 혼동하지 않는다. 대기는 어디를 볼지 알려줄 뿐이다.
- DB Time은 공통 화폐. 실측 CPU 약 17.5%, 나머지가 대기. 모든 진단의 출발점.
- 도구: 실시간 V$ / 지나간 구간 AWR / 자동 진단 ADDM / 스파이크 ASH / 조치 Advisor.
- AWR·ADDM·ASH=Diagnostics Pack, Advisor·SPA=Tuning Pack. 없으면 V$·Statspack.
- 세 가지 경계: 히트율 신앙 · 권고 맹신 · 근거 없는 조치.

실습 01 개요 (Practice Overview)

- 실습 01 — 응답시간과 처리량 지표 확인(VSYS_TIME_MODEL / VSYSMETRIC)
- 실습 02 — 성능 도구 지도 진입점(V$·AWR·ADDM·ASH·Advisor)
- 실습 03 — 라이선스 확인(DBA_FEATURE_USAGE_STATISTICS / CONTROL_MANAGEMENT_PACK_ACCESS)
- 실습 04 — 튜닝 목표 수치 정의(상위 부하 SQL 현재값·목표값)
- 실습 05 — 진료비 스키마 환경 점검(전 챕터 공통 기준선)
- ※ 환경: non-CDB orcl 19.3, 프롬프트 SYS@orcl> / SQLT@orcl>
- ※ 각 실습에 '조치 전 → 조치·조회 → 결과 해석' + '이상 징후→원인→조치' 흐름 포함

Oracle Database 19c 성능 튜닝 실무 과정

성능을 고려한 설계와 개발

Designing and Developing for Performance — 성능은 설계에서 결정된다

확장성 · 바인드 변수와 커서 공유 · 커넥션 비용 · 배열 처리 · 배포 전 워크로드 테스트

ch16

학습 목표 (Objectives)

- 확장성을 정의하고 확장성을 저해하는 요소를 애플리케이션 설계 관점에서 설명한다.
- 하드 파싱과 소프트 파싱의 차이를 알고, 리터럴 SQL이 왜 하드 파싱을 유발하는지 이해한다.
- 바인드 변수로 커서를 공유하는 원리를 실측(리터럴 300 → 300 / 바인드 300 → 0)으로 확인한다.
- CURSOR_SHARING(EXACT/FORCE)의 동작과 부작용을 설명한다.
- 커넥션을 매 요청마다 여는 비용을 이해하고 커넥션 풀링의 필요성을 안다.
- arraysize가 왕복과 논리 읽기에 미치는 영향을 계산으로 설명한다.
- 배포 전 워크로드 테스트의 규칙과 배포 전략(Big Bang·Trickle)을 안다.

이 장의 구성 (Agenda)

- 16.2 확장성 — 무엇이 시스템을 못 늘어나게 하는가
- 16.3 바인드 변수와 커서 공유 — 하드 파싱 비용 (이 장의 핵심)
- 16.4 커넥션 획득 비용 — 접속은 비싸다
- 16.5 배열 처리와 왕복 횟수 — arraysize
- 16.6 그 밖의 애플리케이션 설계 원칙
- 16.7 배포 전 워크로드 테스트
- 요약 · 퀴즈 · 실습 개요

16.2

확장성

- 확장성의 정의 — 자원을 늘린 만큼 처리 능력이 비례해 느는 성질
- 확장성을 떨어뜨리는 요소 — 애플리케이션 설계의 영향이 가장 크다
- 자원 경합과 확장 불가 애플리케이션

확장성이란 무엇인가

Scalability

- 확장성 = 자원 사용을 비례적으로 늘리며 더 많은 워크로드를 처리하는 능력
- 이상적 상태 = 선형 확장성(처리량이 CPU 수에 비례). 현실에선 불가능
- 목표는 설계를 최대한 확장 가능하게 만들어 증설만으로 목표를 달성하는 것
- 인터넷 시대 앱: 24/365 가용, 동시 사용자 예측 불가, 짧은 개발·테스트 시간

확장성을 떨어뜨리는 요소

애플리케이션 설계의 영향이 가장 크다

- 나쁜 스키마 설계 → 확장되지 않는 비싼 SQL
- 나쁜 트랜잭션 설계 → 잠금·직렬화 문제
- 나쁜 커넥션 관리 → 나쁜 응답시간·불안정
- 물리 구현: 나쁜 I/O 전략, 테스트와 다른 실행계획, 과도한 메모리 사용
- 하드웨어·소프트웨어 한계도 있지만 통제하기 쉬움 → 설계가 관건

확장 불가 애플리케이션

자원 경합과 자원 고갈

- 확장 불가 = 자원을 소진해 워크로드를 늘려도 처리량이 안 느는 지점 도달
- 증상: 처리량 고정 + 응답시간 악화
- 자원 고갈 예: 대량 트랜잭션의 전체 스캔 → I/O 부족, 과도한 네트워크 요청
- 워크로드 성장 곡선: 수요는 가속하며 증가 → 설계 결함은 어느 벽에서 폭발
- 부하가 그 지점에 닿기 전엔 보이지 않는다 = 잠복한 시한폭탄

확장성과 용량 계획

튜닝의 배경

- 사전 예방적 모니터링은 대개 용량 계획의 일부
- 자원 소비 추이를 추적해 언제 한계에 닿을지 예측
- 그러나 설계가 확장 불가면 용량 계획도 소용없다
- 자원 2배 넣어도 처리량 2배가 아니면 = 문제를 미룰 뿐
- 이 장의 초점: 용량을 늘리는 법이 아니라 같은 용량으로 더 하는 법

16.3

바인드 변수와 커서 공유

- 하드 파싱과 소프트 파싱 — 파싱은 비싸다
- 리터럴 SQL이 만드는 커서 폭발
- 바인드 변수로 커서 공유 — 리터럴 300 vs 바인드 0 (실측)
- CURSOR_SHARING — 코드를 못 고칠 때의 차선책

파싱이란 무엇인가

하드 파싱 vs 소프트 파싱

- 파싱 = SQL을 해석해 실행계획을 만드는 과정(문법·권한·계획·공유 풀 적재)
- 하드 파싱: 공유 풀에 일치가 없어 모든 작업 수행 → 가장 비싸고 확장성 저해
- 소프트 파싱: 공유 풀에 일치가 있음 → 낫지만 문법·권한 검사로 자원 소모
- 목표: 한 번 파싱하고 여러 번 실행(커서 재사용)

하드 파싱이 확장성을 해치는 이유

직렬화 지점을 만든다

- 하드 파싱은 라이브러리 캐시 래치·뮤텍스를 획득해야 함
- 사용자 1~2명: 비용 안 보임 / 수백 세션 동시: 같은 래치 경합
- 경합은 사용자 수에 선형이 아니라 더 가파르게 증가
- → '개발 땐 빨랐는데 오픈하니 죽는' 현상의 원인

리터럴 SQL이 만드는 커서 폭발

V$SQL — 값마다 커서가 갈라진다

```
SQLT@orcl> SELECT sql_text, COUNT(*) versions
  2         FROM v$sql
  3        WHERE sql_text LIKE
  4              'select count(*) from medical_claims where hosp_id =%'
  5        GROUP BY sql_text ORDER BY sql_text;

SQL_TEXT                                                  VERSIONS
--------------------------------------------------------- ---------
select count(*) from medical_claims where hosp_id = 90024         1
select count(*) from medical_claims where hosp_id = 90036         1
select count(*) from medical_claims where hosp_id = 90037         1
...  (병원 ID마다 별도 커서로 갈라져 30여 개가 쌓였다)
```

바인드 변수로 커서를 공유한다

리터럴 300 vs 바인드 0 (실측, orcl 19.3)

- 리터럴 SQL 300회 실행 → 하드 파싱 델타 = 300, 커서 300개
- 바인드 변수 300회 실행 → 하드 파싱 델타 = 0, 커서 1개(재사용)
- 실행한 일의 양(execute count)은 두 방식이 같다
- → 벌어진 시간 차이 = 파싱 비용
- V$LIBRARYCACHE(SQL AREA) GETHITRATIO 실측 53.3% = 재파싱의 흔적

리터럴이 항상 나쁜 건 아니다

편중 데이터에서의 예외

- 값 종류가 소수로 고정 + 값에 따라 최적 계획이 크게 달라지는 컬럼
- 예: MEDICAL_CLAIMS는 HOSP_ID 1~50에 청구 절반 집중(스큐)
- 이런 편중 컬럼은 리터럴이 값별 다른 계획을 얻어 유리할 수 있음
- 그러나 값 가짓수가 많고 계획이 같은 술어면 리터럴은 백해무익
- 원칙: 실행마다 달라지는 값은 바인드, 계획 좌우 상수는 신중히

CURSOR_SHARING — 차선책

코드를 못 고칠 때만

```
SQLT@orcl> -- 기본값 EXACT: 텍스트가 정확히 같아야 커서 공유
SQLT@orcl> --            → 리터럴은 그대로 하드 파싱
SQLT@orcl> ALTER SESSION SET CURSOR_SHARING = FORCE;
SQLT@orcl> -- 리터럴을 :"SYS_B_0" 형태로 치환 → 커서 공유
SQLT@orcl> ALTER SESSION SET CURSOR_SHARING = EXACT;   -- 원복

-- 주의: 히스토그램 컬럼에서 나쁜 계획이 고정될 수 있음
-- 근본 해법은 바인드 변수를 쓰도록 코드를 고치는 것
```

파싱을 줄이는 세 단계

바인드 위에 한 겹 더

- 1단계: 바인드 변수로 하드 파싱을 없앤다
- 2단계: 커서를 재사용해 소프트 파싱을 줄인다(닫았다 다시 열지 않기)
- 3단계: SESSION_CACHED_CURSORS로 남은 소프트 파싱을 캐시(soft soft parse)
- session cursor cache hits 통계로 3단계 효과 확인
- 대부분은 애플리케이션의 prepare/execute 패턴만으로 달성

16.4

커넥션 획득 비용

- 접속은 비싸다 — 프로세스 생성·메모리·인증·초기화
- 4-CPU 테스트: 무파싱 270 → 재접속 30 (9분의 1)
- 커넥션 풀링과 재사용

접속은 비싸다

4-CPU OLTP 테스트 — 지원 가능한 사용자 수

- 파싱을 전혀 안 함(한 번 파싱 후 재실행): 270명
- 모든 문장 소프트 파싱: 150명
- 모든 문장 하드 파싱: 60명
- 트랜잭션마다 재접속: 30명 (하드 파싱보다도 나쁨)
- → 하드웨어를 안 바꾸고 방식만 고쳐도 4~9배 사용자 수용

커넥션 풀링과 재사용

요청마다 여는 자원이 아니다

- 커넥션 풀: 미리 열어 두고 빌려주고 반납받아 재사용
- 접속 비용을 요청마다가 아니라 기동 시 한 번만 지불
- 재접속은 빈 세션 → 커서 캐시도 없어 파싱 비용과 겹쳐서 옴
- 풀 크기도 설계 대상: 너무 작으면 대기, 너무 크면 자원 낭비
- 전용 vs 공유 서버는 ch18에서 별도로 다룸

접속 통계로 확인

logons cumulative

```
SQLT@orcl> SELECT name, value FROM v$sysstat
  2        WHERE name IN ('logons cumulative','logons current');

NAME                                 VALUE
---------------------------- ----------
logons cumulative                   <환경값>   -- 누적 로그온(재접속마다 증가)
logons current                      <환경값>   -- 현재 접속 수

-- 짧은 시간에 logons cumulative 급증 → 매 요청 재접속 의심
```

16.5

배열 처리와 왕복 횟수

- 한 행씩 주고받는 비용 — 네트워크 왕복
- arraysize와 왕복·논리 읽기의 관계
- SQL*Net 대기 = 네트워크 문제? 오해 주의

한 행씩 주고받는 비용

arraysize와 round-trip

- arraysize = 클라이언트가 한 번에 가져오는 행 수
- 작으면 여러 번 왕복 → 왕복마다 네트워크 지연·논리 읽기
- DML(특히 INSERT)의 단건 반복도 같은 문제 → 배열 인터페이스 사용
- PL/SQL: BULK COLLECT / FORALL이 같은 원리
- 데이터 접근 호출은 비즈니스 로직 옆에 두는 것이 최선

arraysize와 왕복 — 계산

10,000행 반환 시 (전체 행÷arraysize+1)

- arraysize 1: 약 10,001회 왕복 (극단적으로 비효율)
- arraysize 15(SQL*Plus 기본): 약 667회
- arraysize 100: 약 101회 (6분의 1)
- arraysize 1000: 약 11회
- 무한정 크게? OLTP 첫 화면은 작게, 배치·리포트는 크게

autotrace로 왕복 확인

SET ARRAYSIZE로 대조

```
SQLT@orcl> SET AUTOTRACE TRACEONLY STATISTICS
SQLT@orcl> SET ARRAYSIZE 15
SQLT@orcl> SELECT * FROM claim_details;      -- 기본 배열 크기
SQLT@orcl> -- 'SQL*Net roundtrips to/from client', 'consistent gets' 기록
SQLT@orcl> SET ARRAYSIZE 1000
SQLT@orcl> SELECT * FROM claim_details;      -- 배열 크기 확대
SQLT@orcl> -- 왕복 급감 + 논리 읽기 감소를 대조
```

16.6

그 밖의 설계 원칙

- 자주 쓰고 잘 안 바뀌는 데이터는 캐시한다
- 절차는 절차 언어로, 데이터 접근은 SQL로
- 객체지향·Java 개발의 흔한 함정

설계 원칙 — 캐시와 언어

확장 가능한 앱을 만드는 규칙

- 자주 접근·거의 안 바뀌는 데이터는 캐시(단, 원래보다 확실히 싸게)
- SYSDATE FROM DUAL이 워크로드의 60%+를 차지하기도 함
- 절차 로직은 C·Java·PL/SQL로, 데이터 접근/DML은 SQL로
- 참조 무결성은 앱이 아니라 DB의 외래 키 제약으로 강제(빠르고 무네트워크)
- 로컬 캐시는 미들 계층으로 확장 가능(단, 너무 복잡해지면 이득 상실)

객체지향·Java의 함정

데이터 접근을 감추는 설계의 대가

- Java는 이식성 좋지만 인터프리터라 느리고 클라이언트 자원 더 씀
- 데이터 접근을 클래스로 캡슐화 → 효율 모른 채 메서드 호출
- → 최소·거친 인터페이스, WHERE 술어 누락, 앱에서 필터링
- 단건 INSERT로 배열 인터페이스 못 씀
- 종단 간 추적(End to End Tracing)용 액션·모듈 이름 설정 검토

16.7

배포 전 워크로드 테스트

- 워크로드 추정 — 유사 시스템 외삽·벤치마킹·모델링
- 설계 검증 테스트 규칙
- 배포 전략(Big Bang·Trickle)과 성능 체크리스트

워크로드 추정

여러 방법으로 교차 검증

- 유사 시스템 외삽 — 특성 비슷한 기존 시스템을 기준으로 수정
- 벤치마킹 — 자원·시간 많이 들고 운영과 상관 낮을 수 있으나 I/O 파악에 유용
- 벤치마크는 모든 컴포넌트를 한계까지 → 설계 오류가 다 드러남
- 애플리케이션 모델링 — 정밀 모델~어림 계산, 구현 비효율은 미반영
- 비효율을 위한 여유를 미리 두고, 성능 방법(ch17)으로 발견

설계 검증 테스트 규칙

문서로 남기면 용량 계획의 자료

- 현실적 데이터량·분포로 테스트(완전 채운 테이블·운영 인덱스·통계)
- 운영에서 쓸 옵티마이저 모드로 테스트
- 단일 사용자 성능을 먼저 확보(안 되면 다중 사용자는 불가능)
- 모든 SQL의 실행계획을 확보·문서화
- 다중 사용자·DML은 잠금·직렬화 문제 테스트, 운영에 가까운 하드웨어

배포 전략

Big Bang vs Trickle

- Big Bang: 모든 사용자를 한 번에 이전
- — 데이터 변환·동기화 최소, 그러나 규모 테스트에 크게 의존
- Trickle: 사용자를 조금씩 이전
- — 확장성 문제를 부하 증가 중 디버깅, 소수만 영향, 데이터 동기화 필요
- 두 방식 모두 전환 중 중단 위험 있음 → 테스트 지식이 많을수록 유리

성능 체크리스트

배포를 돕는 작업 목록

- SQL이 커서를 효율적으로 쓰는가 — 한 번 파싱·여러 번 실행
- 안 되는 최빈 이유: 바인드 미사용 + 리터럴 술어
- 미들웨어가 반복 로그온·로그오프 안 하는가
- 스키마 객체·통계가 개발→운영으로 올바르게 이관되었는가
- 배포 직후 기준선 통계 확보 → 첫 병목을 예상하고 성능 방법 적용

퀴즈 (Quiz)

- Q1. 하드 파싱과 소프트 파싱의 차이는? 왜 소프트 파싱도 이상적이지 않은가?
- Q2. 리터럴 SQL이 값마다 새 커서를 만드는 이유를 실측(300 vs 0)과 연결해 설명하라.
- Q3. CURSOR_SHARING=FORCE가 근본 해법이 아닌 이유는?
- Q4. 4-CPU 테스트의 270/150/60/30이 뜻하는 바를 확장성 관점에서 설명하라.
- Q5. arraysize를 15→1000으로 올리면 10,000행 조회의 왕복은 어떻게 바뀌는가?
- Q6. SQL*Net message from client가 상위인데 네트워크가 원인이 아닐 수 있는 경우는?

요약 (Summary)

- 확장성은 자원을 늘린 만큼 처리 능력이 비례해 느는 성질. 설계에 가장 큰 영향.
- 리터럴 SQL은 값마다 커서·하드 파싱. 실측 리터럴 300 → 300, 바인드 300 → 0.
- 바인드 변수는 커서 1개 재사용. 코드 못 고칠 때만 CURSOR_SHARING=FORCE.
- 접속은 비싸다. 무파싱 270 → 재접속 30. 커넥션은 풀링해 재사용.
- arraysize를 키우면 왕복·논리 읽기 감소. SQL*Net 대기≠네트워크 문제.
- 배포 전 현실적 데이터로 단일 사용자부터 검증, 배포 직후 기준선 통계 확보.

실습 개요 (Practice Overview)

- 실습 01 — WL_01_parse.sql: 바인드 유무별 커서 수·하드 파싱 델타 대조(300 vs 0)
- 실습 02 — CURSOR_SHARING 설정별 동작(EXACT vs FORCE) 확인
- 실습 03 — 커넥션 획득 비용 측정: 접속 반복 vs 재사용(logons cumulative)
- 실습 04 — arraysize 변경에 따른 SQL*Net 왕복·consistent gets 변화 확인
- 실습 05 — 시퀀스 경합 재현: 오류 없이 대기로만 드러나는 확장성 문제와 CACHE 조정

Oracle Database 19c 성능 튜닝 실무 과정

성능 개선 방법론과 긴급 대응

Performance Improvement Methods — 무엇부터, 어떤 순서로 손대는가

성능 개선 방법론 · 긴급 대응 5단계 · 블로킹 세션 추적 · 조치 전 근거 캡처

ch17

학습 목표 (Objectives)

- 오라클 성능 개선 방법론의 여섯 단계를 순서대로 설명하고, '병목을 확인한 뒤에만 바꾼다'는 전제를 지킨다.
- 증상과 원인을 혼동하지 않고, 자원 대기(wait events)에서 직렬화 지점과 원인으로 좁혀 들어간다.
- 오라클에서 자주 나오는 열 가지 실수를 알고, 눈앞의 증상을 이 목록과 대조해 원인 가설을 세운다.
- 긴급 대응 방법의 다섯 단계를 설명하고, 응급 조치와 근본 원인 분석을 분리해 수행한다.
- 응급 상황에서 CPU 병목인지 대기 병목인지 V$ 뷰로 판별한다.
- V$SESSION.BLOCKING_SESSION·V$LOCK·DBA_BLOCKERS로 블로커와 대기자를 지목한다.
- 근거를 캡처한 뒤에만 세션을 종료한다. 조치 전 캡처 없는 튜닝은 도박임을 안다.

이 장의 구성 (Agenda)

- 17.2 오라클 성능 개선 방법론 — 일하는 방식과 여섯 단계
- 17.3 증상에서 원인으로 — 자주 나오는 열 가지 실수
- 17.4 긴급 대응 — 다섯 단계와 '조치·원인 분석 분리'
- 17.5 응급 진단 체크리스트 — CPU 병목인가 대기 병목인가
- 17.6 블로킹 세션 추적과 해소 (실측 SID 596/597)
- 17.7 조치 전 근거 캡처 — 도박이 아니라 튜닝
- 17.8~17.10 요약 · 퀴즈 · 실습 개요

17.2

오라클 성능 개선 방법론

- 일하는 방식이지 규칙의 집합이 아니다
- 병목을 확인한 뒤에만 바꾼다
- 한 번에 하나만 바꿔 차이를 측정한다
- 가장 큰 병목 하나에 집중한다

방법론은 규칙이 아니라 '일하는 방식'

The Oracle Performance Improvement Method

- 시스템은 저마다 달라 절대 규칙이 없다 — '규칙은 규칙이 없다는 것'
- 데이터를 근거로 측면에서 사고해 병목을 짚는다
- 경험만 믿고 데이터를 무시하면 '신화와 민담에 의한 튜닝'에 빠진다
- ADDM은 이 방법론의 일부를 코드로 구현한 것 — 그래도 권고는 가설이다
- 성능 개선은 본질적으로 반복적(iterative)이다

병목을 확인한 뒤에만 바꾼다

방법론의 전제

- 성능 문제 = 처리량 부족 / 응답시간 불량 / 둘 다
- 확인 없는 변경은 방법론이 아니라 도박(ch15 '근거 없는 조치')
- 반복적이라 첫 병목 제거가 곧바로 개선으로 이어지지 않을 수 있다
- 직렬화 지점이 더 나쁜 방식으로 옮겨 가면 오히려 악화된다
- 그래서 '한 번에 하나만 바꾸고 측정'이라는 황금률이 필요하다

초기 표준 점검 — 증거부터 모은다

통계가 없는 것은 범죄 현장에 증거가 없는 것

- ① 사용자 피드백으로 목표·범위·종료 기준을 업무 언어로 확정
- '3시간 안에 100만 건', '피크 시 5초 이내'처럼 수치로
- ② 좋을 때·나쁠 때 OS·DB·앱 통계를 모두 확보 (비교 가능하게)
- ③ 모든 관련 컴퓨터의 OS를 점검 — 소진된 자원을 증상으로 목록화
- 성공의 척도는 언제나 사용자가 체감하는 성능이다

방법론의 여섯 단계

확인 → 대조 → 모델 → 조치 → 검증 → 반복

- 1 초기 점검: 목표 확정·통계 확보·OS 점검
- 2 흔한 실수 대조: 열 가지 실수와 대조(ADDM이 9개 자동 탐지)
- 3 개념 모델: 증상을 단서로 무슨 일이 벌어지는지 가설
- 4 조치 제안·적용: 예상 효과와 함께, 한 번에 하나만
- 5 효과 검증: 재측정·사용자 체감으로 판정
- 6 반복: 목표 달성 또는 불가능해질 때까지
- 인스턴스 튜닝 효과는 <10%, 앱 비효율 제거는 100%+

개념 모델 — 세 갈래로 좁힌다

A Sample Decision Process

- ① 한가한 시스템에서 단일 사용자도 느린가? → 애플리케이션 문제(설계·SQL)
- ② CPU가 다 쓰이는가? → 고부하 SQL·자원(V$SQL·V$SQLSTATS), OS의 비DB 작업 점검
- ③ CPU는 남는데 느린가? → 직렬화·대기(WAIT_EVENTS). 없으면 DB 밖
- 순서가 중요: ①→②→③
- ③이 이 장의 블로킹 세션 실습으로 직접 이어진다

17.3

증상에서 원인으로

- 자주 나오는 열 가지 실수
- ADDM이 아홉 개를 자동 탐지
- 증상을 원인으로 착각하지 않기
- 가설의 출발점이지 정답표가 아니다

자주 나오는 열 가지 실수 (1~5)

원인 가설의 체크리스트

- 1 잘못된 커넥션 관리 — 매 상호작용 접속·해제, 비확장적
- 2 커서·공유 풀 오용 — 바인드 변수 미사용 = 전면 하드 파싱
- 3 나쁜 SQL — 요구 대비 과도한 자원 사용
- 4 비표준 초기화 파라미터 — 검증된 계획을 덮어쓸 위험
- 5 잘못된 I/O 구성 — 용량 기준 배치, I/O 대역폭 무시

자주 나오는 열 가지 실수 (6~10)

증상을 원인으로 착각하지 말 것

- 6 온라인 리두 로그 설정 — 너무 적거나 작으면 잦은 체크포인트
- 7 버퍼 캐시 블록 직렬화 — free list·INITRANS 부족, ASSM으로 해소
- 8 긴 전체 테이블 스캔 — 인덱스 누락·SQL 문제의 신호
- 9 과도한 재귀(SYS) SQL — 공간 관리 활동 신호, LMT로 완화
- 10 배포·마이그레이션 오류 — 통계·인덱스 이관 실패

증상은 원인이 아니다

통계는 대개 원인이 아니라 증상을 가리킨다

- 물리 I/O가 높다 → 디스크 탓? 튜닝 안 된 SQL 탓?
- 긴 전체 스캔은 결과이지 원인이 아니다
- 래치 경합은 대개 애플리케이션 변경으로 해소
- 그래서 증상(대기)에서 원인(세그먼트·세션·SQL)으로 좁힌다
- 신화·민담에 의한 튜닝을 경계 — 반드시 내 통계로 확인

17.4

긴급 대응

- 방금 멀쩡하던 시스템이 멈췄을 때
- 빠른 진단 + 즉각 조치
- 그러나 사실 기록은 생략하지 않는다
- 응급 조치와 근본 원인 분석은 분리

긴급 대응이란

Emergency Performance Methods

- 신뢰할 수 있던 시스템이 갑자기 예측 불가·요청 처리 불가로 전환
- 무엇이 바뀌었는지 빠르게 판단하고 서비스를 빨리 회복
- 방법은 정공법과 같으나 시급함 때문에 지름길을 택한다
- 생략 금지: 발견한 사실을 상세히 기록한다(의사의 환자 기록처럼)
- 핵심 질문: '마지막으로 성능이 좋았던 이후 무엇이 바뀌었나?'

긴급 대응의 다섯 단계

지름길을 택하되 기록은 남긴다

- 1 증상 수집 — 처리량? 응답시간? 무엇이 바뀌었나? 기준점 확보
- 2 하드웨어 점검 — CPU·디스크·메모리·네트워크, 어느 계층인가
- 3 CPU vs 대기 판별 — V$SESS_TIME_MODEL / V$SESSION_WAIT
- 4 응급 조치 — 오프라인·워크로드 제한·재시작·작업 종료
- 5 안정성 검증 — 기준 통계 수집 후 정공법으로 복귀

응급 조치와 근본 원인 분석을 분리한다

이 장의 핵심 DBA Point

- 응급 조치의 목표: 서비스를 빨리 회복 (증상 멈춤)
- 근본 원인 분석: 회복 후 캡처를 근거로 차분히 (원인 제거)
- 섞으면 둘 다 실패 — 조치가 늦거나, 원인을 못 잡거나
- 순서: 근거 캡처 → 응급 조치 → 회복 검증 → 원인 분석 → 재발 방지
- 조치를 하는 순간 증거가 사라진다 → 캡처가 조치보다 앞선다

17.5

응급 진단 체크리스트

- CPU 병목인가, 대기 병목인가
- 이 판별이 조사 방향을 결정한다
- 원천은 언제나 V$ 뷰
- Idle 이벤트를 병목으로 오판하지 않기

CPU 병목인가 대기 병목인가

긴급 대응 3단계의 갈림길

- CPU 병목: V$SESS_TIME_MODEL(DB CPU), V$SESSTAT·V$SQLSTATS(buffer gets)
- 대기 병목: V$SESSION(EVENT·WAIT_CLASS), V$SESSION_WAIT
- Idle 이벤트(SQL*Net message from client)는 병목이 아니다
- Application 클래스 → 잠금 경합(3.6절), Concurrency → 라이브러리 캐시
- 로그인조차 어려우면 과거 데이터(ASH 🔒 /Statspack)로 진단

CPU 병목 조사 — 세션별 DB CPU

V$SESS_TIME_MODEL

```
SYS@orcl> SELECT s.sid, s.username, stm.value/1e6 db_cpu_sec
  2       FROM v$sess_time_model stm
  3       JOIN v$session s ON s.sid = stm.sid
  4       WHERE stm.stat_name = 'DB CPU'
  5         AND s.type = 'USER'
  6       ORDER BY stm.value DESC FETCH FIRST 5 ROWS ONLY;

       SID USERNAME   DB_CPU_SEC
---------- ---------- ----------
       ...      ...          ...   <- 환경값(부하에 따라 다름)
```

대기 병목 조사 — 활성 대기 집계

V$SESSION, Idle 제외

```
SYS@orcl> SELECT event, wait_class, COUNT(*) sessions
  2       FROM v$session
  3       WHERE type = 'USER' AND status = 'ACTIVE'
  4         AND wait_class <> 'Idle'
  5       GROUP BY event, wait_class
  6       ORDER BY sessions DESC;

EVENT                          WAIT_CLASS    SESSIONS
------------------------------ ------------ ---------
enq: TX - row lock contention    Application          1
...
```

17.6

블로킹 세션 추적과 해소

- CPU는 남는데 느린 직렬화의 전형
- V$SESSION → V$LOCK → DBA_BLOCKERS 세 겹
- 실측: SID 597이 596에 막힘
- 근거를 캡처한 뒤에만 종료한다

행 잠금 경합

enq: TX - row lock contention

- UPDATE 시 그 행에 TX 잠금이 배타 모드(LMODE=6)로 걸린다
- 커밋·롤백 전까지 유지 → 같은 행을 바꾸려는 세션은 대기
- 대기 이벤트 = enq: TX - row lock contention, 클래스 = Application
- 잠금은 정상 동작 — 문제는 잠금 보유 시간(트랜잭션 설계)
- 근본 예방은 짧은 트랜잭션·사용자 대기 중 잠금 미보유(ch16)

V$SESSION.BLOCKING_SESSION (실측)

SID 597이 596에 막힘

```
SYS@orcl> SELECT sid, username, event, blocking_session blk, wait_class
  2       FROM v$session WHERE username = 'SQLT' ORDER BY sid;

       SID USERNAME EVENT                          BLK WAIT_CLASS
---------- -------- ------------------------------ --- -----------
       596 SQLT     SQL*Net message from client        Idle
       597 SQLT     enq: TX - row lock contention  596 Application
```

V$LOCK — 누가 보유하고 누가 요청하나 (실측)

block=1 보유, request=6 대기

```
SYS@orcl> SELECT sid, type, lmode, request, block
  2       FROM v$lock WHERE type = 'TX' ORDER BY sid;

       SID TYPE      LMODE    REQUEST      BLOCK
---------- ---- ---------- ---------- ----------
       596 TX            6          0          1
       597 TX            0          6          0
```

DBA_BLOCKERS · DBA_WAITERS — 요약 뷰

관계를 즉시 나열

```
SYS@orcl> SELECT holding_session blocker, waiting_session waiter
  2       FROM dba_waiters;

   BLOCKER     WAITER
---------- ----------
       596        597

SYS@orcl> SELECT holding_session FROM dba_blockers;

HOLDING_SESSION
---------------
            596
```

블로킹과 교착 상태는 다르다

blocking vs deadlock

- 블로킹: A가 B를 기다리는 한 방향 — 시간이 지나 커밋·롤백하면 풀림
- 블로킹은 오라클이 자동 해소하지 않는다(무한정 유지 가능)
- 교착 상태: A↔B 순환 — 스스로 풀리지 않는다
- 오라클이 순환을 자동 탐지 → 한쪽에 ORA-00060, 그 문장만 롤백
- 실습 4의 행 잠금 경합은 한 방향 블로킹(교착 아님)

해소 — 근거를 캡처한 뒤에만 종료

종료하면 증거가 사라진다

```
-- ★ 반드시 먼저: 아래 근거를 캡처해 파일로 남긴다 ★
--   1) V$SESSION (블로커·대기자·EVENT·BLOCKING_SESSION)
--   2) V$LOCK    (TX 행의 LMODE / REQUEST / BLOCK)
--   3) 대상 세그먼트·행 (V$SESSION의 ROW_WAIT_OBJ# 등)

-- 캡처가 끝난 뒤에만 실행한다.
SYS@orcl> ALTER SYSTEM KILL SESSION '596,<SERIAL#>' IMMEDIATE;

System altered.
```

어느 세그먼트의 어느 행인가

대상 식별 — V$SESSION.ROW_WAIT_*

- ROW_WAIT_OBJ# / FILE# / BLOCK# / ROW# 로 대기 대상 행을 특정
- ROW_WAIT_OBJ#를 DBA_OBJECTS와 조인하면 객체 이름이 나온다
- '어느 테이블의 행이 잠겼는가' → 관련 업무 트랜잭션 추적
- 실습 4: 대기자 597이 REVIEW_LOG_WL의 특정 행을 대기
- 세션 종료 시 이 값들도 사라진다 → 종료 전에 캡처

17.7

조치 전 근거 캡처

- 도박이 아니라 튜닝
- 만성 저하와 긴급 상황 모두에 적용
- 순간 상태는 조치와 함께 소멸한다
- 지금 남기지 않으면 영원히 없다

조치 전 근거 캡처 — 도박이 아니라 튜닝

이 장을 관통하는 원칙

- 조치 전 상태를 캡처하지 않은 튜닝은 튜닝이 아니라 도박이다
- 방법론: '한 번에 하나만 바꾸고 측정'의 전제
- 긴급: '세션을 죽이기 전에 근거를 남긴다'의 규율
- ch15 세 경계 중 '근거 없는 조치'와 같은 뿌리
- V$SESSION·V$LOCK은 순간 상태 — 조치하면 사라진다

캡처가 필수인 이유

성능 데이터의 두 가지 성질

- ① 순간값·누적값 — 순간 상태는 조치와 함께 소멸
- 누적값은 시작·끝 두 번 떠 델타로 계산해야 의미가 생긴다
- ② 조치는 되돌릴 수 없다 — 종료된 세션·롤백된 작업은 복구 불가
- 캡처 내용: 블로커·대기자 SID, 잠금 모드·종류, 세그먼트·행, 대기 이벤트
- 형식보다 '조치 전에 남겼는가'가 중요 (SPOOL로 충분)

퀴즈 (Quiz)

- Q1. 성능 개선 방법론의 '황금률'을 쓰고, 그것을 지키려면 왜 조치 전 캡처가 필요한지 설명하라.
- Q2. 개념 모델 세 갈래에서 각각 먼저 무엇을 의심하는가?
- Q3. 인스턴스 튜닝과 애플리케이션 비효율 제거 중 일반적으로 더 큰 향상을 주는 쪽과 그 이유는?
- Q4. 긴급 대응에서 응급 조치와 근본 원인 분석을 분리해야 하는 이유는?
- Q5. 실측에서 597의 블로커는 596이고 596의 대기 클래스는 Idle이었다. 진짜 블로커와 오판 이유는?
- Q6. V$LOCK의 TX 행에서 block=1과 request=6은 각각 무엇을 뜻하는가? 종료 전 반드시 할 일은?

요약 (Summary)

- 방법론은 규칙이 아니라 일하는 방식 — 병목 확인 후에만, 한 번에 하나만, 반복.
- 인스턴스 튜닝 효과는 <10%, 큰 효과는 애플리케이션 비효율 제거에서.
- 개념 모델 세 갈래: 단일 사용자도 느림=앱 / CPU 소진=자원·SQL / CPU 남음=직렬화.
- 긴급 대응은 방법론의 지름길 버전. 5단계를 밟되 사실 기록은 생략하지 않는다.
- 응급 조치와 근본 원인 분석은 분리한다. 캡처가 조치보다 앞선다.
- 블로킹 실측: 597이 596에 막힘. V$LOCK에서 596 block=1, 597 request=6. 유휴 블로커를 놓치지 않는다.
- 조치 전 상태 캡처 없는 튜닝은 도박이다.

실습 개요 (Practice Overview)

- 실습 01 — 성능 개선 방법론 절차로 현재 인스턴스 1회 진단(CPU/대기 비중)
- 실습 02 — WL_04_lock으로 응급 상황 재현, 자원 독점 세션 식별·차단
- 실습 03 — 라이브러리 캐시 경합에서 원인 SQL 지목(V$SQL·V$SQLSTATS)
- 실습 04 — 블로킹 세션 추적과 해소(실측 SID 596/597) — ch17 핵심
- 실습 05 — 진단 근거를 남기는 절차(SPOOL·V$ 캡처)
- ※ 환경: non-CDB orcl 19.3, 프롬프트 SYS@orcl> / SQLT@orcl>
- ※ 각 실습에 '조치 전 → 조치·조회 → 결과 해석' + '이상→원인→조치→재확인'

Oracle Database 19c 성능 튜닝 실무 과정

성능을 위한 데이터베이스 구성

Configuring a Database for Performance — 처음에 어떻게 놓는가

초기화 파라미터 · Undo와 ORA-01555 · 임시 테이블스페이스와 ORA-01652 · 전용/공유 서버 · 블록 크기와 행 이주

ch18

학습 목표 (Objectives)

- 성능에 영향을 주는 초기화 파라미터를 구분하고 V$PARAMETER로 구성을 읽는다.
- 자동 Undo 관리와 UNDO_RETENTION·RETENTION GUARANTEE가 ORA-01555에 얽히는 방식을 안다.
- 임시 테이블스페이스와 ORA-01652를 이해하고, Undo가 아니라 Temp를 봐야 함을 판별한다.
- 전용 서버와 공유 서버의 프로세스·메모리 차이를 설명하고 워크로드에 맞게 고른다.
- DB_BLOCK_SIZE·PCTFREE와 행 이주·체이닝의 관계를 알고 이주 행을 찾아낸다.
- 구성성 오류(ORA-01555·ORA-01652)를 SQL 문제로 오인하지 않는다.

이 장의 구성 (Agenda)

- 18.2 초기 인스턴스 구성 — 무엇을 처음에 정하는가
- 18.3 성능에 영향을 주는 초기화 파라미터
- 18.4 Undo 보존과 ORA-01555 — 읽기 일관성의 값
- 18.5 임시 테이블스페이스와 ORA-01652 — 정렬이 흘러넘칠 때
- 18.6 전용 서버와 공유 서버 — 프로세스와 메모리의 선택
- 18.7 블록 크기와 행 이주 — 한 블록에 행이 안 들어갈 때
- ※ 실습 환경: non-CDB orcl 19.3. 파라미터 변경·오류 재현은 개인 실습 환경 전용.

이 장의 핵심 메시지

구성 실수는 튜닝으로 덮이지 않는다

- Undo 부족의 ORA-01555는 SQL을 고쳐서 해결할 수 없다
- Temp 부족의 ORA-01652는 인덱스를 추가해서 해결할 수 없다
- 이런 오류를 SQL 문제로 오인하면 엉뚱한 곳에서 시간을 허비한다
- DBA Point: 처음에 제대로 놓아야 한다
- Common Error: Undo·Temp 부족을 SQL 문제로 오인

18.2

초기 인스턴스 구성

- 성능 영향 4갈래: 파라미터·Undo·리두·테이블스페이스
- DBCA 시드는 권고를 이미 충족
- 초기화 파일은 튜닝 첫 자리가 아니다

성능에 영향 없는 필수 파라미터

있어야 뜨는 최소 조건 — 성능 손잡이가 아니다

- DB_NAME / DB_DOMAIN — 이름과 위치
- OPEN_CURSORS — 세션당 커서 상한(권고 500), 키워도 빨라지지 않음
- CONTROL_FILES — 서로 다른 디스크에 2개 이상
- DB_FILES — 할당 가능한 최대 파일 수
- 너무 작으면 ORA-01000, 커도 성능은 그대로

리두 로그와 테이블스페이스

체크포인트와 세그먼트 공간 관리

- 리두 로그: 큰 로그가 일반적으로 유리, 최대 20분에 한 번 스위치
- 작은 로그는 잦은 체크포인트로 성능 저하(DBWR·체크포인트에 영향)
- 최적 크기: V$INSTANCE_RECOVERY.OPTIMAL_LOGFILE_SIZE
- 영구 TS: 로컬 관리 + 자동 세그먼트 공간 관리(ASSM) 권장
- 삽입·수정·삭제가 잦으면 반드시 로컬 익스텐트 관리

18.3

성능 초기화 파라미터

- 성능에 영향을 주는 파라미터는 소수
- PROCESSES가 1차 파라미터 — 다수 값이 파생
- 실측으로 인스턴스 성격을 읽는다

성능 영향이 있는 중요 파라미터

이 값들을 읽으면 인스턴스 성격이 보인다

- COMPATIBLE — 신기능 활성화 경계
- DB_BLOCK_SIZE — OLTP 8192 / DW 더 큼
- SGA_TARGET — 지정 시 버퍼캐시·공유풀 자동 분배
- PGA_AGGREGATE_TARGET — 정렬·해시 작업영역 자동 조정
- PROCESSES(1차) → SESSIONS 파생 / UNDO_MANAGEMENT·UNDO_TABLESPACE

V$PARAMETER 실측 — 이 인스턴스의 구성

성능 관련 항목 필터링 (orcl 19.3, 2026-08-25)

```
SYS@orcl> SELECT name, value FROM v$parameter
  2       WHERE name IN ('db_block_size','sga_target',
  3         'pga_aggregate_target','processes','sessions',
  4         'undo_retention','undo_tablespace','memory_target');

NAME                           VALUE
------------------------------ ----------------
db_block_size                  8192
memory_target                  0
pga_aggregate_target           734003200
processes                      500
sessions                       776
sga_target                     1577058304
undo_retention                 900
undo_tablespace                UNDOTBS1
```

실측 해석과 안전한 변경

값을 읽고, 근거를 남기고, 하나씩 바꾼다

- sga_target 1,577,058,304B ≈ 1.47GB / pga 734,003,200B ≈ 700MB
- memory_target=0은 '설정 안 함' — 오류 아님
- 변경 전 현재값을 반드시 캡처(근거 없는 조치 경계)
- SCOPE: MEMORY(현재) / SPFILE(다음 기동) / BOTH(둘 다)
- 정적 파라미터(PROCESSES)는 SPFILE + 재기동 → 재조회로 확인

18.4

Undo 보존과 ORA-01555

- Undo = 읽기 일관성·복구·롤백의 값
- UNDO_RETENTION은 목표이지 보장이 아니다
- ORA-01555는 SQL 문제가 아니다

Undo가 하는 두 가지 일

롤백과 읽기 일관성 — 둘 다 과거 이미지에서 나온다

- 행이 바뀌면 바뀌기 전 이미지가 Undo 세그먼트에 남는다
- ① 롤백: 커밋 안 된 변경을 되돌린다
- ② 읽기 일관성: 다른 세션이 트랜잭션 시작 시점의 스냅샷을 본다
- 이 과거 이미지가 사라지면 긴 조회가 ORA-01555
- 그래서 Undo는 '읽기 일관성의 값(비용)'이다

자동 Undo 관리

DBA가 신경 쓸 것은 크기와 보존 두 가지

- UNDO_MANAGEMENT=AUTO(기본) — 오라클이 세그먼트 투명 관리
- 수동 롤백 세그먼트는 하위 호환용, 새 구성에서 쓸 이유 없음
- MVCC(다중 버전 읽기 일관성)가 Undo 위에서 동작
- DBA 관심사: ① Undo 테이블스페이스 크기 ② UNDO_RETENTION
- V$UNDOSTAT(10분 통계) / V$ROLLSTAT(세그먼트 동작)

Undo 구성 실측

세 값의 조합이 ORA-01555 가능성을 결정

```
SYS@orcl> SELECT tablespace_name, contents, retention
  2       FROM dba_tablespaces WHERE contents='UNDO';
TABLESPACE_NAME   CONTENTS   RETENTION
----------------- ---------- -------------
UNDOTBS1          UNDO       NOGUARANTEE

SYS@orcl> SELECT tablespace_name, ROUND(SUM(bytes)/1024/1024) mb
  2       FROM dba_data_files
  3       WHERE tablespace_name='UNDOTBS1' GROUP BY tablespace_name;
TABLESPACE_NAME          MB
----------------- ----------
UNDOTBS1                 380
```

UNDO_RETENTION vs RETENTION GUARANTEE

둘 중 무엇을 희생할 것인가

- UNDO_RETENTION: 커밋된 Undo를 최소 N초 보존하려는 '목표'
- NOGUARANTEE: 공간 부족 시 보존 안 지나도 덮어씀 → 긴 조회 ORA-01555
- GUARANTEE: 보존 시간 내 Undo 절대 안 덮어씀 → 대신 DML ORA-30036
- 오래 도는 리포트 필수 완주 → GUARANTEE
- DML 가용성 최우선 → NOGUARANTEE

ORA-01555 snapshot too old의 메커니즘

조회 길이·동시 변경량·Undo 공간의 함수

- 긴 조회 A는 시작 시점의 일관된 스냅샷을 봐야 한다
- 그 사이 다른 세션이 같은 데이터를 바꾸고 커밋
- Undo 공간 부족 → 시작 시점 과거 이미지가 덮어써짐
- A가 그 블록에 도달 → '과거 이미지 없음' → ORA-01555
- 조치: Undo 확장 / UNDO_RETENTION ↑ / RETENTION GUARANTEE

조치와 재확인 — V$UNDOSTAT로 판정

설정값과 실측 필요량을 비교한다

```
-- 조치(개인 실습 환경 전용)
SYS@orcl> ALTER SYSTEM SET undo_retention=3600 SCOPE=BOTH;
SYS@orcl> ALTER TABLESPACE undotbs1 RETENTION GUARANTEE;

-- 판정: 최장 조회 vs 보존, ORA-01555 발생 수
SYS@orcl> SELECT maxquerylen mql, tuned_undoretention tuned,
  2         ssolderrcnt ora1555 FROM v$undostat
  3       ORDER BY begin_time DESC FETCH FIRST 3 ROWS ONLY;
  MQL       TUNED    ORA1555
------- ---------- ----------
 <환경값>     <환경값>          0
-- MAXQUERYLEN > UNDO_RETENTION 이고 ORA1555>0 이면 보존 부족
```

실무 시나리오 — 리포트만 죽는 ORA-01555

'특정 긴 조회만' 실패하면 Undo를 의심한다

- 상황: 월말 마감일 오전, 30분짜리 정산 리포트만 ORA-01555로 실패
- 진단: V$UNDOSTAT의 MAXQUERYLEN > UNDO_RETENTION(15분), SSOLDERRCNT>0
- 원인: 마감 트랜잭션이 Undo를 빠르게 순환 → 과거 이미지 덮어씀
- 판정: 조회 길이(30분) > 보존(15분) → 전형적 ORA-01555, SQL 문제 아님
- 조치: UNDO_RETENTION 3600으로↑ + GUARANTEE(단, Undo 크기 확인)

18.5

임시 테이블스페이스와 ORA-01652

- 정렬·해시·GTT가 흘러넘치는 곳
- 오류 메시지의 TS 이름이 진단의 절반
- Undo가 아니라 Temp를 본다

임시 테이블스페이스

메모리에 안 담기는 작업을 디스크로

- 정렬(ORDER BY·GROUP BY·인덱스 생성)·해시 조인·전역 임시 테이블
- PGA 작업영역 초과 시 임시 TS로 스필(1-pass/multi-pass, ch33)
- 권장: 로컬 관리 + UNIFORM 익스텐트 1MB
- 동시 GTT 다수 → 익스텐트 256K로 작게 / 큰 정렬 → 1MB
- 실측 TEMP = 20MB (대량 정렬엔 부족)

ORA-01652 판별 — Temp인가 SQL인가

두 가지 오인을 피한다

- 오인①: 데이터 TS 부족으로 착각 → 메시지 끝 TS 이름을 반드시 확인
- TEMP이면 임시 공간 / 업무 TS면 세그먼트 확장 실패(성격 다름)
- 오인②: SQL 튜닝으로만 해결하려 함
- 정렬이 업무상 정당하면 → 구성(TEMP 확장)이 정답
- 불필요한 정렬이면 → SQL(정렬 제거·인덱스 회피)

TEMP 확장과 원인 세션 지목

V$TEMPSEG_USAGE로 누가 삼켰는지 본다

```
SYS@orcl> SELECT tablespace_name, ROUND(SUM(bytes)/1024/1024) mb
  2       FROM dba_temp_files GROUP BY tablespace_name;
TABLESPACE_NAME          MB
---------------- ----------
TEMP                     20     <- 확장 전

-- 조치(개인 실습 환경 전용)
SYS@orcl> ALTER TABLESPACE temp ADD TEMPFILE
  2   '.../temp02.dbf' SIZE 512M AUTOEXTEND ON NEXT 64M MAXSIZE 2G;

-- 재확인 → TEMP 532MB. 원인 세션은 V$TEMPSEG_USAGE로 지목
```

18.6

전용 서버와 공유 서버

- 서버 프로세스를 어떻게 배정하는가
- 전용=접속마다 프로세스 / 공유=디스패처로 다중
- 기본은 전용, 공유는 검토 대상

전용 vs 공유 — 구조의 차이

UGA 위치와 적합 워크로드가 갈린다

- 전용: 접속 수만큼 서버 프로세스 / UGA는 PGA
- 공유: 소수 공유 서버+디스패처 / UGA는 SGA(라지 풀)
- 공유 이점: 프로세스·메모리 절감, 고접속률에 유리
- 공유 약점: 병렬 실행 중 세션 묶임, 장시간 작업 부적합
- 기본은 전용 / 간헐 OLTP 다수 접속 → 공유 검토

접속 비용과 사전 생성 프로세스

접속 방식은 어떻게 확인하나

- 전용은 접속마다 프로세스를 순차 초기화 → 고접속률에 비용
- 공유는 디스패처가 대기하다 동시 접속 요청을 바로 받음
- V$SESSION.SERVER: DEDICATED / SHARED / NONE / POOLED
- 19c 사전 생성 프로세스: dedicated broker·threaded 모드
- V$PROCESS_POOL로 관찰, DBMS_PROCESS로 관리

공유 서버 경합 진단과 완화

프로세스를 늘리기 전에 있는 자원을 촘촘히

- V$DISPATCHER_RATE: CUR_ vs AVG_ vs MAX_ 비교
- 현재값이 MAX 근처 → 디스패처 추가 / 한참 낮으면 과다
- V$QUEUE: 요청 큐 대기 꾸준히 증가 → 공유 서버 경합
- V$SHARED_SERVER: STATUS!='QUIT'로 현재 서버 수
- 완화: 디스패처 추가 / 접속 풀링 / 세션 다중화

18.7

블록 크기와 행 이주

- 블록 크기 = I/O 효율과 동시성의 균형
- 이주는 UPDATE로 커진 행이 옮겨지는 것
- 이주와 체이닝의 근본 해법은 다르다

DB_BLOCK_SIZE와 성능

실측 8192 — OLTP의 표준값

- 큰 블록: 한 I/O로 많이 읽음 → DW 대량 스캔 유리
- 큰 블록의 대가: 한 블록에 행 몰림 → buffer busy 경합 ↑
- OLTP 8KB는 I/O 효율과 동시성의 균형점
- 블록 크기는 DB 생성 시 결정, SYSTEM은 변경 불가
- DB_nK_CACHE_SIZE로 비표준 블록 TS 추가 가능

PCTFREE — 이주를 막는 예약 공간

블록에 UPDATE 여유를 남긴다

- PCTFREE: 블록의 몇 %를 삽입에 안 쓰고 남겨 둔다
- UPDATE로 행이 커질 여지 → 이주 예방
- UPDATE 잦고 행이 커지는 테이블 → PCTFREE 높게(20~30)
- INSERT만·갱신 거의 없음 → PCTFREE 낮게(5~10)로 빽빽이
- ASSM에서 PCTUSED는 불필요하지만 PCTFREE는 유효

행 이주 vs 행 체이닝

무엇이 옮겨지고, 무엇이 원래 큰가

- 이주: UPDATE로 커진 행이 원 블록에 안 들어가 이동 + 포인터 잔류
- → 인덱스 조회 시 한 번 더 점프 → 논리·물리 읽기↑
- 체이닝: 행 자체가 블록보다 커서 처음부터 여러 블록에 걸침
- 이주 해법: PCTFREE↑ + ALTER TABLE … MOVE + 인덱스 REBUILD
- 체이닝 해법: 더 큰 블록 크기 테이블스페이스

이주 행을 찾아내기

ANALYZE ... LIST CHAINED ROWS / CHAIN_CNT

```
-- (1) 결과 테이블 생성
SYS@orcl> @?/rdbms/admin/utlchain.sql

-- (2) 이주/체이닝 행 수집
SQLT@orcl> ANALYZE TABLE claim_details LIST CHAINED ROWS
  2         INTO chained_rows;

-- (3) 개수 확인
SQLT@orcl> SELECT COUNT(*) FROM chained_rows
  2         WHERE table_name='CLAIM_DETAILS';

-- 통계에도: DBA_TABLES.CHAIN_CNT (이주+체이닝 합산)
```

실습 환경의 테이블스페이스 지도

어느 오류가 어느 TS와 연결되는지 기준선

```
SYS@orcl> SELECT tablespace_name, ROUND(SUM(bytes)/1024/1024) mb, COUNT(*) files
  2       FROM dba_data_files GROUP BY tablespace_name ORDER BY mb DESC;
TABLESPACE_NAME          MB      FILES
---------------- ---------- ----------
USERS                  2369          2
SYSTEM                  940          1
SYSAUX                  900          1
TS_STG                  422          1
UNDOTBS1                380          1     <- ORA-01555 (4.4)
RTBS                     50          1
TEMP  (temp file)        20          1     <- ORA-01652 (4.5)
```

퀴즈 (Quiz)

- Q1. 성능 파라미터 중 '1차 파라미터'로 다수 값이 파생되는 것은? 파생 예 하나?
- Q2. UNDO_RETENTION=900·NOGUARANTEE에서 Undo 부족 시 무슨 일이? GUARANTEE라면?
- Q3. ORA-01555의 근본 원인을 '조회 길이·동시 변경량·Undo 공간'으로 설명하라.
- Q4. ORA-01652가 임시 공간 문제인지 무엇으로 가르는가? SQL 튜닝이 정답인 경우와 아닌 경우는?
- Q5. 전용 vs 공유 서버의 프로세스·UGA 차이와 각각 유리한 워크로드는?
- Q6. 행 이주와 체이닝의 차이, 근본 해법이 다른 이유(PCTFREE vs 블록 크기)는?

요약 (Summary)

- 초기 구성이 성능의 상한을 정한다. 성능 파라미터는 소수이고 PROCESSES가 1차.
- 실측: 8KB 블록, ASMM(SGA 1.5GB/PGA 700MB), processes 500·sessions 776, TEMP 20MB.
- ORA-01555는 Undo 세그먼트가 작아서가 아니라 과거 이미지를 덮어써서 난다 — SQL 문제 아님.
- ORA-01652는 임시 공간 부족. 메시지의 TS 이름을 먼저 보고 구성/SQL을 가른다.
- 전용=접속마다 프로세스, 공유=디스패처 다중. 간헐 OLTP는 공유, 장시간·배치는 전용.
- 행 이주는 PCTFREE로 예방·CHAIN_CNT로 탐지. 체이닝은 블록 크기 문제.
- 구성성 오류를 SQL 문제로 오인하지 않는다 — 이 장의 Common Error.

실습 개요 (Practice Overview)

- 실습 01 — 성능 관련 초기화 파라미터 일괄 점검(V$PARAMETER 필터링)
- 실습 02 — Undo 보존과 ORA-01555 재현→UNDO_RETENTION·테이블스페이스 조정
- 실습 03 — 임시 테이블스페이스 부족(ORA-01652) 재현과 TEMP 확장
- 실습 04 — 전용 서버 vs 공유 서버 접속 방식 비교(프로세스·메모리)
- 실습 05 — 블록 크기와 행 이주 관찰(ANALYZE ... LIST CHAINED ROWS)
- ※ 모두 개인 실습 환경(non-CDB orcl 19.3) 전용. 조치 전→조치→조치 후 3단 캡처.

측정 기준과 통계 인프라

- TUN ch19 · ch20 · ch21
- 과정의 축, DB Time

Oracle Database 19c 성능 튜닝 실무 과정

성능 측정의 기준 — 통계와 DB Time

Measuring Database Performance — 이 과정의 축

DB Time · 시간 모델(V$SYS/SESS_TIME_MODEL) · 대기 클래스 · 누적값 vs 델타 · STATISTICS_LEVEL

ch19

학습 목표 (Objectives)

- DB Time을 'DB CPU + 대기'로 정의하고, 모든 활성 세션 시간의 합·벽시계 초과 가능성을 설명한다.
- 시간 모델(V$SYS_TIME_MODEL·V$SESS_TIME_MODEL)로 DB Time을 분해하고 시스템/세션 범위를 구분한다.
- 누적값과 델타값의 차이를 이해하고, 관심 구간의 시작·끝을 빼서 부하를 측정한다.
- 짧은 구간에서 시간 모델 분해가 불안정함을 실측으로 확인하고, 수 분 구간(AWR)의 필요성을 설명한다.
- 대기를 12개 클래스로 묶어 V$SYSTEM_WAIT_CLASS로 집계하고 Idle 제외 상위를 지목한다.
- 대기의 실측 증거를 V$SESSION_EVENT에서 확보한다(db file scattered read 120회·65ms).
- 히트율·계산 통계를 지표로 다루고 TIMED_STATISTICS·Idle·원시 카운트 검증 원칙을 적용한다.
- STATISTICS_LEVEL 세 수준을 알고 이 장(무료)과 ch20 AWR(🔒 Diagnostics Pack)의 경계를 구분한다.

이 장의 구성 (Agenda)

- 19.2 데이터베이스 통계와 델타 — 누적값을 그대로 읽지 않는다
- 19.3 DB Time — 이 과정의 공통 화폐
- 19.4 시간 모델 — V$SYS_TIME_MODEL과 V$SESS_TIME_MODEL
- 19.5 대기와 대기 클래스 — V$SYSTEM_WAIT_CLASS
- 19.6 통계 해석의 원칙 — 숫자를 오독하지 않는 법
- 19.7 STATISTICS_LEVEL — 측정을 켜고 끄는 스위치
- 19.8 부하 전후로 DB Time을 재는 절차
- ※ ADMIN 31장의 시간 모델·AWR 개념 소개를 실측·판독으로 확장(non-CDB orcl 19.3).

19.2

데이터베이스 통계와 델타

- 누적값 vs 델타
- 관심 구간의 변화량
- 짧은 구간의 함정 → ch20

누적값은 기동 이후의 합계다

cumulative statistics

- V$SYS_TIME_MODEL·V$SYSSTAT·V$SYSTEM_WAIT_CLASS 값은 기동 이후 누적
- 값은 절대 0이 아니고, 오래 떠 있을수록 커진다
- 절대값 하나로는 좋다·나쁘다를 판정할 수 없다
- 의미를 가지려면 '어느 구간의 값인가'가 붙어야 한다

V$SYS_TIME_MODEL — 누적값 실측

부하 전, 단위 마이크로초

```
SYS@orcl> SELECT stat_name, value FROM v$sys_time_model
  2       WHERE stat_name IN ('DB time','DB CPU', ...)
  3       ORDER BY value DESC;

STAT_NAME                          VALUE
------------------------------ ----------
DB time                          67698738
sql execute elapsed time         29344735
DB CPU                           17487220
parse time elapsed                3907411
hard parse elapsed time           3674923
```

측정 없이 판단하지 않는다

조치 전에 근거를 남긴다

- 성능을 재려면 통계가 '있어야' 한다(수집이 켜져 있어야)
- 구간 시작값을 캡처 안 하면 델타를 만들 재료가 없다
- 조치 전 근거 캡처 = 방법론이 아니라 습관
- 성능 데이터는 부하 없이 생기지 않는다 → 만들어서 잰다

델타와 짧은 구간의 함정

이 장의 핵심 기법

- 델타 = 끝 누적값 − 시작 누적값 (구간의 실제 부하)
- AWR이 하는 일 = 두 스냅샷의 델타 계산
- 구간이 너무 짧으면 CPU/대기 분해가 노이즈로 불안정
- 정확한 분해는 순간이 아니라 수 분 구간으로 → ch20 AWR

⚠ FLUSH BUFFER_CACHE는 개인 실습 전용

실측 재현을 위한 조치 — 운영 금지

- 위 실측은 부하 전 버퍼 캐시를 비워 물리 I/O 대기를 확실히 잡았다
- ALTER SYSTEM FLUSH BUFFER_CACHE = 캐시의 모든 블록 폐기
- 이후 조회가 디스크부터 다시 읽어 db file scattered read 발생
- ★ 운영 DB 절대 금지 — 전체 응답시간 급락, 개인 실습에서만

19.3

DB Time — 공통 화폐

- DB Time = DB CPU + 대기
- 벽시계 초과 가능
- 이질적 문제를 한 저울에

DB Time의 정의

= DB CPU + 대기

- 전경 세션이 DB 호출에 쓴 시간의 총합 = 인스턴스 워크로드 지표
- Idle이 아닌 모든 사용자 세션의 CPU 시간 + 대기 시간
- V$SESS_TIME_MODEL(세션) / V$SYS_TIME_MODEL(시스템)
- 성능 개선 = 같은 워크로드의 DB Time을 줄이는 것

벽시계 초과 · Average Active Sessions

놓치기 쉬운 두 가지

- DB Time은 모든 세션 시간의 합 → 벽시계 시간 초과 가능
- 예: 기동 30분, 활성 세션 4개 → 누적 DB Time ≈ 120분
- AAS = 구간 DB Time ÷ 구간 벽시계 시간(평균 활성 세션 수)
- AAS를 CPU 코어 수와 비교해 붐빈 정도를 가늠

이질적 문제를 한 저울에

시간이 공통 척도인 이유

- 느린 SQL·잠금 경합·I/O 지연은 서로 다른 단위
- 모두 'DB Time의 몇 %'로 환산하면 나란히 비교 가능
- 상위 대기 하나가 아니라 'DB Time 비중과 함께' 본다
- 조치의 가치도 DB Time 비중 감소로 평가한다

19.4

시간 모델

- 시스템 vs 세션 범위
- 델타로 부하 측정
- 짧은 구간 노이즈

V$SESS_TIME_MODEL — 세션 델타 실측

부하 전후 차이 (약 0.3초 구간)

```
-- 부하 전 캡처 → WL_06 축약 부하 → 부하 후 캡처 → 차이

이 세션 DB time 델타 : 275,636 us  (약 0.28초)
이 세션 DB CPU  델타 : 297,047 us  (약 0.30초)
-------------------------------------------------
→ DB time 대비 CPU 107.8% ,  대기 -7.8%
```

왜 CPU가 DB time보다 큰가

짧은 구간의 노이즈 — 핵심 교훈

- 0.3초 구간 → 카운터 갱신 시점 어긋남 + 반올림 = 불안정
- 버그·오차가 아니라 짧은 순간의 특성
- 대기의 실체는 시간 모델이 아니라 V$SESSION_EVENT로 확인
- 정확한 비율은 수 분 구간(AWR)으로 → ch20

시간 모델의 계층

시간이 어디로 갔는가

- 최상위 DB time 아래 sql execute / parse / PL/SQL execution
- parse time elapsed 아래 hard parse elapsed time
- 상위가 하위를 포함 — 단순 합산으로 DB time이 되지 않음
- 위→아래로 좁혀 가며 '시간의 행방'을 짚는 지도로 쓴다

19.5

대기와 대기 클래스

- 12개 대기 클래스
- V$SESSION_EVENT = 증거
- Idle은 항상 제외

대기 이벤트와 12개 클래스

증상이지 원인이 아니다

- 대기 이벤트 = 처리를 멈추고 무언가를 기다린 통계
- 수백 개 이벤트를 12개 클래스로 묶음(같은 해법 기준)
- User I/O · System I/O · Commit · Concurrency · Application ...
- Idle(SQL*Net message from client 등)은 병목 아님 → 제외

V$SESSION_EVENT — 대기의 실측 증거

이 세션이 기다린 것 (Idle 제외)

```
SQLT@orcl> SELECT event, total_waits, time_waited_micro/1000 ms
  2       FROM v$session_event WHERE sid=<내SID>
  3       AND wait_class<>'Idle' ORDER BY time_waited_micro DESC;

EVENT                          TOTAL_WAITS   MS_WAITED
------------------------------ ----------- -----------
db file scattered read                 120          65
db file sequential read                 12          17
log file sync                            1           0
SQL*Net message to client               12           0
```

V$SYSTEM_WAIT_CLASS — 클래스별 집계

기동 이후 누적, Idle 제외

```
SYS@orcl> SELECT wait_class, total_waits, time_waited/100 sec
  2       FROM v$system_wait_class WHERE wait_class<>'Idle'
  3       ORDER BY time_waited DESC;

WAIT_CLASS       TOTAL_WAITS        SEC
---------------- ----------- ----------
Network                 8583        358
System I/O             81525         77
User I/O               13860         52
Other                  19811         27
Application              128          7
```

세션·시스템 통계와 ASH

V$SYSSTAT · V$SESSTAT · ASH

- 논리/물리 읽기·redo·파싱·커밋·정렬 = V$SYSSTAT/V$SESSTAT
- 대기와 짝: db file scattered read ↑ ↔ physical reads ↑
- 이 카운터도 누적 → 델타로 본다(AWR Instance Activity)
- ASH: 활성 세션 1초 샘플링 = 🔒 Diagnostics Pack(개념만, ch25)

19.6

통계 해석의 원칙

- 히트율은 지표
- 대기는 시간과 함께
- 숫자 하나로 결론 X

히트율은 판정자가 아니라 지표

단일 비율의 함정

- 히트율·소프트 파스 비율·래치 히트율 = 지표(indicator)
- 비효율 SQL이 같은 블록 반복 읽으면 히트율은 오히려 상승
- 히트율 99%인데 응답시간이 나쁠 수 있다(ch31 재현)
- 봐야 할 것은 '논리 읽기 자체가 왜 많은가'

대기는 시간과, 계산 통계는 카운트와

TIMED_STATISTICS · Idle · 원시 카운트

- TIMED_STATISTICS로 대기 시간 수집(TYPICAL이면 자동)
- 30초/2시간 vs 30분/45분 — 시간 비중으로 판단
- Idle 이벤트는 병목 아님 → 조사에서 제외
- 계산 통계는 원시 카운트로 검증(소프트 파스 50%의 함정)

통계를 다른 요인과 함께 본다

같은 숫자, 다른 의미

- 대기 비중에 사용자 부하·하드웨어 능력도 함께 고려
- 45분 중 30분 대기라도 2,000명·64노드면 문제 아닐 수 있다
- 사용자 몇 명뿐 시스템에서 같은 비중이면 훨씬 심각
- 환경이 다르면 실측값도 다르다 → 절대값 단정 금지(<환경값>)

19.7

STATISTICS_LEVEL

- 측정을 켜는 스위치
- BASIC/TYPICAL/ALL
- 이 장 무료 · AWR 🔒

STATISTICS_LEVEL 확인

측정 자체를 켜고 끄는 스위치

```
SYS@orcl> SELECT name, value FROM v$parameter
  2       WHERE name = 'statistics_level';

NAME                 VALUE
-------------------- ----------
statistics_level     TYPICAL

-- BASIC  : 자동 통계·AWR·시간 모델 상당수 OFF (비권장)
-- TYPICAL: 기본값·권장. TIMED_STATISTICS 자동 ON
-- ALL    : TYPICAL + OS 통계·계획 통계(오버헤드 큼)
```

라이선스 경계 — 이 장은 무료

🔒 다음 장부터 유상

- 이 장의 뷰(시간 모델·대기 클래스·V$SESSION_EVENT 등) = 팩 불필요
- 라이선스 없이도 델타 기법으로 전부 측정 가능
- AWR·ADDM·ASH·DBA_HIST_*·V$ACTIVE_SESSION_HISTORY = 🔒 Diagnostics Pack
- 대안: V$ 델타 + Statspack(무상). control_management_pack_access 확인

19.8

부하 전후 측정 절차

- 4단계 캡처
- Load Profile 환산
- 교차 검증

부하 전후로 재는 4단계

실습으로 이어지는 절차

- ① 부하 직전 관심 통계 누적값 캡처(시간모델·통계·대기)
- ② 부하를 일정 구간 실행(WL_06_mixed.sql)
- ③ 부하 직후 같은 통계 재캡처
- ④ 델타 계산 → 초당·트랜잭션당으로 환산(Load Profile 형식)

환산과 교차 검증

'느려진 것'과 '일이 늘어난 것'

- 초당·트랜잭션당 환산 → 구간 길이가 달라도 비교 가능
- 트랜잭션당 값이 부하 증가 vs 트랜잭션 무거워짐을 구분(→ch23)
- 물리 읽기 델타↑ ↔ User I/O 대기 델타↑ 여야 앞뒤가 맞음
- 환산 값이 이상하면 먼저 '표본이 충분한가'부터 확인

퀴즈 (Quiz)

- Q1. DB Time의 정의와, DB Time이 벽시계 시간을 초과할 수 있는 이유를 쓰라.
- Q2. 누적값과 델타값의 차이를 설명하고, 왜 판단을 델타로 해야 하는지 쓰라.
- Q3. 0.3초 구간에서 DB CPU 델타(297,047)>DB time 델타(275,636)인 원인과, 대기 확인 방법은?
- Q4. 대기 집계에서 Idle을 제외하는 이유는? 대표 Idle 이벤트 하나를 들어라.
- Q5. 계산 통계 해석 원칙 두 가지를 쓰라(소프트 파스 50% 예 포함).
- Q6. STATISTICS_LEVEL 세 값과, 이 장과 ch20 AWR의 라이선스 경계를 구분하라.

요약 (Summary)

- DB Time = DB CPU + 대기. 모든 비휴지 전경 세션 시간의 합, 이 과정의 공통 화폐.
- 통계는 누적값. 절대값이 아니라 관심 구간의 델타로 본다 — 이 장의 핵심 기법.
- 짧은 구간(0.3초)은 시간 모델 분해가 불안정(CPU 107.8%). 수 분 AWR 구간 필요(→ch20).
- 대기의 실체는 V$SESSION_EVENT — db file scattered read 120회·65ms가 물리 I/O 증거.
- 대기 클래스 12종, V$SYSTEM_WAIT_CLASS로 집계. Idle 항상 제외. 누적≠구간 델타.
- 히트율은 지표, 대기는 시간과, 계산 통계는 원시 카운트와 함께. 숫자 하나로 결론 X.
- STATISTICS_LEVEL=TYPICAL 권장. 이 장 뷰는 무료, AWR부터 🔒 Diagnostics Pack.

실습 개요 (Practice Overview)

- 실습 01 — 시스템 시간 모델로 DB Time 구성 확인(V$SYS_TIME_MODEL)
- 실습 02 — 세션 시간 모델로 부하 구간 분해(V$SESS_TIME_MODEL, 짧은 구간 노이즈)
- 실습 03 — 대기 클래스 집계와 상위 지목(V$SYSTEM_WAIT_CLASS / V$SESSION_EVENT)
- 실습 04 — STATISTICS_LEVEL 수준별 차이와 TIMED_STATISTICS 관계
- 실습 05 — WL_06_mixed 전후 DB Time 델타 측정과 초당·트랜잭션당 환산
- 실습 06 — 누적값 오판 재현과 델타 방식 교정
- ※ 환경: non-CDB orcl 19.3, 프롬프트 SYS@orcl> / SQLT@orcl> (실습 6개)
- ※ 각 실습: '조치 전 → 조치·조회 → 결과 해석' + '이상 징후→원인→조치→재확인'

Oracle Database 19c 성능 튜닝 실무 과정

통계 수집 ① 자동 수집과 AWR 인프라

Gathering Database Statistics — 무엇을 얼마나 오래 기억할 것인가

AWR 스냅샷 주기·보존 · 베이스라인 · SYSAUX 용량 · Statspack 대안 🔒 Diagnostics Pack

ch20

학습 목표 (Objectives)

- AWR이 무엇을 수집해 어디에 저장하는지, 스냅샷·베이스라인·이동 윈도우의 관계를 그린다.
- DBA_HIST_WR_CONTROL로 주기·보존을 확인하고 MODIFY_SNAPSHOT_SETTINGS로 변경한다.
- CREATE_SNAPSHOT으로 부하 구간 앞뒤에 수동 스냅샷 쌍을 확보한다(실습: 295~296).
- 고정·이동 윈도우 베이스라인을 구분하고 필요한 구간을 보호·조회·삭제한다.
- V$SYSAUX_OCCUPANTS로 SM/AWR 점유(281.9MB)를 추적하고 안전하게 정리한다.
- 보존 초과로 데이터가 사라지기 전에 사고 구간을 베이스라인으로 얼려 둔다.
- 라이선스가 없으면 Statspack·V$ 델타로 같은 것을 본다. 🔒 이 장 전체가 Diagnostics Pack 대상.

이 장의 구성 (Agenda)

- 20.2 AWR 인프라 개요 — 무엇을 기억하는가
- 20.3 스냅샷 — 주기와 보존을 정한다
- 20.4 수동 스냅샷 — 부하 구간을 손으로 감싼다
- 20.5 AWR 베이스라인 — 필요한 구간을 보호한다
- 20.6 SYSAUX 용량 관리 — AWR이 차지하는 공간
- 20.7 보존 기간 초과와 데이터 소실
- 20.8 Statspack — 라이선스 없는 환경의 대안
- ※ ADMIN 31장의 AWR·스냅샷 개념 소개를 인프라 운영으로 확장

20.2

AWR 인프라 개요

- V$는 휘발성 · AWR은 영구 기억
- 무엇을 수집해 어디에 저장하는가
- 누적값과 델타
- 스냅샷·베이스라인·이동 윈도우

왜 V$만으로는 부족한가

AWR이 메우는 두 한계

- V$ 뷰 = 지금 이 순간(누적 또는 최근 몇 초)의 상태
- 한계1: 재기동하면 0으로 초기화 → 과거 소실
- 한계2: 살아 있어도 '지난 화요일 3시'를 되짚을 수 없다
- AWR: 주기적으로 V$ 핵심을 떠 디스크에 영구 저장
- 재기동에도 살아남고, 과거 구간을 스냅샷 쌍으로 재구성

AWR이 담는 것 — 누적값과 델타

스냅샷이라는 봉투

- 오브젝트·시간 모델·시스템/세션 통계·상위 SQL·ASH
- 세션 수준 제외 대부분 통계의 누적값+델타를 보존
- 누적값=기동 이후 계속 증가 / 델타=앞 스냅샷과의 차이
- ch19의 '뒤 값-앞 값'을 AWR이 스냅샷마다 자동 저장
- 보고서가 '구간'을 지정하면 값이 나오는 이유

스냅샷 · 베이스라인 · 이동 윈도우

세 개념을 시간 축으로

- 스냅샷 = 시간 선 위에 60분마다 찍히는 점
- 이동 윈도우 = 항상 최근 8일을 덮는(왼쪽 끝이 밀려가는) 구간
- 고정 베이스라인 = 사람이 못 박은, 움직이지 않는 구간
- 이동 윈도우가 지나가며 오래된 점을 지워도…
- 고정 베이스라인이 감싼 점은 못이 박혀 지워지지 않는다

AWR 데이터를 읽는 뷰 — DBA_HIST_*

V$로 지금을, DBA_HIST_로 과거를

- DBA_HIST_SNAPSHOT — 스냅샷 목록
- DBA_HIST_WR_CONTROL — 현재 주기·보존 설정
- DBA_HIST_BASELINE / _DETAILS / _TEMPLATE — 베이스라인
- V$SYSSTAT ↔ DBA_HIST_SYSSTAT (역사가 짝을 이룬다)
- 라이선스 없어 못 쓰면 → V$를 두 번 찍어 델타 계산

20.3

스냅샷 — 주기와 보존

- 기본 60분 주기 · 8일 보존
- DBA_HIST_WR_CONTROL로 확인
- MODIFY_SNAPSHOT_SETTINGS(분 단위)
- 보존은 워크로드 주기에서 나온다

현재 설정 확인

DBA_HIST_WR_CONTROL (실측)

```
SYS@orcl> SELECT snap_interval, retention, topnsql
  2       FROM dba_hist_wr_control;

SNAP_INTERVAL         RETENTION             TOPNSQL
--------------------  --------------------  --------
+00000 01:00:00.0     +00008 00:00:00.0     DEFAULT

-- +00000 01:00:00.0 = 60분 (주기)
-- +00008 00:00:00.0 = 8일  (보존)  → 기본값 그대로
```

설정 변경

MODIFY_SNAPSHOT_SETTINGS (분 단위)

```
SYS@orcl> BEGIN
  2    DBMS_WORKLOAD_REPOSITORY.MODIFY_SNAPSHOT_SETTINGS(
  3        retention => 43200,  -- 분. 30일
  4        interval  => 30,     -- 분. 30분마다
  5        topnsql   => 100);   -- 상위 SQL 100개
  6  END;
  7  /
PL/SQL procedure successfully completed.

-- 필요한 인자만 골라 넘길 수 있다(예: retention만)
-- 변경 후 반드시 DBA_HIST_WR_CONTROL 재조회로 확인
```

보존 기간을 얼마로 잡을 것인가

감이 아니라 워크로드 주기에서

- 원칙: 최소한 하나의 완전한 워크로드 주기를 담을 만큼
- 주중 OLTP·주말 배치 = 주 단위 → 기본 8일이면 충분
- 월말 결산 = 월 단위 피크 → 보존을 한 달로
- 보존↓ 주의: ADDM·SQL/Undo/Segment Advisor 정확도 저하
- 주기·보존·이동윈도우는 한 세트로 설계한다

interval = 0 의 함정

자동 수집을 끄면 무슨 일이

- 주기를 0으로 두면 자동 스냅샷 수집이 멈춘다
- 워크로드·통계 자동 수집 중단 → 자가 관리 기능 대부분 정지
- 수동 스냅샷조차 만들 수 없다
- 그래서 오라클은 자동 수집 비활성화를 강력히 권고하지 않음
- '스냅샷 잠깐 끄자' 요청엔 이 부작용부터 설명한다

20.4

수동 스냅샷

- 부하 구간을 앞뒤로 감싼다
- CREATE_SNAPSHOT
- 실습: 295 → 부하 → 296
- 한 부하, 세 도구의 일관성

부하 구간 감싸기

CREATE_SNAPSHOT (실습 295~296)

```
SYS@orcl> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;   -- 시작
SYS@orcl> SELECT MAX(snap_id) FROM dba_hist_snapshot;  -->  295

-- (SQLT 세션) @WL_06_mixed.sql  ... 종합 부하 ...

SYS@orcl> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;   -- 종료

   SNAP_ID BEGIN_T      END_T
---------- -----------  -----------
       295 08-25 12:00  08-25 12:15
       296 08-25 12:15  08-25 12:17
```

정말 스냅샷이 생겼는가 — 확인 습관

이상 징후 → 원인 → 조치

```
- 이상: CREATE_SNAPSHOT 후에도 MAX(snap_id)가 그대로
SYS@orcl> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_SNAPSHOT;
SYS@orcl> SELECT MAX(snap_id) FROM dba_hist_snapshot;  --> 그대로?

-- 원인: STATISTICS_LEVEL=BASIC 또는
--       CONTROL_MANAGEMENT_PACK_ACCESS=NONE → AWR 꺼짐
SYS@orcl> SHOW PARAMETER statistics_level
SYS@orcl> SHOW PARAMETER control_management_pack_access

-- 조치: TYPICAL / DIAGNOSTIC(+TUNING) 로 되돌린 뒤 재시도
```

한 부하, 세 도구의 일관성

295~296 = 7세션 100초 종합 부하

- 같은 사건을 세 도구가 다른 언어로, 일관되게 기록
- ch22 AWR: DB CPU 95.7% · log file sync · 하드 파싱
- ch24 ADDM: 'Hard Parse Due to Literal Usage' 진단
- ch25 ASH: ON CPU 90.7%
- 구간을 어떻게 잡았는가가 진단의 질을 좌우한다

20.5

AWR 베이스라인

- 필요한 구간을 보호한다
- 고정 vs 이동 윈도우
- CREATE/DROP/RENAME_BASELINE
- DBA Point: 보호해야 살아남는다

베이스라인이란

포함된 스냅샷은 무기한 보존

- 특정 기간 스냅샷 집합을 비교용으로 보존
- 핵심: 포함된 스냅샷은 자동 퍼지에서 제외 → 무기한
- 고정: 사람이 지정한 과거의 '정상' 연속 구간
- 이동 윈도우: 보존 기간 안 전체(시스템 자동, 적응형 임계값용)
- DBA Point — 필요한 구간은 베이스라인으로 보호해야 살아남는다

생성·조회

CREATE_BASELINE / DBA_HIST_BASELINE

```
SYS@orcl> BEGIN
  2    DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE(
  3        start_snap_id => 295, end_snap_id => 296,
  4        baseline_name => 'TUNING_CH6_DEMO',
  5        expiration    => 30);   -- 30일 뒤 만료
  6  END;
  7  /
SYS@orcl> SELECT baseline_name, baseline_type,
                 start_snap_id, end_snap_id FROM dba_hist_baseline;
SYSTEM_MOVING_WINDOW  MOVING_WINDOW   288  296
TUNING_CH6_DEMO       STATIC          295  296
```

삭제 — 실습 후 정리

DROP_BASELINE (cascade FALSE)

```
SYS@orcl> BEGIN
  2    DBMS_WORKLOAD_REPOSITORY.DROP_BASELINE(
  3        baseline_name => 'TUNING_CH6_DEMO',
  4        cascade       => FALSE);  -- 딸린 스냅샷은 보존
  5  END;
  6  /
SYS@orcl> SELECT COUNT(*) FROM dba_hist_baseline
  2       WHERE baseline_name='TUNING_CH6_DEMO';  --> 0

-- cascade FALSE: 베이스라인만 해제, 스냅샷 보존
-- cascade TRUE : 딸린 스냅샷까지 함께 삭제
```

베이스라인 템플릿

미래 구간을 예약한다

- 지나간 구간이 아니라 앞으로 올 구간을 자동 베이스라인화
- 단일 템플릿: 미래의 한 번의 고정 구간
- 반복 템플릿: 매주 월요일 오전처럼 되풀이되는 구간
- CREATE_BASELINE_TEMPLATE (start/end_time, 이름, 만료)
- 월초·분기말 등 반드시 남길 시각을 미리 예약

20.6

SYSAUX 용량 관리

- AWR = 기본 SYSAUX 저장
- V$SYSAUX_OCCUPANTS의 SM/AWR
- 실측 281.9MB (전형적 200~300MB)
- 주기↑·보존↓·DROP_SNAPSHOT_RANGE

점유 확인

V$SYSAUX_OCCUPANTS (실측)

```
SYS@orcl> SELECT occupant_name,
  2         ROUND(space_usage_kbytes/1024,1) mb
  3       FROM v$sysaux_occupants
  4       WHERE occupant_name = 'SM/AWR';

OCCUPANT_NAME          MB
---------------- ----------
SM/AWR              281.9

-- 오라클 기준: 평균 10 활성세션 시스템 ≈ 200~300MB
-- 281.9MB = 전형적 범위 안
```

공간을 줄이는 두 축과 수동 정리

DROP_SNAPSHOT_RANGE

- 정공법1: 주기를 늘린다(덜 자주 찍는다)
- 정공법2: 보존을 줄인다(덜 오래 남긴다) — 정확도 주의
- 급히 회수: DROP_SNAPSHOT_RANGE(low, high)
- 단, 베이스라인에 묶인 스냅샷은 이 방법으로도 안 지워짐
- 자동 퍼지에 맡겨도 보존 지나면 정리됨 → 수동은 응급 수단

SM/AWR이 유난히 커졌다면

원인을 순서대로 좁힌다

- ① 주기가 짧아졌는가 (60분 → 10분이면 6배)
- ② 보존이 길어졌는가 (8일 → 60일)
- ③ topnsql이 MAXIMUM 등 큰 값인가
- ④ 활성 세션이 많은 바쁜 시스템인가
- DBA_HIST_WR_CONTROL과 실제 설정으로 확인

20.7

보존 초과와 데이터 소실

- 8일 지난 스냅샷은 자동 퍼지
- 사고 조사가 늦으면 원본 소실
- 조사 전에 사고 구간을 베이스라인으로
- Common Error를 막는 0번 단계

사고 구간을 즉시 보호하라

조사 전에 얼려 둔다

```
-- (사고 인지 직후) 사고 시각을 포함하는 구간을 찾는다
SYS@orcl> SELECT snap_id, TO_CHAR(end_interval_time,'MM-DD HH24:MI')
  2       FROM dba_hist_snapshot
  3       WHERE end_interval_time BETWEEN :t1 AND :t2
  4       ORDER BY snap_id;   -->  295, 296

-- 그 구간을 즉시 베이스라인으로 보호 (조사 전에!)
SYS@orcl> EXEC DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE(
             295, 296, 'INCIDENT_0825');
```

'어제 데이터가 없다'고 당황하기 전에

소실은 사고가 아니라 예정된 동작

- 보존 넘긴 구간은 보고서가 비거나 지정 불가
- 확인1: DBA_HIST_WR_CONTROL의 보존 기간
- 확인2: DBA_HIST_SNAPSHOT의 가장 오래된 snap_id·시각
- 가장 오래된 스냅샷이 사고보다 뒤면 → 이미 퍼지됨
- 이해하면 다음부터 베이스라인으로 미리 막게 된다

20.8

Statspack — 무상 대안

- 🔒 이 장 전체가 Diagnostics Pack
- 라이선스 없으면 Statspack
- spcreate · STATSPACK.SNAP · spreport
- AWR과의 차이를 안다

설치·수집·보고

spcreate.sql / STATSPACK.SNAP / spreport.sql

```
SYS@orcl> @?/rdbms/admin/spcreate.sql   -- PERFSTAT 설치

PERFSTAT@orcl> EXEC STATSPACK.SNAP;    -- 시작 스냅샷
PERFSTAT@orcl> -- ... WL_06_mixed.sql 부하 ...
PERFSTAT@orcl> EXEC STATSPACK.SNAP;    -- 종료 스냅샷
PERFSTAT@orcl> @?/rdbms/admin/spreport.sql   -- 보고서

-- 레벨 5(기본)/6(계획)/7(세그먼트) · spauto(자동) · sppurge(정리)
-- 라이선스 불필요
```

Statspack의 위치와 한계

후퇴가 아니라 라이선스 경계 안의 진단

- AWR 이전(9i 이전) 시대의 표준 진단 도구
- 10g에서 AWR·ADDM·ASH가 Diagnostics Pack으로 묶임
- Statspack은 무상으로 남아 미보유 환경의 정식 대안
- 한계: ADDM 자동 진단·ASH·베이스라인 없음
- → 보고서를 사람이 읽어야 함 = ch22 판독력이 더 중요

AWR vs Statspack

무엇을 언제 쓰는가

- AWR: Diagnostics Pack 필요 / SYSAUX·DBA_HIST_*
- Statspack: 무상 / PERFSTAT 스키마 STATS$*
- ADDM 자동 진단·ASH·베이스라인 = AWR만
- 라이선스 있으면 AWR, 없으면 Statspack, 한 구간만이면 V$ 델타
- 세 경로 공통 원리: 두 시점의 차이를 부하로 읽는다

퀴즈 (Quiz)

- Q1. AWR 스냅샷은 어디에 저장되고 조회 뷰 접두어는? 담는 통계 다섯 중 셋은?
- Q2. 실측 주기·보존은 각각 얼마? MODIFY_SNAPSHOT_SETTINGS의 interval·retention 단위는?
- Q3. 자동 스냅샷이 있는데도 CREATE_SNAPSHOT을 쓰는 이유는(부하 구간 관점)?
- Q4. 고정 vs 이동 윈도우 베이스라인 차이는? 자동 퍼지 제외가 왜 중요한가?
- Q5. AWR의 occupant_name은? 실측 점유·전형적 범위? 공간 줄이는 두 정공법은?
- Q6. 라이선스 없을 때 두 대안은? Statspack 스냅샷·보고서 명령과 AWR과의 차이 하나는?

요약 (Summary)

- AWR = 휘발성 V$를 스냅샷으로 SYSAUX에 영구 저장 · ch22~ch23의 토대
- 주기·보존 확인=DBA_HIST_WR_CONTROL(실측 60분·8일) / 변경=MODIFY_SNAPSHOT_SETTINGS(분)
- CREATE_SNAPSHOT으로 부하 앞뒤를 감싼다 (실습 295~296, 7세션 100초)
- 베이스라인 스냅샷은 무기한 보존 — 필요한 구간은 보호해야 살아남는다
- SM/AWR 점유=V$SYSAUX_OCCUPANTS(실측 281.9MB) / 주기↑·보존↓·DROP_SNAPSHOT_RANGE
- 사고 구간은 조사 전 즉시 베이스라인으로 · 이 장 전체 Diagnostics Pack, 대안=Statspack/V$ 델타

실습 개요 (Practice Overview)

- 실습 01 — 스냅샷 주기·보존 확인과 변경 (🔒 DBA_HIST_WR_CONTROL)
- 실습 02 — 수동 스냅샷 쌍 확보 (🔒 WL_06 사이 295~296)
- 실습 03 — 베이스라인 생성·조회·삭제 (🔒 TUNING_CH6_DEMO)
- 실습 04 — 보존 초과와 베이스라인 보호 (🔒)
- 실습 05 — SYSAUX 사용량 추적 (🔒 SM/AWR 281.9MB)
- 실습 06 — Statspack 대안 경로 (라이선스 불필요)
- ※ 환경: non-CDB orcl 19.3 · SYS@orcl> / SQLT@orcl> · '조치 전→조치→해석'+'이상→원인→조치→재확인'

Oracle Database 19c 성능 튜닝 실무 과정

히스토그램·확장 통계·시스템 통계

Statistics ② — 옵티마이저가 보는 세계를 정교하게

히스토그램 · METHOD_OPT · 확장 통계 · 시스템 통계 · Preferences · Statistics Advisor

ch21

학습 목표 (Objectives)

- 균등 분포 가정의 한계를 설명하고, 편중 컬럼의 추정 오차를 수치로 제시한다(300 대 3,100).
- 히스토그램 세 종류(FREQUENCY·TOP-FREQUENCY·HYBRID)와 생성 조건을 구분한다.
- METHOD_OPT(FOR ALL COLUMNS SIZE AUTO / FOR COLUMNS)의 동작을 예측한다.
- 컬럼 그룹·표현식 확장 통계로 상관·함수 추정 오류를 바로잡는다.
- NOWORKLOAD와 WORKLOAD 시스템 통계를 구분하고 안전하게 원복한다.
- 통계 Preferences의 네 계층(GLOBAL·DATABASE·SCHEMA·TABLE)과 우선순위를 안다.
- Optimizer Statistics Advisor(무료)의 권고를 해석해 통계 오차를 해소한다.
- 통계를 바꾸는 모든 작업에 조치 전 캡처와 원복 절차를 설계한다.

이 장의 구성 (Agenda)

- 21.2 히스토그램 — 편중된 데이터에서 옵티마이저를 구한다
- 21.3 METHOD_OPT — 어떤 컬럼에 히스토그램을 만들지 결정
- 21.4 확장 통계 — 컬럼 그룹과 표현식
- 21.5 시스템 통계 — 하드웨어를 옵티마이저에게 알린다
- 21.6 통계 Preferences — 수집 정책을 계층으로 관리 (ADMIN 33장 심화)
- 21.7 Optimizer Statistics Advisor — 통계 운영 자동 점검 (ADMIN 33장 심화)
- 21.8 흔한 함정 — 전체 재수집을 만능으로 여기지 않는다
- ※ 실습 7개 모두 non-CDB orcl 19.3 전용, 통계 변경 실습은 원복 필수

21.2

히스토그램

- 균등 분포 가정의 한계 (300 vs 3,100)
- FREQUENCY · TOP-FREQUENCY · HYBRID
- DBA_TAB_COL_STATISTICS 실측
- 접근 경로와 조인에 미치는 영향

균등 분포 가정과 그 한계

히스토그램 없으면 num_rows / NDV

- 옵티마이저 기본 가정: 컬럼 값이 균등하게 분포
- 등식 조건 추정 = 전체 행 수 ÷ NDV
- HOSP_ID: 300,000 ÷ 1,000 = 값당 300건 추정
- 그러나 실제 HOSP_ID=36 청구는 3,100건 → 약 10배 과소추정
- 편중 컬럼에서 균등 가정은 크게 빗나간다

균등 가정 추정치 — 실측

dba_tables / dba_tab_col_statistics

```
SQL> SELECT num_rows,
  2    (SELECT num_distinct FROM dba_tab_col_statistics
  3      WHERE owner='SQLT' AND table_name='MEDICAL_CLAIMS'
  4        AND column_name='HOSP_ID') ndv,
  5    ROUND(num_rows/1000) est_per_value
  6    FROM dba_tables
  7    WHERE owner='SQLT' AND table_name='MEDICAL_CLAIMS';

  NUM_ROWS        NDV EST_PER_VALUE
---------- ---------- -------------
    300000       1000           300

SQL> SELECT COUNT(*) FROM sqlt.medical_claims WHERE hosp_id=36;
  -> 3100   (균등가정 300의 약 10배)
```

히스토그램의 세 종류

NDV·버킷 수·분포가 종류를 결정

- FREQUENCY: NDV ≤ 버킷 수(최대 254) — 값마다 버킷 하나, 가장 정밀
- TOP-FREQUENCY: 상위 값이 전체 대부분 차지 — 롱테일 무시
- HYBRID: NDV 많고 TOP-N 조건 불충족 — 높이균형+빈도 결합
- 버킷 끝값의 반복 횟수까지 기록해 편중·범위를 함께 표현
- 종류는 자동 수집이 데이터에 맞춰 스스로 선택

컬럼별 히스토그램 현황 — 실측

DBA_TAB_COL_STATISTICS

COLUMN_NAME	NUM_DISTINCT	HISTOGRAM	NUM_BUCKETS
CLAIM_ID	300000	HYBRID	254
CLAIM_TYPE	2	FREQUENCY	2
DEPT_CODE	9	FREQUENCY	9
HOSP_ID	1000	HYBRID	254
PAT_ID	50376	NONE	1
RECEIPT_DATE	366	HYBRID	254
REVIEW_STATUS	2	FREQUENCY	2
TOTAL_AMT	189344	HYBRID	254
VISIT_DATE	379	NONE	1

히스토그램이란 무엇인가

값 분포를 버킷으로 저장

- 컬럼 값의 분포를 버킷(bucket)으로 나눠 저장한 통계
- 값 종류가 적으면 값마다 버킷 하나 → 정확한 빈도
- 값 종류가 많으면 여러 값을 한 버킷에 묶어 대표
- '평균의 함정'을 푸는 도구 — 균등 가정이 못 보는 쏠림을 드러냄
- 기본 상한 254버킷(12c 이후 최대 2,048)

히스토그램은 접근 경로와 조인 모두에 영향

카디널리티 오판의 파급

- 카디널리티 오판 300 → 소수라 여겨 인덱스 스캔 선택
- 실제 3,100건 → 3,100번 랜덤 액세스로 전체 스캔보다 느려짐
- 조인: 300으로 오판 → 중첩 루프 선택 → 예상의 10배 탐색
- 정확한 3,100 → 해시 조인 등 대량 처리 방식 선택 가능
- 히스토그램 하나의 정확도가 계획 전체로 파급된다

21.3

METHOD_OPT

- FOR ALL COLUMNS SIZE AUTO 기본값
- SIZE 절: AUTO·SKEWONLY·REPEAT·정수
- FOR COLUMNS로 특정 컬럼 보완
- AUTO_SAMPLE_SIZE의 정확도

현재 통계 Preference 확인 — 실측

DBMS_STATS.GET_PREFS

```
SQL> SELECT
  2   DBMS_STATS.GET_PREFS('METHOD_OPT','SQLT','MEDICAL_CLAIMS') method_opt,
  3   DBMS_STATS.GET_PREFS('ESTIMATE_PERCENT') est_pct,
  4   DBMS_STATS.GET_PREFS('DEGREE') degree
  5   FROM dual;

METHOD_OPT                 EST_PCT                       DEGREE
-------------------------- ----------------------------- ------
FOR ALL COLUMNS SIZE AUTO  DBMS_STATS.AUTO_SAMPLE_SIZE   NULL
```

SIZE 절 — 무엇을, 어떻게

AUTO·SKEWONLY·REPEAT·정수

- AUTO: 컬럼 사용 이력 + 편중을 함께 보고 자동 결정 (기본)
- SKEWONLY: 이력과 무관하게 편중 있는 모든 컬럼에 생성
- REPEAT: 이미 히스토그램 있는 컬럼에만 다시 생성
- 정수 1~2048: 지정 버킷 수 강제 (1 = 히스토그램 없음)
- FOR COLUMNS SIZE 254 HOSP_ID — 특정 컬럼만 보완

실무 — METHOD_OPT를 함부로 바꾸지 않는다

모든 컬럼 254 강제의 위험

- '모든 컬럼 SIZE 254 강제'는 수집 시간·SYSAUX 증가
- 불필요한 히스토그램이 바인드 피킹과 얽혀 계획 불안정
- 먼저 어떤 컬럼이 조건절에 편중과 함께 쓰이는지 확인
- AUTO가 놓친 특정 컬럼만 FOR COLUMNS로 보완
- 테이블 Preference로 관리하고 변경 전후 비교

21.4

확장 통계

- 컬럼 그룹 — 상관 컬럼의 곱셈 오류
- 표현식 통계 — 함수가 컬럼을 가릴 때
- CREATE_EXTENDED_STATS
- 생성 후 반드시 재수집

컬럼 그룹 — 상관된 컬럼의 곱셈 오류

독립 가정이 빗나갈 때

- 옵티마이저는 AND 조건의 선택도를 서로 독립이라 가정하고 곱함
- CLAIM_TYPE(외래 240,489 / 입원 59,511) × DEPT_CODE(9종)
- 상관되면 곱셈 추정이 실제와 크게 어긋남
- 예: 0.80 × 0.111 ≈ 26,700건 추정 — 실제와 다를 수 있음
- 컬럼 그룹 통계 → 조합의 실제 NDV·분포 참조

확장 통계 생성 — 컬럼 그룹

CREATE_EXTENDED_STATS + 재수집

```
SQL> SELECT DBMS_STATS.CREATE_EXTENDED_STATS(
  2         'SQLT','MEDICAL_CLAIMS',
  3         '(CLAIM_TYPE,DEPT_CODE)') AS ext_name
  4    FROM dual;

-- 생성만으로는 값이 비어 있다. 반드시 재수집!
SQL> EXEC DBMS_STATS.GATHER_TABLE_STATS('SQLT','MEDICAL_CLAIMS',-
>   method_opt=>'FOR ALL COLUMNS SIZE AUTO -
>                FOR COLUMNS (CLAIM_TYPE,DEPT_CODE)');

-- 확인: USER_STAT_EXTENSIONS / 제거: DROP_EXTENDED_STATS
```

표현식 통계 — 함수가 컬럼을 가릴 때

기본 선택도 1%의 함정

- WHERE EXTRACT(YEAR FROM receipt_date)=2026 — 원본 통계 못 씀
- 함수 결과 분포를 몰라 기본 선택도(대개 1%)로 추정
- 실제 비율이 크면 카디널리티 크게 어긋남
- 표현식 통계 → 함수 결과의 NDV·분포 저장
- 함수 기반 인덱스를 만들면 표현식 통계도 함께 생성

확장 통계 생성 — 표현식

EXTRACT(YEAR FROM RECEIPT_DATE)

```
SQL> SELECT DBMS_STATS.CREATE_EXTENDED_STATS(
  2         'SQLT','MEDICAL_CLAIMS',
  3         '(EXTRACT(YEAR FROM RECEIPT_DATE))') AS ext_name
  4    FROM dual;

SQL> EXEC DBMS_STATS.GATHER_TABLE_STATS('SQLT','MEDICAL_CLAIMS',-
>   method_opt=>'FOR ALL COLUMNS SIZE AUTO -
>     FOR COLUMNS (EXTRACT(YEAR FROM RECEIPT_DATE)) SIZE AUTO');

-- 함수 기반 인덱스를 만들면 표현식 통계도 함께 생성됨
```

21.5

시스템 통계

- NOWORKLOAD vs WORKLOAD
- SYS.AUX_STATS$ 실측 (현재 NOWORKLOAD)
- MBRC·SREADTIM·MREADTIM
- 수집하면 계획이 바뀔 수 있다 → 원복 필수

시스템 통계란

하드웨어를 옵티마이저에게 알린다

- 데이터 통계 = 카디널리티(몇 건) / 시스템 통계 = 비용(얼마나)
- NOWORKLOAD: 임의 I/O로 측정한 기본 특성 (DB 생성 시 기본)
- WORKLOAD: 실제 부하 구간을 관찰해 측정
- 전체 스캔(다중 블록) vs 인덱스 스캔(단일 블록) 비용 기준
- SYS.AUX_STATS$에 저장 · 인스턴스 전역에 영향

시스템 통계 현황 — 실측

SYS.AUX_STATS$ (NOWORKLOAD 상태)

```
SQL> SELECT sname, pname, pval1 FROM sys.aux_stats$
  2    WHERE pname IN ('CPUSPEEDNW','IOSEEKTIM','IOTFRSPEED',
  3                    'MBRC','SREADTIM','MREADTIM') ORDER BY pname;

SNAME          PNAME              PVAL1
-------------- ------------ -----------
SYSSTATS_MAIN  CPUSPEEDNW   3106.95187
SYSSTATS_MAIN  IOSEEKTIM            10
SYSSTATS_MAIN  IOTFRSPEED         4096
SYSSTATS_MAIN  MBRC              (null)
SYSSTATS_MAIN  MREADTIM          (null)
SYSSTATS_MAIN  SREADTIM          (null)
```

WORKLOAD 수집과 원복

계획이 바뀔 수 있다

- START ~ STOP 사이 실제 I/O 관찰 → MBRC·SREADTIM·MREADTIM 채움
- 대표성 있는 부하 구간을 골라야 함 (한산한 구간은 위험)
- 수집하면 전체 스캔↔인덱스 스캔 상대 비용 변화 → 계획 변경 가능
- 개선일 수도, 회귀일 수도 있다
- 원복: DELETE_SYSTEM_STATS → GATHER_SYSTEM_STATS('NOWORKLOAD')

21.6

통계 Preferences

- GLOBAL · DATABASE · SCHEMA · TABLE
- 좁은 범위가 넓은 범위를 이긴다
- SET_TABLE_PREFS / GET_PREFS
- STALE_PERCENT · INCREMENTAL (ADMIN 33장 심화)

적용 범위와 우선순위

좁은 범위가 우선

- GLOBAL (SET_GLOBAL_PREFS) — Preference 없는 모든 객체
- DATABASE (SET_DATABASE_PREFS) — 현재 모든 사용자 테이블
- SCHEMA (SET_SCHEMA_PREFS) — 해당 스키마 테이블
- TABLE (SET_TABLE_PREFS) — 지정한 한 테이블 (가장 우선)
- 우선순위: TABLE > SCHEMA > DATABASE > GLOBAL

테이블 정책 설정과 원복

SET_TABLE_PREFS / DELETE_TABLE_PREFS

```
-- 조치 전 값 기록
SQL> SELECT DBMS_STATS.GET_PREFS('METHOD_OPT',
  2         'SQLT','MEDICAL_CLAIMS') FROM dual;

-- 변경
SQL> EXEC DBMS_STATS.SET_TABLE_PREFS('SQLT','MEDICAL_CLAIMS',-
>          'ESTIMATE_PERCENT','100');

-- 원복
SQL> EXEC DBMS_STATS.DELETE_TABLE_PREFS('SQLT','MEDICAL_CLAIMS',-
>          'ESTIMATE_PERCENT');
```

자주 쓰는 Preference들

테이블 성격에 맞는 정책

- METHOD_OPT — 히스토그램 대상·버킷 정책
- ESTIMATE_PERCENT — 기본 AUTO_SAMPLE_SIZE 권장
- STALE_PERCENT — 재수집 기준 변경 비율(기본 10%)
- INCREMENTAL — 파티션 증분 수집으로 시간 절약
- 대부분 전역 기본, 초대형·핫 테이블만 예외로 TABLE 설정

21.7

Statistics Advisor

- 통계 운영을 룰로 자동 점검
- Finding · Recommendation · Rationale
- 무료 — Tuning Pack 아님
- 권고는 가설, 근거 확인 후 채택

Advisor 태스크 실행

CREATE / EXECUTE / REPORT

```
SQL> DECLARE v_task VARCHAR2(64);
  2  BEGIN
  3    v_task := DBMS_STATS.CREATE_ADVISOR_TASK('stat_adv_ch7');
  4    v_task := DBMS_STATS.EXECUTE_ADVISOR_TASK('stat_adv_ch7');
  5  END;
  6  /

SQL> SELECT DBMS_STATS.REPORT_ADVISOR_TASK('stat_adv_ch7',
  2         'TEXT','ALL') FROM dual;

-- 자동 적용: DBMS_STATS.IMPLEMENT_ADVISOR_TASK
```

권고 해석과 채택

전체 재수집은 최후의 수단

- 룰: 자동 작업 상태 / 통계 건강(없음·stale·잠금) / 설정 적정성
- 통계 없는 객체 → 그 객체만 GATHER
- 편중인데 히스토그램 없음 → FOR COLUMNS 보완
- 상관 → 확장 통계 / 오래됨 → STALE_PERCENT 조정
- 잠긴 통계가 의도된 것이면 풀지 않는다 — 맥락 확인 후 채택

21.8

흔한 함정

- 전체 재수집을 만능으로 여김
- 진단 → 국소 조치 → 검증
- 통계 변경은 되돌릴 수 있어야 한다

전체 재수집이 만능이 아니다

큰 망치의 세 가지 위험

- ① 진단을 건너뛴다 — 부재·편중·상관·시스템 중 무엇인지 모른 채
- ② 잘 맞던 히스토그램·확장 통계를 덮어써 계획 회귀 가능
- ③ 비용이 크다 — 시간·자원·SYSAUX 소모
- 올바른 순서: 진단(Advisor·계획 비교) → 국소 조치 → 검증
- 원복 안전장치: RESTORE_TABLE_STATS(기본 31일 보관)

퀴즈 (Quiz)

- Q1. 히스토그램 없을 때 HOSP_ID=36 추정치와 실제값의 배수는? 계산식과 함께.
- Q2. FREQUENCY·TOP-FREQUENCY·HYBRID의 생성 조건을 각각 한 줄로.
- Q3. METHOD_OPT의 SIZE AUTO와 SIZE SKEWONLY의 차이는?
- Q4. 컬럼 그룹 확장 통계가 푸는 문제와, 생성 후 반드시 할 일은?
- Q5. 시스템 통계가 NOWORKLOAD임을 어떻게 확인하고, WORKLOAD 실습 후 원복 절차는?
- Q6. Preferences 네 계층과 우선순위, 테이블 정책 프로시저 이름은?

요약 (Summary)

- 균등 가정 300 대 실제 3,100(약 10배)이 히스토그램의 필요성을 증명한다.
- 히스토그램: FREQUENCY(NDV≤버킷)·TOP-FREQUENCY·HYBRID. HOSP_ID는 HYBRID 254.
- METHOD_OPT 기본 AUTO는 사용 이력·편중으로 자동 결정, FOR COLUMNS로 보완.
- 확장 통계: 컬럼 그룹(상관)·표현식(함수). 생성 후 반드시 재수집.
- 시스템 통계: 현재 NOWORKLOAD. WORKLOAD 수집은 계획 변경 가능 → 원복 필수.
- Preferences: GLOBAL·DATABASE·SCHEMA·TABLE, 좁은 범위 우선.
- Statistics Advisor(무료)로 진단. 전체 재수집을 만능으로 여기지 않는다.

실습 개요 (Practice Overview)

- 실습 01 — 히스토그램 유무별 카디널리티 추정 차이(300 vs 3,100, 원복)
- 실습 02 — METHOD_OPT 지정 방식별 히스토그램 생성 결과 비교(원복)
- 실습 03 — 컬럼 그룹 확장 통계 생성(CLAIM_TYPE·DEPT_CODE, 제거)
- 실습 04 — 표현식 통계 생성(RECEIPT_DATE 함수, 제거)
- 실습 05 — 시스템 통계 수집과 계획 변화(NOWORKLOAD↔WORKLOAD, 원복)
- 실습 06 — 통계 Preferences 설정(SET_TABLE_PREFS, 원복)
- 실습 07 — Optimizer Statistics Advisor 실행과 권고 해석
- ※ non-CDB orcl 19.3 · 통계 변경 실습은 조치 전 캡처 + 원복 필수

보고서 판독과 기간 비교

- TUN ch22 · 브리지 02 · ch23(1h) · ch24 · ch25
- AWR 상위 SQL 을 Day 1~4 기법으로

Oracle Database 19c 성능 튜닝 실무 과정

AWR 보고서 읽기

AWR Report Interpretation — 숫자를 엮어 원인을 지목한다

Load Profile · Top 10 Events · Instance Efficiency · SQL 정렬 · Advisory · 실측 스냅샷 295~296

ch22

학습 목표 (Objectives)

- awrrpt.sql로 텍스트 AWR 보고서를 생성하고 섹션 구조를 파악한다.
- Load Profile에서 초당 논리 읽기·파싱·트랜잭션을 판독한다.
- Top 10 Foreground Events를 % DB time 기준으로 읽고 병목을 지목한다.
- '상위 대기 하나'가 아니라 DB Time 비중과 함께 원인을 수렴시킨다.
- Instance Efficiency의 함정(히트율)과 낮은 값의 단서를 구분한다.
- SQL 정렬 섹션에서 진단 가설에 맞는 기준으로 대상 SQL을 선별한다.
- Advisory로 메모리 조정 효과를 근거로 판단한다.
- 부하 없는 구간을 판별하고 라이선스(🔒)·Statspack 대안을 안다.

이 장의 구성 (Agenda)

- 22.2 AWR 보고서 생성 — awrrpt.sql과 섹션 구조
- 22.3 머리말 — DB Time / Elapsed = 평균 활성 세션
- 22.4 Load Profile — 초당 얼마나 일했는가
- 22.5 Top 10 Foreground Events — 상위 대기와 DB Time 비중
- 22.6 Instance Efficiency — 비율의 함정
- 22.7 SQL 정렬 섹션 — 대상 SQL 선별
- 22.8~22.9 Instance Activity / Advisory 판독
- 22.10~22.13 체크리스트 · 부하 없는 구간 · 라이선스 · 도구 연결

왜 AWR 보고서를 읽는가

ch22의 한 문장

- AWR = 두 스냅샷 사이 인스턴스 활동을 한 장으로 요약
- ch19 DB Time·시간모델·대기클래스가 이 보고서 안에 다 있다
- ch22은 '생성'이 아니라 '판독' — 숫자로 원인을 지목
- 핵심: 상위 대기 하나만 보고 결론 내지 않는다
- 언제나 DB Time 비중과 함께 본다
- 소재: 실측 스냅샷 295~296 (7세션 100초 종합 부하)

22.2

AWR 보고서 생성

- awrrpt.sql — text/html, 시작·종료 스냅샷 지정
- 전제: STATISTICS_LEVEL≥TYPICAL, 구간 내 재기동 없음
- 보고서는 '델타' — 두 스냅샷 차이(구간 값)
- 판독은 위(요약)에서 아래(세부)로

awrrpt.sql 실행

텍스트 보고서 생성 절차 (스냅샷 295~296)

```
SYS@orcl> @$ORACLE_HOME/rdbms/admin/awrrpt.sql

Enter value for report_type: text
Enter value for num_days: 1

  Snap Id   Snap Started         Snap Level
  -------   ------------------   ----------
    295     25 Aug 2026 14:20        1
    296     25 Aug 2026 14:22        1

Enter value for begin_snap: 295
Enter value for end_snap:   296
Report written to awrrpt_1_295_296.txt
```

AWR 보고서는 '델타'다

누적값이 아니라 구간 값

- V$ 통계 = 인스턴스 기동 이후 누적값(ch19)
- AWR 보고서 = 종료 스냅샷 – 시작 스냅샷 = 구간 실제량
- Load Profile의 Per Second도 이 100초 구간 한정
- SQL 섹션의 Buffer Gets도 이 구간에서만 발생한 양
- 누적값을 그대로 읽는 ch19의 오판을 자동 교정
- 전제: 두 스냅샷 사이 재기동 없음(있으면 델타 음수)

보고서 섹션 구조 — 읽는 순서

위에서 아래로 좁혀 내려간다

- 머리말 → 구간·DB Time (부하가 있었나)
- Load Profile → 초당 작업량 (부하 성격)
- Top 10 Events → 무엇을 기다렸나 (병목 종류)
- Instance Efficiency → 효율 비율 (함정 주의)
- SQL ordered by … → 어느 SQL이 자원을 썼나
- Instance Activity / Advisory → 세부 확증·메모리 조정

22.3

머리말 판독

- DB Time / Elapsed = 평균 활성 세션(AAS)
- 실측: 574초 / 100초 ≈ 5.7
- AAS를 CPU 코어 수와 비교 → CPU 포화 판정
- 머리말은 방향만 정한다

DB Time부터 본다

실측 구간: Elapsed 100초, DB Time 574초

- Elapsed = 두 스냅샷 사이 벽시계 시간
- DB Time = 모든 활성 세션의 CPU+대기 합
- DB Time / Elapsed ≈ 5.7 → 평균 5.7세션이 동시에 일함
- 7세션 부하 → 세션들이 거의 쉬지 않았음
- AAS를 호스트 코어 수와 비교: 코어보다 크면 CPU 경쟁
- '무엇을' 기다렸나 → Top Events / '누가' → SQL 섹션

22.4

Load Profile 판독

- DB Time vs DB CPU → CPU 바운드?
- 논리 vs 물리 읽기 → 메모리 반복 스캔?
- Parses 대비 Hard parses → 리터럴?
- 절대값이 아니라 항목 간 비율로 읽는다

Load Profile 실측

스냅샷 295~296 (Per Second / Per Transaction)

	Per Second	Per Txn
DB Time(s):	5.5	0.0
DB CPU(s):	5.3	0.0
Redo size (bytes):	140,017.0	677.6
Logical read (blk):	1,185,107.0	5,735.2
Block changes:	916.0	4.4
Physical read (blk):	48.8	0.2
User calls:	1,629.9	7.9
Parses (SQL):	433.7	2.1
Hard parses (SQL):	319.0	1.5
SQL Work Area(MB):	102.5	0.5

Load Profile이 그리는 그림

세 지표가 한 방향을 가리킨다

- DB CPU ≈ DB Time → 대기가 아니라 CPU를 태운다
- 논리 118만 : 물리 48.8 → 메모리 위 반복 스캔
- 하드 파싱 319/s (전체의 73%) → 리터럴 SQL 폭증
- User calls 1,629/s → 짧은 호출 대량 (가벼운 SQL 반복)
- Redo·Block changes → 잦은 커밋(log file sync 배경)
- → CPU 바운드 + 대량 논리읽기 + 리터럴 하드파싱

22.5

Top 10 Foreground Events

- 맨 윗줄은 대개 DB CPU (대기가 아님)
- % DB time으로 가중치를 매겨 읽는다
- 작은 대기들을 한 원인으로 묶는다
- log file sync 3.7%는 주범이 아니다

Top 10 Events 실측

% DB time 기준으로 읽는다

Event	Waits	Time(s)	%DB	Class
DB CPU		549	95.7	
log file sync	21,405	21.1	3.7	Commit
kksfbc child compl.	130	6.4	1.1	Other
db file sequential	1,681	3.2	.6	User I/O
cursor: pin S wait X	609	.9	.1	Concurr
library cache:mutex X	180	.4	.1	Concurr
latch: shared pool	3,506	.3	.1	Concurr
SQL*Net msg to clnt	125,937	.3	.0	Network

상위 대기 하나만 보지 않는다

DB Time 비중과 함께 읽는 법

- DB CPU % DB time이 크면 → CPU가 병목 (SQL·논리읽기 감소)
- 대기는 % DB time으로 우선순위 매김 (Total sec 아님)
- 같은 클래스(Concurrency) 작은 대기 = 하드파싱 경합으로 묶기
- Avg Wait: kksfbc 49.58ms(드물지만 길다) vs lfs 0.99ms(잦다)
- 처방: 잦으면 빈도↓(커밋 묶기), 길면 그 원인(경합) 제거

대기 이벤트의 의미 — 한 방향을 가리킨다

작은 대기들이 하드파싱으로 수렴

- kksfbc child completion: 다른 세션이 만드는 자식 커서 대기
- cursor: pin S wait on X: 커서를 S로 잡으려는데 X로 바뀌는 중
- library cache: mutex X / latch: shared pool: 공유 풀 직렬화 경합
- 네 대기 모두 Concurrency/Other → 공유 풀 커서 경합
- 근본 원인 = 초당 319번의 하드 파싱(Load Profile과 일치)

22.6

Instance Efficiency

- 'Target 100%'를 믿지 않는다
- Buffer Hit 100% = 함정일 수 있다
- 낮은 값(Soft Parse·Library Hit)이 단서
- 단독 판정 금지 — 교차 검증

Instance Efficiency 실측

낮은 값이 진단의 단서다

```
Instance Efficiency (Target 100%)
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   Buffer Nowait %: 100.00  Redo NoWait %: 100.00
   Buffer  Hit   %: 100.00  In-mem Sort  %: 100.00
   Library Hit   %:  59.36  Soft Parse   %:  26.46
Execute to Parse %:  36.72  Latch Hit    %:  99.93
Parse CPU/Elapsd %:  37.71  %Non-Parse CPU: 96.68
```

히트율의 함정과 낮은 값의 단서

비율은 단독으로 좋고 나쁨을 결정하지 않는다

- Buffer Hit 100% → 과도한 논리읽기를 오히려 은폐 (ch31 심화)
- Soft Parse 26% → 하드 파싱 74% = 리터럴 신호(핵심)
- Library Hit 59% → 커서 재사용 실패, 공유 풀 경합 유발
- Execute to Parse 37% → 커서를 몇 번 못 쓰고 버림
- Nowait·Latch·Redo 100%는 '그 자원이 병목 아님'일 뿐
- → Load Profile·Top Events와 교차 검증

22.7

SQL 정렬 섹션

- 무엇이(CPU·파싱) → 누가(어느 SQL)로 좁힌다
- 진단 가설에 맞는 정렬 기준 선택
- 이 구간: Elapsed Time + Gets
- 결론은 인스턴스 층위까지, 재작성은 SQL 튜닝으로

SQL ordered by Elapsed / Gets

같은 SQL이 두 정렬 모두 1위

```
SQL ordered by Elapsed Time
  Elapsed  Exec  /Exec %Total %CPU  SQL Id
  ------- ----- ----- ------ ---- -------------
    190.6 5,912  0.03   33.2 95.8 9vmszgd1ft9x0
  Module: python ...

SQL ordered by Gets  (Total Gets: 122,836,337)
     Gets   Exec   /Exec %Total %CPU  SQL Id
  --------- ----- ------- ----- ---- -------------
  39,149,973 5,912 6,622  31.9 95.8 9vmszgd1ft9x0
```

정렬 기준을 진단에 맞춰 고른다

아무 정렬의 1위나 지목하지 않는다

- CPU·논리읽기 문제 → Elapsed Time, Gets
- 디스크 문제 → Reads (이 구간은 물리읽기 낮아 우선순위↓)
- 파싱 문제 → Parse Calls (리터럴 SQL 직접 지목)
- 너무 자주 도는 SQL → Executions
- Captured SQL 34.6%뿐 → 리터럴이 조각나 순위 밖에 흩어짐
- 두 정렬이 같은 SQL을 가리키면 진단 신뢰도↑

Instance Activity Stats — 확증과 반증

여기서 새 이론을 세우지 않는다

- parse count (hard) 델타 ≈ 319/s×100초 → 하드파싱 확증
- session logical reads ≈ Total Gets 1억2천만 → 논리읽기 확증
- table scans (long) 크면 → 인덱스 못 타는 대량 스캔 확증
- session cursor cache hits 낮음 → 커서 재사용 실패 확증
- physical reads direct·sorts(disk) 크면 → CPU바운드 가설 수정
- 확증만 찾지 말고 반증 통계도 함께 본다(확증 편향 경계)

22.9

Advisory 섹션

- 메모리 크기 변경의 예상 효과를 정량화
- 버퍼 캐시: 물리읽기 낮으면 확대 무효
- 공유 풀: 확대는 증상 완화, 바인드가 원인 제거
- Advisory는 방향이지 명령이 아니다

Advisory 판독 — 늘려서 될 문제인가

기울기가 꺾이는 지점을 찾는다

- Buffer Pool: 물리읽기 48.8/s → 캐시 2배도 효과 ≈ 0
- Shared Pool: 하드파싱엔 유효하나 근본은 바인드 변수
- PGA: SQL Work Area 102MB → multi-pass 있으면 확대 여지(ch33)
- 각 표에서 효과가 평평해지는 지점 이상은 메모리 낭비
- 종합: '메모리 늘려 해결할 문제가 아니다'를 숫자로 증명

판독 체크리스트 — 위에서 아래로

각 단계가 앞 단계 가설을 이어받아 좁힌다

- 1. 머리말 — AAS로 부하 유무 (실측 5.7, 부하 있음)
- 2. Load Profile — CPU 바운드? (DB CPU≈DB Time)
- 3. Load Profile — 논리읽기·하드파싱 비정상? (둘 다)
- 4. Top 10 Events — % DB time 1위? (DB CPU 95.7%)
- 5. Instance Eff. — 낮은 값? (Soft Parse 26% 리터럴)
- 6. SQL 섹션 — 어느 SQL? (9vmszgd1ft9x0)
- 7. Advisory — 메모리 효과? (버퍼 무효, 원인은 SQL)

부하 없는 구간 판정

'문제 없음'도 근거와 함께

- AAS = DB Time/Elapsed ≪ 1 → 진단할 부하 자체가 없음
- 보고서는 유휴 구간에도 Top 10을 채워서 낸다(순위는 상대적)
- Idle 이벤트(SQL*Net msg from client)를 병목으로 오판 금지
- 머리말에서 부하 유무를 먼저 확인 후 Top Events를 읽는다
- 신고 시각 AAS 0.1이면 병목은 DB 밖 → ch34으로 연결

라이선스와 대안 — 🔒 Diagnostics Pack

없는 길도 이 과정은 가르친다

- AWR·ADDM·ASH·DBA_HIST_*·awrrpt.sql = Diagnostics Pack 필요
- CONTROL_MANAGEMENT_PACK_ACCESS ≥ DIAGNOSTIC 이어야 동작
- 대안 1) Statspack: spcreate → SNAP → spreport (무료)
- 대안 2) V$ 직접 조회: V$SYSSTAT·V$SYSTEM_EVENT·V$SQL 델타
- 판독 기술 자체는 도구와 무관 — 표 이름만 다를 뿐

같은 부하를 AWR·ADDM·ASH가 각자 보여준다

확대 배율이 다른 세 렌즈

- AWR(ch22): DB CPU 95.7%, 하드파싱 319/s, 논리읽기 1위 SQL
- ADDM(ch24): 'Hard Parse Due to Literal Usage' 자동 진단
- ASH(ch25): ON CPU 90.7% — 활성 세션이 대부분 CPU에
- AWR=구간 평균 → ADDM=자동 해석 → ASH=순간 확대
- 세 결론이 수렴하면 신뢰도↑ (교차 검증)

실측 구간 한눈에 — 하나의 원인으로 수렴

서로 다른 섹션의 숫자가 한 곳을 가리킨다

- 머리말: AAS 5.7 → 여러 세션이 동시에 경쟁한 실제 부하
- Load Profile: DB CPU≈DB Time + 하드파싱 319/s
- Top Events: DB CPU 95.7%, 나머지는 공유 풀 경합
- Instance Eff.: Soft Parse 26%, Library Hit 59%
- SQL: 9vmszgd1ft9x0 (DB Time 33%, Gets 32%, 95.8% CPU)
- → 결론: 리터럴 하드파싱을 유발하는 단일 SQL

이 장이 경계하는 오판 넷

큰 숫자 하나에 속지 않기

- ① Top Events 1위(대기)를 병목으로 지목 → % DB time을 본다
- ② Buffer Hit 100%를 성능 우수로 해석 → 히트율의 함정
- ③ Instance Activity의 큰 값으로 새 가설 → 확증/반증에만 쓴다
- ④ 부하 없는 구간을 '문제 있다'고 판정 → AAS로 부하 확인
- 공통 처방: 순서(위→아래) + 비중(% DB time) + 수렴

퀴즈 (Quiz)

- Q1. Top Events 1위가 DB CPU 95.7%. 병목은? log file sync 3.7%를 먼저 손대면 안 되는 이유는?
- Q2. 논리읽기 118만/s인데 물리읽기 48.8/s. 이 조합과 Buffer Hit 100%의 관계는?
- Q3. Buffer Hit 100%인데 성능이 나쁠 수 있는 이유(히트율의 함정)는?
- Q4. Soft Parse 26%, Hard parse 319/s가 가리키는 원인과 그것이 나타나는 대기는?
- Q5. Elapsed·Gets 모두 9vmszgd1ft9x0이 1위. 진단 신뢰도가 높은 이유는?
- Q6. 유휴 구간에서 SQL*Net msg from client가 1위. 병목으로 지목하면 안 되는 이유와 부하 판정법은?

요약 (Summary)

- 판독은 위(요약)에서 아래(세부)로. 첫 나눗셈은 DB Time/Elapsed=AAS(5.7)
- Load Profile 3대: DB CPU≈DB Time / 논리 118만:물리 48.8 / 하드파싱 73%
- Top Events는 % DB time으로. DB CPU 95.7%가 병목, log file sync 3.7%는 아님
- Instance Efficiency는 낮은 값이 단서. Buffer Hit 100%는 함정
- SQL: 9vmszgd1ft9x0이 DB Time 33%·Gets 32%를 CPU로 소비 → 주범
- Advisory는 방향이지 명령 아님. 부하 없는 구간(AAS≪1)은 판정 대상 아님
- 🔒 Diagnostics Pack 대상, Statspack·V$ 대안. AWR·ADDM·ASH는 같은 부하의 세 렌즈

실습 개요 (Practice Overview)

non-CDB orcl 19.3 · 프롬프트 SYS@orcl · 🔒 Diagnostics Pack

- 실습 01 — awrrpt.sql로 텍스트 AWR 생성과 섹션 구조 확인
- 실습 02 — Load Profile 판독(초당 논리읽기·파싱·트랜잭션)
- 실습 03 — Top 10 Foreground Events 판독(% DB time)
- 실습 04 — SQL ordered by Elapsed/Gets/Reads 대상 선별
- 실습 05 — Instance Activity Stats 이상 지표 찾기
- 실습 06 — Advisory(버퍼·공유풀·PGA) 판독과 권고 크기
- 실습 07 — 부하 없는 구간 '문제 없음'을 근거와 함께 판정
- ※ 각 실습: 조치 전 → 조회·조치 → 해석 + 이상→원인→조치→재확인, Statspack·V$ 대안 병기

Oracle Database 19c 성능 튜닝 실무 과정

기간 비교 — 어제와 오늘은 무엇이 달라졌는가

Comparing Database Performance Over Time — AWR Compare Periods

Compare Periods 보고서 · Load Profile 대조 · SQL 회귀 탐지 · 베이스라인 · 부하 증가 vs 성능 저하

ch23

학습 목표 (Objectives)

이 장을 마치면 할 수 있는 것

- AWR 보고서 한 장과 Compare Periods 보고서(두 스냅샷 대 네 스냅샷)의 차이를 설명한다.
- awrddrpt.sql·awrddrpi.sql·AWR_DIFF_REPORT_TEXT로 두 구간 차이 보고서를 생성한다.
- Load Profile의 초당값과 트랜잭션당값을 함께 읽어 워크로드 변화를 해석한다.
- DB Time 증가가 부하 증가인지 성능 저하인지 판정하는 절차를 세운다.
- 실행계획 변경으로 느려진 SQL(회귀)을 실행당 지표·plan_hash_value로 식별한다.
- 베이스라인으로 평시를 보존하고, 고정·이동 윈도우 베이스라인을 구분한다.
- 🔒 Diagnostics Pack이 필요함을 알고, Statspack·V$ 대안 경로를 안다.

이 장의 구성 (Agenda)

23.2 ~ 23.9

- 23.2 기간 비교란 무엇인가 — 두 스냅샷이 아니라 두 구간
- 23.3 AWR Compare Periods 보고서 생성
- 23.4 보고서 구조 읽기 — 어디를 먼저 보는가
- 23.5 Load Profile 두 구간 대조
- 23.6 핵심 판정 — 부하 증가인가 성능 저하인가
- 23.7 SQL 단위 회귀 탐지 — 실행계획이 바뀌어 느려진 SQL
- 23.8 베이스라인 대비 비교 — 평시를 저장해 둔다
- 23.9 라이선스와 대안 — Statspack 두 구간 수동 비교

23.2

기간 비교란 무엇인가

- 성능 저하는 서서히 온다 — 정상이던 과거의 기록이 필요
- AWR 보고서(한 구간) vs Compare Periods(두 구간의 차이)
- 스냅샷 네 개 = 보고서 두 장
- 구간 길이가 달라도 되는 이유 — 정규화

성능은 관계적이다

같은 숫자도 비교 대상에 따라 정상·악화·개선

- 초당 논리읽기 100만 = 그 자체로는 좋지도 나쁘지도 않다
- 어제도 100만이면 정상, 어제 10만이면 열 배 악화, 어제 1000만이면 개선
- 진단 성숙도는 '얼마나 좋은 비교 기준을 가졌는가'로 결정된다
- 서서히 오는 저하는 '언제부터 왜'가 기억나지 않는다 → 과거 기록이 답

AWR 보고서 vs Compare Periods

두 스냅샷인가, 네 스냅샷인가

- 일반 AWR 보고서: 두 스냅샷 사이의 '한 구간'을 보여 준다
- Compare Periods 보고서: '두 구간의 차이'를 보여 준다(스냅샷 4개)
- 결과는 %Diff로 정량화된 차이 — 가장 크게 달라진 항목부터 정렬
- 두 구간의 길이가 달라도 됨: 초당·트랜잭션당으로 정규화하기 때문

구간 선정 — 비교 가능성

정규화도 상쇄 못 하는 것: 워크로드 성격

- 정상 구간 = 문제 전, 같은 업무가 같은 시간대에 돌던 때
- 요일·시간대·업무가 다른 두 구간 비교 = '원래 다른 것'을 봄
- 예: 평일 낮 OLTP vs 주말 밤 배치 비교는 무의미
- 정규화는 '길이' 차이를 상쇄하지만 '성격' 차이는 상쇄 못 함
- 비교 가능성(comparability)은 사람이 보장해야 한다

23.3

Compare Periods 보고서 생성

- awrddrpt.sql(로컬) / awrddrpi.sql(특정 DB) / RAC 전역판
- html vs text 형식
- AWR_DIFF_REPORT_TEXT로 SQL에서 직접
- 🔒 Diagnostics Pack 필요

어느 스크립트를 쓰는가

목적에 따라 나뉜다

- awrddrpt.sql — 로컬 인스턴스 두 구간 비교(dbid 안 물음)
- awrddrpi.sql — 특정 dbid·인스턴스 지정 비교
- awrgdrpt.sql / awrgdrpi.sql — RAC 전역(여러 인스턴스)
- 실습(non-CDB 단일 인스턴스)은 awrddrpt.sql이 기준
- 형식: html(브라우저 탐색) / text(자동 처리·트랜스크립트)

awrddrpt.sql 실행

구간A(303→304) vs 구간B(305→306), 실측 스냅

```
SYS@orcl> @?/rdbms/admin/awrddrpt.sql
Enter value for report_type: text
Enter value for num_days: 1
Enter value for begin_snap: 303
Enter value for end_snap: 304
Enter value for num_days2: 1
Enter value for begin_snap2: 305
Enter value for end_snap2: 306
Enter value for report_name: awrdiff_303_304_vs_305_306.txt

Report written to awrdiff_303_304_vs_305_306.txt
```

SQL에서 바로 뽑기

DBMS_WORKLOAD_REPOSITORY.AWR_DIFF_REPORT_TEXT

```
SYS@orcl> SELECT output
  2       FROM TABLE(
  3         DBMS_WORKLOAD_REPOSITORY.AWR_DIFF_REPORT_TEXT(
  4            &dbid, 1, 303, 304,    -- 구간A: dbid,inst,시작,끝
  5            &dbid, 1, 305, 306));  -- 구간B: dbid,inst,시작,끝

... Load Profile / Top Timed Events / SQL Statistics 텍스트 ...
```

23.4

보고서 구조 읽기

- 요약(Summary) → 상세(Details) → 보충(Supplemental)
- 모든 표는 '차이가 큰 순서'로 정렬
- 요약: Load Profile·Top 5 Timed Events부터
- 구성 비교(Host·System)로 환경 변화 먼저 검증

요약(Summary) 읽는 순서

환경 검증 → 부하 → 시간 소비

- Snapshot Sets — 같은 인스턴스인가, 재기동 없었나, 각 구간 DB Time
- Host / System Config Comparison — 하드웨어·파라미터가 그대로인가
- 구성이 바뀌었으면 성능 차이는 워크로드 아닌 '환경'을 먼저 의심
- Load Profile — 두 워크로드를 초당·트랜잭션당으로 대조
- Top 5 Timed Events — 각 구간 DB Time을 가장 많이 쓴 5개

상세(Details)와 보충

차이의 뿌리를 판다

- Time Model — DB Time이 어떤 작업으로 이동했나(파싱? 실행?)
- Wait Events — 대기 클래스·이벤트를 DB Time 차이 순으로
- SQL Statistics — 상위 SQL을 6기준(시간·CPU·읽기·실행·파싱)으로 대조
- I/O·Segment — 파일·세그먼트별 급증 지점
- 보충: Complete List of SQL Text — SQL_ID의 실제 문장 확인

23.5

Load Profile 두 구간 대조

- 초당값과 트랜잭션당값을 함께 본다
- 실측: 초당 논리읽기 +136.8% vs 트랜잭션당 -99.7%
- 원인은 맨 아래 Transactions 행
- Top Timed Events로 교차 검증

Load Profile 실측 발췌

구간A/초 · 구간B/초 · %Diff · A/txn · B/txn · %Diff

	A/초	B/초	%Diff	A/txn	B/txn	%Diff
Logical read (blocks):	577,591.8	1,367,654.2	136.8	1,916,243	4,949	-99.7
Block changes:	183.1	1,249.2	582.4	607.4	4.5	-99.3
User calls:	712.7	1,391.5	95.2	2,364.5	5.0	-99.8
Parses (SQL):	183.5	526.3	186.8	608.9	1.9	-99.7
Hard parses (SQL):	146.4	399.6	173.0	485.6	1.4	-99.7
Executes (SQL):	343.1	760.3	121.6	1,138.3	2.8	-99.8
Transactions:	0.3	276.3	92,000			

두 열이 반대로 움직이는 이유

분자는 시간, 분모는 작업 — 무엇으로 나눴나

- 초당 +136.8% = 구간B가 초당 논리읽기를 2.37배 했다
- 트랜잭션당 -99.7% = 트랜잭션 하나의 논리읽기가 300분의 1로 줄었다
- 모순 아님: 분자(총 읽기) 2.37배, 분모(트랜잭션) 920배 → 나눈 값 급감
- 파싱도 동일 — 초당 하드파싱 +173%지만 트랜잭션당은 -99.7%
- 트랜잭션당이 늘었다면 그때가 진짜 파싱 문제(ch32)

Top Timed Events도 함께 본다

Load Profile='얼마나' / Timed Events='어디로'

- Load Profile은 '얼마나 많은 일을 했나', Timed Events는 '시간이 어디로 갔나'
- 실측 두 구간 모두 DB CPU가 최상단 — CPU 바운드
- 대기 이벤트 순위·비중이 크게 흔들리지 않음 = 같은 성격의 워크로드
- 어느 대기가 갑자기 상위로 치고 오르면 → 성능 저하의 실마리

23.6

핵심 판정 — 부하 증가인가 성능 저하인가

- DB Time 81.8초 → 302.0초 (3.7배)
- 느려진 것 vs 일이 늘어난 것
- 단위 작업당 DB Time으로 나눠 판정
- 흔한 실수: 부하 증가를 성능 저하로 보고

판정의 원리

DB Time을 '한 일'로 나눈다

- 총 DB Time 증가율(3.7배)과 '일의 양' 증가율(트랜잭션 920배)을 비교
- 단위 작업당 DB Time = 총 DB Time ÷ 처리한 일 → 두 구간 비교
- 줄었으면 부하 증가, 늘었으면 성능 저하
- '3.7배'만으로는 좋은지 나쁜지 모른다 — 분모를 알아야 의미가 생김
- 분모는 하나로 고정(OLTP=트랜잭션, 배치=실행) — 두 구간에 똑같이

판정 표 — 실측 종합

총량만 보면 '저하', 분모를 보면 '부하 증가'

- DB Time: 81.8초 → 302.0초 (3.7배) — 이것만 보면 '느려졌다'
- DB CPU: 81.5초 → 293.9초 — 두 구간 모두 CPU 바운드
- Transactions/초: 0.3 → 276.3 — 처리한 일 약 920배
- 논리읽기/txn: 1,916,243 → 4,949 — 단위 작업당은 -99.7%, 효율 개선
- 결론: 성능 저하 아님 → 부하(일) 증가. 실행계획 손대면 과잉 조치

흔한 실수와 교차 검증

부하 증가를 성능 저하로 보고

- 오판 경로: 초당·절대 지표 급증 → '나빠졌다' → SQL 계획 뜯기 → 사이드이펙트
- 근본 원인: 첫 단계에서 분모(처리한 일)를 함께 보지 않음
- 습관: 총량이 늘었다는 보고엔 반드시 트랜잭션·실행을 함께 나눠 본다
- 교차 검증: 대기 구조 유지=부하 증가 / 특정 대기 급증=성능 저하
- CPU 바운드면 대기 사냥은 무의미 — CPU 줄이는 SQL·자원을 본다

실무 시나리오 · '초당 논리읽기 2.4배' 경보

부하 증가를 성능 저하로 오판할 뻔

- 상황: 대시보드가 '초당 논리읽기 어제 대비 2.4배' 경보 → '캐시 폭식 SQL 잡아라'
- 먼저 할 일: 같은 구간의 트랜잭션 수를 본다(920배 폭증)
- 판정: 트랜잭션당 논리읽기 -99.7% → 개별 작업은 더 효율적
- 늘어난 건 효율 나쁜 SQL이 아니라 '들어온 일의 양'
- 교훈: 초당값 하나만 보면 오판 — 반드시 단위 작업당과 함께 본다

23.7

SQL 단위 회귀 탐지

- 집계가 '부하 증가'여도 개별 SQL은 회귀할 수 있다
- 회귀 = 실행 횟수 비슷 + 실행당 시간 급증
- 원인: 실행계획 변경(plan_hash_value)
- DBA_HIST_SQLSTAT으로 확정

SQL Statistics로 후보 좁히기

6기준 표에서 무엇을 보나

- 핵심 신호: 한 구간만 DB Time을 크게 쓴 SQL
- 실행시간·CPU 급등 → 계산량 늘어난 계획(스캔·해시)
- 버퍼읽기 급등 → 인덱스에서 전체 스캔으로 접근 경로 변경
- 물리읽기 급등 → 캐시 미스 대량 / 파싱 급등 → 커서·바인드 문제
- 실행횟수 급등은 '느려진' 게 아니라 '더 불린' 것(부하 증가)

DBA_HIST_SQLSTAT으로 확정

실행당 지표 + plan_hash_value 두 구간 비교

```
SQLT@orcl> SELECT snap_id, plan_hash_value,
  2    executions_delta execs,
  3    ROUND(elapsed_time_delta/NULLIF(executions_delta,0)/1000,2) ms_exec,
  4    ROUND(buffer_gets_delta/NULLIF(executions_delta,0)) gets_exec
  5  FROM dba_hist_sqlstat
  6  WHERE sql_id='&sql_id' AND snap_id IN (304,306) ORDER BY snap_id;

 SNAP_ID   PLAN_HASH  EXECS  MS_EXEC  GETS_EXEC
 -------  ----------  -----  -------  ---------
     304  1388734953    500    12.40       1850   -- 인덱스 계획
     306   983734112    520   112.31      42800   -- 전체스캔 계획(회귀)
```

왜 실행계획이 바뀌는가

원인을 알아야 조치가 정해진다

- 통계 변화 — 카디널리티 추정이 달라져 다른 계획 선택
- 바인드 값 편향(bind peeking) — 첫 파싱 값이 대표적이지 않음
- 적응형 계획·SQL Plan Directives의 개입
- 인덱스·파티션 등 스키마 변경
- 급한 불: 좋았던 계획을 SPM 베이스라인으로 고정(임시 조치임을 기록)

23.8

베이스라인 대비 비교

- 평시 구간을 이름 붙여 보존 — 보존 기간이 지나도 안 지워짐
- CREATE_BASELINE으로 고정 베이스라인
- 고정 vs 이동 윈도우(SYSTEM_MOVING_WINDOW)
- 이동 윈도우 → 적응형 임계값(ch28)

고정 베이스라인 만들기

DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE

```
SYS@orcl> BEGIN
  2    DBMS_WORKLOAD_REPOSITORY.CREATE_BASELINE(
  3       start_snap_id => 303,
  4       end_snap_id   => 304,
  5       baseline_name => 'NORMAL_MORNING_PEAK');
  6  END;
  7  /
PL/SQL procedure successfully completed.

SYS@orcl> SELECT baseline_name, start_snap_id, end_snap_id, baseline_type
  2       FROM dba_hist_baseline;
```

고정 vs 이동 윈도우

사진인가, 따라오는 분포인가

- 고정 베이스라인 = 내가 고른 한 순간의 사진(배포·패치 전후 비교)
- 이동 윈도우 = 최근을 계속 따라오는 평시의 통계 분포
- SYSTEM_MOVING_WINDOW 기본 존재, 크기는 보존 기간과 같음
- 적응형 임계값의 근거 → '평시 대비 비정상' 자동 판정(ch28)
- 윈도우는 업무 주기(주간·월간)를 덮도록 크기를 잡는다

23.9

라이선스와 대안

- 이 장의 모든 도구는 Diagnostics Pack 🔒
- 없으면 Statspack 두 구간 수동 대조
- 또는 V$ 두 시점 캡처 후 델타 비교
- 판정 원리는 도구와 무관하게 같다

라이선스 경계와 대안 경로

🔒 Diagnostics Pack / 무상 대안

- 🔒 Compare Periods·awrddrpt·AWR_DIFF·DBA_HIST_*·베이스라인
- 무상: V$ 직접 조회 / Statspack(spcreate·SNAP·spreport)
- 두 구간 차이 보고서 → Statspack 두 보고서 수동 대조
- 시간 모델 델타 → V$SYS_TIME_MODEL 두 시점 캡처
- '조회만 했다'도 사용 감지됨(DBA_FEATURE_USAGE_STATISTICS)

Statspack 두 구간 대조

무상 대안 — Per Second/Per Txn 대조는 동일

```
-- 설치(최초 1회, PERFSTAT 계정)
SYS@orcl> @?/rdbms/admin/spcreate.sql

PERFSTAT@orcl> EXEC STATSPACK.SNAP;   -- 평시 시작
    ... (평시 워크로드) ...
PERFSTAT@orcl> EXEC STATSPACK.SNAP;   -- 평시 끝
PERFSTAT@orcl> EXEC STATSPACK.SNAP;   -- 문제 시작
    ... (문제 워크로드) ...
PERFSTAT@orcl> EXEC STATSPACK.SNAP;   -- 문제 끝

PERFSTAT@orcl> @?/rdbms/admin/spreport.sql   -- 평시 범위
PERFSTAT@orcl> @?/rdbms/admin/spreport.sql   -- 문제 범위
```

퀴즈 (Quiz)

정답은 실습문제 정답지 참고

- Q1. 일반 AWR 보고서와 Compare Periods의 차이를 스냅샷 개수로 설명하라.
- Q2. 초당 논리읽기 +136.8%인데 트랜잭션당 -99.7%인 상황을 해석하라.
- Q3. DB Time 81.8→302.0초(3.7배)가 부하 증가인지 성능 저하인지 판정 절차는?
- Q4. 특정 SQL 회귀를 확정하려면 어느 뷰에서 어떤 값을 두 구간 비교하나?
- Q5. 고정 베이스라인과 이동 윈도우 베이스라인의 용도 차이는?
- Q6. Diagnostics Pack이 없을 때 대안 두 경로와 공통 판정 원리는?

요약 (Summary)

이 장의 핵심

- Compare Periods는 두 구간의 차이(스냅샷 4개)를 정규화해 보여 준다
- Load Profile은 초당값과 트랜잭션당값을 함께 봐야 한다
- 정수: DB Time 3.7배는 성능 저하 아닌 부하 증가 — 단위 작업당으로 나눠 판정
- 집계가 부하 증가여도 개별 SQL은 회귀 가능(실행당 지표·plan_hash_value)
- 베이스라인으로 평시 보존 / 🔒 없으면 Statspack·V$ 대안

실습 개요 (Practice Overview)

실습 5개 · non-CDB orcl 19.3 · WL_06_mixed

- 실습 01 — Compare Periods 보고서 생성(awrddrpt.sql / AWR_DIFF_REPORT_TEXT)
- 실습 02 — 두 기간 Load Profile 차이 판독(초당 vs 트랜잭션당)
- 실습 03 — SQL 단위 회귀 탐지(DBA_HIST_SQLSTAT·plan_hash_value)
- 실습 04 — 베이스라인 대비 비교(CREATE_BASELINE·고정/이동)
- 실습 05 — 부하 증가 vs 성능 저하 판정(81.8초 vs 302.0초 오판 교정)

Oracle Database 19c 성능 튜닝 실무 과정

ADDM — 자동 진단 결과 해석

Automatic Database Diagnostic Monitor — 권고는 가설이지 결론이 아니다

Finding·Recommendation·Rationale 구조 · 영향도 해석 · ADDM 뷰 직접 조회 · 권고 채택 기준

ch24

학습 목표 (Objectives)

- ADDM이 AWR 위에서 top-down으로 DB Time을 분해하는 자동 진단자임을 설명한다.
- Finding·Recommendation·Rationale 세 층 구조와 Finding의 네 유형을 구분한다.
- 영향도(Impact)와 이득(Benefit)이 각각 무엇을 재는지 구분한다.
- addmrpt.sql / DBMS_ADDM.ANALYZE_DB로 태스크를 실행하고 보고서를 생성한다.
- DBA_ADVISOR_*·DBA_ADDM_* 뷰로 Finding을 영향도 순으로 조회한다.
- 실측 상위 Finding을 ch22 AWR·ch25 ASH의 같은 구간과 연결해 해석한다.
- 권고 채택 전 영향도·근거·부작용을 검토해 "권고 맹신"을 피한다.

이 장의 구성 (Agenda)

- 24.2 ADDM이란 무엇인가 — AWR 위에서 도는 자동 진단자
- 24.3 Finding·Recommendation·Rationale 구조
- 24.4 ADDM 태스크 실행과 보고서 생성
- 24.5 실측 ADDM 보고서 판독 — 스냅샷 295~296
- 24.6 ADDM 뷰 직접 조회와 라이선스 없는 대안
- 24.7 권고를 그대로 따르면 안 되는 이유
- ※ ADMIN 31장의 ADDM 개념 소개를 실측 판독·권고 판단으로 확장

24.2

ADDM이란 무엇인가

- AWR 스냅샷 위에서 도는 자동 진단자
- top-down DB Time 분해
- 자동 실행과 켜는 조건

ADDM의 위치 — V$·AWR·ASH와의 관계

경쟁이 아니라 층위가 다른 협력

- V$ = 지금 이 순간 원천 / AWR = 스냅샷 요약(SYSAUX 영구)
- ADDM = AWR 위 한 겹 — 두 스냅샷 사이 DB Time을 자동 해석
- ASH = 활성 세션 1초 샘플링, 짧은 순간 포착
- ADDM 출력은 진단의 출발점 → 더 좁힐 때 ASH·V$로

왜 사람이 AWR을 읽는데 ADDM이 필요한가

자동화의 세 가지 값

- 속도·일관성 — 수천 줄 보고서를 늘 같은 기준(DB Time)으로 몇 초 만에
- top-down 인과 — "무엇이 높다"가 아니라 증상→근본 원인
- 비문제 영역까지 알려 줌 — 엉뚱한 곳 파는 것 방지
- 한계 — AWR 기반이라 짧은 스파이크는 못 봄(→ ch25 ASH)

자동 실행과 분석 구간

스냅샷마다, 직전 두 개

- AWR 스냅샷 생성 시마다 자동으로 한 번씩 분석·저장
- 분석 구간 = 직전 두 스냅샷 (기본 60분이면 "지난 한 시간")
- 진단 첫 단계는 새 태스크가 아니라 자동 결과 확인
- top-down — 증상 식별 → 근본 원인으로 좁힘 → 영향도 순 정렬

ADDM을 켜는 두 파라미터

기본으로 켜져 있음 — 값부터 확인

```
SYS@orcl> SELECT name, value FROM v$parameter
  2       WHERE name IN ('control_management_pack_access',
  3                      'statistics_level');

NAME                           VALUE
------------------------------ --------------------
control_management_pack_access   DIAGNOSTIC+TUNING
statistics_level                 TYPICAL
```

24.3

Finding·Recommendation·Rationale

- 무엇이 문제 / 어떻게 / 왜
- Finding 4유형
- 영향도와 이득

Finding의 네 가지 유형

조치는 Problem에, Symptom은 단서로

- Problem(문제) — 근본 원인. 영향도가 붙고 권고가 달린다 → 조치 대상
- Symptom(증상) — Problem으로 이어지는 현상. 조치 대상 아님
- Information(정보) — 문제 아닌 영역(예: normal, successful completion)
- Warning(경고) — 분석의 완전성 문제(예: AWR 데이터 누락)

영향도(Impact) vs 이득(Benefit)

무엇을 재는가 — 반드시 구분

- 영향도 = 이 Finding이 유발한 DB Time 비중 (Top SQL 60.78%)
- 이득 = 이 권고를 구현하면 절약될 DB Time 비중
- 이득은 "Finding 영향도"가 아니라 "전체 DB Time"에 대한 %
- 이유 — 서로 다른 Finding의 권고를 같은 잣대로 비교하려고

Recommendation = Action + Rationale

권고 유형 다섯 가지

- 추정 이득을 얻으려면 한 권고의 Action을 모두 적용
- Rationale — 왜 그 조치를 권고했는지, 구현 정보
- 유형: 하드웨어 / DB 구성 / 스키마 / 애플리케이션 / 다른 어드바이저
- 실측: "SQL_ID 9vmszgd1ft9x0에 SQL Tuning Advisor 실행"

영향도 합이 100%를 넘을 수 있다

겹침 — 계산 오류가 아니다

- 한 문제에 여러 원인 → 여러 Problem·Symptom Finding
- Finding들이 같은 DB Time 조각을 중복해 포함할 수 있음
- 예: I/O로 50% SQL + undersized buffer cache 75% = 같은 시간
- 권고 여러 개 = 대안일 수 있음, 전부 적용할 필요 없음

24.4

ADDM 태스크 실행과 보고서

- 자동 결과부터 확인
- DBMS_ADDM / addmrpt.sql
- 분석 모드·Real-Time

실행 경로 — 자동 결과부터

어느 경로든 저장 위치·읽는 법 동일

- 먼저: 스냅샷마다 자동 생성된 ADDM 결과 확인
- 두 스냅샷이 AWR에 남아 있어야(purge 전) 분석 가능
- 대상 인스턴스는 시작 전 기동~종료까지 계속 떠 있어야
- 다른 구간·설정이 필요할 때만 수동 실행/addmrpt.sql

수동 실행 — DBMS_ADDM.ANALYZE_DB

실측 태스크 TUNING_CH9_ADDM_296

```
SYS@orcl> VAR tname VARCHAR2(40);
SYS@orcl> BEGIN
  2         :tname := 'TUNING_CH9_ADDM_296';
  3         DBMS_ADDM.ANALYZE_DB(:tname, 295, 296);
  4       END;
  5       /

SYS@orcl> SET LONG 1000000 PAGESIZE 0 LINESIZE 80
SYS@orcl> SELECT DBMS_ADDM.GET_REPORT(:tname) FROM dual;
```

addmrpt.sql과 분석 모드

대화식 생성 / RAC의 세 모드

- addmrpt.sql — 스냅샷 목록 보여 주고 구간 입력받아 생성(awrrpt.sql의 짝)
- 내부적으로 ANALYZE_DB+GET_REPORT 래퍼 → 결과 동일
- RAC: Database(전체)·Instance(특정)·Partial(부분집합)
- 단일 인스턴스 non-CDB — ANALYZE_DB도 그 한 인스턴스 분석

Real-Time ADDM · DBIO_EXPECTED

스냅샷을 못 기다릴 때 / I/O 기준값

- Real-Time ADDM — MMON이 3초마다 락 없이 통계 수집
- 트리거: High load, CPU bound, 데드락 등 → DBA_HIST_REPORTS
- DBIO_EXPECTED — 단일 블록 읽기 기대 시간(기본 10ms)
- 잘못 잡으면 I/O 진단 왜곡 — 하드웨어에 맞춰 한 번 고정

24.5

실측 ADDM 보고서 판독

- 스냅샷 295~296, DB Time 574초
- 상위 3개 Finding
- ch22·ch24·ch25 같은 부하

Summary of Findings — 실측

영향도 순 정렬이 곧 우선순위

Total database time was 574 seconds.
The average number of active sessions was 5.52.

Summary of Findings

	Description	Active Sessions Percent of Activity		Recomm.
1	Top SQL Statements	3.35	60.78	2
2	Hard Parse Due to Literal Usage	.32	5.82	1
3	Commits and Rollbacks	.2	3.69	2
4	Soft Parse	.13	2.38	2

Finding 1 — Top SQL Statements

영향도 60.78% — 압도적 1위

- SQL_ID 9vmszgd1ft9x0 = ch22 AWR Elapsed 1위(190.6초·33.2%)
- CLAIM_DETAILS 대량 스캔(WL_02) count/sum 집계
- 권고: 이 SQL에 SQL Tuning Advisor 실행 (이득 31.37%)
- Rationale: 5,912회 실행, 평균 0.032초, DB Time 100% CPU·I/O

Finding 1 상세 — 권고와 근거

Action·Rationale의 실제 모습

```
Finding 1: Top SQL Statements
Impact is 3.35 active sessions, 60.78% of total activity.

   Recommendation 1: SQL Tuning
   Estimated benefit is 1.73 active sessions, 31.37%.
   Action
      Run SQL Tuning Advisor on the SELECT statement with
      SQL_ID "9vmszgd1ft9x0".
   Rationale
      Executed 5912 times, avg elapsed time 0.032 seconds.
```

Finding 2 — Hard Parse Due to Literal Usage

WL_01 리터럴이 자동 진단에 잡힌 자리

- ch22 AWR의 숫자(하드 파싱 초당 319, Library Hit 59%)에 이름을 붙임
- 리터럴 → 커서 미공유 → 추가 하드 파싱 (영향도 5.82%)
- 권고: 바인드 변수 사용, 어렵다면 CURSOR_SHARING=FORCE
- DBA_ADVISOR_FINDINGS: Hard Parse=SYMPTOM, Literal Usage=PROBLEM

Finding 3 — Commits and Rollbacks

WL_05 잦은 커밋 (영향도 3.69%)

- 커밋·롤백이 잦아 log file sync가 DB Time 소비(21.15초)
- ch22 AWR Top Events의 log file sync 21.1초·Commit 3.7%와 동일
- 권고: 커밋 빈도 축소(배치 커밋) / 로그 I/O 개선
- 영향도 낮음 → Top SQL 먼저, 여기 매달리지 않는다

세 도구가 같은 사건을 가리킨다

ch22·ch24·ch25의 일관성

- AWR(ch22): DB CPU 95.7% · log file sync · 하드 파싱 경합
- ADDM(ch24): Top SQL 60.78% · Literal 5.82% · Commits 3.69%
- ASH(ch25): ON CPU 90.7% · 상위 SQL 9vmszgd1ft9x0
- 셋은 모순 없음 — ADDM이 관점을 자동 통합해 "어디부터"를 정함

24.6

ADDM 뷰 직접 조회

- 텍스트 대신 뷰로 정렬·자동화
- DBA_ADVISOR_* / DBA_ADDM_*
- 라이선스 없는 대안

핵심 ADDM 뷰

공통 어드바이저 프레임워크 스키마

- DBA_ADVISOR_FINDINGS — 모든 Finding(id·name·type·impact)
- DBA_ADDM_FINDINGS — 그 부분집합, ADDM Finding만
- DBA_ADVISOR_RECOMMENDATIONS — RANK 순 검토, BENEFIT 확인
- DBA_ADVISOR_ACTIONS / _RATIONALE / _TASKS

Finding을 영향도 순으로 조회

실측 — 유형(type)과 영향도가 그대로

```
SYS@orcl> SELECT f.finding_name, f.type,
  2         ROUND(f.impact/1000000,2) AS impact_sec
  3       FROM dba_advisor_findings f
  4       JOIN dba_advisor_tasks t USING (task_id)
  5       WHERE t.task_name = 'TUNING_CH9_ADDM_296'
  6       ORDER BY f.impact DESC;

FINDING_NAME                    TYPE        IMPACT_SEC
Top SQL Statements              PROBLEM         348.72
Hard Parse                      SYMPTOM          35.28
Hard Parse Due to Literal Usage PROBLEM          33.38
Commits and Rollbacks           PROBLEM          21.15
```

라이선스 없이 같은 병목 보기

🔒 대안 경로 — V$ 델타·Statspack

- ADDM·DBA_ADVISOR_*·DBA_ADDM_* = Diagnostics Pack 필요
- Top SQL → V$SQL을 elapsed/gets/executions로 정렬(무료)
- 하드 파싱 → V$SYSSTAT parse count(hard) 델타, V$LIBRARYCACHE
- 커밋 → V$SYSSTAT user commits 델타, V$SYSTEM_EVENT log file sync

24.7

권고를 그대로 따르면 안 되는 이유

- 권고는 가설이지 결론이 아니다
- 영향도·근거·부작용 검토
- 권고 맹신 경계

영향도 낮은 권고에 시간 쓰지 않기

"쉬운 것"이 아니라 "영향 큰 것"부터

- Soft Parse(2.38%)·Commits(3.69%)는 손대기 쉬워 보임
- Top SQL 60.78%인데 3%짜리 없애도 최대 개선 3% (Amdahl)
- 60%짜리를 절반 줄이면 30% 개선 — 어디 쓸지 명백
- ADDM이 영향도 순 정렬 = 이 순서를 지키라는 신호

근거(Rationale) 없이 채택하지 않기

Action만 읽고 Rationale 건너뛰지 말 것

- Rationale = "이 권고가 내 상황에 맞는가"의 판단 재료
- 실측: "5,912회 실행" 근거 → 튜닝 방향(횟수 vs 단가) 결정
- "인덱스 추가" 권고 → 그 SQL이 중요 업무인가, DML 느려지나
- 근거 없는 채택 = "근거 없는 조치" 함정과 만남

권고의 부작용 — CURSOR_SHARING=FORCE

"쉬운 권고"일수록 부작용 검토

- 바인드 변수가 최선이나 앱 수정이 어려움
- CURSOR_SHARING=FORCE는 구현 쉽고 상당한 개선
- 그러나 히스토그램 기반 카디널리티 추정 흐트러짐
- 일부 SQL 실행계획 악화 가능 — 트레이드오프는 사람이 판단

채택 판단 체크리스트

권고 맹신을 절차로 막는다

- ① 상위 영향도 Finding인가? 아니면 상위부터
- ② Rationale을 읽었는가? 내 맥락에 맞는가?
- ③ 부작용(다른 SQL·DML·자원)은 무엇인가?
- ④ 조치 전 캡처했는가? ⑤ 적용 후 재분석으로 영향도 감소 확인?

퀴즈 (Quiz)

- Q1. ADDM Finding의 네 유형을 쓰고, 조치를 어느 유형에 하는지 설명하라.
- Q2. 영향도와 이득은 각각 무엇을 재는가? 이득이 전체 DB Time 기준인 이유는?
- Q3. 여러 Finding의 영향도 합이 100%를 넘을 수 있는 이유를 예로 설명하라.
- Q4. 실측 보고서에서 시간을 먼저 써야 할 Finding과 그 근거는?
- Q5. Hard Parse(SYMPTOM)와 Literal Usage(PROBLEM) 중 조치 대상은?
- Q6. CURSOR_SHARING=FORCE를 적용하기 전 검토할 부작용은?

요약 (Summary)

- ADDM = AWR 두 스냅샷 위 top-down DB Time 분해 자동 진단자, 진단의 출발점
- Finding(Problem·Symptom·Information·Warning)·Recommendation·Rationale 3층
- 조치는 Problem에, 영향도 순 정렬이 곧 우선순위
- 실측: Top SQL 60.78% · Literal 5.82% · Commits 3.69% = ch22·ch25 같은 부하
- DBMS_ADDM.ANALYZE_DB / DBA_ADVISOR_* — 🔒 없으면 V$ 델타·Statspack
- 권고는 가설 — 영향도·근거·부작용 검토 없는 채택은 자동화의 함정

실습 개요 (Practice Overview)

- 실습 01 — ADDM 태스크 실행과 보고서 생성(addmrpt.sql / DBMS_ADDM.ANALYZE_DB) 🔒
- 실습 02 — Finding·Recommendation·Rationale 구조와 영향도 해석 🔒
- 실습 03 — ADDM 분석 모드 확인(Database/Instance, 단일 인스턴스 동작) 🔒
- 실습 04 — ADDM 뷰 직접 조회(DBA_ADDM_FINDINGS, DBA_ADVISOR_*)와 대안 🔒
- 실습 05 — 권고를 그대로 따르면 안 되는 사례 판별 🔒
- ※ 전 실습 Diagnostics Pack 필요 — 라이선스 표기와 V$ 대안 경로 병기

Oracle Database 19c 성능 튜닝 실무 과정

ASH — 샘플링 데이터로 특정 시점 추적

Analyzing Sampled Data — 짧은 스파이크를 잡는 법

샘플링 원리 · V$ACTIVE_SESSION_HISTORY · ashrpt.sql · 세션 재구성 · DBA_HIST · AWR·ADDM·ASH 대조

ch25

학습 목표 (Objectives)

- ASH가 활성 세션을 1초 주기로 샘플링해 순환 버퍼에 저장하는 원리를 설명한다.
- V$ACTIVE_SESSION_HISTORY 컬럼으로 상위 대기·SQL·세션을 직접 도출한다.
- 샘플 수(samples)가 실행 횟수가 아니라 소비 시간의 근사임을 이해한다.
- ashrpt.sql로 보고서를 생성하고 begin_time·duration으로 구간을 지정한다.
- 한 세션이 무엇을 기다렸는지 SAMPLE_TIME 순으로 재구성한다.
- 메모리 ASH(V$)와 디스크 ASH(DBA_HIST_)의 해상도·보존 차이를 안다.
- AWR·ADDM이 놓치는 짧은 스파이크를 ASH로 잡는 논리를 설명한다.
- Diagnostics Pack 경계와 V$SESSION 폴링 대안 경로를 안다.

이 장의 구성 (Agenda)

- 25.2 ASH란 무엇인가 — 샘플링의 원리
- 25.3 ASH가 필요한 순간 — 짧은 스파이크
- 25.4 ASH 보고서 생성 — ashrpt.sql
- 25.5 실측 — 부하 구간 10분을 ASH로 재구성
- 25.6 세션 단위 추적 — 시간순 재구성
- 25.7 DBA_HIST_ACTIVE_SESS_HISTORY — 보존 구간
- 25.8 세 도구가 같은 부하를 본다 — AWR·ADDM·ASH
- 25.9 관리 팩 라이선스와 대안 경로

25.2

ASH란 무엇인가

- 활성 세션을 1초마다 샘플링
- SGA 순환 버퍼에 저장
- V$ACTIVE_SESSION_HISTORY
- samples ≠ 실행 횟수

ASH — 샘플링의 원리

1초마다 활성 세션의 스냅사진

- 매 1초 활성 세션 상태를 SGA 순환 버퍼에 기록
- AWR·ADDM은 구간을 평균 냄 / ASH는 평균 내지 않음
- 1초 원본 샘플 보관 → 특정 초의 활동까지 복원
- 이 시간 해상도가 짧은 스파이크를 잡는 힘

활성 세션만 샘플링한다

ASH가 관리 가능한 크기를 유지하는 비결

- 접속 + Idle 대기가 아닌 세션 = 활성 세션
- CPU를 쓰는 세션(ON CPU)도 활성
- SQL*Net message from client 등 Idle은 제외
- 데이터 크기 ∝ 접속 수가 아니라 실제 작업량

V$ACTIVE_SESSION_HISTORY 구조

한 행 = 한 세션의 1초 샘플 한 장

```
SYS@orcl> SELECT sample_time, session_id, session_state,
  2         event, sql_id
  3       FROM v$active_session_history
  4       WHERE sample_time > SYSTIMESTAMP - INTERVAL '15' SECOND
  5       ORDER BY sample_time DESC FETCH FIRST 8 ROWS ONLY;

SAMPLE_TIME           SESS_ID STATE   EVENT           SQL_ID
--------------------- ------- ------- --------------- -------------
12.16.58.123               45 ON CPU                  9vmszgd1ft9x0
12.16.58.123               61 WAITING log file sync   50kad7809jyj8
```

samples는 실행 횟수가 아니다

이 장의 대표 오판 — 반드시 경계

- samples=186 ≠ 186번 실행
- = 1초 샘플에 186번 '수행 중'으로 포착됨
- 100초짜리 SQL 1회 → 약 100 샘플
- 횟수는 V$SQL.EXECUTIONS / 시간은 ASH

25.3

짧은 스파이크

- AWR·ADDM은 구간을 평균
- 2분 문제 → 평균에 희석
- ASH는 구간만 잘라 포착
- 도구는 지속 시간으로 선택

AWR·ADDM이 놓치는 이유

짧은(transient) 성능 문제

- ADDM: 문제 지속시간 / 스냅샷 간격 비중에 좌우
- 1시간 간격 · 2분 문제 → severity 희석·누락
- AWR: 구간 합계라 2분 스파이크가 미미해짐
- 'AWR에 안 보임' ≠ '문제 없음'

도구별 시간 해상도

같은 부하, 다른 렌즈

- V$ 뷰: 실시간 — 지나가면 사라짐
- AWR: 스냅샷 간격(기본 1h) — 평균에 희석
- ADDM: 스냅샷 간격 — severity 희석
- ASH: 1초 샘플 — 구간만 잘라 스파이크 포착

25.4

ASH 보고서 생성

- ashrpt.sql / ashrpti.sql
- begin_time · duration
- 보고서 섹션 구조
- target로 차원 좁히기

ashrpt.sql 실행과 구간 지정

'10분 전부터 현재까지'를 텍스트로

```
SYS@orcl> @?/rdbms/admin/ashrpt.sql

Enter value for report_type: text      -- 텍스트
Enter value for begin_time:  -10       -- 10분 전 시작
Enter value for duration:              -- (공백) 현재까지
Enter value for report_name:           -- 기본 이름

Using the report name ashrpt_1_0825_1217.txt
```

ASH 보고서 섹션 구조

각 섹션이 무엇을 지목하나

- Top User/Background Events — 상위 대기와 %
- Top SQL — 상위 SQL, Sampled # of Executions
- Top Sessions — 상위 활성·블로킹 세션
- Activity Over Time — 10개 슬롯 skew 분석

target — 무엇을 좁혀 볼 것인가

구간은 '언제', target은 '무엇을'

```
Enter value for target_session_id:            -- 전체
Enter value for target_sql_id: 9vmszgd1ft9x0  -- 이 SQL만
Enter value for target_wait_class:            -- 전체
Enter value for target_module_name:           -- 전체

-- 세션·SQL·대기클래스·서비스·모듈·액션·클라이언트ID로
-- 관심 차원만 남겨 활동을 분리해 본다
```

25.5

실측 — 10분 재구성

- ch22·ch24과 같은 부하 구간
- ON CPU 90.7%
- 상위 두 SQL이 대부분
- AAS 4.48 · CPU per 0.56

상위 대기 이벤트 (실측)

최근 130초 · ON CPU가 EVENT NULL

```
SYS@orcl> SELECT NVL(event,'ON CPU') event, COUNT(*) samples,
  2         ROUND(100*COUNT(*)/SUM(COUNT(*)) OVER(),1) pct
  3       FROM v$active_session_history
  4       WHERE sample_time > SYSTIMESTAMP - INTERVAL '130' SECOND
  5       GROUP BY NVL(event,'ON CPU') ORDER BY samples DESC;

EVENT                       SAMPLES   PCT
-------------------------- ------- -----
ON CPU                         529  90.7
log file sync                   17   2.9
log file parallel write         14   2.4
db file sequential read         10   1.7
```

판독 — CPU가 90%면 어디를 보나

대기가 아니라 CPU 소비가 병목

- SESSION_STATE가 ON CPU인지 WAITING인지부터 가른다
- ON CPU 90.7% → 병목은 '기다림'이 아니라 'CPU 소비'
- log file sync에 먼저 손대면 헛수고
- 다음 수순: CPU를 태우는 상위 SQL_ID로 내려간다

상위 SQL_ID (실측)

누가 CPU를 태웠나

```
SYS@orcl> SELECT sql_id, COUNT(*) samples,
  2         ROUND(100*RATIO_TO_REPORT(COUNT(*)) OVER(),1) pct
  3       FROM v$active_session_history
  4       WHERE sample_time > SYSTIMESTAMP - INTERVAL '130' SECOND
  5         AND sql_id IS NOT NULL
  6       GROUP BY sql_id ORDER BY samples DESC;

SQL_ID          SAMPLES   PCT
------------- --------- -----
9vmszgd1ft9x0       186  52.1
43fd4b2pxfzvm       154  43.1
```

ASH 보고서 발췌 판독

헤더 요약이 직접 조회와 일치하나

```
ASH Report For ORCL/orcl
          Analysis Begin Time:   25-Aug-26 12:15:20
                Sample Count:         582
      Average Active Sessions:        4.48
  Avg. Active Session per CPU:        0.56

Top User Events           Event Class  % Event  AvgAct
------------------------- ----------- -------- -------
CPU + Wait for CPU        CPU            89.69    4.02
log file sync             Commit          2.92    0.13
db file sequential read   User I/O        1.72    0.08
```

Activity Over Time — 슬롯 skew

어느 1분에 부하가 몰렸나

```
SYS@orcl> SELECT TO_CHAR(sample_time,'HH24:MI') slot,
  2         COUNT(*) slot_count,
  3         SUM(CASE WHEN session_state='ON CPU' THEN 1 END) on_cpu
  4       FROM v$active_session_history
  5       WHERE sample_time BETWEEN ... AND ...
  6       GROUP BY TO_CHAR(sample_time,'HH24:MI') ORDER BY slot;

SLOT   SLOT_COUNT   ON_CPU
----- ----------- --------
12:15         198      181
12:16         241      219
12:17         143      129
```

25.6

세션 단위 추적

- 한 세션의 궤적을 시간순으로
- SAMPLE_TIME 순 재구성
- 언제 무엇 때문에 멈췄나
- BLOCKING_SESSION 역추적

한 세션의 10초 재구성

집계로는 안 보이는 사건의 흐름

```
SYS@orcl> SELECT sample_time, session_state st, event, sql_id
  2       FROM v$active_session_history
  3       WHERE session_id = 45
  4         AND sample_time BETWEEN ... AND ... ORDER BY sample_time;

SAMPLE_TIME        ST      EVENT           SQL_ID
------------------ ------- --------------- -------------
12.16.01.128       ON CPU                  9vmszgd1ft9x0
12.16.03.131       ON CPU                  9vmszgd1ft9x0
12.16.04.133       WAITING log file sync   50kad7809jyj8
12.16.05.135       ON CPU                  9vmszgd1ft9x0
```

세션 추적이 주는 것

집계는 %, 추적은 인과의 순서

- 집계: 구간에 log file sync가 몇 % 있었다
- 추적: CPU→커밋 대기→복귀의 사건 흐름
- 원인·결과의 순서가 보이면 조치 대상이 분명
- BLOCKING_SESSION이 차면 블로커를 역추적

25.7

DBA_HIST_ASH

- 메모리 ASH = 유한한 순환 버퍼
- 디스크 ASH = SYSAUX 영구 저장
- 약 1/10 flush · 10초 간격
- 최근은 V$ / 지나간 날은 DBA_HIST

메모리 ASH vs 디스크 ASH

해상도와 보존이 다르다

- 메모리: V$ · 1초 · 수십 분~ (버퍼가 찰 때까지)
- 디스크: DBA_HIST_ · 10초 · 기본 8일(AWR 보존)
- MMON이 약 1/10을 골라 SYSAUX에 저장
- 최근 정밀분석은 V$ / 지나간 날은 DBA_HIST_

DBA_HIST_ASH 보존 구간 조회

같은 구간, 샘플 수는 약 1/10

```
SYS@orcl> SELECT NVL(event,'ON CPU') event, COUNT(*) samples
  2       FROM dba_hist_active_sess_history
  3       WHERE sample_time BETWEEN TIMESTAMP '2026-08-25 12:15:00'
  4                             AND TIMESTAMP '2026-08-25 12:17:30'
  5       GROUP BY NVL(event,'ON CPU') ORDER BY samples DESC;

EVENT                       SAMPLES
--------------------------- --------
ON CPU                            53
log file sync                      2
db file sequential read            1
```

25.8

세 도구가 같은 부하를 본다

- AWR = 무엇이 있었나
- ADDM = 왜 그랬나
- ASH = 언제·누가·무엇으로
- 결론은 한 점으로 모인다

AWR·ADDM·ASH 대조

같은 부하, 세 각도, 한 결론

- AWR(ch22): 구간 누적 → DB CPU 지배
- ADDM(ch24): 자동 진단 → 리터럴 하드파싱
- ASH(ch25): 1초 샘플 → ON CPU 90.7%
- 불일치의 90%는 도구가 아니라 '구간 불일치'

25.9

라이선스와 대안

- 🔒 Diagnostics Pack 필수
- 조회만으로 사용 이력 기록
- 대안: V$SESSION 폴링
- 완전 대체 아닌 절충

관리 팩 라이선스

무심코 쓰면 위반

- V$ACTIVE_SESSION_HISTORY · DBA_HIST · ashrpt 모두 유상
- 조회 순간 DBA_FEATURE_USAGE_STATISTICS에 기록
- CONTROL_MANAGEMENT_PACK_ACCESS ≥ DIAGNOSTIC 필요
- 파라미터 값이 곧 라이선스 보유 증명은 아니다

대안 — V$SESSION 폴링

라이선스 없이 활성 세션 근사 샘플링

```
-- 라이선스 불필요: '지금 활성 세션'을 반복 조회해 적재
SYS@orcl> SELECT NVL(event,'ON CPU') event, COUNT(*) active_now
  2       FROM v$session
  3       WHERE status = 'ACTIVE'
  4         AND wait_class <> 'Idle'
  5       GROUP BY NVL(event,'ON CPU') ORDER BY active_now DESC;

EVENT                     ACTIVE_NOW
------------------------- ----------
ON CPU                             5
log file sync                      1
```

퀴즈 (Quiz)

- Q1. ASH가 '활성 세션만' 샘플링하는 이유와, 데이터 크기가 무엇에 비례하는가?
- Q2. samples=186을 '186번 실행'으로 읽으면 왜 틀리는가? 무엇에 비례하나?
- Q3. 1시간 간격에서 2분 문제가 ADDM에 안 잡히는 이유와 ASH의 역할은?
- Q4. ON CPU 90.7%에서 log file sync에 먼저 손대면 안 되는 이유와 다음 수순은?
- Q5. V$ACTIVE_SESSION_HISTORY와 DBA_HIST_ 의 간격·보존·용도 차이는?
- Q6. 라이선스 없을 때 ASH 대안 경로와 그 한계는?

요약 (Summary)

- ASH = 활성 세션을 1초로 샘플링한 SGA 순환 버퍼. 활성 = Idle 아닌 세션.
- samples는 실행 횟수가 아니라 DB Time 소비의 근사. 횟수는 V$SQL.
- AWR·ADDM은 평균에 스파이크가 희석 / ASH는 구간을 잘라 포착.
- 실측: ON CPU 90.7% · log file sync 2.9% → CPU 지배형, 상위 SQL을 찾는다.
- 메모리 ASH(1초·수십 분) vs 디스크 ASH(10초·기본 8일).
- AWR·ADDM·ASH는 같은 부하를 한 점으로 가리킨다 — 불일치는 구간 불일치.
- ASH 뷰·보고서는 Diagnostics Pack 필요 / 대안은 V$SESSION 폴링.

실습 개요 (Practice Overview)

- 실습 01 — V$ACTIVE_SESSION_HISTORY 구조와 샘플링 주기 확인 🔒
- 실습 02 — ASH 보고서 생성(ashrpt.sql)과 구간 지정 🔒
- 실습 03 — 특정 10분 구간의 상위 세션·SQL·대기 도출(ON CPU 90.7%) 🔒
- 실습 04 — 세션 단위 추적: SAMPLE_TIME 순 재구성 🔒
- 실습 05 — DBA_HIST_ACTIVE_SESS_HISTORY 보존 구간 분석 🔒
- 실습 06 — AWR로 안 보이고 ASH로만 보이는 짧은 스파이크 재현 🔒

Day 8

V$ 진단·대기 이벤트·모니터링·메모리 구조

- TUN ch26 · ch27(3.5h) · ch28(1h) · ch29(1h)
- 8h

Oracle Database 19c 성능 튜닝 실무 과정

V$ 인스턴스 튜닝 ① — 절차와 통계 해석

Instance Tuning Using Performance Views — 절차와 통계

진단 절차 · 세션→대기→SQL→세그먼트 · V$SYSSTAT 델타 · V$SQL 상위 SQL · 절대값 오판 교정

ch26

학습 목표 (Objectives)

- 인스턴스 튜닝 방법론 4단계(문제 정의→통계 검토→변경·측정→반복)를 설명한다.
- 세션→대기→SQL→세그먼트의 실시간 진단 경로를 순서대로 밟는다.
- V$SYSSTAT 값이 누적임을 알고, 델타를 초·트랜잭션당으로 정규화한다.
- session logical reads·parse count·execute count 등 부하 통계를 읽는다.
- V$SQL을 실행시간·논리읽기·실행횟수·실행당으로 정렬해 상위 SQL을 선별한다.
- '절대값 기준 상위 SQL 선별'의 오판을 델타·비율·시그니처로 교정한다.
- V$ 실시간 조회는 라이선스가 필요 없고 DBA_HIST는 Diagnostics Pack이 필요함을 구분한다.

이 장의 구성 (Agenda)

- 26.2 인스턴스 튜닝의 절차 — 무엇을 순서로 하는가
- 26.3 실시간 진단 경로 — 세션 → 대기 → SQL → 세그먼트
- 26.4 V$SYSSTAT — 시스템 통계 읽기(누적값·델타·정규화)
- 26.5 V$SQL — 상위 부하 SQL 선별
- 26.6 절대값을 델타·비율로 교정한다 — 이 장의 축
- 26.7 AWR 없이도 진단은 된다 — V$의 위치
- ※ ADMIN 31장의 성능 뷰 소개를 실시간 진단 절차로 확장한다(non-CDB orcl 19.3).

26.2

인스턴스 튜닝의 절차

- 문제 정의 → 호스트·인스턴스 통계 → 변경·측정 → 반복
- 기준선 없이는 성공을 증명할 수 없다
- 증상과 원인을 혼동하지 않는다

성능 튜닝 방법론 4단계

오라클이 권하는 반복 과정

- ① 문제 정의: 목표·범위·시간대·정량화·변경사항을 수집
- ② 호스트·인스턴스 통계 검토: OS와 오라클 통계를 교차
- ③ 변경과 측정: 한 번에 하나만 바꿔 기준선과 대비
- ④ 목표 미달 시 ②③ 반복 — 은탄환은 없다

① 문제 정의

해결 전에 문제의 성격을 이해한다

- 성능 목표: 시간당 몇 건, 응답 몇 초가 허용 가능한가
- 범위: 인스턴스 전체인가, 특정 앱·프로그램·사용자인가
- 시간대: 피크에만인가, 점진적인가, 갑작스러운가
- 정량화: 재현 가능한 응답시간을 확보 → 조치 전후 비교 기준
- 증상과 원인의 구분: 통계는 대개 증상을 가리킨다

② 호스트·인스턴스 통계 검토

OS와 오라클을 교차 확인

- 호스트: CPU·I/O·네트워크 — 유휴 CPU 많으면 I/O·앱 병목 의심
- I/O: V$SYSTEM_EVENT의 db file 계열, V$IOSTAT_FILE 지연
- CPU: V$SYSSTAT(전체)·V$SESSTAT(세션별)·V$SYS_TIME_MODEL
- 오라클 통계를 OS 통계와 맞춰 진단의 일관성 확보

③ 변경과 측정 · ④ 반복

한 번에 하나만, 그리고 반복

- 한 번에 하나만 바꾼다 → 인과를 잃지 않는다
- 기준선과 대비해 효과를 측정한다
- 분리 불가하면 성격이 다른 변경끼리라도 나눠 측정
- 하나의 병목을 없애면 다음 병목이 드러난다 — 반복
- 멈추는 기준은 통계가 아니라 사용자 체감 성능

26.3

실시간 진단 경로

- 세션 → 대기 → SQL → 세그먼트
- 각 칸이 다음 칸을 가리킨다
- 라이선스 없이 V$만으로 병목까지

왜 세션 → 대기 → SQL → 세그먼트인가

좁은 입구로 들어가 범위를 좁힌다

- 활성 세션 = '지금 실제로 자원을 쓰는 소수'
- 대기 = 어느 자원이 병목인지 방향을 가리킴
- SQL = 원인 문장을 특정
- 세그먼트 = 조치 대상 객체까지 도달
- 통계부터 뒤지면 무엇이 관련 있는지 알 수 없어 길을 잃는다

세션: 지금 누가 무엇을 하는가

V$SESSION

- status='ACTIVE'인 사용자 세션만 추린다
- 핵심 열: SQL_ID·EVENT·WAIT_CLASS·BLOCKING_SESSION·ROW_WAIT_OBJ#
- BLOCKING_SESSION으로 '누가 누구를 막는가' 확인
- ROW_WAIT_OBJ#를 DBA_OBJECTS와 조인해 경합 객체 특정

V$SESSION 실측 — 활성 세션과 대기

부하 구간 SQLT 세션 관측(orcl 19.3)

```
SYS@orcl> SELECT sid, status, SUBSTR(event,1,30) event,
  2         wait_class, SUBSTR(sql_id,1,13) sql_id
  3       FROM v$session
  4       WHERE username='SQLT' AND status='ACTIVE';

SID STATUS   EVENT                         WAIT_CLASS SQL_ID
--- -------- ----------------------------- ---------- -------------
692 ACTIVE   SQL*Net message from client  Idle       9vmszgd1ft9x0
694 ACTIVE   SQL*Net message from client  Idle       9vmszgd1ft9x0
```

대기: 무엇을 기다리는가

V$SESSION_WAIT · V$SESSION

- wait_class <> 'Idle' 세션을 우선한다
- P1·P2·P3로 대기 대상 특정(예: 파일·블록 번호)
- 대기 시간이 가장 큰 이벤트가 다음 조사 지점
- 현재 상태 뷰(WAIT) vs 누적 뷰(EVENT·SYSTEM_EVENT) 구분

SQL: 어느 문장이 원인인가

V$SQL

- 세션이 물던 SQL_ID로 문장을 펼치거나 상위 부하로 정렬
- V$SQL=자식 커서 단위, V$SQLAREA=부모 요약(VERSION_COUNT)
- 부하 중 값이 변하므로 두 번 떠서 델타로 본다
- 누적 상위와 구간 델타 상위가 다를 수 있다

V$SQL 실측 — 논리읽기 상위

SQLT 스키마, buffer_gets 정렬

```
SQL_ID        EXECS BUFFER_GETS PER_EXEC TXT
------------- ----- ----------- -------- -------------------------
9vmszgd1ft9x0  1290     8494906     6585 select /*+ full(d) */ count
ddukvcz4gwcad   631     1946786     3085 select hosp_id, count(*)...
2hwjffuh4cs8t     2        6188     3094 select count(*) from medic
6jwvu6btpjhnu     2        6188     3094 select count(*) from medic
b85cymcwbd1qj     2        6188     3094 select count(*) from medic
13ywm87qcw0by     2        6180     3090 select count(*) from medic
fq4sycqx841ht     2        6180     3090 select count(*) from medic
fxj7b9gx2w1ay     2        6180     3090 select count(*) from medic
```

세그먼트: 어느 객체가 뜨거운가

V$SEGMENT_STATISTICS · V$SEGSTAT

- 대기·SQL로 잡은 방향의 진원지 객체를 특정
- logical reads·physical reads·buffer busy waits 등 세그먼트별
- V$SEGSTAT=가벼움, V$SEGMENT_STATISTICS=소유자·TS까지
- 세션·대기·SQL·세그먼트가 한 줄의 인과로 완성

26.4

V$SYSSTAT — 시스템 통계

- 값은 기동 이후 누적이다
- 절대값이 아니라 델타·비율
- 하드 파싱 비율 같은 파생 지표

누적값의 함정

절대값은 판정 근거가 아니다

- V$SYSSTAT 값은 인스턴스 기동 이후 누적
- session logical reads 3.56억 = 10분치인지 3주치인지 모름
- 판정은 두 시점의 델타를 경과 시간으로 나눈 비율
- 초당(처리량 변화) / 트랜잭션당(한 건의 무게) 정규화

V$SYSSTAT 실측 — 주요 통계

부하 구간 누적값(orcl 19.3)

```
NAME                             VALUE
-------------------------------- ---------
execute count                     338398
parse count (hard)                112198
parse count (total)               185227
physical reads                     70100
session logical reads          356228457
sorts (disk)                           0
sorts (memory)                     60198
user commits                       63992
```

델타 계산 — 두 번 읽어 뺀다

같은 통계, 60초 간격

```
-- (조치 전) 시작 스냅샷
session logical reads = 356,228,457

-- 60초 부하 구간 경과 후 재조회
session logical reads = 362,520,100   ← <환경값>

-- 델타 = 362,520,100 - 356,228,457 = 6,291,643
-- 초당 논리읽기 = 6,291,643 / 60 ≈ 104,861 LIO/s
-- 이 초당 비율만이 판정의 근거가 된다.
```

부하 관련 주요 통계

무엇을 초.트랜잭션당으로 볼 것인가

- session logical reads: 작업량의 대표(논리읽기)
- physical reads: 캐시 미스로 디스크까지 간 양
- execute count / parse count: 실행과 파싱
- user commits: 트랜잭션 수 정규화의 분모
- sorts memory/disk: disk 정렬은 PGA 부족 신호

초·트랜잭션당으로 정규화

분모가 해석의 방향을 정한다

- 초당 = Δ통계 / 경과초 → 처리량(throughput) 변화
- 트랜잭션당 = Δ통계 / Δcommits → 애플리케이션 특성 변화
- AWR Load Profile이 보여 주는 두 정규화를 손으로 재현
- physical read/write total bytes 합 = 초당 I/O(MB/s)

하드 파싱 비율 — 파생 지표의 힘

112,198 / 185,227 ≈ 60.6%

- 하드 파싱 비율 = parse count (hard) / (total)
- 정상은 0에 가까워야 함 → 60.6%는 심각
- 원인: 같은 문장이 리터럴로 흩어진 6개 SQL_ID
- 조치: 바인드 변수(또는 CURSOR_SHARING 검토)
- 절대값 순위표에는 안 보이던 진짜 문제

대기로는 안 잡히는 문제들

추가 통계 — 안전망

- redo log space requests: 체크포인트·DBWR·아카이버 튜닝 신호
- table fetch continued row: 이행·연결 행(1% 넘으면 조사)
- consistent changes/gets: 읽기 일관성 롤백 비용 → AUM 권장
- 이 통계들도 언제나 델타·비율로 읽는다

26.5

V$SQL — 상위 부하 SQL 선별

- 정렬 기준: 실행시간·논리읽기·실행횟수
- 실행당 비용으로 나눠 본다
- 같은 문장을 시그니처로 묶는다

정렬 기준 — 상위의 정의가 바뀐다

한 기준만 보면 그림이 왜곡된다

- elapsed_time: 총 소요 시간이 큰 문장(대개 최우선)
- buffer_gets: 논리읽기가 많은 CPU·캐시 부하원
- executions: 자주 불리는 문장(파싱·왕복 부하)
- buffer_gets/executions: 실행당 효율 — 문장 자체의 무게

실행당으로 나눠 봐야 하는 이유

총량이 큰 두 부류를 가른다

- 무거운 문장: 실행당 비용이 큼 → SQL·인덱스·계획 수정
- 자주 불리는 문장: 실행당 가벼우나 호출이 많음 → 횟수 감소
- 실측 9vmsz: 총량 849만 + 실행당 6,585 = 두 축 모두 나쁨
- elapsed vs buffer_gets 순위 어긋남 = '대기가 문제'라는 단서

절대값 오판 교정 (Common Error)

이 장이 경계하는 대표 오판

- buffer_gets 절대값 1·2위만 손대면 나머지를 무시
- 그럴듯한 답: 그 문장은 빨라지나 근본 문제는 남음
- 하드 파싱 60.6%가 만든 래치 경합은 그대로
- 교정: 절대값 → 실행당 → 시그니처(FORCE_MATCHING)로 묶기

26.6

절대값을 델타·비율로 교정

- 세 가지 오판 패턴
- 부하 증가 ≠ 성능 저하
- 공통 뿌리: 숫자 하나를 한 번 읽는 조급함

세 가지 오판 패턴

맥락 없는 숫자에 의미를 부여하지 않는다

- ① 누적값 오판: 3.56억을 '많다'고 단정
- ② 상위 SQL 오판: 절대값 1·2위만 손댐
- ③ 부하 증가 ↔ 성능 저하 혼동
- + 히트율 신앙: 단일 비율로 건강 판정(같은 부류)

부하 증가 vs 성능 저하

트랜잭션당으로 가른다

```
-- 오전: Δlogical reads=3,000,000  Δcommits=3,000
--   → 트랜잭션당 = 1,000 LIO/txn

-- 오후: Δlogical reads=9,000,000  Δcommits=9,000
--   → 트랜잭션당 = 1,000 LIO/txn

-- 총량 3배지만 트랜잭션당 동일 → 성능 저하 아님, 부하 3배
-- 오후 트랜잭션당이 3,000이면? 그때가 진짜 성능 저하.
```

교정 원칙

보이는 절대값 → 올바른 정규화

- logical reads 3.56억 → 초당·트랜잭션당 LIO
- buffer_gets 상위 SQL → 실행당·시그니처로 재평가
- parse count (hard) → total 대비 비율(60.6%)
- execute count 증가 → 초·트랜잭션당으로 부하/저하 구분

26.7

AWR 없이도 진단은 된다

- V$ 실시간 조회 = 라이선스 무관
- DBA_HIST·AWR·ADDM·ASH = Diagnostics Pack 🔒
- 대안: Statspack · V$ 델타

V$의 위치와 라이선스 경계

'지금'은 무료, '과거'는 라이선스

- 무관: V$SESSION·V$SESSION_WAIT·V$SQL·V$SYSSTAT·V$SEGMENT_STATISTICS
- 🔒 Diagnostics Pack: AWR·ADDM·ASH·DBA_HIST_*·V$ACTIVE_SESSION_HISTORY
- 대안: Statspack(무상) 또는 V$ 두 시점 델타
- AWR이 자동화하던 '구간 정규화'를 손으로 할 뿐 논리는 동일

라이선스 없이 보는 법

보고 싶은 것 → V$ 대안

- 지금 활성 세션·대기 → V$SESSION·V$SESSION_WAIT
- 지금 상위 부하 SQL → V$SQL 기준별 정렬
- 구간 부하율 → V$SYSSTAT 두 시점 델타
- 과거 구간 재구성 → 🔒 대신 Statspack 보고서
- 과거 스파이크 → 🔒 ASH 대신 V$SESSION 반복 샘플링

퀴즈 (Quiz)

- Q1. 인스턴스 튜닝 방법론 4단계를 순서대로 쓰고 각 단계의 수집·측정을 설명하라.
- Q2. 진단 경로 '세션→대기→SQL→세그먼트'가 왜 이 순서인지 설명하라.
- Q3. ACTIVE 세션의 대기가 'SQL*Net message from client'일 때 병목이 아닌 이유는?
- Q4. session logical reads 절대값만으로 판정 못 하는 이유와 올바른 절차는?
- Q5. 하드 파싱 112,198/185,227의 비율을 계산하고 비정상 근거·원인을 쓰라.
- Q6. buffer_gets 절대값 상위만 튜닝할 때 놓치는 것과 교정 방법은?

요약 (Summary)

- 인스턴스 튜닝은 문제 정의→통계 검토→변경·측정→반복이며 기준선이 필수다.
- 실시간 진단은 세션→대기→SQL→세그먼트 순으로 좁힌다.
- ACTIVE라도 대기가 Idle이면 병목이 아니다 — 걸러 낸다.
- V$SYSSTAT은 누적값. 델타를 초·트랜잭션당으로 정규화해야 판정된다.
- V$SQL은 실행시간·논리읽기·실행횟수로 교차하고 실행당·시그니처로 확인.
- V$ 실시간 조회는 라이선스 무관 — AWR 없이도 진단은 가능하다.

실습 개요 (Practice Overview)

- 실습 01 — 실시간 진단 경로: 세션→대기→SQL→세그먼트 추적
- 실습 02 — V$SYSSTAT 델타 계산: 초·트랜잭션당 부하율과 하드 파싱 비율
- 실습 03 — V$SESSION·V$SESSION_WAIT로 현재 활동 파악과 Idle 걸러내기
- 실습 04 — V$SQL 정렬 기준별 상위 SQL 선별과 교차
- 실습 05 — 절대값 오판을 델타·비율·시그니처로 교정
- ※ 모두 non-CDB(orcl 19.3), 라이선스 불필요한 V$ 조회만 사용

Oracle Database 19c 성능 튜닝 실무 과정

V$ 뷰를 이용한 인스턴스 튜닝 ②

대기 이벤트 분석 — 증상에서 원천으로

대기 클래스 · db file 계열 · buffer busy · log file sync · latch/mutex · enq: TX · Idle 오판

ch27

학습 목표 (Objectives)

- V$EVENT_NAME의 wait_class로 대기 이벤트를 분류하고 세 뷰(현재·누적·사전)를 구분한다.
- db file sequential(단일 블록)·scattered(다중 블록) read의 차이를 원리로 설명한다.
- buffer busy waits의 원인 세그먼트를 V$SEGMENT_STATISTICS·ROW_WAIT_OBJ#로 지목한다.
- log file sync를 커밋 빈도와 연결해 디스크 문제와 과도한 커밋을 구분한다.
- latch: shared pool·library cache: mutex X 경합을 하드 파싱과 연결해 해석한다.
- enq: TX - row lock contention을 재현하고 blocking_session으로 블로커를 역추적한다.
- SQL*Net message from client 같은 Idle 이벤트를 병목으로 오판하지 않는다.

이 장의 구성 (Agenda)

- 27.2 대기 이벤트 분류 체계 — V$EVENT_NAME과 대기 클래스
- 27.3 db file sequential read와 scattered read
- 27.4 buffer busy waits — 원인 세그먼트를 지목하라
- 27.5 log file sync — 커밋 빈도가 만드는 대기
- 27.6 latch와 mutex 경합 — 라이브러리 캐시의 자물쇠
- 27.7 enqueue 대기 — enq: TX - row lock contention
- 27.8 Idle 이벤트를 병목으로 오판하지 않기

27.2

대기 이벤트 분류 체계

- 대기 이벤트는 증상이다
- 클래스로 큰 방향
- 현재·누적·사전 세 뷰
- 총 대기 시간으로 정렬

대기 이벤트는 증상이다

이벤트에서 원천으로 한 계단 더

- 이벤트 이름은 '어디를 파라'는 이정표일 뿐
- db file read 상위 = '디스크가 느리다'가 아님
- → '어느 SQL이 그 읽기를 유발하나'가 진짜 질문
- 정렬은 총 대기 시간(time_waited) 기준

세 뷰 — 현재·누적·사전 (실측)

지금 non-Idle 대기 스냅사진

```
SYS@orcl> SELECT SUBSTR(event,1,34) event, wait_class,
  2         COUNT(*) sessions
  3       FROM v$session
  4       WHERE wait_class <> 'Idle' AND wait_class IS NOT NULL
  5       GROUP BY event, wait_class ORDER BY COUNT(*) DESC;

EVENT                          WAIT_CLASS      SESSIONS
------------------------------ --------------- --------
log file single write          System I/O             1
log file switch completion     Configuration          1
buffer busy waits              Concurrency            1
SQL*Net message to client      Network                1
```

대기 클래스 지도

주목 대상은 다섯 클래스

- User I/O — db file read → SQL·실행 계획
- System I/O — log/control write → 백그라운드·디스크
- Concurrency — buffer busy·latch·mutex → 경합·하드파싱
- Commit — log file sync / Application — enq: TX
- Idle(SQL*Net from client) → 병목 아님, 제외

세 지표를 함께 읽는다

횟수·총 시간·평균 시간

- total_waits(빈도) · time_waited(영향) · 평균(성격)
- control file sequential read: 86,867회지만 1초 → 무시
- SQL*Net vector data to client: 3,974회지만 358초 → 주목
- '횟수'로 정렬하면 엉뚱한 곳을 판다 → 총 시간 정렬

27.3

db file 계열

- sequential = 단일 블록
- scattered = 다중 블록
- 이름이 직관과 반대
- 어느 쪽이 큰지로 접근 추론

이름과 실제가 반대

단일 블록 vs 다중 블록

- sequential read = 단일 블록 = 인덱스 접근
- ('버퍼에 순차 배치'라는 뜻, 디스크 순차 아님)
- scattered read = 다중 블록 = 풀 테이블 스캔
- (여러 블록을 흩어진 빈 버퍼에 담아 scattered)

실측 — 어느 쪽이 큰가 (V$SYSTEM_EVENT)

non-Idle, 총 대기 시간 기준

```
SYS@orcl> SELECT SUBSTR(event,1,34) event, total_waits,
  2         ROUND(time_waited/100) sec, wait_class
  3       FROM v$system_event WHERE wait_class<>'Idle'
  4       ORDER BY time_waited DESC;

EVENT                          TOTAL_WAITS  SEC WAIT_CLASS
------------------------------ ----------- ---- ----------
log file sync                        63826   53 Commit
log file parallel write              64602   52 System I/O
db file sequential read              17097   52 User I/O
db file scattered read                1675   13 User I/O
```

판독 — db file 계열을 만나면

이벤트는 방향, SQL이 대상

- scattered 큼 → 풀스캔 과다 → SQL·인덱스 설계
- sequential 큼 → 인덱스 접근 과다 → 조인·반복 조회
- 평균 대기 시간이 크면(수십 ms↑) I/O 서브시스템
- 대상 SQL = V$SQL을 disk_reads·buffer_gets 순 정렬

27.4

buffer busy waits

- 같은 블록에 동시 접근
- 동시성 문제(≠ 디스크)
- V$WAITSTAT로 블록 클래스
- V$SEGMENT_STATISTICS로 세그먼트

왜 같은 블록에 몰리나

I/O 대기가 아니라 동시성 대기

- 여러 세션이 같은 테이블에 동시 INSERT
- 인기 소형 코드 테이블 동시 읽기
- ≠ 디스크 못 가져옴 / = 블록에 줄 섬
- 처방: 경합 분산(역키·해시 파티션·세그먼트 분산)

원인 세그먼트 지목 (V$SEGMENT_STATISTICS)

logical/physical reads 상위가 진원지

```
SYS@orcl> SELECT object_name, statistic_name, value
  2       FROM v$segment_statistics
  3       WHERE owner='SQLT' AND value>0
  4         AND statistic_name IN ('logical reads',
  5             'physical reads','buffer busy waits')
  6       ORDER BY value DESC;

OBJECT_NAME       STATISTIC_NAME      VALUE
----------------- -------------- ----------
MEDICAL_CLAIMS    logical reads   225305696
CLAIM_DETAILS     logical reads   123172432
CLAIM_DETAILS     physical reads      13226
```

지금 막힌 세션의 객체 (ROW_WAIT_OBJ#)

V$SESSION → DBA_OBJECTS

```
SYS@orcl> SELECT sid, row_wait_obj#
  2       FROM v$session
  3       WHERE event = 'buffer busy waits';
       SID  ROW_WAIT_OBJ#
---------- --------------
       596          74213

SYS@orcl> SELECT owner, object_name, object_type
  2       FROM dba_objects WHERE data_object_id = 74213;
OWNER  OBJECT_NAME     OBJECT_TYPE
------ --------------- -----------
SQLT   REVIEW_LOG_WL   TABLE
```

블록 클래스가 대응을 가른다

V$WAITSTAT

- data block → 대상 세그먼트의 동시 DML 분산
- segment header → freelist·ASSM 확인
- undo header/block → 언두 구성 점검
- 먼저 '어느 클래스'인지부터 확인

27.5

log file sync

- 커밋 시 리두 flush 대기
- 실측 63,826회
- 짝 이벤트 = parallel write
- 흔한 원인은 잦은 커밋

log file sync의 원리

커밋의 리두가 디스크에 안착하기까지

- COMMIT/ROLLBACK → 리두를 LGWR가 로그 파일에 flush
- 서버 프로세스는 그 쓰기 완료까지 대기
- = '내 커밋이 안전하게 기록되길 기다리는 시간'
- Commit 클래스의 대표 이벤트

실측 — 커밋 대기의 두 얼굴

log file sync vs parallel write

```
-- log file sync         : 63826 회 / 53초 (Commit)
-- log file parallel write: 64602 회 / 52초 (System I/O)

SYS@orcl> SELECT event, total_waits,
  2         ROUND(time_waited/100) sec
  3       FROM v$system_event
  4       WHERE event LIKE 'log file s%'
  5          OR event = 'log file parallel write';

EVENT                     TOTAL_WAITS  SEC
------------------------- ----------- ----
log file sync                   63826   53
log file parallel write         64602   52
```

평균 대기 시간으로 원인을 가른다

디스크냐, 과도한 커밋이냐

- 평균 대기 큼 → 리두 로그 디스크가 느림
- 평균 대기 작은데 횟수 폭발 → 너무 잦은 커밋
- 실측 63,826회 = row-by-row commit의 흔적
- 처방: 루프 안 COMMIT 제거 → N건마다 배치 커밋

이웃 이벤트도 함께 본다

리두 계열의 다른 대기

- log file switch completion — 로그 그룹 전환 대기(Config)
- → 리두 로그가 작거나 개수 부족 / 체크포인트 지연
- log buffer space — 리두 버퍼가 가득 참
- → 리두 생성 속도가 기록 속도를 앞지름

27.6

latch와 mutex

- 공유 메모리 구조 경량 잠금
- 경합의 배후 = 하드 파싱
- S/X 모드
- 처방은 바인드 변수

latch와 mutex의 차이

라이브러리 캐시를 지키는 자물쇠

- latch: 공유 구조 보호, 마이크로초 단위
- mutex: 더 가볍고 세밀(커서마다 하나) → 경합 분산
- 요즘 라이브러리 캐시 경합 = mutex 이벤트로 표출
- S=공유, X=배타 (cursor: pin S wait on X)

실측 — 경합의 배후는 하드 파싱

latch: shared pool 13,156회

```
EVENT                          WAITS  SEC  CLASS
------------------------------ ------ ---- -----------
latch free                      17174    4  Other
latch: shared pool              13156    1  Concurrency
cursor: pin S wait on X          1077    2  Concurrency
library cache: mutex X            387    1  Concurrency

-- 공유 풀 래치 경합 ↔ 하드 파싱(리터럴 SQL 반복)
-- 근본 처방: 바인드 변수 사용
SYS@orcl> SELECT sql_text, parse_calls, version_count
  2       FROM v$sqlarea ORDER BY parse_calls DESC;
```

근본 처방

증상이 아니라 파싱을 고친다

- 1차 용의자 = 과도한 하드 파싱
- V$SQLAREA에서 parse_calls·version_count 큰 SQL
- 근본 처방 = 바인드 변수(리터럴 제거)
- parse count(hard) 300/초 넘으면 확장성 문제

27.7

enq: TX - row lock contention

- 트랜잭션 잠금(Application)
- 쓰기 대 쓰기에서만
- blocking_session 역추적
- 블로커는 대개 놀고 있다

발생 원리와 ch17과의 관계

같은 경합, 대기 이벤트의 축

- 행 잠금은 읽기를 막지 않음 → '쓰기 대 쓰기'
- 앞 세션 커밋 전까지 뒤 세션이 그 행을 못 잡음
- ch17: '누가 누구를 막았나'(블로킹 597↔596)
- ch27: '이 대기가 어느 세그먼트·SQL에서'

블로커 역추적 (실측)

대기 세션 → 최종 블로커

```
SYS@orcl> SELECT s.sid, s.event, s.blocking_session blocker,
  2         s.row_wait_obj# obj#, s.seconds_in_wait wait_s
  3       FROM v$session s
  4       WHERE s.event='enq: TX - row lock contention';
  SID  EVENT                          BLOCKER  OBJ#  WAIT_S
  ---  ---------------------------- -------- ----- ------
  596  enq: TX - row lock contention      597 74198     46

SYS@orcl> SELECT sid, status, event FROM v$session WHERE sid=597;
  SID  STATUS   EVENT
  ---  -------- --------------------------
  597  INACTIVE SQL*Net message from client
```

실무 시나리오 · 블로커가 놀고 있다

대기 사슬의 맨 끝을 보라

- 여러 세션이 enq: TX로 쌓임
- blocking_session을 타고 올라가면 최종 블로커 1개
- 블로커는 INACTIVE·트랜잭션 방치 상태
- 조치: 커밋/롤백 유도, 짧은 트랜잭션 설계

27.8

Idle 오판 — Common Error

- SQL*Net message from client
- = DB가 노는 시간(Idle)
- from ≠ to client
- wait_class<>'Idle'이 첫 규율

SQL*Net message from client

병목이 아니라 유휴 시간

- = 서버가 클라이언트의 다음 요청을 기다림
- = 사용자 생각 시간 → DB가 놀고 있음(Idle)
- 총 대기 시간이 커도 병목 아님
- from client(Idle) ≠ to client(Network)

Idle을 걸러내는 습관

wait_class<>'Idle' 한 줄

```
SYS@orcl> -- 방법 1: wait_class로 직접 필터(권장)
SYS@orcl> SELECT event, total_waits, time_waited, wait_class
  2       FROM v$system_event
  3       WHERE wait_class <> 'Idle'
  4       ORDER BY time_waited DESC FETCH FIRST 10 ROWS ONLY;

SYS@orcl> -- 방법 2: V$EVENT_NAME으로 Idle을 NOT IN 제외
SYS@orcl> SELECT e.event, e.time_waited FROM v$system_event e
  2       WHERE e.event NOT IN (SELECT name FROM v$event_name
  3                             WHERE wait_class='Idle')
  4       ORDER BY e.time_waited DESC;

-- to client 313388회지만 1초 / vector data 358초 = 대량 전송
```

교훈 · 맨 위가 Idle이면

필터를 빠뜨렸다는 신호

- 'SQL*Net이 70% 최대 병목' 진단서 = 오판
- 네트워크·DB 병목이 아니라 유휴 시간
- wait_class<>'Idle'로 재집계 → 진짜 병목 등장
- 대기 목록 맨 위 Idle = 발견이 아니라 누락 신호

퀴즈 (Quiz)

- Q1. scattered read가 다중 블록(풀스캔)인 이유와 이름이 반대인 까닭은?
- Q2. buffer busy로 멈춘 세션의 객체를 즉시 특정하는 V$SESSION 컬럼은?
- Q3. log file sync 상위 — 디스크 교체 전 확인할 짝 이벤트와 평균 대기 시간은?
- Q4. latch: shared pool 경합의 근본 원인과 처방은?
- Q5. enq: TX 블로커가 INACTIVE·SQL*Net from client면 무엇을 뜻하나?
- Q6. V$SYSTEM_EVENT 맨 위가 SQL*Net message from client — 무엇을 잘못했나?

요약 (Summary)

- 대기 이벤트는 증상 — 클래스로 방향 잡고 원천(SQL·세그먼트·세션)으로 내려간다.
- sequential=단일 블록=인덱스, scattered=다중 블록=풀스캔.
- buffer busy = 동시성. V$SEGMENT_STATISTICS·ROW_WAIT_OBJ#로 세그먼트 지목.
- log file sync = 커밋 대기. parallel write·평균 대기로 디스크 vs 잦은 커밋 판별.
- latch/mutex 경합의 배후는 하드 파싱 — 처방은 바인드 변수.
- enq: TX = 애플리케이션 잠금. blocking_session으로 최종 블로커 역추적.
- SQL*Net message from client는 Idle — wait_class<>'Idle'이 첫 규율.

실습 개요 (Practice Overview)

- 실습 01 — 대기 이벤트 분류 체계 확인(V$EVENT_NAME, 클래스별)
- 실습 02 — WL_02_io.sql로 db file sequential/scattered read 유발·구분
- 실습 03 — buffer busy waits 재현과 원인 세그먼트 지목
- 실습 04 — WL_05_commit.sql로 log file sync 재현 — 커밋 빈도
- 실습 05 — latch/mutex 경합 관찰 — 라이브러리 캐시 경합
- 실습 06 — WL_04_lock.sql로 enq: TX 재현과 블로커 추적
- 실습 07 — Idle 이벤트를 병목으로 오판하는 사례 재현과 교정

Oracle Database 19c 성능 튜닝 실무 과정

경고·메트릭 기반 상시 모니터링

Server-Generated Alerts & Metrics — 얼마로 잡고, 울리면 무엇부터 보는가

서버 생성 알림 · 임계값 산정(이력 분포) · 알림 대응 절차 · 세션·서비스 모니터링 (ADMIN 32장 흡수·심화)

ch28

학습 목표 (Objectives)

- 서버 생성 알림 구조(MMON·ALERT_QUE·딕셔너리 뷰)를 설명하고 Stateful/Stateless를 구분한다.
- DBMS_SERVER_ALERT.SET_THRESHOLD로 임계값을 설정·조회·초기화한다.
- DBA_HIST 이력 분포(p95·p99)에서 근거 있는 임계값을 산정한다. (이 장의 핵심)
- 알림이 울렸을 때 증상에서 원인으로 좁혀 가는 대응 절차를 수행한다.
- V$SESSION_LONGOPS로 장기 작업을, V$SERVICE_STATS·V$SERVICEMETRIC로 업무별 성능을 관찰한다.
- 임계값을 낮게 잡을 때의 알림 피로(alert fatigue)를 이해하고 피한다.

이 장의 구성 (Agenda)

- 28.2 서버 생성 알림의 구조 — 무엇이 언제 울리는가
- 28.3 메트릭 임계값 설정 — DBMS_SERVER_ALERT.SET_THRESHOLD
- 28.4 임계값을 얼마로 잡을 것인가 — 이력 분포 산정 (핵심 · 🔒)
- 28.5 알림 대응 절차 — 울렸을 때 무엇부터 보는가
- 28.6 세션 모니터링 — 활성 세션·장기 실행 작업
- 28.7 서비스 단위 모니터링 — 업무별로 나눠 본다
- 28.8 흔한 함정 — 임계값을 낮게 잡아 알림 피로
- ※ ADMIN 32장이 '설정하는 법'이었다면, 이 장은 '얼마로 잡고 울리면 무엇부터 보는가'다.

28.2

서버 생성 알림의 구조

- 폴링의 약점과 push 알림
- MMON · ALERT_QUE
- Stateful vs Stateless
- 미해소·이력 딕셔너리 뷰

폴링에서 서버 생성 알림으로

감시 로직을 커널 안으로

- 전통적 감시 = DBA 스크립트로 주기적 폴링 → 사각지대·부하·유지 부담
- 서버 생성 알림 = 서버가 스스로 상태를 감시해 문제 시 알림 발생
- 감시 주체 = MMON. 임계값 초과 시 ALERT_QUE(Advanced Queuing)에 등록
- 폴링이 아니라 사건 발생 시점의 즉시 통지(push) 구조

알림 소비 경로와 사전 정의 알림

원천은 하나, 소비는 두 갈래

- 소비 ① EM이 ALERT_QUE 구독 → 콘솔 표시·이메일·페이지 통지
- 소비 ② 딕셔너리 뷰 직접 조회 또는 DBMS_AQ로 큐 디큐
- 사전 정의 알림: Tablespace Space Usage(85/97), Snapshot Too Old 등
- 사용자 알림: 임계값을 지정해야 활성화(응답시간·활성 세션 등)

Stateful vs Stateless 알림

대응 목표가 다르다

- Stateful = 조건이 지속되는 '동안' 유지되는 상태(예: 테이블스페이스 사용률)
- → DBA_OUTSTANDING_ALERTS에 남아 있다가 조건 해소 시 자동 clear
- Stateless = 특정 시점의 사건(예: Snapshot Too Old·인스턴스 up/down)
- → 해소 개념 없이 곧바로 DBA_ALERT_HISTORY로 기록
- 대응: Stateful=원인 제거 / Stateless=사후 원인 확인

미해소 알림 조회 (Stateful)

DBA_OUTSTANDING_ALERTS — 실측

```
SYS@orcl> SELECT reason, message_level,
  2         TO_CHAR(creation_time,'MM-DD HH24:MI') created
  3       FROM  dba_outstanding_alerts WHERE rownum<=10;

REASON                                   MESSAGE_LEVEL CREATED
---------------------------------------- ------------- -----------
Tablespace [RTBS] is [100 percent] full              1 08-10 10:20
```

알림 이력 조회 (Stateless)

DBA_ALERT_HISTORY — 실측

```
SYS@orcl> SELECT SUBSTR(reason,1,44) reason,
  2         TO_CHAR(creation_time,'MM-DD HH24:MI') created
  3       FROM (SELECT reason,creation_time FROM dba_alert_history
  4             ORDER BY creation_time DESC) WHERE rownum<=10;

REASON                                       CREATED
-------------------------------------------- -----------
Instance orcl.orcl. down on node oel7v9.loca 08-24 14:56
Instance orcl.orcl. up on node oel7v9.localdb 08-24 14:56
Operation on resumable session big_load_job  08-20 13:08
```

28.3

메트릭 임계값 설정

- 메트릭 = 표준화된 성능 지표
- DBA_THRESHOLDS 조회
- SET_THRESHOLD 인자
- 연속 관측으로 스파이크 흡수

메트릭이란 무엇이며 무엇을 감시하나

표준화된 성능 지표

- 메트릭 = 누적 절대값을 '초당·호출당·%'로 표준화한 지표
- MMON이 15초·60초 간격으로 계산 → V$SYSMETRIC 등에 노출
- 수준: 시스템 / 세션 / 서비스 / 특정 대상(파일·이벤트)
- 감시 대상 선정 기준: '나빠지면 사용자가 실제로 아파하는가'
- 메트릭 목록·단위는 V$METRICNAME으로 확인

현재 임계값 조회

DBA_THRESHOLDS — 실측

```
SYS@orcl> SELECT metrics_name, warning_value,
  2         critical_value, consecutive_occurrences
  3       FROM dba_thresholds WHERE rownum<=15
  4       ORDER BY metrics_name;

METRICS_NAME            WARNING CRITICAL CONSEC
----------------------- ------- -------- ------
Tablespace Space Usage  <SYS-GEN>      0
Tablespace Space Usage  85            97       1
Tablespace Space Usage  <SYS-GEN>      0
```

임계값 설정 — SET_THRESHOLD

값은 28.4에서 이력으로 산정

```
SYS@orcl> BEGIN
  2    DBMS_SERVER_ALERT.SET_THRESHOLD(
  3      metrics_id       => DBMS_SERVER_ALERT.CPU_TIME_PER_CALL,
  4      warning_operator => DBMS_SERVER_ALERT.OPERATOR_GE,
  5      warning_value    => '<이력 산정 경고값>',
  6      critical_value   => '<이력 산정 위험값>',
  7      observation_period=> 1, consecutive_occurrences=> 3,
  8      instance_name    => 'orcl',
  9      object_type => DBMS_SERVER_ALERT.OBJECT_TYPE_SERVICE,
 10      object_name => 'orcl'); END; /
```

연속 관측으로 알림 피로를 막는다

임계값·주기·연속횟수는 한 세트

- 임계값을 잘 잡아도 순간 스파이크는 늘 발생한다
- consecutive_occurrences=2~3 → 연속 관측될 때만 알림
- 임계값을 낮추는 것보다 연속 횟수를 늘리는 편이 스파이크에 강함
- 임계값 · 관측 주기 · 연속 횟수 = 함께 돌리는 세 손잡이

28.4

임계값을 얼마로 잡을 것인가

- 이 장의 핵심 · ADMIN 미수록
- 감이 아니라 이력 분포에서
- p95=경고 / p99~peak=위험
- 🔒 DBA_HIST 필요(대안 있음)

임계값에 정답 숫자는 이식되지 않는다

그 시스템의 평시에서 나온다

- 'CPU 80% 경고' 같은 값은 그 값을 만든 시스템의 평시일 뿐
- 시스템마다 평시 부하가 다름 → 남의 옷을 입으면 크거나 작다
- 옳은 임계값 = 그 시스템의 '정상 범위를 벗어난 지점'
- 설계 1단계 = 값 정하기가 아니라 '내 평시 측정하기'

현재값만으로는 못 정한다

V$SYSMETRIC 현재값 — 실측

```
SYS@orcl> SELECT metric_name, ROUND(value,2) value, metric_unit
  2  FROM v$sysmetric WHERE intsize_csec BETWEEN 5900 AND 6100
  3    AND metric_name IN (...);

METRIC_NAME                VALUE   METRIC_UNIT
-------------------------- ------- --------------------
Average Active Sessions    4.28    Active Sessions
Database CPU Time Ratio    96.8    % Cpu/DB_Time
Database Wait Time Ratio   3.2     % Wait/DB_Time
Executions Per Sec         916.31  Executes Per Second
Host CPU Utilization (%)   48.77   % Busy/(Idle+Busy)
```

이력 분포를 뽑는다

DBA_HIST_SYSMETRIC_SUMMARY 🔒

```
SYS@orcl> SELECT metric_name,
  2    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP
  3      (ORDER BY average),2) p50,
  4    ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP
  5      (ORDER BY average),2) p95,
  6    ROUND(PERCENTILE_CONT(0.99) WITHIN GROUP
  7      (ORDER BY average),2) p99, ROUND(MAX(maxval),2) peak
  8  FROM dba_hist_sysmetric_summary
  9  WHERE metric_name='Average Active Sessions'
 10  GROUP BY metric_name;

P50    P95    P99    PEAK
<환경값> <환경값> <환경값> <환경값>
```

분포에서 경고·위험을 도출하는 규칙

숫자가 아니라 절차

- 경고(Warning) = 평시 상위 5% 지점(p95) → 드물게만 넘는 선
- 위험(Critical) = p99와 관측 최댓값(peak) 사이 → 거의 안 닿는 선
- 경고~위험 사이에 대응할 물리적 시간 간격을 확보
- consecutive_occurrences 2~3으로 p95 스침 흡수
- 분기마다 이력 재산정 — 평시 자체가 이동한다

평균±표준편차로 보정하기

지표 성격에 맞춰 손잡이 조합

- 정규에 가까우면 평균+2σ≈상위 2~3%(경고), 평균+3σ(위험)
- 백분위수와 두 방법이 일치하면 확신↑, 어긋나면 치우친 분포
- CPU Time Ratio(96.8%): 상한 100% → 값보다 '지속성'을 본다
- Executions Per Sec(916.31): 상한 없음 → 백분위수 방식이 적합
- 표본 구간을 업무 시간대로 분리(낮 vs 배치 새벽)

실무 시나리오 — 4.28로 임계값 정하기

현재값은 분포 위의 한 점

- 상황: AAS 4.28, Host CPU 48.77%. 활성 세션 경고를 걸어야 한다
- ✗ '4.28이 지금 값이니 5쯤에 경고' → 평시 중앙값이면 상시 알림
- ◯ 이력에서 p95·p99를 뽑아 경고=p95, 위험=p99~peak
- Host CPU가 코어 대비 절반 → 세션이 코어 수 넘는 지점이 위험 후보
- 현재값 4.28은 임계값이 아니라 분포 위 좌표로만 쓴다

임계값은 한 번 정하고 끝이 아니다

평시가 이동한다

- 사용자·데이터·업무가 늘면 평시 자체가 위로 이동
- 3개월 전 p95가 오늘은 평시 중앙값일 수 있다 → 다시 알림 피로
- 분기마다·큰 배포 뒤에 이력 재산정, 근거와 함께 기록
- 발생 빈도가 는다 = 임계값이 낮아서가 아니라 한계 접근 신호
- 임계값 발생 추이 = 용량 계획의 입력

28.5

알림 대응 절차

- 울리면 무엇부터 보는가
- 증상 → 원인으로 좁히기
- 위→아래 순서 준수
- 근거를 남긴다

대응 절차 — 위에서 아래로

알림은 증상, 원인은 V$로 판다

- 1) 알림 유형·심각도 — OUTSTANDING/HISTORY
- 2) 전체 부하 — V$SYSMETRIC(AAS·CPU·Wait Ratio)
- 3) 활성 세션 지목 — V$SESSION(ACTIVE)
- 4) 대기·블로커 — BLOCKING_SESSION, V$SESSION_WAIT
- 5) 원인 SQL·세그먼트 — VSQL, VSEGMENT_STATISTICS
- 6) 조치 후 재확인 — 1~2단계 재측정

WL_04_lock 대응 실습 흐름

잠금 경합 유발 → 알림 → 대응

- WL_04_lock: 한 세션이 행을 잡고 커밋 안 함 → 다른 세션들 대기
- 활성 세션↑, 서비스 응답시간 메트릭↑ → 알림 발생(Stateful)
- 2)Wait Ratio↑ 3)enq: TX-row lock contention에 세션 몰림
- 4) BLOCKING_SESSION으로 블로커 지목 → 커밋 유도/응급 종료
- 6) 대기 세션 해소·알림 자동 clear·Wait Ratio 정상 재확인

대응하며 근거를 남긴다

ch17의 원칙을 알림 상황에

- 조치 전 상태 캡처 없으면 조치 효과를 증명 못 함
- 기록: 알림 시각 · V$SYSMETRIC 값 · 활성 세션·대기 · 블로커·SQL
- 그리고 조치 내용 + 조치 후 값을 순서대로
- ① 같은 알림 재발 시 지난 원인과 대조해 빠른 판단
- ② 반복되는 알림의 기록 = 근본 원인(recurring root cause) 증거

28.6

세션 모니터링

- 활성 세션 상시 관찰
- 장기 실행 작업 진척
- V$SESSION_LONGOPS
- work vs wait 구분

장기 실행 작업

V$SESSION_LONGOPS — 실측

```
SYS@orcl> SELECT SUBSTR(opname,1,30) opname, sofar,
  2         totalwork, ROUND(sofar/GREATEST(totalwork,1)*100) pct
  3       FROM v$session_longops WHERE rownum<=6;

OPNAME                         SOFAR  TOTALWORK PCT
------------------------------ ------- --------- ---
RMAN: incremental datafile bac  120320    120320 100
RMAN: incremental datafile bac  104922    104922 100
RMAN: incremental datafile bac  115200    115200 100
RMAN: incremental datafile bac  264800    264800 100
```

활성 세션 — work인가 wait인가

V$SESSION status='ACTIVE'

- 활성 세션을 event로 묶어 보면 병목의 성격이 드러남
- CPU를 쓰느라 활성(work) → 용량 문제에 가까움
- 무언가 기다려 활성(wait) → 경합 문제에 가까움
- enq: TX 몰림 = 잠금 → 다음은 BLOCKING_SESSION 추적(ch27)
- 장기 작업(정상)과 갑자기 몰린 대기(비정상)를 함께 보아 구분

28.7

서비스 단위 모니터링

- 업무별로 나눠 본다
- V$SERVICE_STATS(누적)
- V$SERVICEMETRIC(최근)
- 서비스별 임계값

서비스별 누적 통계

V$SERVICE_STATS — 누적은 델타로

```
SYS@orcl> SELECT service_name, stat_name, value
  2       FROM v$service_stats
  3       WHERE stat_name IN ('DB time','DB CPU',
  4         'user calls','physical reads')
  5         AND service_name NOT LIKE 'SYS$%'
  6       ORDER BY service_name, stat_name;

SERVICE_NAME  STAT_NAME        VALUE
------------- ---------------- --------
orcl          DB CPU           <환경값>
orcl          DB time          <환경값>
orcl          physical reads   <환경값>
orcl          user calls       <환경값>
```

서비스로 나눠 감시하라

V$SERVICE_STATS / V$SERVICEMETRIC

- 인스턴스 전체 지표로는 '어느 업무가' 문제인지 모른다
- 서비스 = 워크로드를 나누는 논리 단위, 통계를 서비스별 집계
- V$SERVICE_STATS: 서비스별 DB time·CPU·읽기(누적 → 델타로)
- V$SERVICEMETRIC: 서비스별 최근 구간 응답시간·CPU
- SET_THRESHOLD object_type=SERVICE → 서비스별 임계값

28.8

흔한 함정 — 알림 피로

- 낮으면 소음, 높으면 지각
- 무뎌지면 진짜도 무시
- 해법 = 분포 기반 산정
- 울리면 반드시 조치할 것만

임계값을 낮게 잡으면

alert fatigue의 진짜 위험

- 선의로 낮게 잡음 → 정상 운영 중에도 수시로 울림
- 며칠 뒤 '또 그 알림' → 아무도 열어 보지 않음
- 무뎌진 사람은 '진짜 위험 알림'까지 함께 무시 = 감시 무력화
- 반대로 너무 높으면 문제가 커진 뒤에야 울림(RTBS 100% 사례)
- 해법: 경고=p95 · 위험=거의 안 닿는 선 · 연속 관측으로 흡수

퀴즈 (Quiz)

- Q1. Stateful과 Stateless 알림의 차이를 해소 방식·조회 뷰로 설명하라.
- Q2. 임계값을 이력 분포에서 산정해야 하는 이유를 한 문장으로.
- Q3. 경고 p95·위험 p99~peak 규칙에서 각 값의 의미는?
- Q4. consecutive_occurrences를 늘리면 무엇이 좋아지는가?
- Q5. 알림이 울렸을 때 활성 세션보다 먼저 볼 것은 무엇이며 왜인가?
- Q6. 임계값을 낮게 잡을 때 알림 피로의 가장 큰 위험은?

요약 (Summary)

- 서버 생성 알림 = MMON이 감시→ALERT_QUE 등록하는 push. EM 없이 딕셔너리 뷰로 관리.
- Stateful(지속·자동 해소) / Stateless(사건·이력 직행) — 대응 목표가 다르다.
- 설정은 SET_THRESHOLD, 확인은 DBA_THRESHOLDS. 값·연산자·주기·연속·대상.
- 임계값은 감이 아니라 DBA_HIST 분포에서: 경고=p95, 위험=p99~peak, 연속으로 흡수(핵심).
- 울리면 유형→전체부하→활성세션→대기·블로커→SQL·세그먼트→재확인 순으로.
- 세션=LONGOPS·활성 관찰, 업무=V$SERVICE_STATS·V$SERVICEMETRIC로 분리.
- 낮으면 알림 피로, 높으면 지각. 울리면 반드시 조치할 알림만 남긴다.

실습 개요 (Practice Overview)

- 실습 01 — 서버 생성 알림 구조 확인(Stateful/Stateless, OUTSTANDING/HISTORY)
- 실습 02 — 메트릭 임계값 설정(SET_THRESHOLD)과 DBA_THRESHOLDS 조회
- 실습 03 — 임계값 산정: DBA_HIST 이력 분포(p95·p99) (🔒 ·대안 경로)
- 실습 04 — 테이블스페이스 사용률 알림 발생·해제 전 과정 재현
- 실습 05 — WL_04_lock 경합 유발 → 알림 대응 절차(무엇부터 보는가)
- 실습 06 — 세션 모니터링·장기 실행 작업(V$SESSION_LONGOPS)
- 실습 07 — 서비스 단위 모니터링(V$SERVICE_STATS·V$SERVICEMETRIC)
- ※ 환경: non-CDB orcl 19.3, SYS@orcl>/SQLT@orcl>. '조치 전→조회→해석'+'이상→원인→조치→재확인'.

Oracle Database 19c 성능 튜닝 실무 과정

데이터베이스 메모리 할당 구조

Database Memory Allocation — AMM · ASMM · 수동

메모리 관리 방식 판별 · 파라미터 체계 · 자동 조정 이력 · 메모리 어드바이저 · ORA-04031

ch29

학습 목표 (Objectives)

- 오라클 메모리 구조(SGA 캐시·풀, PGA, In-Memory)를 열거하고 역할을 설명한다.
- 메모리 관리 세 방식(AMM·ASMM·수동)을 파라미터 값 조합만으로 판별한다.
- MEMORY_TARGET·SGA_TARGET·PGA_AGGREGATE_TARGET의 목표·상한·동적/정적 관계를 구분한다.
- V$SGAINFO·V$SGA_DYNAMIC_COMPONENTS로 실제 배분과 그래뉼을 읽는다.
- V$SGA_RESIZE_OPS·V$MEMORY_RESIZE_OPS로 자동 조정 이력을 추적한다.
- V$SGA_TARGET_ADVICE 곡선으로 '더 줘도 평평한 지점'과 '절벽'을 판정한다.
- ORA-04031을 재현하고 원인(부족·단편화)과 해소책을 설명한다. In-Memory는 라이선스 조건만.

이 장의 구성 (Agenda)

- 29.2 오라클 메모리 구조 개요 — 무엇을 배분하는가
- 29.3 메모리 관리 방식 세 가지 — AMM · ASMM · 수동
- 29.4 파라미터 체계 — 목표·상한·동적/정적
- 29.5 SGA 동적 구성과 자동 조정 이력
- 29.6 메모리 어드바이저 — 지금 크기가 적절한가
- 29.7 ORA-04031 — 메모리 부족의 대표 오류
- 29.8 In-Memory Column Store — 존재와 라이선스(실습 제외)
- ※ 실습 DB는 ASMM(orcl 19.3, non-CDB). 파라미터 변경·재기동은 개인 환경 전용.

29.2

오라클 메모리 구조 개요

- 공유 영역 SGA vs 프로세스 사설 PGA
- 버퍼 캐시·공유 풀이 성능의 핵심
- 무엇이 자동 조정 대상인가

메모리가 성능을 좌우하는 이유

물리 I/O는 메모리 접근보다 압도적으로 느리다

- 디스크 접근(물리 I/O)은 대개 10ms 안팎 — 메모리보다 수십만 배 느림
- 물리 I/O는 드라이버·OS 스케줄러를 거쳐 CPU도 더 씀
- 자주 쓰는 데이터를 메모리에 붙들면 물리 I/O·CPU를 함께 줄인다
- 메모리 관리 = 전체 크기 결정 + 구성 요소별 배분

SGA의 주요 구성 요소

공유 영역 — 모든 세션이 함께 쓴다

- 버퍼 캐시(DB_CACHE_SIZE): 데이터 블록 캐싱 → 물리 I/O 감소
- 공유 풀(SHARED_POOL_SIZE): 라이브러리 캐시·딕셔너리 캐시·결과 캐시
- 라지 풀·자바 풀·스트림 풀: 병렬·RMAN·JVM·AQ용
- 리두 로그 버퍼(LOG_BUFFER): 자동 조정 대상 아님
- In-Memory 영역(INMEMORY_SIZE): 컬럼 저장(별도 라이선스)

SGA vs PGA — 공유냐 사설이냐

배분 전략을 가르는 기준

- SGA: 모든 세션이 공유 → 캐시가 클수록 재사용 이득
- PGA: 세션마다 사설(정렬·해시 작업 영역) → 세션 수에 비례해 증가
- OLTP(동시 접속 多): PGA 여유를 고려
- DW(대량 데이터): 큰 작업 영역(PGA)+큰 버퍼 캐시(SGA)

29.3

메모리 관리 방식 세 가지

- AMM: MEMORY_TARGET>0
- ASMM: MEMORY_TARGET=0, SGA_TARGET>0
- 수동: 둘 다 0 · 실습 DB는 ASMM

자동 메모리 관리(AMM)

SGA+PGA를 통째로 오라클이 조정

- MEMORY_TARGET(동적) 하나로 총 메모리만 지정
- SGA↔PGA, SGA 내부 풀 사이를 필요에 따라 재분배
- 가장 자동화 — 오라클 권장, 소규모·개발에 편리
- 제약: 리눅스 HugePages를 활용 못 함 → 대형 SGA는 ASMM 선호

자동 공유 메모리 관리(ASMM)

SGA는 자동 분배, PGA는 별도 관리

- MEMORY_TARGET=0, SGA_TARGET>0일 때 작동
- SGA_TARGET 안에서 버퍼 캐시·공유 풀·라지 풀 등 자동 분배
- 개별 풀에 0 아닌 값을 주면 '최소 보장치(하한)'로 작동
- PGA는 PGA_AGGREGATE_TARGET로 자동 관리 → 실습 DB가 이 상태

수동 관리와 판별 규칙

세 줄로 방식을 가른다

- 수동: MEMORY_TARGET=0 그리고 SGA_TARGET=0 → 개별 풀 직접 지정
- 판별 ① MEMORY_TARGET>0 이면 AMM(확정)
- 판별 ② =0이고 SGA_TARGET>0 이면 ASMM
- 판별 ③ 둘 다 0이면 수동
- 방식을 모른 채 파라미터를 바꾸면 의도와 다른 결과가 난다

실측 — 파라미터로 방식 판별

V$PARAMETER (orcl 19.3, non-CDB)

```
NAME                    DISPLAY_VALUE
----------------------- --------------
db_cache_size           0
memory_max_target       0
memory_target           0         <-- AMM 아님
pga_aggregate_limit     2G
pga_aggregate_target    700M
sga_max_size            2G
sga_target              1504M      <-- >0 → ASMM
shared_pool_size        0
```

세 방식 한눈에 비교

자동화 수준과 관리 노력의 트레이드오프

- AMM: 관리 노력 최소 / 오라클이 SGA+PGA 전부 조정 / HugePages 제약
- ASMM: SGA만 자동 + PGA 별도 / HugePages 가능 / 운영 환경 선호
- 수동: 완전한 제어 / 워크로드 변화마다 손봐야 해 노력 최대
- 공통: 자동이라도 상한을 잘못 주면 자동으로 나빠진다

29.4

파라미터 체계

- 목표(동적) vs 상한(정적)
- 목표는 상한을 넘을 수 없다
- MEMORY_TARGET과 SGA_TARGET 혼동 주의

목표·상한과 동적/정적

파라미터는 짝으로 이해한다

- SGA_TARGET(1504M, 동적) ≤ SGA_MAX_SIZE(2G, 정적)
- 목표는 상한 이하여야 하고, 상한까지 동적으로 키울 수 있다
- PGA_AGGREGATE_TARGET(700M, 목표) vs PGA_AGGREGATE_LIMIT(2G, 하드 한도)
- 상한은 성장 여유를 고려해 넉넉히, 목표만 동적으로 조절

Common Error — 자동을 두 번 켜기

MEMORY_TARGET과 SGA_TARGET 동시 설정 혼란

- 증상: 재기동 후 SGA가 기대와 다르거나 ORA-00838/00844로 기동 막힘
- 원인: MEMORY_TARGET>0이면 AMM, 이때 SGA_TARGET은 '하한'으로만
- 관계를 어기면(MEMORY_TARGET<SGA_TARGET 등) 오류
- 판정: '자동'을 두 번 켜지 말고 한 방식을 고른다
- ASMM이면 MEMORY_TARGET=0 + SGA_TARGET만 설정

29.5

SGA 동적 구성과 조정 이력

- 파라미터=지시, 실제 배분=결과
- 그래뉼(16MB) 단위 조정
- GROW/SHRINK는 짝을 이룬다

실측 — SGA 구성 (V$SGAINFO)

실제로 어떻게 쪼개졌는가

```
NAME                                 MB    RESIZEABLE
-------------------------------- ------ ----------
Maximum SGA Size                   2048   No
Buffer Cache Size                  1024   Yes
Free SGA Memory Available           544
Shared Pool Size                    416   Yes
Startup overhead in Shared Pool   249.3   No
Shared IO Pool Size                  64   Yes
Granule Size                         16   No
Large Pool Size                      16   Yes
Fixed SGA Size                      8.5   No
In-Memory Area Size                   0   No
```

그래뉼과 Free SGA 해석

숫자가 무엇의 합인지 확인한다

- 그래뉼=16MB (SGA>1GB이므로) — 모든 조정은 16MB 배수
- 그래뉼은 기동 시 계산되어 수명 동안 불변
- Buffer Cache Size 1024 = DEFAULT+KEEP+4K 등의 합
- 그중 DEFAULT buffer cache는 864MB(뒤 화면과 대조)
- 뷰마다 '이 숫자가 무엇의 합인가'를 확인하는 습관

실측 — 동적 구성 (V$SGA_DYNAMIC_COMPONENTS)

현재·최소·최대로 움직인 범위를 본다

COMPONENT	CUR_MB	MIN_MB	MAX_MB
DEFAULT buffer cache	864	832	864
shared pool	416	416	416
Shared IO Pool	64	64	64
KEEP buffer cache	48	48	48
DEFAULT 4K buffer cache	48	48	48
streams pool	32	32	32
large pool	16	16	48

실측 — 조정 이력 (V$SGA_RESIZE_OPS)

GROW/SHRINK가 짝을 이룬다

```
COMPONENT             OPER   FROM  TO   T
--------------------- ------ ----- ---- -----------
large pool            GROW    16    32  08-25 17:28
DEFAULT buffer cache  SHRINK 864   848  08-25 17:28
large pool            SHRINK  32    16  08-25 17:28
DEFAULT buffer cache  GROW   848   864  08-25 17:28
large pool            SHRINK  48    16  08-25 14:34
DEFAULT buffer cache  GROW   832   864  08-25 14:34
-- 총량(SGA_TARGET) 고정 → 한 풀이 커지면 다른 풀이 내놓는다
```

조정 이력 해석 — 정상과 신호

GROW/SHRINK가 잦다고 다 나쁜 것은 아니다

- 짧은 왕복(thrashing 없이) = 건강한 자동 조정
- 문제 신호 ① 한 풀이 한 방향으로 계속 밀림 → 하한 검토
- 문제 신호 ② 조정이 지나치게 잦아 오버헤드 발생
- 문제 신호 ③ 조정에도 특정 풀에서 ORA-04031 반복
- 조정 이력은 '판정 근거'이지 그 자체가 '문제'는 아니다

어느 뷰가 언제 채워지나

방식에 맞는 뷰를 고른다

- V$SGA_RESIZE_OPS: SGA 풀 조정(ASMM·AMM 공통)
- V$MEMORY_RESIZE_OPS: SGA+PGA 통합 조정(주로 AMM)
- ASMM(실습 DB)에서는 SGA_RESIZE_OPS가 자연스럽다
- V$MEMORY_RESIZE_OPS는 AMM에서 SGA↔PGA 이동까지 포함

실무 시나리오 · '특정 시간대만 느리다'

메모리 관점으로 좁히기

- 1단계 판별: V$PARAMETER로 MEMORY_TARGET=0·SGA_TARGET=1504M → ASMM
- 2단계 이력: V$SGA_RESIZE_OPS를 그 시간대로 필터 — 어느 풀이 압력?
- 3단계 어드바이저: 곡선이 평평하면 SGA 증설 요구를 근거로 기각
- 4단계 진짜 원인: 대량 정렬이면 PGA(ch33), 리터럴이면 공유 풀(29.7)
- 판정: '늘리자'는 감이 아니라 판별→이력→어드바이저→진단으로 답한다

29.6

메모리 어드바이저

- 감이 아니라 곡선으로 결정
- 무릎(knee)과 절벽(cliff)
- 라이선스 무관 기본 기능

실측 — SGA 어드바이저 (V$SGA_TARGET_ADVICE)

크기 배수 대 예상 DB Time

```
SGA_MB   FACTOR   ESTD_DB_TIME
------   ------   ------------
  752    0.5         2213
 1128    0.75        1636
 1504    1.0         1635     <-- 현재
 1880    1.25        1635
 2256    1.5         1635
 3008    2.0         1635
-- 늘려도 평평(1635), 1128까지 줄여도 거의 동일(1636)
-- 752로 반토막 내면 2213으로 급증 (절벽)
```

곡선 읽기 — 무릎과 절벽

두 원칙

- ① '더 줘도 평평'하면 늘리지 않는다 → 그 메모리는 PGA로
- ② '줄이면 급증'하는 절벽 바로 위 + 여유 한 칸이 최적
- 어드바이저는 관측 워크로드 기반 추정 — 대표 구간에서 관측
- 실측: 1128MB까지 줄여도 DB Time 거의 그대로(과대 배정)

어드바이저의 함정

평평하다고 튜닝할 게 없는 건 아니다

- 값은 STATISTICS_LEVEL=TYPICAL 이상 + 충분한 워크로드에서 수집
- 방금 기동한 인스턴스의 곡선은 표본이 빈약
- '크기로는' 개선 안 됨 ≠ '튜닝할 게 없음'
- 진짜 병목: 비효율 SQL·하드 파싱·PGA·I/O → ch26~ch30 진단

29.7

ORA-04031 — 메모리 부족의 대표 오류

- 절대 부족 vs 단편화
- 증상과 원인의 구분
- FLUSH는 임시, 근본은 구성·바인드

ORA-04031이란

공유 메모리 연속 청크 할당 실패

- 메시지: unable to allocate N bytes of shared memory
- 원인 ① 절대 부족: 풀이 워크로드에 비해 너무 작다
- 원인 ② 단편화: 총량은 있으나 작은 조각으로 흩어짐
- 리터럴 SQL 폭증이 라이브러리 캐시를 잘게 채워 단편화 심화
- 예측 어려움 — 며칠 멀쩡하다 특정 크기 할당에서만 실패

원인 감별과 해소

키울 것인가, 애플리케이션을 고칠 것인가

- 감별: 메시지의 풀 이름 + 발생 시점(기동 직후=부족, 며칠 뒤=단편화)
- V$SGASTAT의 free memory가 넉넉한데 실패 → 단편화
- 해소 ① 부족: SGA_TARGET(또는 풀 하한)을 어드바이저 근거로 상향
- 해소 ② 단편화: 바인드 변수·CURSOR_SHARING → 커서 공유 회복
- FLUSH SHARED_POOL은 임시책(하드 파싱 급증) — 근본 아님

⚠ 파라미터 변경·재기동은 개인 환경 전용

실습 02·03·05 공통 안전 수칙

- SGA_TARGET·MEMORY_TARGET·SHARED_POOL_SIZE 변경과 재기동은 개인 DB만
- 공유 서버에서 잘못 주면 기동 실패(ORA-00838 등)·타 세션 즉시 영향
- ORA-04031 재현은 격리 환경에서, 재현 후 반드시 원복
- 절대값(1504M 등)은 환경 의존 <환경값> — 외울 것은 절차

29.8

In-Memory Column Store

- 존재와 적용 조건만
- 별도 유상 라이선스 옵션
- 메모리 파라미터·어드바이저는 라이선스 무관

In-Memory와 라이선스 경계

이 장은 존재·조건만, 실습 제외

- IM column store: 테이블을 컬럼 형식으로 SGA에 저장 → 스캔·집계 가속
- 버퍼 캐시를 대체하지 않고 보완(행·컬럼 병존), INMEMORY_SIZE로 설정
- 적합: 분석성 워크로드 / OLTP엔 이득 작고 유지 비용
- 🔒 별도 유상 옵션(Oracle Database In-Memory)
- 메모리 파라미터·어드바이저·V$SGA* 진단 뷰는 라이선스 무관

퀴즈 (Quiz)

- Q1. MEMORY_TARGET·SGA_TARGET 값으로 AMM·ASMM·수동을 판별하는 규칙과 실측 판정은?
- Q2. ASMM에서 DB_CACHE_SIZE=0이 '메모리가 없다'는 뜻이 아닌 이유는?
- Q3. 목표(target)와 상한(max), 동적/정적의 관계를 SGA_TARGET·SGA_MAX_SIZE로 설명하라.
- Q4. V$SGA_RESIZE_OPS에서 large pool GROW와 buffer cache SHRINK가 짝을 이루는 이유는?
- Q5. SGA 어드바이저 곡선(1504=1635, 3008=1635, 1128=1636, 752=2213)으로 어떤 결정을?
- Q6. ORA-04031의 두 원인(부족·단편화)과 각 해소책, FLUSH가 근본이 아닌 이유는?

요약 (Summary)

- 메모리 관리는 AMM(MEMORY_TARGET>0)·ASMM(=0, SGA_TARGET>0)·수동(둘 다 0) 세 방식.
- 실습 DB는 ASMM — SGA_TARGET=1504M, 개별 풀 0(자동), PGA_AGGREGATE_TARGET=700M.
- 파라미터는 목표(동적)·상한(정적)의 짝. MEMORY_TARGET과 SGA_TARGET을 혼동하면 기동 실패.
- V$SGAINFO·동적 구성으로 실제 배분·그래뉼(16MB), V$SGA_RESIZE_OPS로 GROW/SHRINK 짝 추적.
- 어드바이저 곡선으로 무릎·절벽 판정 — 실측은 1128MB까지 줄여도 DB Time 거의 그대로.
- ORA-04031은 부족·단편화에서 온다. 구성 조정·바인드 변수화로 해소. In-Memory는 라이선스 옵션.

실습 개요 (Practice Overview)

- 실습 01 — 메모리 관리 방식 판별(AMM/ASMM/수동)과 파라미터 확인(실측=ASMM)
- 실습 02 — AMM 구성과 MEMORY_TARGET 조정 후 재기동 확인(개인 환경 전용)
- 실습 03 — ASMM 전환과 SGA_TARGET·PGA_AGGREGATE_TARGET 동작 확인(개인 환경 전용)
- 실습 04 — 자동 조정 이력 추적(V$SGA_RESIZE_OPS·V$MEMORY_RESIZE_OPS)
- 실습 05 — 메모리 부족(ORA-04031) 재현 → 구성 조정으로 해소(개인 환경 전용)
- ※ 모두 non-CDB(orcl 19.3). 파라미터 변경·재기동은 반드시 개인 실습 환경에서만.

Day 9

어드바이저와 SGA

- TUN ch30 · 브리지 03 · ch31 · ch32
- 8h — 손 튜닝 SQL 을 STA 권고와 대조

Oracle Database 19c 성능 튜닝 실무 과정

Advisor 프레임워크와 자동 튜닝

자동 유지관리 작업 · SQL Tuning/Access Advisor · SPA · 권고 해석과 채택 판단 (ADMIN 29·30·33장 흡수·심화)

ch30

학습 목표 (Objectives)

- 세 가지 자동 유지관리 작업을 구분하고 DBA_AUTOTASK_*로 무엇이 언제 도는지 확인한다.
- 유지관리 윈도우가 업무 시간과 겹칠 때 윈도우 조정·리소스 할당으로 통제한다.
- Advisory Framework의 공통 구조(태스크·Finding·Recommendation)와 라이선스 경계를 안다.
- SQL Tuning Advisor 권고를 해석하고, '권고 없음'도 유효한 결과임을 이해한다.
- SQL Access Advisor·SPA로 워크로드 기준 설계·변경 검증을 수행한다.
- SQL Plan Directives·Adaptive Plans를 관찰하고 비활성화 시점을 판단한다.
- 어드바이저는 항상 권고하지 않으며, 권고는 검증할 가설임을 체득한다.

이 장의 구성 (Agenda)

- 30.2 자동 유지관리 작업과 유지관리 윈도우
- 30.3 Advisory Framework 전반
- 30.4 SQL Tuning Advisor — 두 실측 사례의 대조
- 30.5 SQL Access Advisor
- 30.6 SQL Performance Analyzer (SPA)
- 30.7 SQL Plan Directives와 Adaptive Plans
- 요약 · 퀴즈 · 실습 개요

이 장의 두 축과 ADMIN 심화

무엇이 흡수되고 무엇이 더해지나

- 축 1 — 자동으로 도는 것을 통제한다(자동 작업·윈도우·자원)
- 축 2 — 권고를 해석하고 채택을 판단한다(어드바이저)
- ADMIN 29·30장 = 자동 작업 '설정' → 여기선 업무 영향 '통제'
- ADMIN 33장 = 어드바이저 '소개' → 여기선 권고 '해석·채택 판단'
- 실측 두 사례로 '권고 맹신 경계'를 가르친다

30.2

자동 유지관리 작업과 유지관리 윈도우

- 세 가지 자동 작업 · 유지관리 윈도우
- 업무 시간 영향 통제 — 윈도우 조정·리소스 할당
- ADMIN 29·30장 '설정'을 '통제·판단'으로 심화

세 가지 자동 유지관리 작업

사전 예방적 유지관리 인프라

- auto optimizer stats collection — 낡은/없는 통계 자동 수집(가장 중요)
- auto space advisor — 세그먼트 공간 낭비 분석(권고만)
- sql tuning advisor — 고부하 SQL 자동 튜닝 분석 🔒 Tuning Pack
- 통계 수집·세그먼트 어드바이저는 팩 불필요 / SQL 튜닝만 Tuning Pack

무엇이 켜져 있나 — DBA_AUTOTASK_CLIENT

라이선스 무관 조회 (실측)

```
SYS@orcl> SELECT client_name, status, attributes
  2       FROM dba_autotask_client ORDER BY client_name;

CLIENT_NAME                     STATUS   ATTRIBUTES
------------------------------- -------- --------------------
auto optimizer stats collection   ENABLED  ON BY DEFAULT, ...
auto space advisor                ENABLED  ON BY DEFAULT, ...
sql tuning advisor                ENABLED  ONCE PER WINDOW, ...
```

언제 도나 — 유지관리 윈도우

DBA_AUTOTASK_WINDOW_CLIENTS (실측)

```
SYS@orcl> SELECT window_name, optimizer_stats,
  2         sql_tune_advisor, autotask_status
  3       FROM dba_autotask_window_clients WHERE rownum<=8;

WINDOW_NAME       OPTIMIZER_STATS  SQL_TUNE_ADVISOR  AUTOTASK
----------------- ---------------- ----------------- --------
MONDAY_WINDOW     ENABLED          ENABLED           ENABLED
...(요일별 7개)   ENABLED          ENABLED           ENABLED
```

업무 시간 영향 통제

윈도우가 업무·배치와 겹칠 때

- 방법 1 — 윈도우 시간을 옮긴다(DBMS_SCHEDULER.SET_ATTRIBUTE)
- 방법 2 — 자원을 조인다: DEFAULT_MAINTENANCE_PLAN의 ORA$AUTOTASK
- 기본 배분: ORA$AUTOTASK 25 / ORA$DIAGNOSTICS 5 / OTHER_GROUPS 70
- 리소스 변경은 대기 영역(Pending Area) 4단계 워크플로우 필수
- mgmt_p1은 상한이 아니라 '경합 시 보장 최소치'

자동 작업이 업무를 방해할 때의 순서

끄기 전에 확인·조정·조임

- ① 확인 — 윈도우가 언제 열리고 업무·배치와 겹치는가
- ② 조정 — 겹치면 윈도우 시간을 옮긴다(통계는 계속 최신)
- ③ 조임 — 못 옮기면 ORA$AUTOTASK 배분을 낮춘다
- ④ 최후에만 끈다 — 끄면 통계가 낡아 계획 품질 저하
- DBA_AUTOTASK_JOB_HISTORY로 실행 시각·소요를 대조한다

언제 얼마나 돌았나 — 실행 이력

DBA_AUTOTASK_JOB_HISTORY

```
SYS@orcl> SELECT client_name,
  2         TO_CHAR(window_start_time,'MM-DD HH24:MI') started,
  3         job_status, job_duration
  4       FROM dba_autotask_job_history
  5       WHERE window_start_time > SYSTIMESTAMP - 7
  6       ORDER BY window_start_time DESC FETCH FIRST 5 ROWS ONLY;

auto optimizer stats collection  <환경값>  SUCCEEDED  +000 00:0X
sql tuning advisor               <환경값>  SUCCEEDED  +000 00:0X
```

30.3

Advisory Framework 전반

- 어드바이저 종류 — DBA_ADVISOR_DEFINITIONS
- 공통 구조 — 태스크·Finding·Recommendation
- 라이선스 경계 — 무엇이 Tuning Pack인가

어드바이저 종류

DBA_ADVISOR_DEFINITIONS (실측)

```
SYS@orcl> SELECT advisor_name FROM dba_advisor_definitions
  2       ORDER BY advisor_name;

ADDM                          SQL Tuning Advisor
Compression Advisor           SQL Workload Manager
SPM Evolve Advisor            Segment Advisor
SQL Access Advisor            Statistics Advisor
SQL Performance Analyzer      Tune MView
SQL Repair Advisor            Undo Advisor

12 rows selected.
```

공통 아키텍처

모든 어드바이저가 같은 수명주기

- ① 태스크 생성 → ② 대상(SQL/워크로드) 연결 → ③ 실행 → ④ 결과 조회
- 결과 = Finding(무엇을 발견) + Recommendation(무엇을 하라)
- DBA_ADVISOR_TASKS / FINDINGS / RECOMMENDATIONS / ACTIONS / RATIONALE
- 패키지만 다름: DBMS_SQLTUNE · DBMS_ADVISOR · DBMS_SQLPA

🔒 라이선스 경계

- SQL Tuning/Access Advisor · SPA → Tuning Pack(Diagnostics 선행)
- 자동 SQL 튜닝 어드바이저 결과 조회 → Tuning Pack 경계
- DBA_AUTOTASK_* 조회 · 윈도우 조정 · 리소스 변경 → 팩 무관
- 대안: 인덱스 설계는 실행계획 분석, 계획 안정은 SQL Plan Baseline

30.4

SQL Tuning Advisor — 두 실측 사례

- 권고 종류: 통계·프로파일·인덱스·재작성
- 사례 1: '권고 없음'(이미 최적) / 사례 2: benefit 99.64%
- 핵심 — 어드바이저는 항상 권고하지 않는다

무엇을 권고하는가

네 종류, 우선순위로 읽는다

- 통계 — 가장 먼저, 가장 안전 (통계만 바꿔도 계획 개선)
- SQL 프로파일 — SQL 안 고치고 계획만 보정 (근본 해결 아님)
- 인덱스 — 단일 문장 기준, 워크로드 전체는 Access Advisor로
- 재작성 — 개발 협의 필요, 이 과정의 종착점(SQLT로 이관)

SQL 프로파일 — 매력과 함정

가장 손쉬워 보이지만 가장 신중히

- SQL 텍스트를 안 고치고 카디널리티 보정 힌트를 딕셔너리에 저장
- 즉시 효과 → 매력적. 그러나 데이터 분포 바뀌면 보정이 틀어짐
- SQL 자체의 나쁜 구조는 그대로 남는다 → 근본 해결 아님
- Common Error: SQL 프로파일을 근본 해결로 착각
- 응급 처치로 쓰고 근본 원인(통계·인덱스·구조)은 별도 추적

사례 1 — 권고가 없는 경우

SQL 9vmszgd1ft9x0 (전체 집계) 실측

```
SYS@orcl> t := DBMS_SQLTUNE.CREATE_TUNING_TASK(
             sql_id=>'9vmszgd1ft9x0', scope=>'COMPREHENSIVE');
SYS@orcl> DBMS_SQLTUNE.EXECUTE_TUNING_TASK(...);
SYS@orcl> SELECT DBMS_SQLTUNE.REPORT_TUNING_TASK(...) FROM dual;

SQL Text : select /*+ full(d) */ count(*), sum(d.amt)
           from claim_details d
-------------------------------------------------------------
There are no recommendations to improve the statement.
```

사례 2 — 인덱스 권고 (benefit 99.64%)

선택 조건 SQL, PK만 있어 전체 스캔

```
FINDINGS SECTION (1 finding)
1- Index Finding
  Recommendation (estimated benefit: 99.64%)
  create index SQLT.IDX$$_02860001 on SQLT.MEDICAL_CLAIMS
   ("HOSP_ID","RECEIPT_DATE","CLAIM_ID","TOTAL_AMT");

EXPLAIN PLANS SECTION – 1- Original
|*  1 |  TABLE ACCESS FULL| MEDICAL_CLAIMS | cost 856 |
```

두 사례의 대조 — 핵심 교훈

어드바이저는 항상 권고하지 않는다

- 같은 어드바이저·같은 실행 → 한쪽은 침묵, 한쪽은 99.64%
- 차이는 SQL 성격 — 전체 집계는 인덱스 무익, 소수 조회는 결정적
- 오판 1: '권고 없음 = 쓸모없음' (아니다, 이미 최적의 확인)
- 오판 2: 'benefit 99.64% = 무조건 생성' (단일 문장 기준일 뿐)
- 권고는 결론이 아니라 검증할 가설이다

benefit을 그대로 믿었다가

실무 시나리오

- 개발팀이 benefit 99.64%만 보고 인덱스를 운영에 생성
- 조회는 빨라졌으나 대량 INSERT 배치가 20% 느려짐
- 빠진 것: 인덱스 유지 비용(쓰기)을 보지 않음
- 어드바이저 rationale: '대표 워크로드로 Access Advisor를 돌려라'
- 실습은 권고 확인까지만 — 인덱스 미생성(Before 상태 유지)

자동 SQL 튜닝과 자동 채택

밤마다 도는 SYS_AUTO_SQL_TUNING_TASK

- 자동 작업이 고부하 SQL을 골라 같은 분석 수행
- REPORT_AUTO_TUNING_TASK로 결과 조회 (🔒 Tuning Pack)
- ACCEPT_SQL_PROFILES 기본 FALSE — 자동 채택 꺼짐(권장)
- 발견된 프로파일은 검토 후 수동 ACCEPT — 그래도 응급 처치

30.5

SQL Access Advisor

- SQL Tuning Advisor와의 차이 — 단일 vs 워크로드
- 인덱스·구체화 뷰 권고와 DDL 스크립트
- 채택 판단 — 유지 비용·공간·중복

SQL Tuning Advisor와 무엇이 다른가

층위가 다르다

- SQL Tuning: SQL 한 문장을 어떻게 잘 돌게 할까
- SQL Access: 워크로드 전체에 어떤 물리 구조가 최적일까
- 권고: 인덱스 + 구체화 뷰(MV) + MV 로그 + 파티셔닝
- 인덱스 유지 비용·공간까지 종합 고려 (DBMS_ADVISOR)

실행 흐름과 채택 판단

QUICK_TUNE / GET_TASK_SCRIPT (🔒 Tuning Pack)

```
SYS@orcl> EXEC DBMS_ADVISOR.QUICK_TUNE(
>   advisor_name=>DBMS_ADVISOR.SQLACCESS_ADVISOR,
>   task_name=>'QT_CLAIMS', attr1=>:sql);
SYS@orcl> SELECT DBMS_ADVISOR.GET_TASK_SCRIPT('QT_CLAIMS')
  2       FROM dual;   -- 권고 DDL 확인(적용 안 함)

채택 판단: 기존 인덱스와 중복? 쓰기 빈도? MV 갱신 주기?
         대표 워크로드로 돌렸나? 테스트 후 before/after 판정
```

30.6

SQL Performance Analyzer (SPA)

- 변경 전후를 SQL 집합으로 비교
- before / after / COMPARE PERFORMANCE
- 회귀 SQL만 골라 대응

변경 전후 비교 흐름

DBMS_SQLPA, STS 기반 (🔒 Tuning Pack)

```
-- (1) 분석 태스크 생성 (STS 기반)
EXEC :t:=DBMS_SQLPA.CREATE_ANALYSIS_TASK(sqlset_name=>'CLAIMS_STS',...);
-- (2) before 측정  execution_type=>'TEST EXECUTE'
-- (3) 환경 변경   ALTER SESSION SET optimizer_index_cost_adj=20;
-- (4) after 측정
-- (5) COMPARE PERFORMANCE (comparison_metric=elapsed_time)
-- (6) REPORT_ANALYSIS_TASK(type=>'TEXT', section=>'SUMMARY')
  Improved:<환경값>  Regressed:<환경값>  Unchanged:<환경값>
```

회귀 SQL 대응

전체 이득은 취하고 회귀만 막는다

- 변경이 전체 이득이어도 몇몇 SQL은 느려질 수 있다
- SPA의 가치: 변경 적용 전에 회귀 SQL을 미리 골라냄
- 대응: 회귀 SQL만 SQL Plan Baseline으로 기존 계획 고정
- Baseline은 SQL Plan Management 기능 — 팩 불필요

30.7

SQL Plan Directives와 Adaptive Plans

- 옵티마이저의 자기 보정 — 실행 중/이후
- Adaptive Plans(19c 기본 ON) · Directives
- 계획이 흔들릴 때의 대응 순서

Adaptive Plans — 실행 중 결정

19c 기본값 확인

```
SYS@orcl> SHOW PARAMETER optimizer_adaptive

NAME                                 TYPE      VALUE
------------------------------------ --------- -------
optimizer_adaptive_plans             boolean   TRUE
optimizer_adaptive_statistics        boolean   FALSE

계획(PLANS) ON / 통계(STATISTICS) OFF = 권장 기본
```

SQL Plan Directives — 다음 실행 메모

DBA_SQL_PLAN_DIRECTIVES

```
SYS@orcl> SELECT directive_id, type, state, reason
  2       FROM dba_sql_plan_directives WHERE ... MEDICAL_CLAIMS;

DIRECTIVE_ID  TYPE             STATE   REASON
------------  ---------------- ------- ----------------------
<환경값>      DYNAMIC_SAMPLING USABLE  SINGLE TABLE CARDINALITY
                                       MISESTIMATE

근본 해결 = 상관관계 컬럼에 컬럼 그룹 확장 통계(ch21)
```

계획이 흔들릴 때의 대응 순서

전역 비활성화는 최후

- ① 근본 해소 — 확장 통계·통계 재수집으로 원인 제거
- ② 특정 SQL만 고정 — SQL Plan Baseline(팩 불필요)
- ③ 전역 비활성화는 최후 — 끄면 다른 SQL 이득도 사라짐
- 판단 기준은 언제나 '한 SQL 편의'가 아니라 '워크로드 전체'

퀴즈 (Quiz)

- Q1. 세 자동 유지관리 작업과 그중 Tuning Pack이 필요한 것은?
- Q2. 윈도우가 업무와 겹칠 때 대응 두 방향과 ORA$AUTOTASK 기본 배분값은?
- Q3. 'no recommendations'와 benefit 99.64%의 차이가 왜 생기나?
- Q4. benefit 99.64%를 곧바로 인덱스로 만들면 안 되는 이유는?
- Q5. SQL Tuning Advisor와 SQL Access Advisor의 입력·관점 차이는?
- Q6. SQL 프로파일이 근본 해결이 아닌 이유와 ACCEPT_SQL_PROFILES 권장값은?

요약 (Summary)

- 자동 작업 셋은 DBA_AUTOTASK_*로 무엇이 언제 도는지 먼저 확인한다.
- 업무와 겹치면 윈도우 조정→자원 조임→최후에 끄기(대기 영역 4단계).
- Advisory Framework는 태스크→Finding→Recommendation 공통 구조.
- 실측: 전체 집계 SQL은 '권고 없음', 선택 조건 SQL은 인덱스 99.64%.
- 권고 없음도 결과, 큰 benefit도 단일 문장 기준 — 워크로드로 판단.
- Access Advisor=워크로드 설계 / SPA=변경 검증·회귀 탐지.
- Adaptive/Directives는 자기 보정 — 확장 통계→Baseline→전역 끄기 순.

실습 개요 (Practice Overview)

- 실습 01 — 자동 유지관리 작업 구성 확인(DBA_AUTOTASK_*)
- 실습 02 — 유지관리 윈도우 조정·리소스 할당 변경
- 실습 03 — Advisory Framework 전반(DBA_ADVISOR_DEFINITIONS)
- 실습 04 — SQL Tuning Advisor ① '권고 없음' 확인 🔒
- 실습 05 — SQL Tuning Advisor ② 인덱스 권고(99.64%) 🔒
- 실습 06 — 자동 SQL 튜닝 결과·채택 정책 🔒
- 실습 07 — SQL Access Advisor / SPA 🔒
- 실습 08 — SQL Plan Directives·Adaptive Plans 관찰

Oracle Database 19c 성능 튜닝 실무 과정

SGA 튜닝 ① 버퍼 캐시와 리두 로그 버퍼 Tuning the Database Buffer Cache — 히트율의 함정

버퍼 캐시 구조 · 히트율의 함정 · V$DB_CACHE_ADVICE · KEEP/RECYCLE 풀 · 리두 로그 버퍼

ch31

학습 목표 (Objectives)

- 버퍼 캐시 구조(DEFAULT·KEEP·RECYCLE, LRU, DBWR)와 논리·물리 읽기를 구분한다.
- 히트율의 함정 — 100%에 가까워도 좋은 상태가 아닐 수 있음을 실측으로 설명한다.
- 반복 스캔이 히트율을 부풀리는 원리와 대량 논리 읽기의 비효율을 판별한다.
- V$DB_CACHE_ADVICE로 크기 변경 효과를 예측하고 수렴 구간을 읽는다.
- 다중 버퍼 풀(KEEP=소형 참조, RECYCLE=대량 스캔) 배치 기준을 세운다.
- 리두 로그 버퍼 동작과 log buffer space 대기·재시도 비율을 판독한다.

이 장의 구성 (Agenda)

- 31.2 버퍼 캐시 구조 — 무엇을, 왜 캐싱하는가
- 31.3 히트율의 함정 — 이 장의 핵심
- 31.4 V$DB_CACHE_ADVICE — 크기 변경 효과 예측
- 31.5 다중 버퍼 풀 — KEEP과 RECYCLE
- 31.6 캐싱 모드와 대량 스캔의 캐시 동작
- 31.7 리두 로그 버퍼 — log buffer space 대기

31.2

버퍼 캐시 구조

- 논리 읽기 vs 물리 읽기
- LRU·touch count·DBWR
- DEFAULT/KEEP/RECYCLE 풀

SGA 안에서 버퍼 캐시의 자리

이 장이 다루는 두 구조

- SGA = 버퍼 캐시 + 공유 풀 + 라지/자바/스트림 풀 + 리두 버퍼
- 버퍼 캐시와 공유 풀이 대개 가장 큰 비중
- ASMM/AMM가 자동 조정해도 각 구조의 원리는 알아야 판단 가능
- 이 장: 버퍼 캐시(읽기) + 리두 로그 버퍼(쓰기)

무엇을, 왜 캐싱하는가

버퍼 캐시의 목적

- 버퍼 캐시 = SGA 안 데이터 블록의 사본 저장소
- 캐시 히트 → 메모리에서 바로(논리 읽기)
- 캐시 미스 → 데이터 파일에서 읽어 올림(물리 읽기)
- 목적: 자주 쓰는 블록을 잡아 둬 물리 I/O를 줄인다

논리 읽기와 물리 읽기

이 장의 토대가 되는 구분

- 논리 읽기 = db block gets + consistent gets (요청 총량)
- 물리 읽기 = 캐시에 없어 디스크까지 내려간 부분
- 물리 읽기는 언제나 논리 읽기의 부분집합
- 히트율은 '물리 회피율'만 말할 뿐, 논리 읽기 낭비는 못 본다

LRU와 DBWR

누구를 캐시에 남길 것인가

- LRU + 접촉 횟수(touch count)로 남길 블록을 결정
- 자주 쓰면 앞(MRU), 오래 안 쓰면 뒤(LRU)로 밀려 재사용
- 변경 블록(dirty)은 DBWR가 데이터 파일에 쓴 뒤 재사용
- WAL 원칙: 리두를 먼저 기록하고 블록을 나중에 쓴다

풀별 크기·히트율 실측

V$BUFFER_POOL_STATISTICS (별칭 충돌 수정 후)

```
SYS@orcl> SELECT name, set_msize buf_cnt,
  2    ROUND(set_msize*block_size/1024/1024,1) mb,
  3    ROUND(100*(1-physical_reads/
  4      GREATEST(db_block_gets+consistent_gets,1)),2) hit_pct
  5  FROM v$buffer_pool_statistics WHERE set_msize>0;

NAME       BUF_CNT     MB    HIT_PCT
-------- --------- ------ ----------
DEFAULT     11268     44        100
DEFAULT    105732    826      99.98
KEEP         5874   45.9        100

-- 첫 시도는 별칭 buffers가 예약어 충돌로 ORA-00904 → buf_cnt로 수정.
```

31.3

히트율의 함정

- 100%인데 나쁠 수 있다
- 나쁜 SQL이 히트율을 올린다
- 실측 99.98% / 논리 3.7억

히트율의 역설

이 장에서 가장 중요한 것

- 히트율 = 1 – 물리 읽기 / 논리 읽기
- 100%에 가까워도 시스템은 나쁜 상태일 수 있다
- 심지어 나쁜 SQL일수록 히트율은 더 높아진다
- 히트율은 잘못 읽으면 문제를 가려 버리는 지표

실측으로 계산하기

V$SYSSTAT 원자료 → 99.98%

```
SYS@orcl> SELECT name, value FROM v$sysstat
  2  WHERE name IN ('db block gets','consistent gets',
  3                 'physical reads','session logical reads');

NAME                      VALUE
--------------------- -----------
consistent gets       374267229
db block gets            618971
physical reads            75765
session logical reads 374886200

-- 히트율 = 1 - 75,765 / (618,971 + 374,267,229) = 99.98%
-- 그런데 논리 읽기가 3.7억 회! 물리는 7.5만. 무엇이 문제인가?
```

나쁜 SQL이 히트율을 올린다

반복 스캔의 산수

- 100블록을 1번 읽으면: 논리 100, 물리 100, 히트율 0%
- 같은 100블록을 1,000번 읽으면: 논리 100,000, 물리 100
- → 히트율 = 1 − 100/100,000 = 99.9%로 치솟음
- 반복이 심할수록 히트율은 100%에 수렴한다

왜 대량 논리 읽기가 느린가

물리 I/O가 없어도 느리다

- 논리 읽기 한 번도 CPU와 버퍼 래치 비용이 든다
- 초당 수십만 논리 읽기 → CPU가 묶인다
- 인기 블록 동시 접근 → cache buffers chains·buffer busy
- 히트율 100% = 디스크를 안 쓴다 ≠ 빠르다

히트율 신앙은 왜 끈질긴가

그리고 함께 봐야 할 것

- 이유1: 단 하나의 숫자라 직관적이고 보고하기 쉽다
- 이유2: 메모리가 귀하던 시절엔 물리 I/O가 지배적 병목이었다
- 지금은 대부분 OLTP가 이미 99%+ → 변별력 상실
- 함께 볼 것: 논리 읽기 총량·추세 / 상위 buffer_gets SQL / CPU 비중

실무 시나리오

히트율 99.99%인데 화면이 느리다

- 상황: 히트율 99.99% 확인 후 '애플리케이션 문제'로 결론
- 놓친 것: 이 SQL의 실행당 논리 읽기가 수십만
- 반복 읽는 비효율 실행계획 → 논리 폭증, 히트율은 100% 수렴
- 조치: 캐시 확장이 아니라 실행계획을 고친다

31.4

버퍼 캐시 어드바이저

- 크기→물리 읽기를 함수로
- 물리 읽기 계수
- 480MB에서 수렴

왜 히트율이 아니라 예측인가

결과값 vs 함수형 데이터

- 히트율: '지금 이 크기에서 얼마나 적중했나' — 결과값 하나
- 어드바이스: '크기를 바꾸면 어떻게 될까' — 크기의 함수
- 튜닝은 '무엇을 바꾸면 무엇이 달라지나'를 다루는 일
- 기여하지 않는 버퍼는 낭비 — 그 구간을 어드바이스가 보여준다

V$DB_CACHE_ADVICE 실측

크기별 물리 읽기 예측 (DEFAULT 8K)

```
SYS@orcl> SELECT ROUND(size_for_estimate) cache_mb,
  2    size_factor factor, estd_physical_read_factor rd_factor,
  3    estd_physical_reads
  4  FROM v$db_cache_advice
  5  WHERE name='DEFAULT' AND block_size=8192
  6  ORDER BY size_for_estimate;

 CACHE_MB   FACTOR  RD_FACTOR  ESTD_PHYSICAL_READS
--------- -------- ---------- --------------------
       80   0.0926   446.8807             33631793
      160   0.1852     2.6614               200293
      320   0.3704     1.1634                87559
      480   0.5556     1.0000                75259  <= 수렴
      864   1.0000     1.0000                75259  (현재)
     1600   1.8519     1.0000                75259
```

어드바이스를 읽는 법

세 구간으로 나눠 읽는다

- 위험 구간(80MB): 계수 447배 — 절대 줄이면 안 됨
- 개선 구간(160~400MB): 계수 2.66→1.0, 키우면 물리 읽기 감소
- 수렴 구간(480MB↑): 계수 1.0, 더 키워도 소용없음
- 결론: 현재 864MB는 이미 충분. 회수해도 물리 읽기 불변

31.5

다중 버퍼 풀

- KEEP=소형 참조 상주
- RECYCLE=대량 스캔 격리
- DRUG_MASTER 사례

왜 풀을 나누는가

대형 스캔이 따뜻한 블록을 밀어낸다

- 하나의 풀 → 성격 반대인 객체가 같은 LRU에서 경쟁
- 대형 세그먼트 스캔이 소형 마스터의 따뜻한 블록을 축출
- 비순차 물리 읽기의 10% 초과 세그먼트 = '아주 크다'
- 해법: KEEP·RECYCLE로 물리적 분리

KEEP / RECYCLE / DEFAULT

각 풀의 역할

- KEEP: 항상 상주, 미스 최소화 — 소형 코드·마스터 테이블
- RECYCLE: 빨리 버림, 격리 — 대량 스캔 대형 세그먼트
- DEFAULT: 나머지 일반 객체
- 실측: db_keep_cache_size=48M 이미 설정, recycle=0

풀 파라미터 실측

KEEP은 명시, DEFAULT는 ASMM 자동

```
SYS@orcl> SELECT name, display_value FROM v$parameter
  2  WHERE name IN ('db_cache_size','db_keep_cache_size',
  3                 'db_recycle_cache_size') ORDER BY name;

NAME                      DISPLAY_VALUE
------------------------- --------------
db_cache_size             0
db_keep_cache_size        48M
db_recycle_cache_size     0

-- db_cache_size=0 → ASMM가 DEFAULT를 자동 관리(sga_target>0)
-- KEEP/RECYCLE은 자동 대상이 아니라 명시적으로 크기를 준다.
```

DRUG_MASTER를 KEEP에

조치 전 → 조치 → 조치 후 예열

```
SYS@orcl> -- (조치 전) 현재 풀 확인
SYS@orcl> SELECT buffer_pool FROM dba_tables
  2  WHERE owner='SQLT' AND table_name='DRUG_MASTER';
BUFFER_POOL --> DEFAULT

SYS@orcl> -- (조치) KEEP 풀로 지정
SYS@orcl> ALTER TABLE sqlt.drug_master
  2         STORAGE (BUFFER_POOL KEEP);
Table altered.

SYS@orcl> -- (조치 후) 재확인 + 전체 스캔으로 예열
SYS@orcl> SELECT /*+ FULL(d) */ COUNT(*) FROM sqlt.drug_master d;
```

RECYCLE로 대량 스캔 격리

DEFAULT 풀 오염 방지

```
SYS@orcl> -- 대량 스캔 대형 세그먼트를 RECYCLE로 격리
SYS@orcl> ALTER TABLE sqlt.claim_detail_archive
  2         STORAGE (BUFFER_POOL RECYCLE);
Table altered.

SYS@orcl> ALTER INDEX sqlt.ix_claim_arch_dt
  2         STORAGE (BUFFER_POOL RECYCLE);
Index altered.

-- 판정: 이 세그먼트의 넓은 스캔이 DEFAULT의 '따뜻한' 블록을
--   밀어내는 현상이 사라지는지 관찰(상위 SQL 물리 읽기 비교).
```

풀 배치 기준

Best Practice

- KEEP: 작고 자주 참조되는 코드·마스터, 예측 가능한 응답시간
- RECYCLE: 비순차 물리 읽기 10% 초과 대형 세그먼트
- 먼저 SQL 튜닝 — 비선택적 인덱스는 선택적 인덱스로
- 풀 크기는 각 풀에 대해 V$DB_CACHE_ADVICE로 따로 산정

31.6

캐싱 모드와 대량 스캔

- CACHE / NOCACHE
- LRU 꼬리 배치
- 직접 경로 읽기

같은 100%, 다른 의미

맥락이 해석을 바꾼다

- DEFAULT 풀 99.98%: 반복 스캔으로 부풀려졌을 수 있다
- KEEP 풀 100%: 작고 자주 쓰는 것만 담아 항상 적중(정상)
- 직접 경로 읽기 대형 스캔: 히트율에 아예 안 잡힘
- CACHE는 LRU 위치만 바꿈 / KEEP은 전용 풀로 격리

31.7

리두 로그 버퍼

- LGWR 기록 조건
- log buffer space 대기
- 재시도 비율로 판단

리두 로그 버퍼 동작

LGWR는 언제 쓰는가

- 데이터 블록 변경 → 리두가 리두 로그 버퍼에 기록됨
- LGWR 기록 조건: ①버퍼 1/3 참 ②COMMIT/ROLLBACK ③DBWR 요청
- 공간 없으면 서버 프로세스가 log buffer space로 대기
- 버퍼 캐시와 반대: 대개 작아도 충분, 키우면 손해일 수도

리두 지표 실측

재시도를 리두 항목과 비교

```
SYS@orcl> SELECT 'log_buffer' name, display_value val
  2  FROM v$parameter WHERE name='log_buffer'
  3  UNION ALL
  4  SELECT stat.name, TO_CHAR(stat.value) FROM v$sysstat stat
  5  WHERE stat.name IN ('redo size','redo entries',
  6                      'redo buffer allocation retries');

NAME                           VAL
------------------------------ ----------
log_buffer                     7360K
redo size                      129961816
redo entries                   249457
redo buffer allocation retries 3

-- retries 3 / entries 249,457 = 0.0012% → 버퍼 충분!
```

직접 경로 읽기와 소형 테이블 임계

실행계획은 그대로인데 느려졌다면

- 소형 테이블 임계 = max(버퍼 캐시의 2%, 20블록)
- 임계 초과 대형 스캔 → LRU 꼬리 or 직접 경로(PGA)로 우회
- 19c 적응적 직접 경로 읽기: 상황 따라 경로 전환
- 경로 전환 시 물리 읽기·direct path read 대기가 갑자기 증가

리두 버퍼 판정 규율

버퍼냐 I/O냐를 가른다

- retries / entries 비율로 충분성 판단(경험칙 1% 미만이면 충분)
- log buffer space 상위 → log file parallel write도 함께 확인
- 후자가 높으면 병목은 버퍼가 아니라 리두 I/O(디스크)
- 근거 없이 log_buffer 키우지 않는다 / 커밋 빈도 최적화 우선

퀴즈 (Quiz)

- Q1. 히트율 공식을 쓰고, 99.98%인데도 좋은 상태가 아닐 수 있는 이유를 실측으로 설명하라.
- Q2. 비효율 SQL의 반복 스캔이 히트율을 '오히려 올리는' 원리를 설명하라.
- Q3. V$DB_CACHE_ADVICE 계수가 480MB에서 1.0으로 수렴했다. 어떤 결론을 내릴 수 있는가?
- Q4. KEEP과 RECYCLE의 용도를 쓰고, DRUG_MASTER를 어느 풀에 둘지 이유와 함께 답하라.
- Q5. BUFFER_POOL KEEP 지정 직후 전체 스캔으로 예열하는 이유는?
- Q6. redo buffer allocation retries를 무엇과 비교해 리두 버퍼 크기를 판단하는가?

요약 (Summary)

- 히트율 = 1 − 물리/논리. 물리 회피율일 뿐, 논리 읽기 낭비는 못 본다.
- 실측 99.98%지만 논리 읽기 3.7억. 나쁜 SQL이 히트율을 오히려 올린다.
- 크기 판단은 히트율이 아니라 V$DB_CACHE_ADVICE로. 480MB에서 수렴.
- 다중 풀: 소형 참조는 KEEP(48M 설정), 대량 스캔은 RECYCLE로 격리.
- 같은 100%도 KEEP과 DEFAULT는 의미가 다르다. 직접 경로는 히트율에 안 잡힌다.
- 리두 버퍼: retries/entries 비율(실측 3/249,457)로 충분성을 판단한다.

실습 개요 (Practice Overview)

- 실습 01 — 버퍼 캐시 구성·크기·사용률 확인(V$BUFFER_POOL_STATISTICS, ORA-00904 수정)
- 실습 02 — 히트율의 함정 재현(WL_02_io, 조치 전후 히트율·논리 읽기 비교)
- 실습 03 — 버퍼 캐시 어드바이저 판독(V$DB_CACHE_ADVICE, 수렴 구간)
- 실습 04 — 다중 버퍼 풀 KEEP 구성(DRUG_MASTER 배치·예열)
- 실습 05 — RECYCLE 풀과 대량 스캔 분리
- 실습 06 — 리두 로그 버퍼와 log buffer space 대기(재시도 비율)

Oracle Database 19c 성능 튜닝 실무 과정

SGA 튜닝 ② 공유 풀과 라지 풀

Shared Pool & Large Pool — 부족의 진짜 원인은 파싱이다

라이브러리 캐시 · 하드 파싱 · CURSOR_SHARING · V$SHARED_POOL_ADVICE · ORA-04031 · 라지 풀 분리

ch32

학습 목표 (Objectives)

- V$SGASTAT으로 공유 풀 구성을 읽고 파편화·부풀림을 판별한다.
- V$LIBRARYCACHE의 히트율·RELOADS·INVALIDATIONS로 커서 재사용을 진단한다.
- 파싱 통계로 하드 파싱 비율을 계산하고 리터럴 SQL을 지목한다.
- CURSOR_SHARING이 공유 풀 점유를 어떻게 바꾸는지 이해하고 한계를 안다.
- V$SHARED_POOL_ADVICE로 '크기 확대가 유효한가'를 판정한다.
- ORA-04031을 메시지로 원인 영역부터 식별하고 순서대로 해소한다.
- 라지 풀을 분리해 병렬·RMAN 버퍼가 공유 풀을 침범하지 않게 한다.

이 장의 구성 (Agenda)

- 32.2 공유 풀은 무엇으로 차 있는가 — V$SGASTAT
- 32.3 라이브러리 캐시 판독 — V$LIBRARYCACHE
- 32.4 하드 파싱 — 공유 풀 부족의 진짜 원인
- 32.5 커서 공유 — CURSOR_SHARING
- 32.6 공유 풀 어드바이저 — V$SHARED_POOL_ADVICE
- 32.7 ORA-04031 재현과 해소
- 32.8 라지 풀 분리 — 공유 풀을 지키는 방벽
- 32.9~32.13 체크리스트 · 오판 · 라이선스 · 요약

공유 풀을 왜 따로 보는가

ch32의 한 문장

- 버퍼 캐시=데이터, 공유 풀=데이터를 다루는 '방법'(SQL·커서)
- 공유 풀이 아프면 디스크가 아니라 파싱이 CPU를 태운다
- 핵심: 공유 풀 부족의 진짜 원인은 대개 크기가 아니라 파싱
- ORA-04031을 크기만 키워 임시 봉합하지 않는다
- 도구 3층: V$SGASTAT → V$LIBRARYCACHE·파싱통계 → 어드바이저
- ch29~ch32 SGA 3부작의 마지막 · 다음은 ch33 결과캐시·PGA

32.2

공유 풀 구성 — V$SGASTAT

- 공유 풀을 구성 요소 단위로 쪼개 본다
- SQLA=공유 SQL, KGLH0/KGLS=라이브러리 캐시 힙
- free memory 총량이 넉넉해도 파편화로 ORA-04031
- row cache=데이터 사전 캐시도 공유 풀에 있다

공유 풀 구성 실측

V$SGASTAT (pool='shared pool' 상위)

```
NAME                           MB
------------------------- ------
free memory                67.22
SQLA                       30.71
KGLH0                      25.82
ksunfy_meta 1              19.74
ASH buffers                16.00
SO private sga             13.49
row cache mutex             8.60
KGLS                        7.49
```

free memory가 넉넉한데 ORA-04031?

총량이 아니라 파편화가 문제

- ORA-04031 = 요청한 크기의 '연속' 공간 할당 실패
- 리터럴 커서 수만 개 → 공유 풀을 잘게 파편화
- 합계 여유 60MB인데 5KB 연속 청크 하나가 없다
- → free memory 총량은 ORA-04031을 배제하지 못한다
- 크기 확대는 며칠 봉합, 근본은 바인드 변수

32.3

라이브러리 캐시 판독

- GET_HIT/PIN_HIT=커서 재사용 성공률
- RELOADS=공간 압박, INVALIDATIONS=객체 변경
- SQL AREA 낮고 TABLE/PROCEDURE 높으면 리터럴 신호
- 누적값이므로 델타로 진단한다

라이브러리 캐시 실측

V$LIBRARYCACHE (gets 상위)

```
NAMESPACE          GETS  GET_HIT  PIN_HIT RELOADS INVALID
--------------- ------- -------- -------- ------- -------
SQL AREA         304311     13.9     44.6    2441     699
TABLE/PROCEDURE  163666     95.2     97.2    4318       0
SCHEMA            10179     99.7    100.0       0       0
BODY               2174     93.6     97.1     106       0
INDEX              1216     84.4     70.8     163       0
```

RELOADS와 INVALIDATIONS는 다르다

원인이 다르니 조치도 갈린다

- RELOADS = 공간 압박: 커서가 밀려났다 재적재(재파싱)
- → 공유 풀이 좁거나 리터럴이 유효 커서를 밀어냄
- INVALIDATIONS = 변경: DDL·통계 재수집으로 커서 무효화
- → 업무 시간 통계수집·DDL 남발 점검
- 네임스페이스로 '어느 종류 객체가 압박받나' 좁힘
- BODY RELOADS 튀면 → 대형 패키지 KEEP 고정 검토

GET과 PIN, 그리고 낮은 값의 의미

이중고: 새 커서 + 밀려남

- GET=핸들 찾기, PIN=실행하려 고정하기
- PINS(70만) > GETS(30만): 한 번 찾아 여러 번 실행 (자연)
- GET_HIT 낮음 = 찾기 자체 실패(새 SQL이라 없음)
- PIN_HIT 낮음 = 찾았는데 재파싱 필요(밀려났다/무효화)
- 실측은 둘 다 낮음 → 리터럴+밀려남 이중고

32.4

하드 파싱 — 진짜 원인

- 소프트=커서 재사용, 하드=처음부터 파싱
- 하드 파싱 비율 = hard/total, 정상은 0에 수렴
- 실측 134042/218056 ≈ 61.5% → 리터럴 폭증
- 악순환의 종착역이 ORA-04031

파싱 통계 실측

V$SYSSTAT

```
NAME                           VALUE
---------------------------- ---------
parse count (hard)              134042
parse count (total)             218056
parse time cpu                    6804
session cursor cache hits       179634
```

하드 파싱이 공유 풀을 갉는 악순환

리터럴 → 일회용 커서 → 파편화 → 재파싱

- 리터럴 'id=1','id=2',... → 전부 다른 SQL로 각각 하드파싱
- ① CPU가 파싱에 낭비 (parse time cpu ↑)
- ② 공유 풀이 일회용 커서로 부풀음 (SQLA·KGLH0 ↑)
- ③ 새 커서 공간 위해 유효 커서 밀어냄 (RELOADS ↑)
- → 밀려난 커서 재파싱 → 악순환 → 종착역 ORA-04031
- 크기 확대는 악순환 속도만 늦출 뿐, 방향은 그대로

파싱 단계와 경합 지점

공유 풀 문제의 세 얼굴

- 하드 파싱 5단계: 구문→의미→커서탐색(실패)→최적화→할당
- 할당 단계에서 라이브러리 캐시 뮤텍스·shared pool 래치 잡음
- 여러 세션 동시 하드파싱 → 이 직렬화 지점에서 줄서기
- 대기로 나타남: library cache: mutex X / cursor: pin S wait on X
- 세 얼굴: 낮은 히트율 + 높은 하드파싱% + 뮤텍스·래치 경합

32.5

커서 공유 — CURSOR_SHARING

- EXACT(기본): 텍스트 다르면 다른 커서 → 하드파싱
- FORCE: 리터럴을 시스템 바인드로 치환 → 하드파싱 급감
- SIMILAR: 12c부터 폐기, 쓰지 않는다
- FORCE는 임시방편, 근본은 앱의 바인드 변수

바인드 변수 vs CURSOR_SHARING

정공법과 사후 보정

- 바인드 변수 = 앱이 값을 :id로 넘김 → 커서 하나만 생김(정공법)
- CURSOR_SHARING=FORCE = 오라클이 실행 시점에 바꿔치기(사후 보정)
- FORCE는 '앱 못 고칠 때'의 차선책
- 성능·계획 품질은 앱 수정 > FORCE
- ch16 바인드 변수를 '공유 풀 점유' 관점에서 다시 봄

CURSOR_SHARING=FORCE 치환 확인

리터럴이 시스템 바인드로 바뀐 흔적

```
SYS@orcl> ALTER SESSION SET cursor_sharing=FORCE;

SQLT@orcl> SELECT sql_text FROM v$sql
  2       WHERE sql_text LIKE '%patient_id=%' AND ROWNUM<=1;

SQL_TEXT
----------------------------------------
SELECT ... WHERE patient_id=:"SYS_B_0"
```

FORCE의 함정 — 편중 컬럼

급한 불은 끄되 계획을 재검증

- FORCE는 하드파싱·공유풀 점유를 줄이는 강력한 임시방편
- 그러나 편중(skew) 컬럼에선 하나의 계획 공유 → 성능 저하 가능
- 예: HOSP_ID처럼 값마다 건수 다른 컬럼
- 원칙: FORCE는 세션·시스템 임시조치, 근본은 바인드 전환
- 적용 후 반드시 주요 SQL 계획 재검증

32.6

공유 풀 어드바이저

- 크기를 바꾸면 파싱 절약이 얼마나 변할지 시뮬레이션
- SAVED 곡선의 무릎(knee)을 찾는다
- 실측: 416MB에서 무릎 — 줄이면 손해, 늘려도 이득 없음
- '크기로 풀 문제가 아니다'를 숫자로 증명

공유 풀 어드바이저 실측

V$SHARED_POOL_ADVICE

```
   SP_MB   FACTOR    SAVED
-------- -------- --------
     368   0.8846   0.6424
     384   0.9231   0.8626
     400   0.9615   0.9363
     416   1.0000   1.0000   <-- 현재
     464   1.1154   1.0000
     560   1.3462   1.0000
     608   1.4615   1.0007
```

어드바이저가 말하는 것과 아닌 것

곡선의 모양이 방향을 정한다

- 말하는 것: 416MB 적정 — 줄이면 손해, 늘려도 이득 없음
- 말 안 하는 것: 왜 파싱이 많은지(리터럴)는 모름
- 무릎 이전=확대 유효, 무릎 이후=낭비 (ch31 V$DB_CACHE_ADVICE와 동일)
- 곡선 가파르면 확대, 평평하면 파싱 제거 — 데이터가 방향 결정
- ORA-04031 대응의 관문: 확대가 유효한지부터 판정

32.7

ORA-04031 재현과 해소

- 성능 저하가 아니라 '기능 실패' — SQL이 못 돎
- 메시지가 영역·대상·하위영역·함수를 담는다
- 원인 영역부터 식별 → 어드바이저로 확대 유효성 판정
- 순위: 바인드 → CURSOR_SHARING → 예약풀 → 크기(최후)

ORA-04031 메시지 해부

괄호 네 조각을 읽는다

```
ORA-04031: unable to allocate 4160 bytes of shared memory
  ("shared pool","SELECT ... WL01_LIT ...",
   "SQLA","kkscsAddChildNode")

   shared pool  = 실패한 영역 (large pool이면 라지 풀)
   SELECT WL01  = 실패 유발 대상 (리터럴 커서)
   SQLA         = 공유 SQL 영역 하위에서 할당 시도
   kkscsAdd...  = 자식 커서 추가 함수
```

해소 순서 — 근본부터

크기 확대는 최후, 처음이 아니다

- 1(근본) 하드파싱 제거 — 앱 바인드 변수 전환
- 2(완화) CURSOR_SHARING=FORCE — 커서 폭증 차단
- 3(완충) 예약 풀(SHARED_POOL_RESERVED_SIZE) 조정
- 4(최후) SHARED_POOL_SIZE/SGA_TARGET 확대 — 어드바이저가 유효 보일 때만
- 재현→원인영역 식별→확증→해소→재확인이 실습5의 뼈대

32.8

라지 풀 분리

- 병렬·RMAN·공유서버의 큰 버퍼 담당
- 없으면 큰 할당이 공유 풀 침범 → 커서 밀어냄
- 라지 풀엔 LRU 없음 — 큰 할당에 파편화 적음
- PX msg pool이 라지 풀에 있다 = 공유 풀 보호 중

라지 풀 실측

V$SGASTAT (pool='large pool')

```
NAME                MB
-------------- ------
PX msg pool     15.00
free memory      0.69
session heap     0.31
miscellaneous    0.06
```

라지 풀을 언제 명시적으로 구성하나

병렬·RMAN·공유서버가 있으면

- 병렬 실행: PX 메시지 버퍼 → 없으면 공유 풀 점유
- RMAN: 백업·복구 I/O 버퍼 → 없으면 백업시간대 파편화·ORA-04031
- 공유 서버: 세션 UGA → 없으면 세션 메모리가 공유 풀 잠식
- ASMM: LARGE_POOL_SIZE를 하한으로 잡으면 자동조정이 그 위에서
- 순수 OLTP(병렬·백업 없음)면 라지 풀 작아도 무방

공유 풀 진단 체크리스트

각 단계가 앞 가설을 이어받아 좁힌다

- 1. V$SGASTAT — SQLA·KGLH0 부풀고 free 조각났나(free 67MB나 파편화)
- 2. V$LIBRARYCACHE — GET/PIN_HIT 낮은가(13.9%/44.6%)
- 3. 파싱 통계 — 하드파싱 비율 높은가(61.5%)
- 4. V$SHARED_POOL_ADVICE — 확대 유효한가(곡선 평평→아님)
- 5. 원인 조치 — 바인드→CURSOR_SHARING→예약풀→크기
- 6. V$SGASTAT(large) — 병렬·RMAN 버퍼 분리됐나(PX msg 15MB)

실측 한눈에 — 하나의 원인으로 수렴

서로 다른 뷰의 숫자가 한 곳을 가리킨다

- V$SGASTAT: SQLA 30.71MB·KGLH0 25.82MB로 커서가 공유 풀 점유
- V$LIBRARYCACHE: SQL AREA GET_HIT 13.9%, RELOADS 2441
- 파싱 통계: 하드파싱 비율 61.5% (134042/218056)
- V$SHARED_POOL_ADVICE: 확대 곡선 평평(SAVED 1.0 고정)
- V$SGASTAT(large): PX msg pool 15MB 분리로 공유 풀 보호
- → 결론: 리터럴 하드파싱이 공유 풀을 압박 · 답은 바인드 변수

이 장이 경계하는 오판 넷

공유 풀 튜닝의 함정

- ① ORA-04031을 크기만 키워 봉합 → 원인이 파싱이면 재발
- ② free memory 넉넉하니 공유 풀 충분 → 파편화 무시
- ③ 라이브러리 캐시 히트율 낮으니 무조건 확대 → 어드바이저부터
- ④ CURSOR_SHARING=FORCE면 만사형통 → 편중 컬럼 계획 저하
- 공통 처방: 원인 영역 식별 + 어드바이저 판정 + 순서 준수

라이선스 노트

공유 풀 진단은 대부분 라이선스 무관

- 무관: V$LIBRARYCACHE·V$SGASTAT·V$SYSSTAT·V$SQLAREA
- 무관: V$SHARED_POOL_ADVICE 어드바이저 뷰도 조회 가능
- 무관: CURSOR_SHARING·LARGE_POOL_SIZE 조정
- 🔒 Diagnostics Pack 필요: DBA_HIST_* 이력, AWR 어드바이저 섹션
- 대안: V$ 뷰를 두 시점에 조회해 델타 계산(ch26·ch27 경로)

퀴즈 (Quiz)

- Q1. free memory 67MB나 남았는데 ORA-04031. 총량이 충분한데 왜?
- Q2. SQL AREA GET_HIT 13.9% vs TABLE/PROCEDURE 95.2%. 이 차이가 가리키는 원인은?
- Q3. 하드파싱 비율 61.5%. 정상은 어디에 수렴해야 하고 61.5%는 무엇을 뜻하나?
- Q4. 어드바이저 SAVED가 416·512·560MB에서 모두 1.0. ORA-04031에 크기를 키워야 하나?
- Q5. ORA-04031 메시지에 shared pool/SQLA/kkscsAddChildNode. 원인 영역과 진짜 원인은?
- Q6. FORCE로 하드파싱 급감했는데 일부 SQL이 느려졌다. 무엇을 확인·대응하나?

요약 (Summary)

- 공유 풀=SQL·커서·실행계획·데이터사전. 부족의 진짜 원인은 대개 파싱
- V$SGASTAT: free 67MB여도 파편화로 ORA-04031 가능
- V$LIBRARYCACHE: SQL AREA GET_HIT 13.9%=커서 재사용 실패(하드파싱)
- 파싱 통계: 하드 61.5%=리터럴. CURSOR_SHARING=FORCE는 임시방편
- V$SHARED_POOL_ADVICE: 416MB 무릎, 확대 평평→크기론 해결 안 됨
- ORA-04031: 메시지로 영역 식별, 바인드→FORCE→예약풀→크기(최후)
- 라지 풀: PX msg 15MB 분리로 공유 풀 보호. 진단은 라이선스 무관

실습 개요 (Practice Overview)

non-CDB orcl 19.3(ASMM) · SYS@orcl / SQLT@orcl

- 실습 01 — 라이브러리 캐시(V$LIBRARYCACHE)와 재파싱 지표 판독
- 실습 02 — WL_01_parse로 하드 파싱 유발 후 공유 풀 변화 관찰
- 실습 03 — 커서 공유(EXACT/FORCE)별 공유 풀 점유·커서 개수 비교
- 실습 04 — 공유 풀 어드바이저(V$SHARED_POOL_ADVICE) 판독
- 실습 05 — ORA-04031 재현·원인 영역 식별·예약풀/크기 조정(개인환경 전용)
- 실습 06 — 라지 풀 구성과 병렬·RMAN 버퍼 분리 확인
- ※ 각 실습: 조치 전→조회·조치→해석 + 이상→원인→조치→재확인

Day 10

결과 캐시·PGA·I/O, 총정리, 종합 평가

- TUN ch33 · ch34 · 총정리 · 종합 진단 시나리오
- 8h — 평가 60%, 3h50m

Oracle Database 19c 성능 튜닝 실무 과정

결과 캐시와 PGA 튜닝

Result Cache & PGA Tuning — 재계산을 없애고, 정렬을 메모리에서 끝낸다

서버·클라이언트 결과 캐시 · 무효화 · 적용 판별 · 작업 영역 · optimal/one-pass/multi-pass · PGA_AGGREGATE_LIMIT · 실측 orcl 19.3

ch33

학습 목표 (Objectives)

- 서버·클라이언트 결과 캐시를 구분하고 RESULT_CACHE 힌트로 결과를 캐시에 올린다.
- 결과 캐시 무효화 과정을 관찰하고 V$RESULT_CACHE_STATISTICS로 건강 상태를 읽는다.
- 결과 캐시가 이득인 질의와 손해인 질의를 판별한다(변경 잦은 테이블은 손해).
- PGA 작업 영역의 optimal / one-pass / multi-pass 세 크기를 구분한다.
- V$PGASTAT·V$SQL_WORKAREA_HISTOGRAM으로 작업 영역 분포를 진단한다.
- 세션 수준 축소로 1-pass·multi-pass 정렬(디스크 정렬)을 재현한다.
- PGA_AGGREGATE_TARGET(목표)과 PGA_AGGREGATE_LIMIT(한도)의 차이·ORA-04036을 안다.

이 장의 구성 (Agenda)

- 33.2 결과 캐시란 무엇인가 — 서버·클라이언트
- 33.3 활성화와 RESULT_CACHE 힌트 · 요건과 제약
- 33.4~33.5 무효화 · 모니터링(V$RESULT_CACHE_STATISTICS)
- 33.6 적용 부적합 판별 — 변경 잦은 테이블은 손해
- 33.7~33.8 PGA 작업 영역 · optimal/one-pass/multi-pass
- 33.9 PGA 모니터링 — V$PGASTAT·작업 영역 히스토그램
- 33.10 1-pass·multi-pass 정렬 재현(세션 수준 축소)
- 33.11 PGA_AGGREGATE_LIMIT — 절대 한도와 ORA-04036

왜 결과 캐시와 PGA인가

ch33의 두 문장

- 같은 공유 풀 안, 성격이 정반대인 두 메모리를 다룬다
- 결과 캐시 = 공유되는 '질의 결과' 캐시(재계산 제거)
- PGA = 서버 프로세스의 사적 작업 공간(정렬·해시)
- 결과 캐시: 읽기 위주·소량 반환·반복 호출에서만 이득
- PGA: 작업 영역이 optimal로 도느냐가 전부
- 실측 orcl 19.3 — PGA 700M, 디스크 정렬 0인 '건강한 기본 상태'

33.2

결과 캐시란 무엇인가

- 서버 결과 캐시 — 공유 풀 안의 메모리 풀
- 클라이언트 결과 캐시 — OCI 클라이언트 메모리
- 실행계획의 RESULT CACHE 연산으로 확인

세 가지 캐시, 서로 다른 층위

무엇을 저장해 무엇을 아끼는가

- 버퍼 캐시(ch31): '데이터 블록' 저장 → 물리 I/O 절감
- 라이브러리 캐시(ch32): '실행계획' 저장 → 하드 파싱 절감
- 결과 캐시(ch33): '질의의 최종 결과' 저장 → 실행 자체 제거
- 결과 캐시가 이득이 가장 크다 — 히트하면 실행 단계 생략
- 그만큼 '기반 데이터가 안 바뀐다'는 조건이 까다롭다
- 서버 캐시=공유 풀 / 클라이언트 캐시=OCI 프로세스 메모리(별개)

실행계획에서 결과 캐시 확인

RESULT_CACHE 힌트와 RESULT CACHE 연산

```
SELECT /*+ RESULT_CACHE */ department_id, AVG(salary)
  FROM hr.employees
 GROUP BY department_id;

-----------------------------------------------------------
| Id | Operation           | Name                        |
-----------------------------------------------------------
|  0 | SELECT STATEMENT    |                             |
|  1 |  RESULT CACHE       | 8fpza04gtwsfr6n595au15yj4y |  <= 여기
|  2 |   HASH GROUP BY     |                             |
|  3 |    TABLE ACCESS FULL| EMPLOYEES                   |
-----------------------------------------------------------
```

클라이언트 결과 캐시 — OCI 프로세스 안

서버 캐시와 완전히 별개

- 클라이언트 프로세스 안의 메모리 — 그 프로세스 세션들이 공유
- OCCI·JDBC OCI 드라이버·ODP.NET 지원
- 이점 ① 서버 왕복 없이 응답 → 응답시간↓
- 이점 ② 서버 CPU·I/O 절약 → 확장성↑ / ③ 값싼 클라이언트 메모리
- 활성화: CLIENT_RESULT_CACHE_SIZE ≥ 32768바이트(기본 0=비활성)
- 서버가 변경 감지 시 다음 왕복에 무효화 전송 → 일관성 유지

33.3

활성화와 RESULT_CACHE 힌트

- 결과 캐시 모드 — MANUAL(기본·권장) vs FORCE
- 우선순위 — 힌트 > 테이블 주석 > 모드
- 요건과 제약 · DRUG_MASTER가 좋은 후보인 이유

결과 캐시 모드 — MANUAL이 기본이자 정답

RESULT_CACHE_MODE

- MANUAL: 힌트·테이블 주석으로 명시한 결과만 캐시(기본·권장)
- FORCE: 모든 결과를 캐시 시도 → NO_RESULT_CACHE로만 제외
- 실측: result_cache_mode = MANUAL (기본)
- FORCE 비권장 ① 모든 질의 캐시 시도 → 래치·성능 오버헤드
- FORCE 비권장 ② 비결정적 함수까지 캐시 → 낡은 결과 위험
- '캐시가 좋다니 전부 켜자'가 정확히 이 함정

우선순위: 힌트 > 테이블 주석 > 모드

같은 테이블도 질의별로 캐시 여부를 뒤집는다

```
-- 테이블 주석: 이 테이블 질의는 캐시 대상으로 강제
ALTER TABLE drug_master RESULT_CACHE (MODE FORCE);

-- 자주 쓰고 소량 반환 → 주석 덕분에 캐시 (이득)
SELECT category, COUNT(*) FROM drug_master GROUP BY category;

-- 일회성·대량 반환 → 힌트로 캐시 제외 (힌트가 최우선)
SELECT /*+ NO_RESULT_CACHE */ *
  FROM drug_master ORDER BY apply_date DESC;
```

DRUG_MASTER — 왜 좋은 후보인가

후보를 고르는 세 가지 질문

- ① 읽기 위주인가? — DRUG_MASTER(수가·약가 마스터)는 거의 안 변함
- ② 반환 행이 적은가? — 분류별 집계는 수십 줄 (MAX_RESULT 5% 이내)
- ③ 반복 호출되는가? — 같은 값으로 자주 실행되어야 Find가 쌓임
- 적합: DRUG_MASTER(수천 건, 읽기 위주 참조)
- 부적합: CLAIM_DETAILS(100만+, 계속 적재·변경)
- 요건: 읽기 일관성·바인드 파라미터화 / 금지: SYSDATE·시퀀스·임시테이블

캐시될 자격 — 요건과 제약

힌트를 붙였는데 왜 캐시가 안 되지?

- 읽기 일관성: 가장 최근 커밋 상태(또는 플래시백 시점)여야 함
- 활성 트랜잭션이 참조 객체를 걸고 있으면 캐시 대상 아님
- 파라미터화: 바인드·USER·SYS_CONTEXT(상수)·NLS면 같은 값에 재사용
- 금지 ① 임시 테이블, SYS·SYSTEM 스키마 테이블
- 금지 ② 시퀀스 CURRVAL·NEXTVAL
- 금지 ③ SYSDATE·SYSTIMESTAMP·CURRENT_DATE·SYS_GUID 등

33.4 ~ 33.5

무효화와 모니터링

- 무효화 — 기반 테이블이 바뀌면 결과가 낡는다
- V$RESULT_CACHE_STATISTICS로 건강 상태 판독
- Find ↑ · Invalidation≈0 · Failure≈0 = 건강

무효화 — 정확성의 대가

기반 객체가 바뀌면 결과는 무효

- 캐시된 결과가 의존하는 객체가 DML·DDL로 바뀌면 즉시 무효
- 다음 실행은 히트 대신 재계산 → 캐시 갱신 (자동·투명)
- 무효화 단위는 '결과 하나' — 같은 테이블 의존 결과는 함께 무효
- 커밋 기준 — 변경만 하고 커밋 안 하면 다른 세션 캐시는 유효
- V$RESULT_CACHE_OBJECTS.STATUS: Published(유효)/Invalid(무효)
- 이 무효화가 축복이자 저주 — 잦으면 손해(33.6)

V$RESULT_CACHE_STATISTICS 판독 (실측)

Find는 높게, Invalidation·Failure는 낮게

```
NAME                          VALUE    -- 해석
------------------------------------------------------------
Block Size (Bytes)             1024
Block Count Maximum            7712     -- 최대 확보 가능
Block Count Current              32     -- 현재 사용(여유 큼)
Create Count Success              1     -- 생성 성공
Create Count Failure              0     -- 낮을수록 좋음
Find Count                        5     -- 캐시 히트(높을수록 좋음)
Invalidation Count                0     -- 낮을수록 좋음
Delete Count Valid                0     -- 낮을수록 좋음
```

메모리 사용 요약 — MEMORY_REPORT

DBMS_RESULT_CACHE로 한눈에

```
SQL> SET SERVEROUTPUT ON
SQL> EXECUTE DBMS_RESULT_CACHE.MEMORY_REPORT

R e s u l t   C a c h e   M e m o r y   R e p o r t
[Parameters]
Block Size          = 1024 bytes
Maximum Cache Size  = 7712K bytes (7712 blocks)
Maximum Result Size = 385 blocks
[Memory]  Total Memory = ...  Cache Memory = 32 blocks
```

33.6

적용 부적합 판별

- 변경 잦은 테이블에서는 손해다 (DBA Point)
- 생성-무효화-재생성 반복 → 관리 비용만, 이득 없음
- 판별 한 문장 + 무효화 통계로 사후 검증

결과 캐시가 손해가 되는 순간

변경 잦은 테이블은 붙이지 않는다

- 이득의 원천 = '한 번 계산해 여러 번 재사용'
- 기반 테이블이 자주 바뀌면 캐시가 자꾸 무효화됨
- 무효화 다음 실행 = 히트 아니라 재계산 → 다시 저장
- 생성→짧은 재사용→무효화→재생성 반복 = 관리 비용만
- 심하면 결과 캐시 래치 경합이라는 새 병목 발생
- 조치: 캐시를 키우지 말고 그 질의에서 캐시를 '뗀다'

적합 vs 부적합 — 한눈에

진료비 스키마로 대조

- 적합: 읽기 위주(DRUG_MASTER) / 적은 반환 / 반복 호출
- 부적합: 쓰기 잦음(CLAIM_DETAILS) / 대량 반환 / 일회성
- 신호(적합): Find ↑ · Invalidation≈0 · Create Failure≈0
- 신호(부적합): Invalidation ↑ · Create Failure ↑ · Find 낮음
- 판별 한 문장: '읽기 위주 테이블에서 적은 결과를 반복 반환하는가'
- 붙인 뒤에도 Invalidation Count 관찰 → 손해면 즉시 제거

33.7 ~ 33.8

PGA와 작업 영역

- PGA — 서버 프로세스의 사적 작업 공간
- 정렬 영역·해시 영역이 작업 영역을 쓴다
- optimal / one-pass / multi-pass 세 크기

PGA와 작업 영역

공유되지 않는 사적 메모리

- PGA = 서버 프로세스 하나의 사적 메모리(그 프로세스만 접근)
- 정렬(ORDER BY·GROUP BY·윈도우) → 정렬 영역(sort area)
- 해시 조인 → 해시 영역(hash area)
- 작업 영역 크면 빠르지만 메모리 소비↑, 작으면 디스크로 넘침
- SGA와 달리 공유 안 됨 → 접속 세션 늘면 PGA 수요도 늘어
- 동시 작업 영역↑ → global memory bound↓ (하나에 줄 최대치 감소)

optimal · one-pass · multi-pass

PGA 진단의 공통 언어

- optimal: 전부 메모리에서 끝남 — 이상적, 디스크 접근 없음
- one-pass: optimal보다 작아 디스크 1회 경유 — 응답시간 증가
- multi-pass: 여러 번 디스크 경유 — 응답시간 급증(치명적)
- 목표: optimal 90%+ (OLTP 100%), one-pass 10%-, multi-pass 회피
- 핵심 감각: one-pass는 감수, multi-pass는 제거
- one-pass 메모리 요구는 작다 — 1GB 정렬을 22MB로 one-pass

33.9

PGA 모니터링

- V$PGASTAT — target·over allocation·cache hit
- V$SQL_WORKAREA_HISTOGRAM — 크기별 optimal 분포
- 실측: 전부 optimal, 디스크 정렬 0 (건강)

V$PGASTAT 판독 (실측)

target은 목표이지 한도가 아니다

```
NAME                                 MB   -- 해석
------------------------------------------------------------
aggregate PGA target parameter       700   -- 목표(한도 아님)
total PGA inuse                      297.41
total PGA allocated                  487.82  -- target 700 이내
maximum PGA allocated                510.74  -- 기동 이후 최대
over allocation count                  0   -- 초과 할당 0(건강)
cache hit percentage                 100   -- 전부 optimal(이상적)
```

작업 영역 히스토그램 (실측)

모든 버킷에서 onepass·multipass = 0

```
LOW_KB | HIGH_KB | OPTIMAL | ONEPASS | MULTIPASS
-----------------------------------------------
     2 |      4  |   25677 |    0    |    0
    64 |    128  |   20190 |    0    |    0
   512 |   1024  |   10872 |    0    |    0
  1024 |   2048  |   10690 |    0    |    0
 32768 | 65536   |       2 |    0    |    0   <= 큰 정렬도 optimal

-- V$SYSSTAT: sorts(disk)=0, sorts(memory)=67163
```

PGA_AGGREGATE_TARGET 튜닝 — 추측하지 않는다

V$PGA_TARGET_ADVICE로 근거 있게

```
SELECT ROUND(pga_target_for_estimate/1024/1024) target_mb,
       estd_pga_cache_hit_percentage cache_hit_perc,
       estd_overalloc_count
  FROM v$pga_target_advice;

-- 절차 ① over allocation=0 되는 지점 이상으로 target
--      ② 응답시간·메모리 제약 안에서 cache hit 최대화
--      ③ 조정 후 실제 optimal/one/multi 분포로 검증
```

범인 좁히기 — 세션·커서 수준

히스토그램이 이상하면 여기서 지목

- V$PROCESS_MEMORY: 프로세스별 PGA를 범주(SQL·PL/SQL·기타)로 분해
- → 조정 불가능 PGA 폭주 세션 지목(ORA-04036 원인 추적)
- V$SQL_WORKAREA: 커서별 작업 영역 누적 통계
- → V$SQL·V$SQL_PLAN 조인 시 어느 연산자가 one/multi-pass인지
- one/multi-pass 커서만 뽑는 질의 = 실측에선 '빈 결과'(건강 증명)
- V$SQL_WORKAREA_ACTIVE: 실행 중 NUMBER_PASSES·TEMPSEG_SIZE 실시간

33.10

1-pass·multi-pass 정렬 재현

- 건강한 상태에서 나쁜 상태를 일부러 만든다
- 세션 수준 축소 — 인스턴스는 건드리지 않는다
- 디스크 정렬·임시 테이블스페이스 사용 관찰

세션 수준 축소로 디스크 정렬 재현

인스턴스 파라미터는 절대 건드리지 않는다

```
-- (개인 실습 환경 전용) 이 세션의 작업 영역만 축소
ALTER SESSION SET WORKAREA_SIZE_POLICY = MANUAL;
ALTER SESSION SET SORT_AREA_SIZE = 65536;   -- 64KB
ALTER SESSION SET HASH_AREA_SIZE = 65536;

-- 대량 정렬 (CLAIM_DETAILS 100만 행 규모)
SELECT drug_code, COUNT(*), SUM(claim_amount)
  FROM claim_details GROUP BY drug_code ORDER BY 3 DESC;
-- 관찰: sorts(disk)↑, 히스토그램 onepass↑, TEMPSEG 사용
```

재현의 목적 — 정상값을 몸에 익힌다

전후 대비가 곧 학습

- 목적은 '느리게 만들기'가 아니라 '지표 반응을 눈에 익히기'
- 출발: optimal(sorts disk=0, 히스토그램 전부 optimal)
- 축소 후: sorts(disk) 0→튐, 히스토그램 onepass 잡힘
- 더 축소: one-pass → multi-pass, 응답시간 급격히 악화 체감
- 임시 공간 부족 시 ORA-01652(임시 세그먼트 확장 실패)
- 인스턴스 축소는 개인 환경 전용 — 조치 전 값 캡처·원복 필수

33.11

PGA_AGGREGATE_LIMIT

- target(목표) vs limit(절대 한도)
- 조정 가능 메모리 vs 조정 불가능 메모리
- ORA-04036 — 메모리 부족으로만 읽지 않는다

target은 목표, limit은 천장

ORA-04036의 진짜 의미

- target은 목표이지 한도 아님 — 소폭·잠깐 초과는 정상
- target은 '조정 가능한' 메모리만 통제(정렬·해시 영역)
- '조정 불가능한' 메모리(PL/SQL 컬렉션 등)는 통제 밖
- limit 초과 시 조정 불가능 PGA 최대 세션의 콜 중단→세션 종료
- 그 오류가 ORA-04036 — '안전장치가 작동했다'는 신호
- 오판 금지: 메모리 부족이 아니라 특정 세션 폭주가 진짜 원인

ORA-04036 대응 — 올바른 순서

한도를 키우는 건 처음이 아니라 마지막

- Common Error: ORA-04036을 메모리 부족으로만 읽고 한도만 키움
- 진짜 원인: 특정 세션의 조정 불가능 PGA 폭주(거대 컬렉션 등)
- ① V$PROCESS_MEMORY로 범인 세션·범주 지목
- ② 그 SQL·PL/SQL을 고친다
- ③ 필요하면 근거 있게 한도 조정 (마지막 수단)
- 세션 단위 한도: 리소스 관리자 SESSION_PGA_LIMIT → ORA-10260

퀴즈 (Quiz)

스스로 점검

- Q1. 서버 결과 캐시와 버퍼 캐시가 저장하는 대상의 차이는?
- Q2. RESULT_CACHE_MODE를 FORCE로 두면 안 되는 이유 두 가지는?
- Q3. DRUG_MASTER는 적합하고 CLAIM_DETAILS는 부적합한 이유(무효화 관점)?
- Q4. V$RESULT_CACHE_STATISTICS에서 '건강한 캐시' 통계 조합은?
- Q5. optimal·one-pass·multi-pass 정의와 실무 목표는?
- Q6. cache hit 100%인데도 1-pass를 세션 수준으로 재현하는 이유·방법은?

요약 (Summary)

ch33 핵심 정리

- 결과 캐시 = 완성된 결과 저장 → 재계산 제거. 서버/클라이언트 별개
- 모드는 MANUAL(기본·권장). 힌트 > 테이블 주석 > 모드
- 무효화가 정확성을 지키지만 변경 잦으면 손해 → 판별 필수
- 판별 한 문장: 읽기 위주·소량 반환·반복 호출인가
- PGA 작업 영역은 optimal 90%+ 목표, multi-pass 회피
- 실측: 700M·over alloc 0·cache hit 100%·디스크 정렬 0(건강)
- target(목표) vs limit(한도). ORA-04036은 세션 폭주가 진짜 원인

실습 개요 (Practice Overview)

6개 실습 · non-CDB orcl 19.3 · 조치 전→조회→해석

- 실습 01 — 서버 결과 캐시 활성화와 RESULT_CACHE 힌트(DRUG_MASTER)
- 실습 02 — 결과 캐시 무효화 관찰(기반 테이블 변경)
- 실습 03 — 결과 캐시 모니터링과 적용 부적합 판별
- 실습 04 — PGA 작업 영역 확인(V$PGASTAT·V$SQL_WORKAREA_HISTOGRAM)
- 실습 05 — 1-pass·multi-pass 정렬 재현(세션 축소·임시 테이블스페이스)
- 실습 06 — PGA_AGGREGATE_LIMIT 초과(ORA-04036) 재현과 조정

Oracle Database 19c 성능 튜닝 실무 과정

I/O 구성과 OS 자원 관리

I/O Configuration & Managing OS Resources — 병목이 DB 안인가 밖인가

파일별 I/O 분포 · CALIBRATE_IO · 비동기 I/O · 다중 블록 읽기 · V$OSSTAT CPU·메모리 · 네트워크 오판

ch34

학습 목표 (Objectives)

- V$FILESTAT·V$IOSTAT_FILE로 데이터파일별 I/O 분포를 읽고 편중을 판별한다.
- 시스템 통계가 NOWORKLOAD(MBRC·SREADTIM NULL)인지 확인한다.
- CALIBRATE_IO의 전제조건·실행·결과(max_iops·max_mbps·latency)를 이해한다.
- 비동기 I/O(FILESYSTEMIO_OPTIONS·DISK_ASYNCH_IO)와 다중 블록 읽기를 점검한다.
- V$OSSTAT로 CPU 사용률을 계산하고 DB 통계와 대조해 병목 위치를 가른다.
- 스와핑 서버에서 SGA를 키우면 안 되는 이유를 설명한다.
- SQL*Net message from client를 Idle로 올바르게 해석한다.

이 장의 구성 (Agenda)

- 34.2 데이터파일별 I/O 분포 — 편중 판별
- 34.3 I/O 캘리브레이션 — CALIBRATE_IO
- 34.4 비동기 I/O와 다중 블록 읽기
- 34.5 병목이 DB 안인가 밖인가 — OS 자원 판별
- 34.6 네트워크 지연 오판 — SQL*Net message from client
- 34.7 측정 → 진단 → 조치 → 재측정 (과정 전체 종합)

34.2

데이터파일별 I/O 분포

- V$FILESTAT — 파일별 읽기·쓰기·평균 지연
- 편중 판별: 절대값이 아니라 상대 비교·델타
- V$IOSTAT_FILE — 파일 유형별 소량/대량 읽기

V$FILESTAT가 보여 주는 것

파일별 물리 I/O의 분포와 지연

- PHYRDS(읽기)·PHYWRTS(쓰기)·READTIM·WRITETIM 누적
- 평균 읽기 지연 ≈ READTIM / PHYRDS
- DBA_DATA_FILES와 조인해 파일 이름별로 본다
- 누적값이므로 구간은 델타로 계산

V$FILESTAT 실측 — 파일별 I/O

읽기 횟수 내림차순 (orcl)

```
SQL> SELECT d.file_name, f.phyrds reads, f.phywrts writes,
  2         ROUND(f.readtim/GREATEST(f.phyrds,1),2) avg_rd_ms
  3    FROM v$filestat f JOIN dba_data_files d ON f.file#=d.file_id
  4    ORDER BY f.phyrds DESC;

FILE_NAME                 READS  WRITES  AVG_RD_MS
------------------------- ------ ------- ---------
system01.dbf              13917    1344      0.34
sysaux01.dbf               7222    7584      0.47
users01.dbf                2340       0      0.97
ts_stg01.dbf                404       0      0.54
undotbs01.dbf                68    2007      0.38
```

편중 판별 — 무엇이 문제인가

용도로 설명되면 정상, 아니면 조사

- SYSTEM=읽기, UNDO=쓰기, SYSAUX=균형 → 정상 편중
- 문제는 '용도로 설명되지 않는 편중'
- 판정은 절대 임계값이 아니라 같은 인스턴스 다른 파일 대비 배수
- 읽기 병목(캐시·인덱스)과 쓰기 병목(DBWR·리두)은 조치가 다르다

V$IOSTAT_FILE 실측 — 파일 유형별

소량(랜덤)·대량(순차) 읽기 구분

```
SQL> SELECT filetype_name,
  2      ROUND(small_read_megabytes,1) sm_rd_mb,
  3      ROUND(large_read_megabytes,1) lg_rd_mb,
  4      small_read_reqs, large_read_reqs
  5    FROM v$iostat_file ...;

FILETYPE_NAME  SM_RD_MB LG_RD_MB SM_REQS  LG_REQS
-------------- -------- -------- -------  -------
Control File       1776       40  113662       48
Data File           113      988   12940     1023
Data File            92      888    6356      913
```

실무 — '디스크가 느리다'를 분포로 번역

증설 전에 파일 분포부터

- 신고: 야간 배치가 느려짐, 인프라팀은 '스토리지 교체' 주장
- 1차: V$FILESTAT를 배치 시작·종료에 조회해 델타
- 발견: 한 업무 파일의 PHYRDS·대량 읽기만 급증
- 해석: 스토리지 전체가 아니라 한 세그먼트 전체 스캔
- 조치: 세그먼트→SQL 계획 점검, 그래도면 CALIBRATE_IO로 근거 후 판단

34.3

I/O 캘리브레이션

- NOWORKLOAD 확인 — MBRC·SREADTIM NULL
- CALIBRATE_IO — 랜덤 I/O로 상한 실측
- 결과: max_iops·max_mbps·latency

먼저 상태 확인 — NOWORKLOAD?

AUX_STATS$ (실측: 미수행)

```
SQL> SELECT pname, pval1 FROM sys.aux_stats$
  2    WHERE pname IN ('IOSEEKTIM','IOTFRSPEED','MBRC',
  3       'SREADTIM','MREADTIM','MAXTHR','SLAVETHR');

PNAME            PVAL1
---------------- ----------
IOSEEKTIM        10
IOTFRSPEED       4096
MBRC
MREADTIM
SREADTIM
```

시스템 통계 두 종류를 구분

목적이 다르다

- GATHER_SYSTEM_STATS → 옵티마이저 비용용 SREADTIM·MREADTIM·MBRC
- CALIBRATE_IO → 스토리지 상한 max_iops·max_mbps
- 실측: 둘 다 미수행(NOWORKLOAD)
- 실습 목표: 미수행 확인 + 실행 절차 습득

CALIBRATE_IO — 전제조건과 절차

DBMS_RESOURCE_MANAGER.CALIBRATE_IO

- 전제 1: SYSDBA 권한
- 전제 2: TIMED_STATISTICS = TRUE
- 전제 3: 데이터파일 비동기 I/O 활성(FILESYSTEMIO_OPTIONS=SETALL 등)
- 1단계 8KB 랜덤 → max_iops(OLTP), 2단계 1MB 랜덤 → max_mbps(DW)
- 무거운 I/O 유발 → 반드시 유휴/비피크 시간에만

CALIBRATE_IO 실행

결과는 스토리지 의존 → <환경값>

```
SQL> SET SERVEROUTPUT ON
SQL> DECLARE
  2    lat NUMBER; iops INTEGER; mbps INTEGER;
  3  BEGIN
  4    DBMS_RESOURCE_MANAGER.CALIBRATE_IO(2,10,iops,mbps,lat);
  5    DBMS_OUTPUT.PUT_LINE('iops='||iops||' mbps='||mbps);
  6  END;
  7  /
max_iops = <환경값>
max_mbps = <환경값>
latency  = <환경값>
```

결과 확인과 해석

V$IO_CALIBRATION_STATUS / DBA_RSRC_IO_CALIBRATE

- 상태: NOT AVAILABLE → IN PROGRESS → READY
- 결과 테이블: DBA_RSRC_IO_CALIBRATE (max_iops·max_mbps·latency)
- max_iops 낮고 sequential read 크다 → 랜덤 I/O 병목(OLTP)
- max_mbps 낮고 scattered read 크다 → 순차 대역폭 병목(DW)
- Orion은 소개만 — DB 내부 CALIBRATE_IO 중심

34.4

비동기 I/O와 다중 블록 읽기

- 동기 vs 비동기 I/O
- FILESYSTEMIO_OPTIONS · DISK_ASYNCH_IO
- DB_FILE_MULTIBLOCK_READ_COUNT · 직접 I/O

동기 vs 비동기 I/O

가능하면 비동기로 병목 회피

- 동기: 요청 제출 후 완료까지 프로세스가 멈춰 대기
- 비동기: 처리되는 동안 다른 작업을 계속
- DBWR 병렬 쓰기·전체 스캔 겹친 읽기에서 효과
- 플랫폼별로 기본 활성 여부가 다름

I/O 파라미터 실측

V$PARAMETER (orcl)

```
SQL> SELECT name, display_value FROM v$parameter
  2    WHERE name IN ('filesystemio_options','disk_asynch_io',
  3      'db_file_multiblock_read_count','db_writer_processes');

NAME                           DISPLAY_VALUE
------------------------------ -------------
db_file_multiblock_read_count  128
db_writer_processes            1
disk_asynch_io                 TRUE
filesystemio_options           none
```

FILESYSTEMIO_OPTIONS와 다중 블록 읽기

값의 의미와 조정 판단

- ASYNCH / DIRECTIO / SETALL / NONE 네 값
- SETALL = 비동기 + 직접 I/O 모두 활성(캘리브레이션 전제)
- DB_FILE_MULTIBLOCK_READ_COUNT=128 → 한 번에 최대 약 1MB 순차 읽기
- 클수록 전체 스캔↑ 그러나 옵티마이저가 스캔 선호 가능
- 다중 블록 읽기 효과는 db file scattered read로 관찰

OS 파일 시스템 캐시와 직접 I/O

이중 캐싱을 피한다

- 파일 시스템 데이터파일은 기본적으로 FS 캐시를 거침
- 오라클 버퍼 캐시 + FS 캐시 = 이중 캐싱(메모리·CPU 낭비)
- 직접 I/O는 FS 캐시 우회 → CPU 절약
- 단, TEMP·NOCACHE LOB·병렬 직접 읽기는 OS 버퍼가 유리할 수도
- Windows는 항상 FS 캐시 우회 → 이 문제 없음

34.5

병목이 DB 안인가 밖인가

- V$OSSTAT — CPU·메모리 직접 조회
- CPU 사용률 계산과 DB 통계 대조
- 메모리 스와핑 징후와 SGA 재검토

왜 DB 밖을 봐야 하는가

OS 지표는 원인이 아니라 위치를 가른다

- 인스턴스를 튜닝해도 부족하면 시스템 시간을 본다
- OS 튜닝의 효과 기대는 낮다 — 구성·앱 변경이 더 크다
- OS 지표는 대개 증상(예: buffer busy → 시스템 콜 증가)
- 목적: OS 튜닝이 아니라 '병목의 위치 판별'

V$OSSTAT 실측 — CPU·메모리

라이선스 무관 (orcl)

```
SQL> SELECT stat_name, value FROM v$osstat
  2    WHERE stat_name IN ('NUM_CPUS','NUM_CPU_CORES',
  3      'PHYSICAL_MEMORY_BYTES','LOAD','IDLE_TIME','BUSY_TIME');

STAT_NAME               VALUE
---------------------- ---------------
BUSY_TIME                       653492
IDLE_TIME                     35695635
LOAD                    0.2099609375
NUM_CPUS                             8
PHYSICAL_MEMORY_BYTES       8044179456
```

CPU 사용률 계산과 기대치

숫자 하나로 좋다·나쁘다 못 정한다

- 사용률 = BUSY / (BUSY+IDLE) = 653492 / 36349127 ≈ 1.8%
- 누적 평균이므로 구간은 델타로 재계산
- 피크 90%는 정상일 수 있고, 평상시 높으면 위험
- 실측 1.8% = 무부하 → 병목을 논할 수 없음

판정 논리 — 병목의 위치

OS CPU와 DB Time을 대조

- CPU 높고 오라클이 씀 + DB CPU 비중 높음 → 병목 DB 안(SQL 튜닝)
- CPU 높지만 비오라클 프로세스가 씀 → 병목 DB 밖
- CPU 한가한데 DB Time 대부분 대기 → I/O·잠금 대기
- 실측은 무부하 → WL_02_io 부하 후 델타로 판정(실습 05)

실무 — 스와핑 서버에서 SGA 확대 유혹

순서가 거꾸로다

- 제안: '히트율이 낮다' → SGA_TARGET 확대로 캐시 늘리자
- 그러나 서버 메모리는 이미 빠듯
- 먼저 볼 것: PHYSICAL_MEMORY vs SGA+PGA, 스왑 si/so
- 판정: 스와핑 관측되면 SGA 확대는 스왑 악화
- 조치: 스와핑 해소·프로세스 정리 → 필요시 메모리 증설 후 확대

메모리 스와핑 징후와 SGA 재검토

최악의 오조치를 피한다

- 물리 메모리 부족 → 페이징·스와핑 → 성능 급락
- PHYSICAL_MEMORY_BYTES vs SGA+PGA 합 대조
- 실측: 물리 약 7.5GiB, ASMM 운영(ch29)
- ★ 스와핑 중 SGA 확대는 최악 — 스왑 악화
- 조치 순서: 스와핑 해소 또는 메모리 증설 → 그 뒤 캐시 확대

34.6

네트워크 지연 오판

- SQL*Net message from client = Idle
- 한가한 시스템일수록 크게 보인다
- 진짜 네트워크는 전송 이벤트로 의심

SQL*Net message from client는 Idle

네트워크 병목이 아니다

- DB가 클라이언트의 다음 요청을 기다리는 유휴 시간
- 사용자 생각·앱 처리 시간이 여기 쌓임 → 대개 상위
- 한가한 시스템일수록 비중이 큼 → 병목으로 세면 오판
- 대기 클래스가 Idle → DB Time 병목 순위에서 제외

진짜 네트워크를 의심할 근거

전송 이벤트와 전송량 대비 시간

- SQL*Net more data to/from client 큼 → 대량 전송 의심
- SQL*Net message from dblink 큼 → 원격 DB·링크 지연
- 전송량 대비 시간이 길 때만 네트워크 조사
- 유휴 뒤에 숨은 비유휴 대기(느린 SQL·잠금·I/O)가 진짜 병목

34.7

측정 → 진단 → 조치 → 재측정

- 20개 장을 하나의 순환으로
- 병목이 DB 안인가 밖인가를 가른다
- 근거를 가지고 옳은 팀에 넘긴다

과정 전체를 한 번 꿰다

ch15에서 ch34까지 하나의 방법론

- 측정: DB Time → 대기 분해 → I/O면 V$FILESTAT, 자원이면 V$OSSTAT
- 진단: 병목이 DB 안(SQL·구성·메모리)인가 밖(스토리지·CPU·메모리·네트워크)인가
- 조치: 원인별 처방 — 스와핑이면 SGA를 키우지 않는다
- 재측정: 같은 지표를 델타로 다시 재어 판정 (조치 전 캡처 필수)
- 경계 긋기: SQL은 SQL 튜닝으로, 자원 부족은 인프라 증설로

퀴즈 (Quiz)

- Q1. undo=쓰기·system=읽기 편중이 문제가 아닌 이유는?
- Q2. MBRC·SREADTIM·MREADTIM이 NULL이면 무엇을 뜻하고 무엇을 하는가?
- Q3. CALIBRATE_IO 전제조건 셋과 유휴 시간에만 수행하는 이유는?
- Q4. BUSY=653492, IDLE=35695635의 CPU 사용률과 '한가하다' 전 확인할 것은?
- Q5. 스와핑 서버에서 SGA를 키우면 안 되는 이유와 올바른 조치 순서는?
- Q6. SQL*Net message from client를 네트워크 병목으로 보면 안 되는 이유는?

요약 (Summary)

- I/O 편중은 V$FILESTAT·V$IOSTAT_FILE로 상대 비교·델타로 판정한다.
- NOWORKLOAD(MBRC·SREADTIM NULL)이면 CALIBRATE_IO로 스토리지 상한을 실측한다.
- 비동기 I/O(disk_asynch_io=TRUE, filesystemio_options=none)·다중 블록 읽기(128)를 점검한다.
- V$OSSTAT로 CPU 사용률(실측 약 1.8%, 무부하)을 계산해 DB Time과 대조한다.
- 스와핑 중 SGA 확대는 금지 — 스와핑부터 해소한다.
- SQL*Net message from client는 Idle — 네트워크 오판을 교정한다.
- 전 과정은 측정→진단→조치→재측정의 한 순환이다.

실습 개요 (Practice Overview)

- 실습 01 — 데이터파일별 I/O 분포와 편중 판별(V$FILESTAT, WL_02_io)
- 실습 02 — I/O 캘리브레이션과 결과 해석(AUX_STATS$·CALIBRATE_IO)
- 실습 03 — 비동기 I/O 설정 확인(FILESYSTEMIO_OPTIONS·DISK_ASYNCH_IO)
- 실습 04 — 다중 블록 읽기와 전체 스캔(DB_FILE_MULTIBLOCK_READ_COUNT, WL_02_io)
- 실습 05 — OS CPU 사용률과 DB 통계 대조(V$OSSTAT)
- 실습 06 — 메모리 스와핑 징후와 SGA 재검토
- 실습 07 — 네트워크 지연 판별(SQL*Net message from client)
- ※ 환경: non-CDB orcl 19.3 · 프롬프트 SYS@orcl>/SQLT@orcl> · 조치 전→조회→해석 + 오류 흐름